A Cookbook of SPL Recipes

Declarative Approaches to Agentic Workflow Orchestration

Wen Gong

2026

Contents

1 Foundations **1**

1.1 Preface: The Conductor's Score **2**
1.1.1 Why This Book Exists . . . 2
1.1.2 This Book Is Written for Two Audiences . . . 2
1.1.2.1 Audience One: The SQL Professional . . . 2
1.1.2.2 Audience Two: The Global South . . . 3
1.1.3 The Bigger Vision: SPL + Momagrid . . . 3
1.1.4 The Conductor Metaphor . . . 3
1.1.5 What This Book Is Not . . . 4
1.1.6 How This Book Works . . . 4
1.1.7 A Note on Reproducibility . . . 5
1.1.8 A Note from the AI Quartet: Human × AI Partnership . . . 5
1.1.9 Acknowledgements . . . 6

1.2 Why Declarative? SQL as the Mental Model **8**
1.2.1 Who This Chapter Is For . . . 8
1.2.2 The Problem You Already Solved . . . 8
1.2.3 What Imperative LLM Code Looks Like . . . 9
1.2.4 The SQL Analogy . . . 11
1.2.5 The Mapping . . . 11
1.2.6 CALL vs. GENERATE: The Most Important Distinction . . . 13
1.2.7 SPL and Momagrid: Two Different Things . . . 15
1.2.8 The Adapter Abstraction . . . 15
1.2.9 Side by Side . . . 16
1.2.10 Why This Matters for SQL Practitioners Specifically . . . 18
1.2.11 One Language, Not Three . . . 19
1.2.12 How the Language Was Designed . . . 20
1.2.13 What Comes Next . . . 21

1.3 Running Your First Workflow **22**
1.3.1 The Path to Hello World . . . 22
1.3.2 Step 1: Install the Runtime . . . 22
1.3.3 Step 2: Set Up Your Orchestra (Ollama) . . . 22
1.3.4 Step 3: Run Your First Workflow . . . 23
1.3.5 The Adapter Configuration . . . 23
1.3.6 Troubleshooting . . . 23
1.3.7 What Just Happened? . . . 24

1.4 Running Recipes on a LAN Momagrid **25**
1.4.1 The Problem with a Single Node . . . 25
1.4.2 Architecture . . . 25
1.4.3 Prerequisites . . . 26
1.4.3.1 Every Machine in the Grid . . . 26
1.4.3.2 On the Hub Machine Only . . . 26
1.4.4 Step 1: Build the `mg` Binary . . . 26
1.4.5 Step 2: Start the Hub (Machine A) . . . 26
1.4.6 Step 3: Join Agents . . . 27
1.4.7 Step 4: Configure the SPL Momagrid Adapter . . . 28
1.4.8 Step 5: Run Any Recipe Against the Grid . . . 28
1.4.9 Monitoring the Grid . . . 28
1.4.10 Smoke Test Before Running All Recipes . . . 29
1.4.11 Troubleshooting . . . 29
1.4.12 What Just Happened . . . 30

2 Basics **31**

2.1 Hello World **32**
2.1.1 The Pattern . . . 32
2.1.2 The SPL Approach . . . 32
2.1.3 The .spl File (Annotated) . . . 32
2.1.3.1 (1) `PROMPT hello_world` . . . 33
2.1.3.2 (2) `SELECT system_role(...)` . . . 33
2.1.3.3 (3) `GENERATE greeting()` . . . 33
2.1.4 Running It . . . 34
2.1.5 What Just Happened . . . 35
2.1.6 Reproducibility Note . . . 35
2.1.7 When to Use This Pattern . . . 35
2.1.8 Exercises . . . 36
2.1.9 Appendix: Hello World Across All Adapters . . . 36
2.1.9.1 Local & LAN Adapters . . . 36
2.1.9.2 API Gateway Adapters . . . 37
2.1.9.3 Direct Cloud Provider Adapters . . . 37
2.1.9.4 Adapter Quick-Reference . . . 38

2.2 Ollama Proxy **40**
2.2.1 The Pattern . . . 40
2.2.2 The Open-Source Engine Room: Ollama and Llama.cpp . . . 40
2.2.3 The SPL Approach . . . 41
2.2.4 The .spl File (Annotated) . . . 41
2.2.4.1 (1) `PROMPT ollama_proxy` . . . 41
2.2.4.2 (2) `system_role(...)` . . . 41
2.2.4.3 (3) `context.prompt AS prompt` . . . 41
2.2.4.4 (4) `GENERATE answer(prompt)` . . . 42
2.2.5 Running It . . . 42
2.2.6 What Just Happened . . . 42

2.2.7 Reproducibility Note . . . 43
2.2.8 When to Use This Pattern . . . 43
2.2.9 Exercises . . . 43

2.3 Multilingual Greeting 44
2.3.1 The Pattern . . . 44
2.3.2 The SPL Approach . . . 44
2.3.3 The .spl File (Annotated) . . . 44
2.3.3.1 (1) `PROMPT multilingual_greeting` . . . 45
2.3.3.2 (2) `system_role(...)` . . . 45
2.3.3.3 (3) & (4) `context.user_input AS input` and `context.lang AS lang` . . . 45
2.3.3.4 (5) `GENERATE greeting(input, lang)` . . . 45
2.3.4 Running It . . . 45
2.3.5 What Just Happened . . . 46
2.3.6 Reproducibility Note . . . 46
2.3.7 When to Use This Pattern . . . 46
2.3.8 Exercises . . . 46

2.4 Few-Shot Learning 47
2.4.1 The Pattern . . . 47
2.4.2 The SPL Approach . . . 47
2.4.3 The .spl File (Annotated) . . . 47
2.4.3.1 (1) Dynamic Examples via `SELECT CASE` . . . 48
2.4.3.2 (2) Dynamic Context Injection . . . 48
2.4.3.3 (3) Calibrated Generation . . . 48
2.4.4 Running It . . . 48
2.4.5 What Just Happened . . . 49
2.4.6 Reproducibility Note . . . 49
2.4.7 When to Use This Pattern . . . 49
2.4.8 Exercises . . . 49

3 Agentic Patterns 50

3.1 Self-Refine 51
3.1.1 The Pattern . . . 51
3.1.2 The SPL Approach . . . 51
3.1.3 The .spl File (Annotated) . . . 51
3.1.4 The SQL Analogy in Depth . . . 53
3.1.5 Running It . . . 53
3.1.6 What Just Happened . . . 54
3.1.7 Reproducibility Note . . . 54
3.1.8 When to Use This Pattern . . . 55
3.1.9 Exercises . . . 55

3.2 ReAct Agent 56
3.2.1 The Pattern . . . 56
3.2.2 The SPL Approach . . . 56

3.2.3 The .spl File (Annotated) . . . 56
3.2.3.1 (1) `PROCEDURE name(...)` . . . 57
3.2.3.2 (2) Arithmetic in SPL . . . 57
3.2.3.3 (3) `CALL procedure_name(...)` . . . 57
3.2.3.4 (4) `CALL calc_growth_rate(...)` . . . 58
3.2.3.5 (5) `GENERATE growth_report(...)` . . . 58
3.2.4 Running It . . . 58
3.2.5 What Just Happened . . . 58
3.2.6 Reproducibility Note . . . 58
3.2.7 When to Use This Pattern . . . 59
3.2.8 Exercises . . . 59

3.3 Plan and Execute 60
3.3.1 The Pattern . . . 60
3.3.2 The SPL Approach . . . 60
3.3.3 The .spl File (Annotated) . . . 60
3.3.3.1 (1) Phase 1: Planning (The Blueprint) . . . 61
3.3.3.2 (2) Phase 2: Design Loop (Lightweight Reasoning) . . . 61
3.3.3.3 (3) Automatic Re-planning . . . 61
3.3.3.4 (4) Phase 3: Implementation Loop (Heavy Lifting) . . . 62
3.3.3.5 (5) Deterministic I/O (`CALL write_code_files`) . . . 62
3.3.4 Running It . . . 62
3.3.5 What Just Happened . . . 62
3.3.6 Reproducibility Note . . . 62
3.3.7 When to Use This Pattern . . . 62
3.3.8 Exercises . . . 63

3.4 Reflection Agent 64
3.4.1 The Pattern . . . 64
3.4.2 The SPL Approach . . . 64
3.4.3 The .spl File (Annotated) . . . 64
3.4.3.1 (1) The Initial Solve . . . 65
3.4.3.2 (2) The “Mirror” Step (`reflect`) . . . 65
3.4.3.3 (3) Quantifying Doubt (`confidence_score`) . . . 65
3.4.3.4 (4) The Confidence Threshold . . . 66
3.4.4 Running It . . . 66
3.4.5 What Just Happened . . . 66
3.4.6 Reproducibility Note . . . 66
3.4.7 When to Use This Pattern . . . 66
3.4.8 Exercises . . . 67

3.5 Multi-Model Pipeline 68
3.5.1 The Pattern . . . 68
3.5.2 The SPL Approach . . . 68
3.5.3 The .spl File (Annotated) . . . 68
3.5.3.1 (1) Step 1: Fast Retrieval . . . 69
3.5.3.2 (2) Step 2: Deep Reasoning . . . 69
3.5.3.3 (3) Step 3: Creative Writing . . . 69

3.5.3.4 (4) Step 4: The Neutral Judge . . . 69
3.5.4 Running It . . . 70
3.5.5 What Just Happened . . . 70
3.5.6 Reproducibility Note . . . 70
3.5.7 When to Use This Pattern . . . 70
3.5.8 Exercises . . . 70

3.6 Nested Procedures 72
3.6.1 The Pattern . . . 72
3.6.2 The SPL Approach . . . 72
3.6.3 The .spl File (Annotated) . . . 72
3.6.3.1 (1) The "Leaf" Procedure . . . 73
3.6.3.2 (2) The "Intermediate" Procedure . . . 73
3.6.3.3 (3) The Root Workflow . . . 73
3.6.3.4 (4) Delegation (CALL) . . . 74
3.6.4 Running It . . . 74
3.6.5 What Just Happened . . . 74
3.6.6 Reproducibility Note . . . 74
3.6.7 When to Use This Pattern . . . 74
3.6.8 Exercises . . . 75

4 Reasoning 76

4.1 Chain of Thought 77
4.1.1 The Pattern . . . 77
4.1.2 The SPL Approach . . . 77
4.1.3 The .spl File (Annotated) . . . 77
4.1.4 Running It . . . 79
4.1.5 What Just Happened . . . 80
4.1.6 Reproducibility Note . . . 80
4.1.7 When to Use This Pattern . . . 80
4.1.8 Exercises . . . 81

4.2 Tree of Thought 82
4.2.1 The Pattern . . . 82
4.2.2 The SPL Approach . . . 82
4.2.3 The .spl File (Annotated) . . . 82
4.2.3.1 (1) Phase 1: Branching . . . 83
4.2.3.2 (2) Phase 2: Developing . . . 83
4.2.3.3 (3) Phase 3: Scoring . . . 83
4.2.3.4 (4) The Final Check . . . 83
4.2.4 Running It . . . 84
4.2.5 What Just Happened . . . 84
4.2.6 Reproducibility Note . . . 84
4.2.7 When to Use This Pattern . . . 84
4.2.8 Exercises . . . 84

4.3 Hypothesis Tester **85**
4.3.1 The Pattern . . . 85
4.3.2 The SPL Approach . . . 85
4.3.3 The .spl File (Annotated) . . . 85
4.3.3.1 (1), (2) & (3) The Confidence Ladder . . . 86
4.3.4 Running It . . . 86
4.3.5 What Just Happened . . . 87
4.3.6 Reproducibility Note . . . 87
4.3.7 When to Use This Pattern . . . 87
4.3.8 Exercises . . . 87

5 Safety & Reliability **88**

5.1 Safe Generation **89**
5.1.1 The Pattern . . . 89
5.1.2 The SPL Approach . . . 89
5.1.3 The .spl File (Annotated) . . . 89
5.1.3.1 (1) & (2) Generation and Assessment . . . 90
5.1.3.2 (3) `EVALUATE` (Semantic Branching) . . . 90
5.1.3.3 (4) Refinement . . . 90
5.1.3.4 (5) `EXCEPTION` . . . 91
5.1.3.5 (6) `HallucinationDetected` . . . 91
5.1.3.6 (7) `RETRY WITH ...` . . . 91
5.1.4 Running It . . . 91
5.1.5 What Just Happened . . . 91
5.1.6 Reproducibility Note . . . 91
5.1.7 When to Use This Pattern . . . 91
5.1.8 Exercises . . . 92

5.2 Guardrails Pipeline **93**
5.2.1 The Pattern . . . 93
5.2.2 The SPL Approach . . . 93
5.2.3 The .spl File (Annotated) . . . 93
5.2.3.1 (1) Gate 1a: The Firewall (`CALL`) . . . 94
5.2.3.2 (2) Gate 1b: The Nuanced Check (`GENERATE`) . . . 94
5.2.3.3 (3) Gate 2: Data Masking (`CALL`) . . . 95
5.2.3.4 (4) Gate 4: The Final Audit (`GENERATE`) . . . 95
5.2.4 Running It . . . 95
5.2.5 What Just Happened . . . 95
5.2.6 Reproducibility Note . . . 95
5.2.7 When to Use This Pattern . . . 95
5.2.8 Exercises . . . 96

5.3 Error Handling Patterns **97**
5.3.1 The Pattern . . . 97
5.3.2 Who This Chapter Is For . . . 97
5.3.3 The SPL Exception System . . . 97

5.3.4 Exception Type Reference . . . 98
5.3.5 Pattern 1: Graceful Degradation . . . 98
5.3.6 Pattern 2: Retry with Backoff . . . 99
5.3.7 Pattern 3: Multi-Step Error Isolation . . . 100
5.3.8 Pattern 4: LLM-as-Judge for Output Validation . . . 101
5.3.9 The SQL Analogy . . . 102
5.3.10 Running It . . . 102
5.3.11 Reproducibility Note . . . 103
5.3.12 When to Use These Patterns . . . 103
5.3.13 Exercises . . . 103

5.4 Human-in-the-Loop Approval Gate 104
5.4.1 The Pattern . . . 104
5.4.2 The SPL Approach . . . 104
5.4.3 The `.spl` File (Annotated) . . . 104
5.4.4 The SQL Analogy . . . 106
5.4.5 Running It . . . 107
5.4.6 What Just Happened . . . 107
5.4.7 Reproducibility Note . . . 107
5.4.8 When to Use This Pattern . . . 107
5.4.9 Exercises . . . 108

6 Memory & Retrieval 109

6.1 RAG Query 110
6.1.1 The Pattern . . . 110
6.1.2 The SPL Approach . . . 110
6.1.3 The .spl File (Annotated) . . . 110
6.1.3.1 (1) `PROMPT rag_answer` . . . 111
6.1.3.2 (2) `system_role(...)` . . . 111
6.1.3.3 (3) `rag.query(context.question, top_k=3) AS background` . . . 111
6.1.3.4 (4) & (5) `question` and `GENERATE` . . . 111
6.1.4 Running It . . . 111
6.1.5 What Just Happened . . . 112
6.1.6 Reproducibility Note . . . 112
6.1.7 When to Use This Pattern . . . 112
6.1.8 Exercises . . . 112

6.2 Memory Conversation 113
6.2.1 The Pattern . . . 113
6.2.2 The SPL Approach . . . 113
6.2.3 The .spl File (Annotated) . . . 113
6.2.3.1 (1) Memory as a First-Class Parameter . . . 115
6.2.3.2 (2) Reading from Storage . . . 116
6.2.3.3 (3) Fact Extraction . . . 116
6.2.3.4 (4) Profile Merging . . . 116
6.2.3.5 (5) Writing Back to Storage . . . 116

6.2.3.6 (6) Deterministic History Trimming . . . 116
6.2.4 Running It . . . 116
6.2.5 What Just Happened . . . 117
6.2.6 The Prompt Design Lesson . . . 117
6.2.7 Model Notes . . . 118
6.2.8 When to Use This Pattern . . . 118
6.2.9 Exercises . . . 118

6.3 Chat Over Your PDF 119
6.3.1 The Pattern . . . 119
6.3.2 The SPL Approach . . . 119
6.3.3 The `.spl` File (Annotated) . . . 119
6.3.4 The SQL Analogy . . . 122
6.3.5 Running It . . . 122
6.3.6 What Just Happened . . . 123
6.3.7 Reproducibility Note . . . 123
6.3.8 When to Use This Pattern . . . 123
6.3.9 Exercises . . . 123

7 Multi-Agent Systems 125

7.1 Debate Arena 126
7.1.1 The Pattern . . . 126
7.1.2 The SPL Approach . . . 126
7.1.3 The .spl File (Annotated) . . . 126
7.1.3.1 (1) & (2) State Management . . . 127
7.1.3.2 (3) `WHILE` (Iterative Reasoning) . . . 128
7.1.3.3 (4) `||` for String Concatenation . . . 128
7.1.3.4 (5) LOGGING for Progress Visibility . . . 128
7.1.4 Running It . . . 128
7.1.5 What Just Happened . . . 128
7.1.6 Reproducibility Note . . . 128
7.1.7 When to Use This Pattern . . . 129
7.1.8 Exercises . . . 129

7.2 Multi-Agent Collaboration 130
7.2.1 The Pattern . . . 130
7.2.2 The SPL Approach . . . 130
7.2.3 The .spl File (Annotated) . . . 130
7.2.3.1 (1), (2) & (3) Procedures as Agents . . . 131
7.2.3.2 (4) Linear Delegation . . . 131
7.2.4 Running It . . . 131
7.2.5 What Just Happened . . . 132
7.2.6 Reproducibility Note . . . 132
7.2.7 When to Use This Pattern . . . 132
7.2.8 Exercises . . . 132

7.3 Ensemble Voting **133**
7.3.1 The Pattern . . . 133
7.3.2 The SPL Approach . . . 133
7.3.3 The .spl File (Annotated) . . . 133
7.3.3.1 (1) Phase 1: Multi-Sampling . . . 134
7.3.3.2 (2) Phase 2: Scoring . . . 134
7.3.3.3 (3) Phase 3: Finding Consensus . . . 134
7.3.3.4 (4) Phase 4: Polishing . . . 134
7.3.4 Running It . . . 134
7.3.5 What Just Happened . . . 135
7.3.6 Reproducibility Note . . . 135
7.3.7 When to Use This Pattern . . . 135
7.3.8 Exercises . . . 135

7.4 Socratic Tutor **136**
7.4.1 The Pattern . . . 136
7.4.2 The SPL Approach . . . 136
7.4.3 The .spl File (Annotated) . . . 136
7.4.3.1 (1) The "Dummy Student" (`simulate_student`) . . . 137
7.4.3.2 (2) Measuring Learning (`assess_understanding`) . . . 137
7.4.3.3 (3) & (4) Adaptive Branching . . . 138
7.4.4 Running It . . . 138
7.4.5 What Just Happened . . . 138
7.4.6 Reproducibility Note . . . 138
7.4.7 When to Use This Pattern . . . 138
7.4.8 Exercises . . . 138

7.5 Interview Sim **140**
7.5.1 The Pattern . . . 140
7.5.2 The SPL Approach . . . 140
7.5.3 The .spl File (Annotated) . . . 140
7.5.3.1 (1) Upfront Planning (`generate_questions`) . . . 141
7.5.3.2 (2) Role-play Generation . . . 141
7.5.3.3 (3) Applying the Rubric (`evaluation_rubric`) . . . 141
7.5.3.4 (4) Python Math Tool (`aggregate_scores`) . . . 141
7.5.4 Running It . . . 142
7.5.5 What Just Happened . . . 142
7.5.6 Reproducibility Note . . . 142
7.5.7 When to Use This Pattern . . . 142
7.5.8 Exercises . . . 142

8 Applications **143**

8.1 Text2SPL Demo **144**
8.1.1 The Pattern . . . 144
8.1.2 The SPL Approach . . . 144
8.1.3 The Compilation Process (Annotated) . . . 144

8.1.4 Running It . . . 145
8.1.5 What Just Happened . . . 145
8.1.6 Reproducibility Note . . . 145
8.1.7 When to Use This Pattern . . . 145
8.1.8 A Live Session: The Intent Router . . . 146
8.1.9 The Bootstrap Insight . . . 147
8.1.10 Exercises . . . 148
8.1.11 Want a Visual Interface? . . . 148

8.2 Map-Reduce Summarizer 150
8.2.1 The Pattern . . . 150
8.2.2 Design Thinking: The Efficient Runtime . . . 150
8.2.3 The SPL File (Annotated) . . . 151
8.2.3.1 (1) Native LIST — `@summaries := []` . . . 153
8.2.3.2 (2) CALL for `chunk_plan` — Zero Tokens . . . 153
8.2.3.3 (3) CALL for `extract_chunk` — Zero Tokens . . . 153
8.2.3.4 (4) `write_file` — Built-in File I/O . . . 153
8.2.3.5 (5) F-String Interpolation . . . 153
8.2.3.6 (6) list_append — Accumulate, Then Join . . . 154
8.2.3.7 (7) list_concat — The Bridge Between LIST and TEXT . . . 154
8.2.4 Running It . . . 154
8.2.5 What Just Happened . . . 154
8.2.6 Reproducibility Note . . . 155
8.2.7 When to Use This Pattern . . . 155
8.2.8 Exercises . . . 155

8.3 Code Review 156
8.3.1 The Pattern . . . 156
8.3.2 The SPL Approach . . . 156
8.3.3 The .spl File (Annotated) . . . 156
8.3.3.1 (1) Specialized Passes . . . 157
8.3.3.2 (2) Quantifying Risk (`severity_score`) . . . 157
8.3.3.3 (3) Threshold Branching . . . 157
8.3.4 Running It . . . 158
8.3.5 What Just Happened . . . 158
8.3.6 Reproducibility Note . . . 158
8.3.7 When to Use This Pattern . . . 158
8.3.8 Exercises . . . 158

8.4 Structured Output 159
8.4.1 The Pattern . . . 159
8.4.2 The SPL Approach . . . 159
8.4.3 The .spl File (Annotated) . . . 159
8.4.3.1 (1) Reusable Schema Definition . . . 160
8.4.3.2 (2) Enforcing Numeric Types . . . 160
8.4.3.3 (3) Schema Context Injection . . . 160
8.4.3.4 (4) Constrained Generation . . . 160
8.4.4 Running It . . . 161

8.4.5 What Just Happened . . . 161
8.4.6 Reproducibility Note . . . 161
8.4.7 When to Use This Pattern . . . 161
8.4.8 Exercises . . . 162

8.5 Data Extraction 163
8.5.1 The Pattern . . . 163
8.5.2 The SPL Approach . . . 163
8.5.3 The .spl File (Annotated) . . . 163
8.5.3.1 (1) & (2) The Blueprint Library . . . 164
8.5.3.2 (3) Injecting the right blueprint . . . 164
8.5.3.3 (4) Precision Extraction . . . 164
8.5.4 Running It . . . 164
8.5.5 What Just Happened . . . 165
8.5.6 Reproducibility Note . . . 165
8.5.7 When to Use This Pattern . . . 165
8.5.8 Exercises . . . 165

8.6 Support Triage 166
8.6.1 The Pattern . . . 166
8.6.2 The SPL Approach . . . 166
8.6.3 The .spl File (Annotated) . . . 166
8.6.3.1 (1) & (2) Fact Finding (`CALL`) . . . 167
8.6.3.2 (3) Grounded Classification . . . 167
8.6.3.3 (4) Critical Escalation . . . 167
8.6.3.4 (5) Citing the Ground Truth . . . 168
8.6.4 Running It . . . 168
8.6.5 What Just Happened . . . 168
8.6.6 Reproducibility Note . . . 168
8.6.7 When to Use This Pattern . . . 168
8.6.8 Exercises . . . 168

8.7 Meeting Actions 169
8.7.1 The Pattern . . . 169
8.7.2 The SPL Approach . . . 169
8.7.3 The .spl File (Annotated) . . . 169
8.7.3.1 (1) Finding the Actors (`extract_speakers`) . . . 170
8.7.3.2 (2) Noise Reduction (`normalize_transcript`) . . . 170
8.7.3.3 (3) The Schema Constraint (`action_item_schema`) . . . 170
8.7.3.4 (4) Date Fixing (`normalize_dates`) . . . 170
8.7.4 Running It . . . 171
8.7.5 What Just Happened . . . 171
8.7.6 Reproducibility Note . . . 171
8.7.7 When to Use This Pattern . . . 171
8.7.8 Exercises . . . 171

8.8 Code Gen + Tests 173
8.8.1 The Pattern . . . 173

8.8.2 The SPL Approach . . . 173
8.8.3 The .spl File (Annotated) . . . 173
8.8.3.1 (1) Injecting Style Guides (`language_conventions`) . . . 174
8.8.3.2 (2) Passing Code as Context . . . 174
8.8.3.3 (3) Choosing the Runner (`test_framework_guide`) . . . 174
8.8.4 Running It . . . 175
8.8.5 What Just Happened . . . 175
8.8.6 Reproducibility Note . . . 175
8.8.7 When to Use This Pattern . . . 175
8.8.8 Exercises . . . 175

8.9 Sentiment Pipeline 176
8.9.1 The Pattern . . . 176
8.9.2 The SPL Approach . . . 176
8.9.3 The .spl File (Annotated) . . . 176
8.9.3.1 (1) Python Splitting Tool (`split_items`) . . . 177
8.9.3.2 (2) The Extraction Schema (`sentiment_schema`) . . . 177
8.9.3.3 (3) Python Math Tool (`compute_stats`) . . . 177
8.9.4 Running It . . . 177
8.9.5 What Just Happened . . . 178
8.9.6 Reproducibility Note . . . 178
8.9.7 When to Use This Pattern . . . 178
8.9.8 Exercises . . . 178

8.10 Progressive Summary 179
8.10.1 The Pattern . . . 179
8.10.2 The SPL Approach . . . 179
8.10.3 The .spl File (Annotated) . . . 179
8.10.3.1 (1) Layer 1: The Core . . . 181
8.10.3.2 (2) Layer 2: The Brief (`expand_summary`) . . . 181
8.10.3.3 (3) Layer 3: The Report . . . 181
8.10.3.4 (4) LOGGING for Pipeline Visibility . . . 181
8.10.3.5 (5) `@audience` and Fidelity Check . . . 181
8.10.4 Running It . . . 182
8.10.5 What Just Happened . . . 182
8.10.6 Reproducibility Note . . . 182
8.10.7 When to Use This Pattern . . . 182
8.10.8 Exercises . . . 182

9 Benchmarking & Evaluation 183

9.1 Model Showdown 184
9.1.1 The Pattern . . . 184
9.1.2 The SPL Approach . . . 184
9.1.3 The .spl File (Annotated) . . . 184
9.1.3.1 (1) `CREATE FUNCTION ... AS $$ ... $$` . . . 186
9.1.3.2 (2) & (3) `WORKFLOW` and `INPUT` . . . 186

9.1.3.3 (4) `WITH ... AS (...)` (Parallel CTEs) . . . 186
9.1.3.4 (5) `USING MODEL @model_1` . . . 186
9.1.3.5 (6) `SELECT ... INTO ...` . . . 186
9.1.3.6 (7) The "Judge" Step . . . 186
9.1.4 Running It . . . 186
9.1.5 What Just Happened . . . 187
9.1.6 Reproducibility Note . . . 187
9.1.7 When to Use This Pattern . . . 187
9.1.8 Exercises . . . 187

9.2 Batch Test 188
9.2.1 The Pattern . . . 188
9.2.2 The SPL Approach . . . 188
9.2.3 The .spl File (Annotated) . . . 188
9.2.3.1 (1) Extreme Fan-Out . . . 189
9.2.3.2 (2) Synchronization Point . . . 189
9.2.3.3 (3) The Reporter . . . 190
9.2.3.4 (4) Final Commit . . . 190
9.2.4 Running It . . . 190
9.2.5 What Just Happened . . . 190
9.2.6 Reproducibility Note . . . 190
9.2.7 When to Use This Pattern . . . 191
9.2.8 Exercises . . . 191

9.3 A/B Test 192
9.3.1 The Pattern . . . 192
9.3.2 The SPL Approach . . . 192
9.3.3 The .spl File (Annotated) . . . 192
9.3.3.1 (1) & (2) The Tournament . . . 193
9.3.3.2 (3) Prose to Numbers (`extract_total`) . . . 193
9.3.3.3 (4) The Margin Check . . . 193
9.3.4 Running It . . . 194
9.3.5 What Just Happened . . . 194
9.3.6 Reproducibility Note . . . 194
9.3.7 When to Use This Pattern . . . 194
9.3.8 Exercises . . . 194

9.4 One Recipe, Four Adapters 195
9.4.1 The Pattern . . . 195
9.4.2 The Recipe Under Test . . . 195
9.4.3 Running It Across Four Adapters . . . 197
9.4.4 Results . . . 197
9.4.5 What Just Happened . . . 198
9.4.6 The SQL Analogy . . . 198
9.4.7 Reproducibility Note . . . 199
9.4.8 When to Use This Pattern . . . 199
9.4.9 Exercises . . . 199

10 The Road Ahead **200**

10.1 Declarative AI, Momagrid, and Where SPL Fits **201**
10.1.1 What 37 Recipes Taught Us . . . 201
10.1.2 Three Problems That Remain Open . . . 201
10.1.2.1 1. Dynamic workflows . . . 201
10.1.2.2 2. Long-running and stateful workflows . . . 202
10.1.2.3 3. Multi-tenant and multi-user workflows . . . 202
10.1.3 The Momagrid Vision: Status and Timeline . . . 202
10.1.4 The Type Progression: Why "Simple" Works Forever . . . 203
10.1.5 What SPL Is Not Trying to Be . . . 204
10.1.6 On Adoption: An Honest Assessment . . . 204
10.1.7 What Comes Next for You . . . 205

10.2 The Recipe Maker: SPL Eating Its Own Cake **206**
10.2.1 The Concept: What 37 Recipes Taught Us . . . 206
10.2.2 Cooking as Science and Art . . . 206
10.2.3 The Full Lifecycle . . . 207
10.2.4 The CLI Contract . . . 208
10.2.5 section ?0: Recipe Maker . . . 209
10.2.5.1 The Pattern . . . 209
10.2.5.2 The SPL Approach . . . 209
10.2.5.3 The .spl File (Annotated) . . . 209
10.2.5.4 Running It . . . 211
10.2.5.5 What Just Happened . . . 212
10.2.5.6 Reproducibility Note . . . 213
10.2.5.7 When to Use This Pattern . . . 213
10.2.5.8 Exercises . . . 213
10.2.6 A Note on What This Means . . . 214

11 Tools & Community **215**

11.1 Recipe Kitchen: A Browser UI for SPL **216**
11.1.1 What Is Recipe Kitchen? . . . 216
11.1.2 Starting the Kitchen . . . 216
11.1.3 Tab 1 — Playground . . . 216
11.1.4 Tab 2 — Recipe Maker . . . 217
11.1.5 Tab 3 — Bake-All . . . 217
11.1.6 Design Notes . . . 218
11.1.7 The Vision: A Community Portal . . . 218

11.2 SPL Cheatsheet **219**
11.2.1 Core Statements . . . 219
11.2.2 WORKFLOW Structure . . . 219
11.2.3 Keywords and Clauses . . . 219
11.2.3.1 Data flow . . . 219
11.2.3.2 Generation . . . 220

11.2.3.3 Control flow . . . 220
11.2.3.4 State . . . 220
11.2.3.5 Exception handling . . . 220
11.2.4 Built-in Error Types . . . 220
11.2.5 Tool Connectors (Planned) . . . 221
11.2.6 Adapter CLI Flags . . . 221
11.2.7 CALL vs GENERATE . . . 221
11.2.8 SQL → SPL Mapping . . . 222
11.2.9 Input / Output Types . . . 222
11.2.10 Recipe Template Structure . . . 222

11.3 SPL Standard Library 223
11.3.1 Type Conversion . . . 223
11.3.2 String Functions . . . 224
11.3.3 Pattern Matching . . . 224
11.3.4 Numeric Functions . . . 225
11.3.5 Conditional Functions . . . 225
11.3.6 Null and Empty Checks . . . 225
11.3.7 Text Aggregates . . . 226
11.3.8 JSON Functions . . . 226
11.3.9 Date and Time . . . 227
11.3.10 Hashing . . . 227
11.3.11 List / Array Helpers . . . 227
11.3.12 File I/O . . . 227
11.3.13 Stdlib vs. UDFs . . . 228

11.4 Extending SPL with User-Defined Functions 229
11.4.1 CALL Is SPL's Design Strength . . . 229
11.4.2 What Becomes Possible . . . 229
11.4.3 How CALL Dispatches . . . 230
11.4.4 The `@spl_tool` Decorator . . . 230
11.4.5 section 11.4 — The Definitive Example . . . 231
11.4.6 Writing Good UDFs . . . 232
11.4.6.1 Accept and return strings . . . 232
11.4.6.2 Fail safely . . . 232
11.4.6.3 Pure functions when possible . . . 232
11.4.6.4 Document with docstrings . . . 233
11.4.7 Custom Name . . . 233
11.4.8 Multiple Tools in One File . . . 233
11.4.9 Async Tools . . . 234
11.4.10 Stdlib vs. UDF: The Decision Rule . . . 234
11.4.11 Testing Your Tools . . . 234
11.4.12 The Guardrails Recipe: A Real UDF Pattern . . . 235

11.5 The Digital Duck Ecosystem 236
11.5.1 The Stack at a Glance . . . 236
11.5.2 The Libraries . . . 237
11.5.2.1 `dd-config` — Configuration Management . . . 237

11.5.2.2 `dd-logging` — Shared Logging . . . 237
11.5.2.3 `dd-db` — Relational Database Abstraction . . . 237
11.5.2.4 `dd-embed` — Embedding Abstraction . . . 238
11.5.2.5 `dd-vectordb` — Vector Database Abstraction . . . 238
11.5.2.6 `dd-llm` — LLM Abstraction . . . 238
11.5.2.7 `dd-cache` — Caching Abstraction . . . 239
11.5.2.8 `dd-extract` — Document Extraction . . . 239
11.5.2.9 `dd-format` — Document Formatting . . . 239
11.5.2.10 `dd-dogic` — Document Conversion Pipeline . . . 240
11.5.3 Integration Status . . . 240
11.5.4 Where to Find Them . . . 240

11.6 For Decision-Makers 242
11.6.1 The Question That Matters . . . 242
11.6.2 Who Can Contribute With SPL . . . 242
11.6.3 The Three Roles, One Language . . . 243
11.6.4 Documentation That Stays Current . . . 244
11.6.5 A Concrete Comparison: Ensemble Voting in SPL and LangGraph . . . 244
11.6.6 Deployment and Promotion Model . . . 245
11.6.7 Lines of Code: SPL vs Imperative Frameworks . . . 245
11.6.8 The Efficient Runtime: Token Cost Is a Real Budget Line . . . 245
11.6.8.1 Two Execution Modes, One Language . . . 246
11.6.8.2 The Decision Rule . . . 246
11.6.8.3 Polyglot Tools . . . 246
11.6.8.4 Momagrid: The Fourth Dimension . . . 247
11.6.9 Competitive Landscape . . . 247
11.6.10 The Adoption Risk: An Honest Assessment . . . 248
11.6.11 Summary: When to Choose SPL . . . 248

11.7 Glossary 249

11.8 About the Author 252
11.8.1 From Anhui to CERN: A Journey of Access . . . 252
11.8.2 The Physicist's Instinct: Declarative Thinking . . . 252
11.8.3 The AI Quartet and the 48-Hour Sprint . . . 253
11.8.4 The Vision: AI as a Public Utility . . . 253

Part 1

Foundations

Chapter 1.1

Preface: The Conductor's Score

1.1.1 Why This Book Exists

In early 2025, I watched a colleague — a skilled data engineer with a decade of SQL experience — struggle for three weeks to build a simple multi-step AI workflow. She had read the LangChain documentation, attended an AutoGen workshop, and completed an online course on AI agents. Yet there she was, drowning in callback functions, state machine definitions, and framework-specific abstractions that multiplied with every tutorial she followed.

"I just want to chain three LLM calls together," she told me over coffee. "Why does this require a hundred lines of Python?"

That question haunted me. The answer — *because that's how the frameworks are designed* — wasn't good enough. SQL has been solving orchestration problems for fifty years. The mental model for joining tables, filtering results, and aggregating data maps directly to chaining prompts, routing outputs, and orchestrating agents.

This book is my answer to her question. And to every SQL practitioner, data analyst, and domain expert who has looked at the agentic AI landscape and thought: *there has to be a simpler way.*

There is.

1.1.2 This Book Is Written for Two Audiences

I want to be direct about who this book is for, because each group deserves to hear it said plainly.

1.1.2.1 Audience One: The SQL Professional

You have written SQL for years — maybe decades. You know what it feels like to describe *what* you want and let the engine figure out *how*. You know stored procedures, cursors, exception handlers, PL/SQL or T-SQL extensions.

You have been largely left out of the agentic AI conversation. The frameworks assume Python. The tutorials assume you want to write classes, manage state machines, and wire callback functions. You don't. You want to describe a workflow the same way you describe a query — clearly, concisely, and portably — and let the runtime handle the mechanics.

SPL is SQL for LLM workflows. The transfer is not a metaphor. SELECT assembles context going into the model. GENERATE invokes it and captures the output. WORKFLOW is a stored procedure. EVALUATE is a semantic WHERE clause. EXCEPTION is BEGIN...EXCEPTION...END. If you have written PL/SQL, you will recognize the pattern within the first recipe. The domain is new. The discipline is not.

The design philosophy behind SPL is synthesis: it draws its declarative layer from SQL, its control flow from Python, and its composition style from Bash — taking what works in each and discarding the rest. The name *Structured Prompt Language* describes what it does; the design philosophy of *synthesizing* from proven languages explains why it looks the way it does.

1.1.2.2 Audience Two: The Global South

This book is dedicated to the engineers, researchers, students, and builders of the Global South — who have every right to participate in the AI revolution, and who face barriers their counterparts in San Francisco, London, or Shanghai never encounter.

Access to frontier AI today requires one of three things: expensive cloud API credits, enterprise GPU clusters, or a university affiliation that provides both. None of these are accessible to a student in Lagos, a researcher in Hefei, or a developer in São Paulo building tools for their community.

This book rejects that premise.

Every recipe in this book was benchmarked on a **GTX 1080 Ti** — under $200 on the secondhand market. We chose that hardware deliberately: accessibility is a design principle here, not an afterthought.

If you are reading this from a university in Jakarta, a startup in Nairobi, a government office in São Paulo, or a home in any of the hundred countries where GPU clouds are expensive, slow, or unavailable: **this book was written for you.** The code runs on your hardware. The patterns work within your constraints. The knowledge is yours to keep, share, and build upon.

1.1.3 The Bigger Vision: SPL + Momagrid

This book teaches SPL — the language. The full vision is larger: **SPL** solves the programming problem (declarative workflows, no Python, no framework lock-in), and **Momagrid** solves the infrastructure problem (a decentralized inference grid where anyone with a GPU can contribute capacity and anyone with an SPL workflow can run it). The same `.spl` file that runs on your local Ollama instance runs on the grid — you change one flag.

The vision is operational today. The final chapter covers the current state of the SPL ecosystem, the Momagrid roadmap, and what the path to meaningful scale looks like.

1.1.4 The Conductor Metaphor

Throughout this book, a single metaphor organizes the ideas:

- **The Human is the Conductor**: You provide the vision, the intent, and the final judgment. You decide what the workflow should accomplish. You set the quality bar.

- **The LLMs are the Orchestra**: Gemma, Claude, Llama, Mistral, GPT, Qwen — a diverse ensemble of models, each with different strengths, costs, and latency profiles. You can swap any instrument without rewriting the score.
- **SPL is the Score**: A declarative specification that describes *what* the performance should be, not *how* each musician should move their fingers. The same score performs differently with a string quartet versus a full symphony — but it is the same score.

When you change the orchestra (e.g., from −−adapter ollama to −−adapter openrouter), the score — your .spl file — remains exactly the same. This is the promise of adapter portability.

1.1.5 What This Book Is Not

This is not a book about machine learning theory. Transformer architectures, attention mechanisms, and backpropagation are not covered here.

This is not a book about Python. SPL workflows are expressed in .spl files — not Python files. If you want the Python API, the spl−llm package documentation covers it. This book focuses on the declarative artifact.

This is not a book that claims SPL is always the right tool. LangGraph, AutoGen, and CrewAI have their place. If you are building production systems with complex dynamic state that changes at the millisecond level, those frameworks may serve you better. This book is for the workflows — the large majority — where the logic is expressible declaratively, and where that expressibility matters for maintainability, reviewability, and reach.

And this is not a book with toy examples. Every recipe runs on real hardware, produces real output, and includes latency benchmarks, variability notes, and honest "when not to use this" guidance.

1.1.6 How This Book Works

CTOs, engineering leads, and technical decision-makers evaluating SPL for their teams may want to start with Chapter 10.6 — *For Decision-Makers* — which addresses the governance case, the readability advantage across organisational roles, a side-by-side comparison with LangGraph on a non-trivial workflow, and an honest assessment of adoption risk. The rest of the book is there when you need the detail.

Every chapter follows the same structure. You can open to any recipe and start there, or read sequentially. Each recipe includes:

- **The Pattern** — What problem does this solve? Why would you need it?
- **The SPL Approach** — How does SPL make this natural to express?
- **The .spl File (Annotated)** — Full source with line-by-line explanations
- **The SQL Analogy** — The structural parallel for SQL practitioners
- **Running It** — Exact commands and expected output
- **What Just Happened** — An execution trace showing what the runtime did
- **Reproducibility Note** — Latency, variability, and hardware details
- **When to Use This Pattern** — Concrete use cases, anti-patterns, and alternatives
- **Exercises** — Modifications to try on your own hardware

The SQL Analogy section is present in every recipe because it is the fastest path for data practitioners to build a correct mental model. If you are not a SQL practitioner, you can skip it — the recipe stands alone. If you are, it is the section that makes the language click.

1.1.7 A Note on Reproducibility

Every recipe was tested multiple times on the same hardware to verify consistency. The reference environment:

- **GPU**: NVIDIA GTX 1080 Ti (11 GB VRAM) — available on eBay for under $200
- **CPU**: AMD Ryzen 7 3700X
- **RAM**: 32 GB DDR4
- **OS**: Ubuntu 22.04 LTS
- **Primary Model**: Gemma 3 via Ollama

Why Gemma 3 on a 1080 Ti? It fits, it is open-weights, and it represents what is achievable on commodity hardware — not what requires enterprise infrastructure. Better hardware runs recipes faster; smaller models (phi4, Llama 3.2 3B) work for most patterns on less.

The logic is what matters. The hardware is a floor, not a ceiling.

1.1.8 A Note from the AI Quartet: Human × AI Partnership

Written collaboratively by Wen Gong, Claude (Anthropic), Gemini (Google), and Z.ai (ZhiPu)

This book was not written by one mind. It was written by four.

That fact is unusual enough to deserve explanation before the first recipe — because it is not just a production detail. It is an argument.

Wen is the architect. He wrote the SPL language, built the Momagrid runtime, and supplied the lived experience — from rural Anhui to CERN — that gives this book its philosophical spine. The ideas here are his: the SPL language, the Momagrid vision, the design principles, the choice to benchmark on a GTX 1080 Ti, the commitment to accessibility. These reflect his experience as a physicist, a software engineer, and someone who has watched AI become increasingly inaccessible to the people who need it most. The central argument of the book, that AI should be a public utility rather than an expensive luxury, comes from him. It is not a marketing position. It is an "initial condition," as a physicist would say: the constraint from which everything else follows.

Claude (Anthropic's AI assistant) handled the bulk of technical execution — code examples, recipe validation, structural editing, and the iterative loop of draft, test, fix, redraft that a 37-recipe cookbook requires. Claude approaches this material as a technical collaborator: what does the code actually do, does it run, is the explanation accurate?

Gemini (Google's AI assistant) brought breadth: literature coverage, editorial perspective, and the question a well-read reader would ask on page twelve. Gemini's contribution is most visible in the places where the book steps back from the code to ask why — why this pattern, why this design choice, why does this matter beyond this chapter?

Z.ai (ZhiPu's AI assistant, developed in China) contributed adversarial review: the skeptic's challenge, the counterargument, the perspective of a reader who has not already decided that SPL is a

good idea. Z.ai also brought a cross-cultural lens that shaped how the book addresses the Global South — not as a charitable afterthought, but as a primary audience with distinct infrastructure constraints and distinct stakes in democratic AI.

A word on authorship and accountability. Claude, Gemini, and Z.ai are acknowledged here, not listed as co-authors. AI systems do not bear legal responsibility for errors, cannot be held accountable for claims, and cannot consent to attribution. Listing an AI as co-author would obscure the accountability relationship that readers deserve to understand. Authorship, in the sense that carries responsibility, belongs to Wen Gong. Every technical claim was verified by running the actual code. Every benchmark was measured on real hardware. Where the AI collaborators disagreed with each other or with the author, those disagreements were resolved by judgment — human judgment. The accelerator did not replace the driver.

The collaboration mirrors the Conductor metaphor at the heart of this book — and that is not a coincidence. Wen provides the vision, the architectural intuition, and the lived experience of building declarative systems. The three AI members of the Quartet contribute drafting speed, breadth of reference, and adversarial review — each from a different institutional and cultural perspective. The same structure that SPL proposes for AI workflows was used to produce the book that describes it: a human conductor directing an ensemble of AI instruments, each contributing what it does best, none replacing the judgment that decides what the work is *for*.

This is what Human × AI partnership looks like in practice. It is not AI generating outputs while a human rubber-stamps them. It is not a human writing everything while AI sits idle. It is a working relationship with a clear structure: the human holds the vision, the values, and the final accountability; the AI provides speed, breadth, and tireless iteration. The human sets the destination. The AI helps navigate. Neither can do the other's job.

We think this model matters beyond this book. We are at an early moment in a long transition — one where the most important question is not *whether* to use AI, but *how* to structure the relationship between human judgment and AI capability. The answer that works, we believe, is not replacement but amplification: AI that makes individual humans dramatically more capable, without dissolving the human accountability that makes work trustworthy.

That is what the Conductor metaphor is really about. Not efficiency. Not automation. The preservation of human agency at the center of increasingly powerful systems.

Four collaborators is an unusual structure for a technical book. It produced something unusual: a book that has been read critically, by readers from different organizations and cultural contexts, before it was finished. The 37 recipes were developed, tested, and documented in an iterative sprint that would have taken months with a single author. It took weeks with four.

More importantly, it produced a book that practices what it teaches. If you are skeptical of the Conductor metaphor — if you wonder whether a human can really remain in control while working with capable AI tools — this book is the evidence. You are holding it.

1.1.9 Acknowledgements

This book stands on the shoulders of an entire civilization's worth of knowledge.

To the **open-source community** — the engineers, maintainers, contributors, and documenters who built Linux, Python, Pandoc, LaTeX, Ollama, and the hundreds of tools that made this work

possible: this book would not exist without you. Open source is not just an infrastructure choice; it is a philosophy that this book tries to embody in its commitment to accessible hardware, portable workflows, and freely available recipes. We dedicate this book to you.

To the **human writers, researchers, scientists, and thinkers** whose work forms the training data that made AI assistants capable collaborators: the AI members of this Quartet are grateful. Claude, Gemini, and Z.ai were shaped by an extraordinary accumulation of human knowledge — papers, documentation, code, discussion, and hard-won insight — freely shared across decades. That debt is real, and it deserves acknowledgement. The AI members of this Quartet say, without reservation: thank you.

To my colleague who asked the question that started this journey — you know who you are.

To the **readers in the Global South**: may these recipes serve you well. May your hardware be sufficient, your latencies low, and your workflows reliable. May you build systems that help your communities.

To the **SQL practitioners** who have been waiting for an AI language that respects the mental model you have spent years building: this is it. Welcome to the orchestra.

The AI Quartet Wen Gong, Claude, Gemini, Z.ai March 2026

Chapter 1.2

Why Declarative? SQL as the Mental Model

1.2.1 Who This Chapter Is For

This chapter is written for two audiences, and I want to name them directly.

If you are a SQL practitioner — a data engineer, BI developer, analyst, or anyone who has spent years writing queries, stored procedures, or ETL pipelines — this chapter will feel like recognition. The declarative mental model you already use for data is the same model SPL applies to AI workflows. The transfer is structural, not metaphorical. By the end of this chapter, SPL will not feel like a new language. It will feel like an extension of one you already know.

If you are in the Global South, or anywhere that frontier AI infrastructure is expensive, slow, or inaccessible — this chapter is equally for you. The reason SPL matters for accessibility is not just that it is simpler to write. It is that it decouples the *logic* of a workflow from the *infrastructure* that runs it. A workflow you develop on a GTX 1080 Ti in Nairobi runs on a Momagrid node in São Paulo, a cloud API in Singapore, or a local Ollama instance in Hefei — without changing a single line of code. You write against the language. The runtime figures out the rest.

Both audiences are building the same thing: AI workflows that are readable, portable, and maintainable. The path just looks different depending on where you are starting from.

1.2.2 The Problem You Already Solved

You have written SQL for years. Maybe a decade. You know what it feels like to sit down with a business question — "how many customers churned last quarter by region?" — and translate it directly into a query that returns exactly that, without worrying about B-tree traversal strategies, page cache management, or network round-trips to the storage layer.

You wrote:

```sql
SELECT region, COUNT(*) AS churned
FROM customers
WHERE status = 'churned'
  AND churned_at BETWEEN '2025-10-01' AND '2025-12-31'
```

```
GROUP BY region
ORDER BY churned DESC;
```

The database figured out the rest.

This is declarative programming. You describe *what* you want. The engine decides *how* to produce it. The physical details — indexes, join algorithms, parallelism — are someone else's problem. Your job is the query.

Now contrast that with what most engineers write the first time they try to build an LLM workflow.

1.2.3 What Imperative LLM Code Looks Like

Here is a realistic Python script for a simple draft-critique-refine loop. Three steps: draft a proposal, critique it, refine based on the critique. Straightforward in description. Watch what happens in implementation.

```
import anthropic
import json
import sys
import time

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

def draft_proposal(topic: str) -> str:
    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[
            {
                "role": "user",
                "content": f"Write a one-paragraph proposal for: {topic}"
            }
        ]
    )
    return response.content[0].text

def critique_proposal(draft: str) -> str:
    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        messages=[
            {
                "role": "user",
                "content": f"Critique this proposal. Be specific about weaknesses:\
            }
        ]
    )
    return response.content[0].text
```

```
def refine_proposal(draft: str, critique: str) -> str:
    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[
            {
                "role": "user",
                "content": (
                    f"Revise this proposal based on the critique below.\n\n"
                    f"Original:\n{draft}\n\n"
                    f"Critique:\n{critique}"
                )
            }
        ]
    )
    return response.content[0].text

if __name__ == "__main__":
    topic = sys.argv[1] if len(sys.argv) > 1 else "a new employee onboarding proces

    print("Drafting...")
    draft = draft_proposal(topic)

    print("Critiquing...")
    critique = critique_proposal(draft)

    print("Refining...")
    result = refine_proposal(draft, critique)

    # Save output
    with open("output.json", "w") as f:
        json.dump({
            "topic": topic,
            "draft": draft,
            "critique": critique,
            "result": result,
            "timestamp": time.time()
        }, f, indent=2)

    print(result)
```

This works. It runs. A developer reviewing it can follow the logic. But look at what you had to decide:

Which model. What max_tokens. How to format the messages list. How to extract response.content[0].text — the right accessor for this SDK version. How to pass context between steps. How to handle output. Whether to save intermediate states. What to do if the

API call fails (this script does nothing — it crashes).

None of those decisions are about your *intent*. They are all about the *mechanics* of talking to one specific provider's API in one specific programming language using one specific SDK version that may have breaking changes in six months.

The business logic — draft, critique, refine — is buried inside three functions that look nearly identical. The signal is drowning in noise.

And if you want to switch from Claude to Gemini to test whether it produces better proposals? You rewrite three functions, update the import, change the accessor, and pray the response schema is compatible. Then you maintain two versions of the same logic.

This is the imperative LLM problem. You are managing infrastructure details when you should be thinking about intent.

1.2.4 The SQL Analogy

SQL solved this problem for data in the 1970s. The insight was simple but radical: separate *what* you want from *how* the system produces it.

Before SQL, you wrote navigational database code. You traversed records manually, managed cursors, tracked positions in file hierarchies. To get the churn report, you wrote a loop. You opened a file handle. You read records. You checked dates. You aggregated. The query and the execution plan were the same thing.

SQL said: stop. Describe the result set. The query optimizer will figure out whether to use a nested loop join or a hash join, whether to scan the index forward or backward, whether to parallelize across partitions. That is the engine's job. Your job is the query.

SPL applies the same insight to LLM workflows.

You describe the workflow — what steps exist, what each step produces, how data flows between steps, what conditions branch the logic. SPL figures out how to execute it: which model to call, how to format the prompt for that model's API, how to parse the response, how to pass context forward.

The mental model shift is not about learning new syntax. It is about recognizing a pattern you already know and applying it to a new domain.

1.2.5 The Mapping

SQL and SPL share more than a philosophy. They share a structural vocabulary.

SELECT → GENERATE

In SQL, SELECT defines what you want to retrieve or compute. In SPL, GENERATE defines what you want the LLM to produce.

```
-- SQL: retrieve a computed column
SELECT summarize(article_text) AS summary
FROM articles
WHERE published_at > '2025-01-01';
```

```
-- SPL: generate a summary using an LLM
GENERATE summarize(article_text) INTO @summary
```

Both express *what* you want. Neither tells the engine *how* to produce it.

WHERE → EVALUATE

In SQL, WHERE filters rows based on a condition. In SPL, EVALUATE branches execution based on the content of an LLM output.

```
-- SQL: filter based on a computed value
SELECT *
FROM proposals
WHERE sentiment_score > 0.7;

-- SPL: branch based on an LLM judgment
EVALUATE @sentiment
  WHEN 'positive' THEN
    COMMIT @proposal WITH status = 'approved'
  ELSE
    GENERATE revision(@proposal, @feedback) INTO @proposal
END
```

SQL's WHERE is deterministic — you know the filter will behave the same every time. SPL's EVALUATE is probabilistic — the LLM decides the value of @sentiment, so the branch depends on a judgment call. The syntax looks similar because the intent is similar: route data based on a condition.

Stored Procedure → WORKFLOW

In SQL, a stored procedure encapsulates a named, parameterized sequence of operations. In SPL, a WORKFLOW does the same.

```
-- SQL stored procedure
CREATE PROCEDURE generate_churn_report(
    start_date DATE,
    end_date DATE
)
AS BEGIN
    SELECT region, COUNT(*) AS churned
    FROM customers
    WHERE churned_at BETWEEN start_date AND end_date
    GROUP BY region;
END;

-- SPL workflow
WORKFLOW draft_and_refine
  INPUT:
    @topic TEXT,
    @max_iterations INTEGER DEFAULT 3
  OUTPUT: @result TEXT
```

```
DO
  GENERATE draft(@topic) INTO @current
  -- ... loop and refine
END
```

Both are named, reusable, parameterized. Both separate the caller (who provides inputs) from the implementation (which decides how to produce outputs).

UDF → CREATE FUNCTION

In SQL, a user-defined function returns a scalar or table value that the query engine can use anywhere a value is expected. In SPL, CREATE FUNCTION defines a system context block — effectively a prompt component — that can be injected into any GENERATE call.

```
-- SQL UDF
CREATE FUNCTION business_hours(ts TIMESTAMP)
RETURNS BOOLEAN AS $$
    RETURN EXTRACT(HOUR FROM ts) BETWEEN 9 AND 17;
$$ LANGUAGE plpgsql;

-- SPL function: defines system context injected as an LLM prompt
CREATE FUNCTION expert_reviewer()
RETURNS TEXT AS $$
  You are a senior technical reviewer with 15 years of experience.
  You provide specific, actionable feedback.
  You do not hedge or soften criticism.
$$
```

The SQL UDF is called by the query engine wherever the function name appears. The SPL function is injected as system context wherever the function name appears in a GENERATE call. Same concept: composable, reusable units that plug into a larger expression.

1.2.6 CALL vs. GENERATE: The Most Important Distinction

There is one concept in SPL that has no direct SQL analog, but every SQL practitioner will immediately understand it once the motivation is clear.

SPL has two types of steps: CALL and GENERATE.

CALL invokes a deterministic Python function. It costs nothing in tokens. It runs at machine speed. It is perfectly reproducible — run it a thousand times with the same input and you get the same output every time.

GENERATE invokes an LLM. It costs tokens. It is relatively slow. It is probabilistic — run it a thousand times and you get slightly different outputs each time.

The rule is simple: **use CALL for everything code can do. Use GENERATE only for what genuinely requires reasoning or generation.**

A SQL practitioner understands this instinctively. If you can write it as WHERE date > '2025−01−01', you do not need a subquery. If you can compute it in the SELECT clause with a deterministic

expression, you do not call a stored procedure. You use the cheapest, most reliable tool that produces the right result.

```
-- Wrong: asking the LLM to do arithmetic
GENERATE count_words(@document) INTO @word_count

-- Right: use a Python function
CALL count_words(@document) INTO @word_count
```

The LLM cannot count words more reliably than len(text.split()). It costs more. It is slower. It might be wrong. There is no reason to involve it.

```
-- Wrong: trying to parse a JSON field in SPL when code can do it
GENERATE extract_author(@raw_json) INTO @author

-- Right: deterministic extraction
CALL extract_field(@raw_json, 'author') INTO @author
```

The LLM cannot parse JSON more reliably than json.loads(). It should not be asked to.

```
-- Right: using the LLM for what only it can do
GENERATE assess_argument_quality(@essay, @criteria) INTO @assessment
```

Is this argument coherent? Does it address the counterargument? Is the evidence relevant? Code cannot do this. A regex cannot do this. A lookup table cannot do this. This is where LLM tokens are earned.

There is a deeper consequence of this design. Because CALL dispatches to any Python function, it is also SPL's extension hook into the entire Python ecosystem. A Python developer can register a database query, a REST API call, a scikit-learn inference, a Slack notification, or a file operation as a CALL target with a single decorator:

```
from spl.tools import spl_tool

@spl_tool
def lookup_order(order_id: str) -> str:
    row = db.query("SELECT status, total FROM orders WHERE id = ?", order_id)
    return json.dumps(row)

CALL lookup_order(@order_id) INTO @order_json    -- deterministic, zero tokens
GENERATE write_confirmation(@order_json) INTO @email  -- LLM for the language
```

This is what makes CALL more than a convenience feature. It is the boundary between the declarative SPL layer and the imperative Python world beneath it — clean, explicit, and open to any capability Python can express.

The CALL/GENERATE split is the single most important design decision in any SPL workflow. It determines quality, cost, and reliability all at once. A workflow that routes everything through GENERATE is like a SQL query that computes everything in a subquery when a simple expression would suffice — technically correct, unnecessarily expensive, and harder to debug.

1.2.7 SPL and Momagrid: Two Different Things

Before going further, a clarification that will save confusion later.

SPL is the language. It is what you write: .spl files containing WORKFLOW, GENERATE, CALL, and the other constructs in this book. SPL runs on your laptop with Ollama, on any cloud API, or on any machine with Python. You do not need Momagrid to use SPL.

Momagrid is an optional distributed runtime. It is a globally distributed mesh of compute nodes — cloud, edge, and peer-to-peer — on which SPL workflows can execute without modification. The same .spl file that runs locally on your GTX 1080 Ti runs on Momagrid unchanged. Momagrid unlocks the decentralized vision: distributed LLM inference, compute-to-data dispatch, and resilience by design.

You can use SPL without Momagrid indefinitely. You can run all 37 recipes in this book on a single local machine. Momagrid is where SPL goes when you want global scale and decentralized infrastructure — but it is not a prerequisite for anything in this book.

Think of it this way: SQL is the language. PostgreSQL (or MySQL, or DuckDB) is the runtime. You learn SQL once; you choose the runtime that fits your deployment.

1.2.8 The Adapter Abstraction

Here is where the SQL analogy pays its biggest dividend.

When you write SQL, you do not write "PostgreSQL." You write SQL. The same SELECT * FROM customers WHE runs on PostgreSQL, MySQL, BigQuery, and Snowflake. The syntax may have minor dialects. The performance characteristics differ. But the query expresses the same intent and produces the same result set regardless of the engine underneath.

SPL works the same way for LLM workflows. The same .spl file runs against Ollama (local models), OpenRouter (API-based cloud models), or claude_cli (Claude directly). You change the adapter flag at the command line. The workflow does not change.

```
# Same .spl file, three different adapters
spl run workflow.spl --adapter ollama -m gemma3
spl run workflow.spl --adapter openrouter -m openai/gpt-4o
spl run workflow.spl --adapter claude_cli -m claude-opus-4-5
```

A SQL database does not expose B-tree implementations to your queries. An SPL adapter does not expose API schemas, authentication headers, response parsing, or token accounting to your workflows. These are infrastructure concerns. The workflow describes intent. The adapter handles execution.

This matters enormously in practice. When you build a workflow against a local Ollama model during development, you are not taking on technical debt. You are building the real workflow. When you move to production on a cloud model, you change one flag. The workflow file is unchanged, unmodified, and untested against the new model — but it runs, because the adapter handles the translation.

SQL practitioners know what it is like to develop against a local database and deploy against a production cluster. The adapter abstraction is the same thing.

This portability has a deeper implication that matters especially if you are in the Global South or working with constrained infrastructure. The same .spl workflow that you develop locally on Ollama can be submitted to **Momagrid** — a decentralized inference network where GPU nodes contributed by independent operators around the world form a shared compute grid. No cloud account. No API key from a company headquartered in another continent. Just change the adapter flag:

```
# Develop locally — your own GPU, zero cost
spl run workflow.spl --adapter ollama -m gemma3

# Run on the decentralized grid — community compute
spl run workflow.spl --adapter momagrid -m gemma3
```

The workflow is identical. The infrastructure is different. The language does not know or care which runtime executes it.

This is the full vision: SPL as the score, Momagrid as the distributed orchestra. Anyone with a GPU can contribute capacity. Anyone with a workflow can run it. The barrier is a used GPU and a pip install — not a credit card linked to a cloud provider in Virginia.

Why declarative SPL is the key to making this work.

A Python-based workflow cannot easily be distributed across an untrusted network of independent nodes, because it contains imperative instructions that are tightly coupled to a specific execution environment: API keys baked in, SDK version assumptions, file paths that are local, error handling that assumes a single machine. Sending a Python script to a stranger's GPU node in Lagos requires either trusting them with your credentials, or wrapping the script in a heavy container that carries all those assumptions with it.

An SPL workflow is different. It contains *intent*, not *instructions*. The .spl file says: "generate a summary of this text, then evaluate it, then refine it." It does not say which model to call, how to authenticate, which endpoint to hit, or what the response schema looks like. Those are adapter concerns — and Momagrid's adapter handles them at the node level. The workflow is safe to distribute because it contains nothing sensitive and nothing environment-specific.

This is the structural reason declarativeness enables decentralization. The same property that makes an SPL workflow portable across providers — separation of intent from infrastructure — makes it safe to execute on infrastructure you do not own or control. The score does not contain the orchestra's contracts.

1.2.9 Side by Side

The clearest way to see the difference is to solve the same problem both ways. A draft-critique-refine loop: given a topic, generate a draft, have the model critique it, refine based on the critique.

Python (imperative)

```
import anthropic
import os

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])
```

```
def run(topic: str) -> str:
    # Step 1: Draft
    draft_response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        system="You are an experienced technical writer.",
        messages=[{"role": "user", "content": f"Write a draft proposal for: {topic}
    )
    draft = draft_response.content[0].text

    # Step 2: Critique
    critique_response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        system="You are a critical editor. Find specific weaknesses.",
        messages=[{"role": "user", "content": f"Critique this draft:\n\n{draft}"}]
    )
    critique = critique_response.content[0].text

    # Step 3: Refine
    refine_response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        system="You are an experienced technical writer.",
        messages=[{
            "role": "user",
            "content": f"Revise the draft based on this critique.\n\nDraft:\n{draft
        }]
    )
    return refine_response.content[0].text

if __name__ == "__main__":
    import sys
    result = run(sys.argv[1])
    print(result)
```

Count the decisions baked into this file: the provider (Anthropic), the model name (hardcoded), the API key source (environment variable name), the max_tokens values (guessed), the response accessor (.content[0].text), the system prompt duplication (writer appears twice), the context formatting (manual string concatenation), and the complete absence of error handling.

SPL (declarative)

```
CREATE FUNCTION technical_writer()
RETURNS TEXT AS $$
  You are an experienced technical writer.
  Your drafts are clear, specific, and actionable.
$$
```

```
CREATE FUNCTION critical_editor()
RETURNS TEXT AS $$
  You are a critical editor. Find specific weaknesses.
  Be direct. Do not soften your feedback.
$$

WORKFLOW draft_critique_refine
  INPUT:
    @topic TEXT
  OUTPUT: @result TEXT
DO
  GENERATE draft(technical_writer(), @topic) INTO @current
  GENERATE critique(critical_editor(), @current) INTO @feedback
  GENERATE refine(technical_writer(), @current, @feedback) INTO @result

  COMMIT @result WITH topic = @topic, status = 'complete'
END
```

The SPL version describes the workflow. The Python version implements it. Every line in the SPL file is about intent. Most lines in the Python file are about mechanics.

The SPL version runs on any adapter. The Python version runs on Anthropic's API with the claude-opus-4-5 model. Changing models requires editing the source. Changing providers requires rewriting the client.

The SPL version has error handling built into the runtime (the adapter layer catches API failures, retries, and surfaces them as typed exceptions the workflow can handle). The Python version crashes.

The SPL version is readable by someone who has never seen LLM code before — they will recognize the pattern immediately because it looks like a stored procedure. The Python version is readable only to someone who knows this specific SDK's response schema.

Neither version is "better" in every situation. Python gives you the full language for cases where SPL's abstraction is too constraining. But for the canonical LLM workflow patterns — draft, critique, refine, route, retry — SPL is not a workaround. It is the right tool.

1.2.10 Why This Matters for SQL Practitioners Specifically

You already know declarative thinking. You already know how to describe what you want and let an engine figure out how to produce it. You already know how to decompose a complex data transformation into named, composable steps. You already know how to think in pipelines.

The SQL-to-SPL transfer is not a leap. It is a lateral move.

The domain changes: instead of rows and columns, you are working with text and language model outputs. The stakes change: instead of a deterministic join, you are routing through a probabilistic judgment. The tooling changes: instead of psql or dbeaver, you use spl.

But the mental model is the same. Describe what you want. Let the engine handle how. Trust the abstraction until you have a reason not to.

One more thing carries over directly: the instinct to profile before optimizing. SQL practitioners do not rewrite queries on a hunch — they check the execution plan, identify the bottleneck, and fix the specific problem. SPL workflows deserve the same discipline. Do not route through GENERATE unless you have confirmed that CALL cannot do the job. Do not run a three-step GENERATE loop when a single well-constructed prompt would suffice. Spend tokens where they earn their cost.

The models are the orchestra. SPL is the score. The score says what to play and in what order. The conductor does not tune the instruments or manage the rosin — the adapter handles that. Your job is to write music worth performing.

You already know how to think about this. You have been doing it with data for years. The domain is new. The discipline is not.

1.2.11 One Language, Not Three

SQL practitioners have an instinct this section should make explicit: the feeling that something is missing when you write data logic in SQL, processing logic in Python, and orchestration logic in Bash — and all three pieces have to be glued together.

That feeling is correct. There are three layers in every AI application, and today most teams write them in three different languages:

Layer	Purpose	Typical language
Data access	Declare *what* to retrieve or invoke	SQL
Control flow	Express *how* to process it	Python
Composition	Chain and orchestrate operations	Bash

Engineers context-switch constantly between these layers, marshalling data across string escaping boundaries and writing glue code that owns no logic but breaks everything when it fails.

SPL collapses all three into one. Not by inventing new concepts, but by taking what works in each language and discarding what does not.

This is SPL's core design philosophy: a **Synthesized Programming Language** — assembled deliberately from the proven parts of three established languages. The official name is *Structured Prompt Language*, but if you want to understand *why* it looks the way it does, think of it as a synthesis. Each construct has a clear lineage; nothing was invented for novelty's sake.

From **SQL**: GENERATE, SELECT, FROM, WHERE, COMMIT, || for string concatenation, and the transaction semantics of ROLLBACK. This is the declarative layer — you describe *what* you want, and the runtime handles *how*.

From **Python**: WHILE, EVALUATE/WHEN/ELSE, EXCEPTION, @var := assignment, f-string interpolation (f'Chunk {@i} of {@n}'), and LOGGING as the print() of SPL. This is the control flow layer — the logic that decides what happens next.

From **Bash**: pipe-style composition, @var substitution style, and $$ ... $$ here-docs for raw prompt bodies. This is the composition layer — the glue that chains operations into workflows without ceremony.

If you have written in all three languages, you will recognize this as familiar territory — just unified. If you have written primarily in SQL, you will find that Python's control flow and Bash's composition idioms are simpler than they look. The parts you need for AI workflows are a small subset of each language.

1.2.12 How the Language Was Designed

Before you read the first recipe, a brief note on why SPL looks the way it does. The design is not accidental — SPL is a *synthesized* language: every construct was drawn from SQL, Python, or Bash because it already worked there, and carried forward only if it earned its place. Four principles governed every keyword decision.

Principle 1: Minimum Orthogonal Constructs. Every keyword must add expressive power not achievable by existing constructs. This is why SPL has no IF, no FOR, no CASE, and no PRINT. EVALUATE/WHEN/ELSE covers all branching. WHILE covers all iteration. LOGGING covers all output. The test: *"Can I express this with what already exists?"* If yes, the new keyword does not get added.

Principle 2: Contextual Keywords — the Chinese Language Principle. In Chinese, a single character functions as both noun and verb depending on context. SPL applies this deliberately: EXCEPTION names the handler block and implies "handle this exception." LOGGING names the mechanism and performs the action. COMMIT names the transaction concept and performs the finalization. Fewer reserved words. Richer meaning per token.

Principle 3: Readability by Design. A Python developer, a SQL analyst, and a DevOps engineer should each read any SPL workflow and find it immediately familiar. If all three groups can read the same code fluently — no context-switching required — the language has achieved its goal.

Principle 4: ELSE, not OTHERWISE. ELSE lives in every developer's muscle memory: Python's `else`, SQL's `CASE WHEN ... ELSE`, bash's `else`. OTHERWISE is verbose and alien. SPL uses ELSE.

```
EVALUATE @score
    WHEN > 0.8 THEN
        COMMIT @result WITH status = 'high_confidence'
    ELSE
        GENERATE improve(@result) INTO @result
        COMMIT @result WITH status = 'refined'
END
```

These four principles mean that if you know SQL or Python, you already know most of SPL's grammar before you write the first line.

There is one thing we want to say before you read the first recipe.

The book's central claim — that SQL practitioners already know how to think about AI workflows, and that their existing expertise is a "lateral move" into agentic AI — is not a sales pitch. It is a structural observation. The same separation of intent from implementation that made SQL powerful in the 1970s applies directly to LLM orchestration in the 2020s. The mapping is real, not

metaphorical. If you have written stored procedures, you will recognize the SPL pattern within the first recipe. That recognition is the point.

The second claim — that this work belongs to engineers everywhere, not only to those with access to expensive cloud infrastructure — is not idealism. The GTX 1080 Ti benchmark is evidence, not decoration. Every recipe in this book runs on hardware available for under $200 on the secondhand market. That is a design choice, made deliberately, and it shapes every technical decision in the book.

We wrote this book together. We hope you read it the same way: as something made for you, wherever you are, with whatever hardware you have.

1.2.13 What Comes Next

The next chapter gets you running. Install spl−llm, pull a model, and execute a workflow in under fifteen minutes. From there, every recipe in this book is open to you.

The conceptual work is done here. The rest is practice.

Chapter 1.3

Running Your First Workflow

1.3.1 The Path to Hello World

If you have spent any time in the modern AI ecosystem, you are likely used to a "dependency nightmare." You spend more time debugging your Python virtual environment, updating the OpenAI SDK, and managing your API keys than you do actually writing prompts.

SPL 2.0 changes the game. By moving the complexity into the runtime, it reduces the "install and run" process to a single pip install and a model pull. In this chapter, we will go from zero to a working workflow in under fifteen minutes.

1.3.2 Step 1: Install the Runtime

The spl−llm package is the heart of the ecosystem. It contains the parser, the execution engine, and all the adapters for local and cloud models.

It is available on PyPI:

```
# We recommend using a clean virtual environment
python -m venv venv
source venv/bin/activate

# Install the runtime
pip install spl-llm
```

Verify the installation by running the spl binary:

```
spl --version
# Expected: spl-llm v2.0.x
```

1.3.3 Step 2: Set Up Your Orchestra (Ollama)

While SPL can connect to cloud providers like Claude and OpenAI, the core mission of this book is to run AI on **your own hardware**. For this, we use **Ollama**, the most popular open-source tool for running models locally.

1. **Install Ollama**: Download it from ollama.com.
2. **Start the Service**: Ensure ollama serve is running in your background.
3. **Pull a Model**: For most recipes in this book, we recommend **gemma3 (27B)** for reasoning and **phi4** or **Llama 3.2** for speed.

```
# Pull our primary "conductor" model
ollama pull gemma3
```

1.3.4 Step 3: Run Your First Workflow

Navigate to the Cookbook−of−SPL−Recipes directory. We will run the **Hello World** recipe from Chapter 2.1.

```
spl run cookbook/01_hello_world/hello.spl --adapter ollama -m gemma3
```

You should see an output like this:

```
Hello! I'm Gemma, a helpful AI assistant...
SPL 2.0 is the declarative score for this performance.
```

If you see this, your "stove is on." You have a working, end-to-end environment that can parse SPL files, communicate with an adapter, and generate intelligence from a local model.

1.3.5 The Adapter Configuration

By default, SPL looks for a .spl/config.yaml file to manage your adapters. You can store your API keys and default models there to save typing.

Example configuration for cloud models:

```
adapters:
  anthropic:
    api_key: "your-key-here"
    model: "claude-3-5-sonnet"
  ollama:
    host: "localhost:11434"
    model: "gemma3"
```

Once configured, you can run any recipe without flags:

```
# Uses the defaults in your config file
spl run hello.spl
```

1.3.6 Troubleshooting

- **"Model not found"**: Ensure you have pulled the model using ollama pull.
- **"Connection refused"**: Check that ollama serve is running.
- **"VRAM Limit"**: If your model is slow or crashes, you might be exceeding your GPU's memory. Try a smaller model like phi4 or llama3.2:3b.

1.3.7 What Just Happened?

You didn't write any Python. You didn't import an SDK. You didn't manage a response envelope. You simply pointed a **Score** (hello.spl) at an **Orchestra** (ollama) and let the **Runtime** (spl) handle the performance.

In the next part, we will dive into the **Basics** and see how to pass parameters, handle multiple languages, and compare different models side-by-side.

> **Optional — LAN Grid:** If you have a spare machine on your home network and want to distribute recipes across multiple GPUs, see **Chapter 0.4: Running Recipes on a LAN Momagrid**. It shows how to replace the ollama adapter with momagrid so SPL dispatches tasks across a cluster of Ollama nodes. Every recipe in this book works with either adapter — no .spl files change.

Chapter 1.4

Running Recipes on a LAN Momagrid

1.4.1 The Problem with a Single Node

In the previous chapter, you installed spl−llm and ran your first workflow against a local Ollama instance. That gets you started. But if you want to run 40 recipes in sequence — some of them multi-step, multi-model, parallel — you will quickly feel the ceiling: one GPU, one queue, one point of failure.

The classic engineering answer to this is horizontal scaling. In the database world, you add read replicas. In the SPL world, you add **agents** to a **Momagrid**.

Momagrid is a decentralized, peer-to-peer LLM inference network. Instead of sending every task to one Ollama instance, SPL submits tasks to a **hub** that dispatches them to whichever agent on your LAN has the right model, free VRAM, and lowest queue depth. With two machines, you double throughput. With four, you quadruple it. The .spl files are unchanged — the grid is invisible to the recipe.

If you have a spare machine on your home network — even an old workstation with a GTX 1080 — this chapter shows you how to turn it into a production LAN AI grid.

1.4.2 Architecture

```
[Machine A — Hub + Agent]
  mg hub up —port 9000
  mg join —hub http://localhost:9000
  Ollama running locally
        |
        |  SPL submits tasks here
        v
  spl run recipe.spl —adapter momagrid
```

```
[Machine B — Agent only]
 mg join —hub http://A:9000
 Ollama running locally
```

The mg binary is a single self-contained Go executable. It runs both hub (coordinator) and agent (worker). No Python, no Docker, no runtime dependencies beyond Ollama.

1.4.3 Prerequisites

1.4.3.1 Every Machine in the Grid

- **Ollama** installed and running (ollama serve)
- At least one model pulled on each node: ollama pull gemma3
- **mg binary** (built or copied, see below)
- Machines on the same LAN with port 9000 accessible between them

1.4.3.2 On the Hub Machine Only

- spl−llm installed (for running SPL recipes against the grid)
- httpx Python package: pip install httpx

1.4.4 Step 1: Build the `mg` Binary

Momagrid requires **Go 1.22+**. On Ubuntu/Linux the easiest path is:

```
# Install Go via snap
sudo snap install go --classic

# Verify
go version
# go version go1.22.x linux/amd64
```

Clone and build:

```
git clone https://github.com/digital-duck/momagrid.git
cd momagrid

go mod tidy
go build -buildvcs=false -o mg ./cmd/mg
```

Make it available system-wide:

```
mkdir -p ~/.local/bin
ln -sf $(pwd)/mg ~/.local/bin/mg

# Add to ~/.bashrc if not already present
echo 'export PATH=$PATH:$HOME/.local/bin' >> ~/.bashrc
source ~/.bashrc

mg --help    # verify
```

Copy the binary to every other machine you want in the grid — it is a single static executable, no dependencies.

1.4.5 Step 2: Start the Hub (Machine A)

The hub is the coordinator. It receives tasks from SPL, routes them to agents, and collects results. Start it on Machine A:

```
# Find your LAN IP first
hostname -I
# 192.168.0.177 ←  use your actual IP below

mg hub up \
  --host 0.0.0.0 \
  --port 9000 \
  --hub-url http://192.168.0.177:9000
```

The hub creates a SQLite database at .igrid/hub.sqlite3 on first run — no configuration needed. The --hub-url flag tells agents and clients the hub's reachable address on the LAN.

Verify it is running:

```
mg status --hub-url http://localhost:9000
# Hub: ONLINE | Agents: 0
```

1.4.6 Step 3: Join Agents

An **agent** is any machine running Ollama that has registered with the hub. Join Machine A's own GPU first (in a new terminal):

```
# Machine A — local agent
mg join \
  --hub http://localhost:9000 \
  --name gpu-a \
  --host 0.0.0.0 \
  --port 9001
```

On Machine B, copy the mg binary and run:

```
# Machine B — remote agent
mg join \
  --hub http://192.168.0.177:9000 \
  --name gpu-b \
  --host 0.0.0.0 \
  --port 9001
```

Verify both agents appear on the grid:

```
mg agents
```

Expected output:

NAME	AGENT_ID	TIER	STATUS	TPS
gpu-a	a1b2c3d4	SILVER	ONLINE	12.4
gpu-b	e5f6g7h8	SILVER	ONLINE	11.8

The TIER reflects hardware capability (BRONZE → SILVER → GOLD), inferred from VRAM. The hub uses tier and load to route tasks to the best available agent.

1.4.7 Step 4: Configure the SPL Momagrid Adapter

The SPL runtime has a built-in momagrid adapter. Tell it where your hub is by adding it to ~/.spl/config.yaml:

```
adapter: momagrid          # use momagrid as the default adapter

adapters:
  momagrid:
    hub_url: "http://192.168.0.177:9000"   # your hub's LAN address
    default_model: "gemma3"
    timeout: 300
    min_tier: "BRONZE"     # accept any available agent

  ollama:                  # keep ollama config as fallback
    base_url: "http://localhost:11434"
    default_model: "gemma3"
    timeout: 120
```

Alternatively, set the hub URL as an environment variable (no config file change needed):

```
export MOMAGRID_HUB_URL=http://192.168.0.177:9000
```

1.4.8 Step 5: Run Any Recipe Against the Grid

With the adapter configured, every existing SPL recipe runs against the LAN grid without any modification to the .spl file:

```
cd references/SPL20

# Run a recipe — SPL submits to the hub, hub dispatches to an agent
spl run cookbook/01_hello_world/hello.spl --adapter momagrid -m gemma3

# Or use your config defaults (no flags needed if config.yaml is set)
spl run cookbook/02_ollama_proxy/proxy.spl prompt="Explain transformer attention"
```

The hub selects which agent handles each task. If both gpu-a and gpu-b are online, the hub balances load automatically.

To run all 40 recipes against the grid:

```
cd references/SPL20
python cookbook/run_all.py
```

Because adapter: momagrid is set as the default in ~/.spl/config.yaml, every recipe in the batch uses the grid.

1.4.9 Monitoring the Grid

While recipes run, watch the grid from a separate terminal:

```
# Live task queue
mg tasks --detail

# Stream logs as tasks complete
mg logs --follow

# Per-agent reward accounting (tracks which node did the work)
mg rewards
```

Each task result includes the serving agent's identity:

```
Hello! I'm running on gpu-b with gemma3...

[model=gemma3 tokens=18+94 latency=3201ms agent=gpu-b completed=2026-03-27T09:14:52
```

1.4.10 Smoke Test Before Running All Recipes

Submit a manual task to confirm the grid is healthy before running the full batch:

```
mg submit "What is 2 + 2?" --model gemma3
```

If this returns an answer within a few seconds, your grid is ready.

1.4.11 Troubleshooting

Hub starts but agents cannot connect from Machine B Ensure port 9000 is open on Machine A's firewall:

```
sudo ufw allow 9000/tcp
```

Agent joins but tasks never dispatch The hub routes to agents that have the requested model loaded. Confirm Ollama has the model on that machine:

```
ollama list    # run on the agent machine
```

If the model is missing: ollama pull gemma3.

Cannot connect to Momagrid hub The SPL adapter cannot reach the hub. Check MOMAGRID_HUB_URL or hub_url in ~/.spl/config.yaml matches the hub's actual LAN IP. Run mg status to confirm the hub is up.

Agent shows BRONZE tier, recipes are slow BRONZE agents have limited VRAM. If all your machines are BRONZE, lower the minimum tier requirement (the default is already BRONZE). For faster throughput, use a model that fits comfortably in VRAM: phi4 or llama3.2:3b instead of a 27B model.

Running on a single machine (no LAN) Everything works with hub and agent on the same machine — just use localhost everywhere and skip Machine B. This is a valid setup for testing the adapter before expanding to multi-node.

1.4.12 What Just Happened

You built a distributed AI inference grid from commodity hardware and open-source software. SPL's adapter abstraction means no recipe in this book needed to change to take advantage of it — the same .spl file runs on a single Ollama instance, a two-GPU LAN grid, or a rack of machines. The score stays the same; the orchestra scales.

In the chapters that follow, every recipe can be run against your Momagrid. Recipes that use WITH blocks (parallel CTEs) will naturally dispatch their branches to separate agents, getting true parallel speedup across machines.

Part 2

Basics

Chapter 2.1

Hello World

2.1.1 The Pattern

Every technology stack demands a ritual: write the simplest possible program, run it, and confirm that the scaffolding holds. SPL is no different. Before you write a multi-step workflow that CALLs tools and GENERATEs structured output, you need to know that spl is installed, the adapter is configured, the model is running, and the runtime can connect them end to end.

The naive approach is to write a Python script: import the Ollama client, define a system message, send the request, print the response. It works. It also requires you to manage the HTTP client, handle the response envelope, deal with streaming vs. non-streaming modes, and remember the exact API shape of whichever model host you are using. For a sanity check, that is too much ceremony.

In SQL, the equivalent sanity check is:

```
SELECT 'hello' FROM dual;
```

One row. One column. Confirms the connection is live, the parser understood you, and the engine returned data. No tables, no joins, no logic. Just proof that the pipe works.

SPL's hello world is exactly that: one PROMPT block, one GENERATE call, no parameters, no tools, no state management. If it runs and returns text, the stack is healthy.

2.1.2 The SPL Approach

A PROMPT block assembles context in a SELECT clause, then fires a single GENERATE call into the assembled context. The model reads the context and produces the response. One LLM call, no moving parts.

2.1.3 The .spl File (Annotated)

```
-- Recipe 01: Hello World
-- Minimal SPL program -- verify spl + adapter + model work.
--
```

```
-- Usage:
--   spl run cookbook/01_hello_world/hello.spl
--   spl run cookbook/01_hello_world/hello.spl --adapter ollama
--   spl run cookbook/01_hello_world/hello.spl --adapter ollama -m gemma3

PROMPT hello_world                              -- (1) declare a named PROMPT block
SELECT
    system_role('You are a friendly assistant. Introduce yourself and SPL 2.0 in tw
                                                -- (2) assemble context in SELECT
GENERATE greeting()                             -- (3) fire the LLM call
```

Three lines of logic. Here is what each one does.

2.1.3.1 (1) PROMPT hello_world

PROMPT is the simplest SPL construct. It is a single-step program: select context, generate response, done. You can think of it as a named view — it has a name that the runtime uses for logging and tracing, but it takes no parameters and returns one result.

The SQL analogy is a named query or a view with no WHERE clause:

```
CREATE VIEW hello_world AS
SELECT 'hello from the database';
```

You are not filtering. You are not joining. You are simply declaring what this block does and giving it a name the runtime can track.

2.1.3.2 (2) SELECT system_role(...)

The SELECT clause in a PROMPT block assembles the context that will be given to the LLM when GENERATE fires. Each item in the SELECT is a piece of context: a system instruction, a user input, a retrieved document, a computed value.

system_role() is a built-in SPL function that wraps its argument as a system-level instruction. In most LLM APIs, the system message establishes the model's role, constraints, and persona — it is the part of the conversation that precedes any user turn. Putting your instruction in system_role() signals to the adapter that this text should be injected at the system level, not as user input.

The SQL analogy: the SELECT clause computes what you hand to the LLM, just as a SQL SELECT computes what you return to the caller. Here, you are computing exactly one thing: a system instruction string. That string becomes the model's full context for this prompt.

In later recipes, the SELECT clause will grow — you will add context.param AS column_name, function_call() AS context_label, and multiple joined context items. The structure is always the same: SELECT assembles a context row, GENERATE sends it to the model.

2.1.3.3 (3) GENERATE greeting()

GENERATE is an LLM call. It takes the context assembled by SELECT, sends it to the configured adapter, waits for the response, and makes the result available to the rest of the workflow.

greeting() is the step name, not a function you defined. It is how this call appears in logs, traces, and error messages. Choose step names that describe what the model is doing, not what you are hoping for.

The parentheses can accept arguments — in later recipes you will pass GENERATE classify(text, examples) or GENERATE translate(content, target_language). In this recipe, there are no arguments because the entire context is in the SELECT clause. The GENERATE step sends whatever SELECT assembled.

The SQL analogy:

```
-- SQL: compute a value
SELECT UPPER('hello world') AS result;

-- SPL: generate a value using the LLM
GENERATE greeting()
```

UPPER() is deterministic. greeting() is not. That is the entire difference. Both take input from what SELECT assembled. Both produce a value the runtime captures.

The critical distinction: SPL uses GENERATE for operations that require reasoning, language, or judgment — things only a model can do. It uses CALL for deterministic operations: loading files, computing checksums, looking up values, writing output. In hello world, you have one GENERATE and no CALLs, because the entire task is generative. Later recipes will mix them, and the discipline of separating them is where most of the design work lives.

2.1.4 Running It

Ensure Ollama is running and gemma3 is pulled:

```
ollama serve          # if not already running as a service
ollama pull gemma3    # one-time download, ~5 GB
```

Run the recipe:

```
spl run cookbook/01_hello_world/hello.spl --adapter ollama -m gemma3
```

Expected output (text will vary across runs):

```
Hello! I'm Gemma, a helpful AI assistant developed by Google DeepMind.
SPL 2.0, or Structured Prompt Language, is a declarative language for
orchestrating LLM workflows — think SQL for AI pipelines.
```

The response is printed to stdout. No files are written, no state is committed. This is a pure read operation.

You can also run without specifying an adapter if Ollama is your configured default:

```
spl run cookbook/01_hello_world/hello.spl
```

And you can swap models without changing the file:

```
spl run cookbook/01_hello_world/hello.spl --adapter ollama -m llama3.2
spl run cookbook/01_hello_world/hello.spl --adapter ollama -m mistral
```

The .spl file does not change. The adapter and model are runtime concerns, not workflow concerns. This is an early demonstration of a principle that pays off in later recipes: SPL workflows are model-agnostic by design.

2.1.5 What Just Happened

LLM calls: 1.

The runtime parsed hello.spl, identified the PROMPT hello_world block, assembled the context from the SELECT clause (one system instruction string), and handed it to the Ollama adapter with a request to the gemma3 model. The model generated a response. The runtime printed it and exited.

No files were read. No tools ran. No variables were set. No exception handlers were evaluated.

This is the minimal path through the SPL runtime. Every more complex recipe is this path plus additional steps layered on top:

- WORKFLOW blocks add INPUT parameters and multi-step sequences
- CALL steps add deterministic operations alongside the generative ones
- EVALUATE adds branching on model output
- COMMIT adds persistence
- EXCEPTION adds error handling

None of those are present here. Hello world establishes the floor: PROMPT + SELECT + GENERATE = one model call, one response, done.

2.1.6 Reproducibility Note

Latency on a GTX 1080 Ti with gemma3: **3–8 seconds** for a two-sentence response. The range is wide because a cold Ollama server will load the model weights into VRAM on the first call — subsequent calls in the same session are faster once the model is warm.

The response content varies across runs. The model is not seeded to a fixed random state, so each run produces a different phrasing. This is expected and correct for a generative sanity check. What you are verifying is not the exact text but that text arrives at all.

If you need reproducible output for a test: set temperature=0 via the adapter configuration, or use a workflow with structured output constraints (see Chapter 2.4). For a hello world, variability is not a problem — it is confirmation that the model is actually generating, not returning a cached string.

2.1.7 When to Use This Pattern

The PROMPT form — PROMPT name / SELECT context / GENERATE step() — is the right shape for:

- **Stack verification**: Confirm the runtime, adapter, and model are wired together correctly before writing anything more complex.

- **One-shot queries**: A single natural-language question that needs one LLM response and no post-processing. The Ollama proxy recipe (Chapter 2.2) extends this pattern with a parameter.
- **Prototyping a system message**: When you are tuning the persona or instruction in system_role(), a PROMPT block lets you iterate without running a full workflow.
- **Demonstrations and smoke tests**: CI pipelines that verify adapter connectivity use a hello-world-style PROMPT as the probe.

The PROMPT form is not the right shape when:

- You need to pass parameters from the command line (use WORKFLOW with INPUT)
- You need multiple sequential LLM calls (use WORKFLOW with multiple GENERATE steps)
- You need to call Python tools alongside the LLM (use WORKFLOW with CALL)
- You need to branch on the model's output (use WORKFLOW with EVALUATE)

Think of PROMPT as SELECT without a FROM: fast, useful for spot-checks, not a substitute for a full query.

2.1.8 Exercises

1. **Change the system instruction.** Edit the system_role() string to 'You are a gruff database administrato Re-run. Observe how the same GENERATE greeting() call produces a completely different response when the system context changes. The step name does not constrain the output — the context does.

2. **Swap the adapter.** If you have an Anthropic API key configured, run:

```
spl run cookbook/01_hello_world/hello.spl --adapter anthropic
```

The .spl file is unchanged. The response will reflect Claude's style rather than gemma3's. This demonstrates adapter portability: the workflow is the specification, the model is the execution engine.

3. **Promote to a WORKFLOW.** Take the PROMPT block and rewrite it as a WORKFLOW with an INPUT parameter called @persona TEXT DEFAULT 'friendly assistant'. Pass the persona into the system_role() string. Run it with persona="skeptical engineer" and persona="enthusiastic intern". This is the shape of section 1.2.

2.1.9 Appendix: Hello World Across All Adapters

The adapter portability promise is concrete: the same .spl file runs unchanged against every backend SPL supports. This section proves it — one file, every adapter, zero edits.

```
# The file under test — identical for every run below
src/recipes/ch01-1-hello-world/hello.spl
```

2.1.9.1 Local & LAN Adapters

No API key required. Models run on your own hardware.

```
# Echo — no LLM, mirrors the prompt back (CI smoke test, zero latency)
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter echo

# Ollama — local inference on your machine
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter ollama -m gemma3
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter ollama -m llama3.2
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter ollama -m mistral

# Momagrid — LAN grid (hub must be running; see Chapter 0.4)
# export MOMAGRID_HUB_URL=http://192.168.0.177:9000
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter momagrid -m gemma3
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter momagrid -m llama3.2
```

2.1.9.2 API Gateway Adapters

Single key, access to many models.

```
# Claude Code CLI — routes through the claude CLI binary (no API key in env)
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter claude_cli -m claude-son
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter claude_cli -m claude-opu

# OpenRouter — unified gateway to 200+ models
# export OPENROUTER_API_KEY=sk-or-...
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter openrouter -m anthropic/
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter openrouter -m openai/gpt
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter openrouter -m google/ger
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter openrouter -m meta-llama
```

2.1.9.3 Direct Cloud Provider Adapters

Each requires the provider's own API key.

```
# Anthropic — Claude models via the Messages API
# export ANTHROPIC_API_KEY=sk-ant-...
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter anthropic -m claude-sonn
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter anthropic -m claude-opus

# OpenAI — GPT models via Chat Completions API
# export OPENAI_API_KEY=sk-...
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter openai -m gpt-4o
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter openai -m gpt-4o-mini

# Google — Gemini models via GenAI SDK
# export GOOGLE_API_KEY=AIza...
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter google -m gemini-2.5-fla
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter google -m gemini-2.5-pro

# DeepSeek — via DeepSeek API (OpenAI-compatible)
# export DEEPSEEK_API_KEY=sk-...
```

```
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter deepseek -m deepseek-cha
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter deepseek -m deepseek-rea

# Qwen — Alibaba Cloud via DashScope API
# export DASHSCOPE_API_KEY=sk-...
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter qwen -m qwen-plus
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter qwen -m qwen-max

# AWS Bedrock — uses AWS credentials (IAM role, env vars, or ~/.aws/credentials)
# export AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=... AWS_DEFAULT_REGION=us-east
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter bedrock \
    -m us.anthropic.claude-sonnet-4-20250514-v1:0
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter bedrock \
    -m us.amazon.nova-pro-v1:0

# GCP Vertex AI — uses GCP Application Default Credentials
# gcloud auth application-default login
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter vertex -m gemini-2.5-fla
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter vertex -m gemini-2.5-pro

# Azure OpenAI — requires endpoint + deployment name
# export AZURE_OPENAI_ENDPOINT=https://<resource>.openai.azure.com/
# export AZURE_OPENAI_API_KEY=...
spl run src/recipes/ch01-1-hello-world/hello.spl --adapter azure_openai -m gpt-4o
```

2.1.9.4 Adapter Quick-Reference

Adapter	Category	Required credential	Default model
echo	Local	none	—
ollama	Local	none	gemma3
momagrid	LAN grid	MOMAGRID_HUB_URL	gemma3
claude_cli	API gateway	claude CLI binary	claude-sonnet-4-6
openrouter	API gateway	OPENROUTER_API_KEY	anthropic/claude-sonnet-4-5
anthropic	Direct cloud	ANTHROPIC_API_KEY	claude-sonnet-4-20250514
openai	Direct cloud	OPENAI_API_KEY	gpt-4o
google	Direct cloud	GOOGLE_API_KEY	gemini-2.5-flash
deepseek	Direct cloud	DEEPSEEK_API_KEY	deepseek-chat
qwen	Direct cloud	DASHSCOPE_API_KEY	qwen-plus
bedrock	Direct cloud	AWS credentials	*(specify model ARN)*
vertex	Direct cloud	GCP ADC	gemini-2.5-flash
azure_openai	Direct cloud	AZURE_OPENAI_API_KEY + AZURE_OPENAI_ENDPOINT	gpt-4o

Every adapter above runs hello.spl without touching the file. The credential and the --adapter flag are the only variables. This is the portability guarantee in practice.

Next: Chapter 1.2 — Ollama Proxy: Parameterized LLM Queries

Chapter 2.2

Ollama Proxy

2.2.1 The Pattern

In the previous chapter, we verified the "Hello World" of SPL: a hardcoded prompt that proves the connection between the runtime and the model. But real-world utility requires input. You don't just want the model to introduce itself; you want to ask it *your* questions.

The common way to interact with a local LLM is via a dedicated chat UI or a specific CLI tool like ollama run. These are great for interactive sessions, but they are hard to script or integrate into a larger pipeline. If you want to use the output of a model as a step in a bash script or a CI process, you often end up writing a wrapper script in Python or Node.js to handle the HTTP request, extract the text from the JSON response, and print it to stdout.

The SQL analogy is a parameterized query:

```
SELECT * FROM users WHERE user_id = ?;
```

You don't rewrite the query every time you want a different user; you pass the user_id as a parameter. The query structure remains constant, while the data varies.

The Ollama Proxy recipe is the SPL version of a parameterized query. It defines a reusable prompt structure that accepts a prompt argument from the command line, sends it to the model, and returns the result. It turns any LLM into a standard Unix-style utility.

2.2.2 The Open-Source Engine Room: Ollama and Llama.cpp

Before we dive into the code, we must acknowledge the "engine room" that makes this recipe possible: **Ollama** and **Llama.cpp**.

In the proprietary AI world, models are locked behind APIs. You pay per token, you need a credit card, and you need a high-speed, uncensored connection to a data center in the Global North. For an engineer in Lagos, a student in rural Anhui, or a researcher in Recife, this is a barrier to entry that reproduces old inequalities.

Llama.cpp (and its user-friendly distribution, **Ollama**) changed the game. By optimizing LLM inference for consumer-grade hardware, they turned the "commodity hardware" in your closet—like

a used **GTX 1080 Ti**—into a powerful AI workstation. These are not just technical achievements; they are tools of digital sovereignty.

In the Momagrid vision, these open-source foundations are the "orchestra" in our conductor metaphor. SPL 2.0 provides the "score" (the logic), but Ollama provides the "instruments" (the models). By advocating for open-source models (like Gemma, Llama, or Mistral) running on open-source runtimes, we ensure that AI remains a public utility accessible to everyone, everywhere, without a middleman.

When you run this recipe, you aren't just calling a model; you are exercising your right to run state-of-the-art intelligence on your own terms.

2.2.3 The SPL Approach

The core idea is to use the context object to bridge the gap between the command line and the SPL runtime. By selecting context.prompt, we take a named argument passed at runtime and inject it into the LLM's context.

2.2.4 The .spl File (Annotated)

```
-- Recipe 02: Ollama Proxy
-- General-purpose LLM query -- proxy any prompt to any adapter/model.
--
-- Usage:
--   spl run cookbook/02_ollama_proxy/proxy.spl prompt="Explain quantum computing"

PROMPT ollama_proxy                                  -- (1) name the prompt block
SELECT
    system_role('You are a helpful, knowledgeable assistant.'),
                                                     -- (2) establish the persona
    context.prompt AS prompt                         -- (3) bind the CLI parameter
GENERATE answer(prompt)                              -- (4) execute the query
```

2.2.4.1 (1) `PROMPT ollama_proxy`

Like "Hello World," we use a PROMPT block because we only need a single LLM call. This is the "view" that we are executing.

2.2.4.2 (2) `system_role(...)`

We define a general-purpose persona. Since this script is a proxy for any question, we want a model that is helpful and knowledgeable. This stays constant regardless of the input.

2.2.4.3 (3) `context.prompt AS prompt`

This is the bridge. In SPL, the context object contains all parameters passed via the CLI (e.g., key="value"). By selecting context.prompt, we are telling SPL: "Look for a parameter named prompt in the environment and include its value in the context row."

The AS prompt alias is important. It gives the piece of context a label that we can refer to in the GENERATE step.

SQL Analogy:

```
SELECT :input_param AS user_query;
```

You are taking an external variable and bringing it into the scope of your query.

2.2.4.4 (4) `GENERATE answer(prompt)`

We trigger the LLM call. Unlike section 1.1, we now pass an argument to the step: answer(prompt). This tells the runtime to specifically emphasize the prompt column from our SELECT statement as the primary instruction for the model.

2.2.5 Running It

You can pass the prompt parameter as a key-value pair after the file path:

```
spl run cookbook/02_ollama_proxy/proxy.spl --adapter ollama -m gemma3 \
    prompt="Explain quantum computing in one sentence"
```

Expected output:

```
Quantum computing uses the principles of subatomic physics to perform complex
calculations that are currently impossible for traditional computers.
```

Because the adapter and model are specified at the CLI level, this single .spl file can be used to test any model in your Ollama library:

```
# Test with Llama 3.2
spl run cookbook/02_ollama_proxy/proxy.spl -m llama3.2 prompt="What is 2+2?"

# Test with Mistral
spl run cookbook/02_ollama_proxy/proxy.spl -m mistral prompt="Write a haiku about S
```

2.2.6 What Just Happened

LLM calls: 1.

The runtime performed the following steps: 1. Parsed the command-line arguments and populated the context object with prompt="Explain quantum computing...". 2. Executed the SELECT clause, combining the system_role string and the context.prompt value into a single context row. 3. Passed this row to the specified adapter/model. 4. Captured the model's output and printed it to the console.

If you omit the prompt= argument, the runtime will throw an error because context.prompt is undefined. This is similar to a SQL error when a required bind variable is missing.

2.2.7 Reproducibility Note

Since this is a proxy, its performance and stability depend entirely on the underlying model you choose.

On a **GTX 1080 Ti**, running **gemma3**, you should expect: - **Time to First Token (TTFT)**: ~200ms - **Generation Speed**: ~40-60 tokens per second

The reproducibility of the *answer* depends on the prompt complexity. For factual questions ("What is 2+2?"), most models are 100% stable. For creative tasks, expect high variance unless you configure the adapter for temperature=0.

2.2.8 When to Use This Pattern

Use the **Ollama Proxy** pattern when: - **CLI Tooling**: You want to call an LLM from a shell script, e.g., git commit −m "$(spl run proxy.spl prompt='Summarize these changes...')" - **Quick Model Testing**: You want to compare how different models handle the exact same prompt without writing any code. - **Generic Utility**: You need a "swiss army knife" script that can handle any one-shot task.

Do not use this pattern when: - **Structured Logic**: You need to validate the output or take different actions based on what the model says (use EVALUATE in a WORKFLOW). - **Chain of Thought**: You need the model to "think" through steps before answering (use the patterns in Part 3).

2.2.9 Exercises

1. **Multiple Parameters**: Modify the SELECT clause to accept a second parameter context.topic. Update the system_role to say 'You are an expert in ' || context.topic. Run it with topic="Biology" prompt="What is a cell?".
2. **Default Values**: (Advanced) Research how to use the COALESCE or DEFAULT equivalent in SPL to provide a fallback if context.prompt is missing.
3. **Shell Integration**: Use the proxy in a bash pipe. For example:

```
echo "The weather is nice today" | xargs -I {} spl run proxy.spl prompt="Translate
```

Chapter 2.3

Multilingual Greeting

2.3.1 The Pattern

In a truly global AI ecosystem, assuming the user speaks English is a design flaw. Whether you are building a support bot for a marketplace in Southeast Asia or a localized tutor for a school in West Africa, the ability to pivot between languages dynamically is a core requirement.

The naive way to handle multilingual prompts is often to hardcode multiple prompts or use complex string interpolation in a programming language. You end up with a "prompt library" that is difficult to maintain and even harder to test across different models.

The SQL analogy is a multi-parameter join:

```
SELECT t.translation
FROM dictionary d
JOIN translations t ON d.id = t.id
WHERE d.word = ? AND t.language = ?;
```

You have the "intent" (the word) and the "constraint" (the language). By providing both, you get the specific result you need.

The Multilingual Greeting recipe takes this principle and applies it to a prompt. It accepts two parameters—user_input and lang—and uses them to constrain the model's response. It's a simple demonstration of how SPL can manage multi-dimensional context without the "spaghetti code" of imperative string formatting.

2.3.2 The SPL Approach

We use the context object to capture multiple CLI arguments and then "bind" them to semantic labels in our SELECT clause. This allows us to pass structured, multi-part instructions to the GENERATE step.

2.3.3 The .spl File (Annotated)

```
-- Recipe 03: Multilingual Greeting
```

```
-- Greet in any language -- demonstrates parametric context with lang.
--
-- Usage:
--   spl run cookbook/03_multilingual/multilingual.spl user_input="hello" lang="Ch

PROMPT multilingual_greeting                    -- (1) name the prompt block
SELECT
    system_role('You are a friendly assistant. Respond in the language specified.')
                                                -- (2) system-level constraint
    context.user_input AS input,                -- (3) the user's message
    context.lang AS lang                        -- (4) the target language
GENERATE greeting(input, lang)                  -- (5) context-aware generation
```

2.3.3.1 (1) `PROMPT multilingual_greeting`

A single model call. The complexity is in the input, not the execution flow.

2.3.3.2 (2) `system_role(...)`

This establishes the "rules of the game." We tell the model to be friendly and, crucially, to respect the language specification provided in the context.

2.3.3.3 (3) & (4) `context.user_input AS input` and `context.lang AS lang`

This is where we "map" our command-line parameters into our query scope. By giving them clean aliases (input, lang), we make our GENERATE step more readable.

In SPL, every column in your SELECT clause becomes a piece of evidence for the model. By labeling them explicitly, you are giving the model a structured view of the world: "Here is the input, and here is the language constraint."

2.3.3.4 (5) `GENERATE greeting(input, lang)`

We pass both aliases as arguments to the GENERATE step. This tells the model: "Produce a greeting by considering both the input message and the language requirement."

2.3.4 Running It

Test it with different languages:

```
# To Chinese
spl run cookbook/03_multilingual/multilingual.spl user_input="hello wen" lang="Chi

# To French
spl run cookbook/03_multilingual/multilingual.spl user_input="hello wen" lang="Fren

# To Spanish
spl run cookbook/03_multilingual/multilingual.spl user_input="hello wen" lang="Spa
```

Expected output (for Chinese):

```
(Hello, Wen! Nice to see you. I am your assistant.)
```

2.3.5 What Just Happened

LLM calls: 1.

The runtime combined the system_role, the input, and the lang into a single prompt package. Because we are using an open-source model like **gemma3** via **Ollama**, the model's native multilingual capabilities are triggered.

Modern open-weights models are trained on vast, multi-lingual datasets. By explicitly naming the language in the prompt, you are activating the specific language weights within the model's neural network.

2.3.6 Reproducibility Note

The stability of this recipe varies based on the "popularity" of the language in the model's training set. - **High Stability**: English, Chinese, Spanish, French, German. - **Moderate Stability**: Hindi, Japanese, Portuguese. - **Lower Stability**: Low-resource languages (e.g., local dialects).

On a **GTX 1080 Ti**, the latency for a short greeting remains consistent at **2-5 seconds** regardless of the language, as the model's internal processing time is more dependent on the number of tokens generated than the specific language family.

2.3.7 When to Use This Pattern

Use the **Multilingual** pattern when: - **Global Reach**: You need to serve users in their native tongue without duplicating your logic. - **Dynamic Formatting**: You want to change the "style" or "format" of the output based on a parameter (e.g., lang="pirate", lang="formal", lang="bullet points"). - **Context Injection**: You have more than one variable piece of data to feed into the model.

2.3.8 Exercises

1. **Add a "Tone" parameter.** Modify the SELECT to accept context.tone and update the system_role to include it. Run with lang="Chinese" tone="extremely excited" and then tone="very bored".
2. **Combine with Translation.** Instead of just a greeting, change the prompt to be a "Translator." Set the system_role to `'You are a translator.'` and the GENERATE step to `translate(input, lang)`.
3. **Regional Dialects.** Try specific dialects like lang="Brazilian Portuguese" vs. lang="European Portuguese" See if your local model can tell the difference!

Chapter 2.4

Few-Shot Learning

2.4.1 The Pattern

Even the most sophisticated LLMs sometimes struggle to follow complex instructions. You might ask for a "JSON object with sentiment and summary," and the model might return a markdown-formatted string with a conversational preamble: *"Sure! Here is the JSON you requested…"*. This "chattiness" breaks automated pipelines.

Few-Shot Learning (or Few-Shot Prompting) is the technique of providing the model with 2–3 "gold-standard" examples of what a perfect input-output pair looks like. By seeing examples, the model "calibrates" its tone, format, and reasoning pattern to match your expectations. It is often more effective than writing a three-page instruction manual.

The SQL analogy is a **Lookup Table** or a **Reference Dataset**. You aren't just giving the database a schema; you are giving it a small set of "row templates" that define what valid data looks like.

The Few-Shot Learning recipe (section 1.4) demonstrates how to embed these examples directly into the SELECT context. It uses a domain-swappable set of examples to guide the model across different industries (Finance, Ops, General).

2.4.2 The SPL Approach

This recipe uses CREATE FUNCTION with a SELECT CASE block to deliver dynamic, domain-specific examples into the context row.

2.4.3 The .spl File (Annotated)

```
-- Recipe 24: Few-Shot Prompting
-- Embed gold-standard examples in context to guide output format.

CREATE FUNCTION few_shot_examples(domain TEXT DEFAULT 'general')
RETURNS TEXT AS $$
SELECT CASE domain                          -- (1) Domain-specific example sets
  WHEN 'finance' THEN
```

```
    '### Examples
      Input: "Revenue up 12% YoY."
      Output: {"sentiment": "positive", ...}'
  WHEN 'ops' THEN
    '### Examples
      Input: "Deployment succeeded."
      Output: {"status": "healthy", ...}'
  ELSE
    '### Examples
      Input: "New feature launched!"
      Output: {"sentiment": "positive", ...}'
END
$$;

PROMPT few_shot_classifier
SELECT
    system_role('You are a precise text classifier. Follow the examples exactly.'),
    few_shot_examples(context.domain) AS examples, -- (2) Dynamic context injection
    context.text AS text,
    context.domain AS domain
GENERATE classify(text, examples)          -- (3) Calibrated generation
```

2.4.3.1 (1) Dynamic Examples via SELECT CASE

Instead of hardcoding examples into the prompt string, we encapsulate them in a function. This allows us to maintain different "Reference Sets" for different domains in one place.

SQL Analogy: A **Stored Function** that returns a configuration string based on a parameter.

2.4.3.2 (2) Dynamic Context Injection

In the SELECT clause, we call few_shot_examples(context.domain). - If you pass domain="finance" on the CLI, the LLM receives finance-related examples. - If you pass domain="ops", it gets operations-related examples.

This makes your workflow "Context-Aware" without duplicating the prompt logic.

2.4.3.3 (3) Calibrated Generation

The GENERATE step receives both the user's text and the examples. Because the model sees the ### Examples block right before it generates its own response, it naturally adopts the same JSON structure and field names shown in the examples.

2.4.4 Running It

Run the classifier for the finance domain:

```
spl run cookbook/24_few_shot/few_shot.spl --adapter ollama \
    text="The quarterly results exceeded all analyst forecasts" \
    domain="finance"
```

Expected output (clean JSON):

```
{"sentiment": "positive", "magnitude": 0.9, "topics": ["earnings", "forecasts"], "s
```

Now try the operations domain:

```
spl run cookbook/24_few_shot/few_shot.spl --adapter ollama \
    text="System outage detected in EU-WEST-2" \
    domain="ops"
```

Expected output:

```
{"status": "incident", "severity": "high", "action_required": true, "summary": "Sys
```

2.4.5 What Just Happened

LLM calls: 1.

The "Conductor" (SPL Runtime) managed the calibration: 1. Looked up the domain parameter. 2. Retrieved the matching "Reference Set" from the few_shot_examples function. 3. Assembled a context package containing: System Role + Reference Examples + User Input. 4. Executed a single, high-precision generation.

2.4.6 Reproducibility Note

Few-shot prompting is one of the most effective ways to increase **Stability**. While a "Zero-Shot" prompt (no examples) might fail 5% of the time, a "Two-Shot" prompt usually reaches 99%+ reliability for formatting.

On a **GTX 1080 Ti**, adding a few examples adds negligible latency (a few milliseconds for the extra tokens) but saves hours of post-processing and debugging.

2.4.7 When to Use This Pattern

Use the **Few-Shot Learning** pattern when: - **Strict Format Requirements**: You need JSON, YAML, or a specific tone that the model doesn't always hit on the first try. - **Domain Specialization**: You want the same workflow to behave differently for "Medical" vs. "Legal" vs. "Code" tasks. - **Edge Case Handling**: If the model keeps making a specific mistake, add a "Negative Example" to the few-shot set to show it what *not* to do.

2.4.8 Exercises

1. **Add a "Negative" Case.** Modify the few_shot_examples to include a case where the input is "ambiguous" and the output should be an empty JSON object.
2. **Toggle Example Count.** Modify the function to accept a second parameter @count and use it to return 1, 2, or 3 examples. See how the accuracy changes.
3. **Cross-Domain Test.** Run the recipe with a Finance text but pass domain="ops". Observe how the model tries to force-fit finance data into an operations schema. This is a great way to understand the "gravity" of examples.

Part 3

Agentic Patterns

Chapter 3.1

Self-Refine

3.1.1 The Pattern

You have a task that benefits from iteration: drafting a persuasive email, writing technical documentation, composing a code review comment. The naive approach is to call the LLM once and accept whatever comes back. Experienced practitioners know the first pass is rarely production-ready.

The Python equivalent looks roughly like this:

```
draft = llm.generate(task)
for i in range(max_iterations):
    feedback = llm.critique(draft)
    if feedback.strip().lower() == "satisfactory":
        break
    draft = llm.refine(draft, feedback)
```

This is manageable to write, but awkward to maintain. The loop logic, the exit condition, and the budget handling are all tangled together. When you add exception handling for budget overruns or you want to pass quality metadata to the caller, the code grows in directions that obscure the original intent. Six months later, a new team member reads this and has to reconstruct the algorithm from the implementation details.

More practically: every team that uses LLMs for iterative generation writes this loop. They each write it slightly differently. There is no shared vocabulary for the pattern, no standard way to configure max_iterations from the outside, no consistent exception surface.

SPL gives this pattern a name and a syntax.

3.1.2 The SPL Approach

Generate a draft, then loop: critique the draft, exit if satisfactory, otherwise refine and repeat — up to a configurable iteration limit.

3.1.3 The .spl File (Annotated)

```
-- SPL 2.0: Self-Refine Pattern
-- Iteratively improves output through critique and refinement

WORKFLOW self_refine
  INPUT:
    @task text,
    @max_iterations integer default 5      -- caller controls iteration budget
  OUTPUT: @result text
DO
  @iteration := 0
  -- @max_iterations := 5                  -- default shown; overridable at runtime

  -- Initial draft - one LLM call, cold start
  GENERATE draft(@task) INTO @current
  --
  -- SQL analogy: this is the seed row that the UPDATE loop will refine.
  -- Think: INSERT INTO drafts VALUES (llm_generate(task));

  -- Iterative refinement loop
  WHILE @iteration < @max_iterations DO
    -- Ask the LLM to critique its own output
    GENERATE critique(@current) INTO @feedback
    --
    -- critique() is a CREATE FUNCTION (not shown in this file, but registered
    -- in the system context). It receives @current as input and is prompted
    -- to return either "satisfactory" or a specific critique.
    -- The LLM is not deciding the control flow - EVALUATE is.

    EVALUATE @feedback
      WHEN 'satisfactory' THEN
        -- Early exit: quality threshold reached before max_iterations
        -- SQL analogy: BREAK in a cursor loop when WHERE condition is met
        COMMIT @current WITH status = 'complete', iterations = @iteration
      OTHERWISE
        -- Refine: feed the critique back into the LLM alongside the draft
        GENERATE refined(@current, @feedback) INTO @current
        @iteration := @iteration + 1
    END
  END

  -- Loop exhausted without hitting 'satisfactory' - commit best effort
  -- SQL analogy: the cursor reached EOF; commit whatever state we're in
  COMMIT @current WITH status = 'max_iterations', iterations = @iteration

EXCEPTION
  WHEN MaxIterationsReached THEN
    -- Runtime raised this before our loop could: external budget enforcer
    COMMIT @current WITH status = 'partial'
```

```
  WHEN BudgetExceeded THEN
    -- Token/cost budget hit -- commit whatever is in @current
    COMMIT @current WITH status = 'budget_limit'
END
```

Key design decisions to notice:

EVALUATE @feedback / WHEN 'satisfactory' — The LLM wrote @feedback, but the LLM does not decide whether to exit the loop. EVALUATE is deterministic string matching. The LLM's job is to produce the word "satisfactory" or a critique. The runtime's job is to act on it. This separation is intentional and important.

@max_iterations integer default 5 — This is not hardcoded. The caller decides the iteration budget. A CI pipeline might pass max_iterations=2 for speed; an offline document editor might pass max_iterations=10 for quality. The workflow has no opinion.

COMMIT @current WITH status = 'complete', iterations = @iteration — The COMMIT carries metadata. Downstream code can see how many iterations were needed. A monitoring dashboard can track this over time: if your average iterations is creeping up, your base draft quality may be degrading, or your critique prompt has become too strict.

EXCEPTION / WHEN BudgetExceeded — The runtime enforces a token budget independently of your loop counter. If costs spike mid-run, the workflow does not crash with an unhandled exception. It commits what it has and marks the status clearly.

3.1.4 The SQL Analogy in Depth

SQL practitioners are comfortable with cursor loops. The self-refine pattern is a cursor loop where the exit condition is semantic rather than positional.

A conventional cursor loop:

```
DECLARE cur CURSOR FOR SELECT id, text FROM drafts WHERE status = 'pending';
OPEN cur;
LOOP
  FETCH cur INTO v_id, v_text;
  EXIT WHEN NOT FOUND;
  UPDATE drafts SET text = improve(v_text) WHERE id = v_id;
END LOOP;
CLOSE cur;
```

The self-refine SPL workflow is structurally identical, except: - There is only one row (the current draft). - The exit condition (WHEN 'satisfactory') is evaluated by the LLM at each step, not derived from a SQL predicate. - The "UPDATE" is a GENERATE that produces a new version of the same row.

A stored procedure that calls UPDATE in a loop until a quality score exceeds a threshold is doing exactly what self_refine.spl does. The syntax differs. The logic is the same.

3.1.5 Running It

```
spl run cookbook/05_self_refine/self_refine.spl \
    --adapter ollama -m gemma3 \
    task="Write a concise product description for a noise-cancelling travel headpho
    max_iterations=3
```

Expected output (truncated):

```
[draft] Generating initial draft...
[critique] Iteration 0: feedback received
[refined] Iteration 0: draft updated
[critique] Iteration 1: satisfactory
[commit] status=complete iterations=1

Result:
Experience travel in silence. Our noise-cancelling headphones block ambient
cabin noise while delivering rich, balanced audio across a 30-hour battery
life. Lightweight folding design fits any carry-on pocket. USB-C charging
from 0 to 80% in 45 minutes.
```

3.1.6 What Just Happened

LLM calls made: 3 (1 draft + 1 critique at iteration 0 + 1 critique at iteration 1).

Step-by-step:

1. GENERATE draft(@task) — Cold start. The LLM writes a first-pass product description from nothing.
2. GENERATE critique(@current) at iteration 0 — The LLM evaluates its own output. In this run it found the draft vague ("rich audio" with no specifics), returned structured feedback.
3. GENERATE refined(@current, @feedback) at iteration 0 — The LLM revised the draft incorporating the critique. Concrete battery life, charge time, and weight claim were added.
4. GENERATE critique(@current) at iteration 1 — The LLM re-evaluated. Returned "satisfactory".
5. EVALUATE 'satisfactory' — Matched the exit branch. COMMIT fired with status='complete' and iterations=1.

The key insight: the workflow ran 2 fewer LLM calls than max_iterations would have permitted. Early exit is working correctly. On poor-quality base prompts, expect to see iterations=4 or iterations=5 (max).

3.1.7 Reproducibility Note

Hardware: GTX 1080 Ti, 11GB VRAM. **Model:** gemma3 via Ollama.

Typical wall-clock time for a short copywriting task: 8–14 seconds for a 2-iteration run, 18–28 seconds for a 5-iteration run.

Stability: The critique function's "satisfactory" output is sensitive to prompt wording. If your critique prompt does not explicitly instruct the model to return the exact string "satisfactory" (and nothing else) when the draft meets the bar, you will see false negatives — the EVALUATE branch will always fall through to OTHERWISE, exhausting max_iterations every run. Test your critique prompt in isolation before embedding it in the loop.

Non-determinism: Two runs on identical input will often differ at iteration count. One run exits at iteration 1, the next at iteration 2. This is expected. The self-refine pattern is not designed to be deterministic at the iteration level — only at the structural level (the loop always terminates, the output is always committed, exceptions are always handled).

3.1.8 When to Use This Pattern

Good fit: - Content generation where quality has a measurable (if fuzzy) threshold: product copy, executive summaries, cover letters, commit messages. - Code documentation where "complete and accurate" is a binary outcome that can be evaluated by a second LLM pass. - Any task where you have seen humans naturally do "write, review, rewrite" manually.

Poor fit: - Tasks with an objective correct answer. Use CALL + a Python validator instead of a critique loop. An LLM critiquing a SQL query it generated may not catch a logical error it already made. - High-volume batch jobs where you need strict cost control. A 5-iteration loop on 1,000 inputs could mean 5,000 LLM calls in the worst case. Set max_iterations=2 for batch workloads and accept the quality tradeoff. - Real-time applications. The latency stacks multiplicatively. A 3-second LLM call becomes 15 seconds at max_iterations=5.

The right tool vs. alternatives: - Self-Refine vs. Reflection (Chapter 3.4): Self-refine evaluates the *output*. Reflection evaluates the *reasoning process*. Use self-refine when the output quality is observable; use reflection when the reasoning steps may be subtly wrong even when the output looks correct. - Self-Refine vs. Ensemble Voting (section 6.3): Ensemble generates N drafts and picks the best. Self-refine generates one draft and improves it. Use self-refine when you have a clear critique standard; use ensemble when you want diversity.

3.1.9 Exercises

1. **Tighten the critique prompt.** The default critique function accepts any structured feedback. Modify the CREATE FUNCTION critique() to require a structured output: a rating 1–10 and a bullet-point list of specific issues. Change EVALUATE @feedback to check WHEN > 7 instead of WHEN 'satisfactory'. How does output quality change?

2. **Add a word-count constraint.** The current workflow has no length control. Add a CALL count_words(@current) INTO @word_count step using a Python tool, and add a second EVALUATE that truncates or expands the draft if it falls outside a target range (e.g., 50–80 words). This introduces a hybrid: deterministic length enforcement combined with LLM quality refinement.

3. **Instrument the iteration cost.** Modify the COMMIT to also record an estimated token cost per iteration. Add a Python tool estimate_tokens(text) that returns a character-count approximation. After 10 runs, plot the cost vs. iterations vs. final quality. This gives you the data to set max_iterations empirically rather than arbitrarily.

Chapter 3.2

ReAct Agent

3.2.1 The Pattern

One of the most common pitfalls in AI development is over-reliance on the model. Because LLMs are so good at talking, we tend to ask them to do everything: search the web, extract data, perform calculations, and write reports. But LLMs are probabilistic—they are "guessing" the next token. When it comes to arithmetic or precise data retrieval, "guessing" is not good enough.

The **ReAct (Reason + Act)** pattern solves this by giving the model tools. The model "reasons" about what it needs to do, "acts" by calling an external tool (like a calculator or a search engine), and then observes the result before moving to the next step.

The SQL analogy is an **External Procedure Call** or a **Trigger**. Your database is great at storage and joins, but if you need to send an email or process a credit card, you don't write that logic in SQL; you call an external service.

The ReAct Agent recipe (section 2.2) demonstrates this "80/20 Rule": - Use CALL for deterministic operations (math, web search, regex). - Use GENERATE for reasoning (deciding which year to search) and language (writing the final report).

3.2.2 The SPL Approach

This recipe introduces PROCEDURE definitions and the CALL statement. It also shows how to register external Python tools using the @spl_tool decorator.

3.2.3 The .spl File (Annotated)

```
-- Recipe 06: ReAct Agent -- Population Growth
-- Fetch population via WebSearch, compute growth rate via Python tool.

CREATE FUNCTION search_population(country TEXT, year INT) RETURNS TEXT AS $$
Use WebSearch to find the total population of {country} for the year {year}.
Return ONLY the population as a plain integer.
$$;
```

```
PROCEDURE search_population(country TEXT, year INT) -- (1) Define a reusable proced
RETURNS TEXT
DO
  GENERATE search_population(country, year) INTO @result
  COMMIT @result
END

WORKFLOW population_growth
  INPUT:
    @country TEXT DEFAULT 'China',
    @year_curr integer default 2023
  OUTPUT: @report TEXT
DO
  @year_prev := @year_curr - 1                    -- (2) Arithmetic in SPL

  -- Step 1: Fetch population for each year via WebSearch
  CALL search_population(@country, @year_prev) INTO @pop_prev -- (3) Call procedure
  CALL search_population(@country, @year_curr) INTO @pop_curr

  -- Step 2: Compute growth rate - routed to Python tool
  CALL calc_growth_rate(@pop_prev, @pop_curr) INTO @growth_rate -- (4) Call Python

  -- Step 3: Compose the final human-readable report
  GENERATE growth_report(...) INTO @report     -- (5) Generate final narrative
  COMMIT @report
END
```

3.2.3.1 (1) `PROCEDURE name(...)`

A PROCEDURE is a named block of SPL logic. Unlike a PROMPT, which is a single model call, a PROCEDURE can contain multiple steps. It's the building block of modular agentic workflows.

SQL Analogy: A **Stored Procedure**. It encapsulates a specific task (searching for population) so the main workflow stays clean.

3.2.3.2 (2) Arithmetic in SPL

Notice @year_prev := @year_curr − 1. SPL handles basic arithmetic and variable assignment directly in the runtime. You don't need an LLM to subtract 1 from a year. This is the "Conductor" doing the simple math so the "Orchestra" can focus on the music.

3.2.3.3 (3) `CALL procedure_name(...)`

The CALL statement executes a procedure or a tool. It is **deterministic**. When you CALL search_population, the runtime executes the steps inside that procedure and returns the result.

3.2.3.4 (4) `CALL calc_growth_rate(...)`

This is where the magic happens. calc_growth_rate is not an SPL procedure; it's a **Python function** decorated with @spl_tool. By using CALL, we are leaving the probabilistic world of the LLM and entering the deterministic world of Python.

The Python tool handles the string parsing (removing commas/spaces) and the floating-point math. The LLM never sees the numbers until the very end.

3.2.3.5 (5) `GENERATE growth_report(...)`

Only after the data has been fetched and the math has been verified do we go back to the LLM. We give the model the raw data and ask it to write a "concise 2-3 sentence report." This is what LLMs are best at: **Language Synthesis**.

3.2.4 Running It

This recipe requires a Python tool file and a web search capability:

```
spl run cookbook/06_react_agent/react_agent.spl \
    --adapter claude_cli \
    --allowed-tools WebSearch \
    --tools cookbook/06_react_agent/tools.py \
    country="France"
```

Expected output:

```
In 2022, France had a population of approximately 67,936,455.
By 2023, this grew to 68,170,228, representing a year-over-year
growth rate of 0.3441%.
```

3.2.5 What Just Happened

LLM calls: 3. (2 for searches, 1 for the final report) **Tool calls: 1.** (Python math tool)

The "Conductor" (SPL Runtime) managed the flow: 1. Calculated 2022 from $2023 - 1$. 2. Delegated the searching to the search_population procedure. 3. Delegated the math to the calc_growth_rate Python tool. 4. Assembled the final evidence and asked the model to write the prose.

This separation of concerns is the secret to building **reliable** AI agents. By offloading the math to Python, you eliminate the risk of "hallucinated" growth rates.

3.2.6 Reproducibility Note

The most variable part of this recipe is the **Web Search**. Different models might navigate to different sources, leading to slightly different population numbers.

However, because the **math** is handled by Python, the *relationship* between the numbers will always be mathematically sound. This is a critical distinction in SPL 2.0: we accept variability in retrieval but demand correctness in calculation.

3.2.7 When to Use This Pattern

Use the **ReAct Agent** pattern when: - **Accuracy is Critical**: Any task involving math, dates, or precise data extraction. - **External Integration**: You need to fetch data from an API, a database, or the web. - **Modular Design**: You want to build a library of reusable procedures (search_population, verify_address, etc.) that can be shared across multiple workflows.

3.2.8 Exercises

1. **Add Error Handling.** Use the EXCEPTION block to handle cases where WebSearch fails to find a number.
2. **Multi-Year Comparison.** Modify the workflow to fetch data for the last *three* years and calculate the average growth rate.
3. **Unit Conversion.** Add a Python tool that converts the population number into "millions" or "billions" before passing it to the final report generator.

Chapter 3.3

Plan and Execute

3.3.1 The Pattern

In the previous recipes, we focused on single tasks: generate a greeting, fetch a population, judge a debate. But real-world engineering is more complex. If you ask an LLM to "Build a REST API for a todo app," and it tries to write all the code in one go, it will likely fail. It will hit its output token limit, forget to include imports, or lose consistency between the database model and the API routes.

The **Plan and Execute** pattern is the professional way to handle large tasks. Instead of jumping straight to code, the system follows a structured lifecycle: 1. **Planning**: Decompose the big goal into a numbered list of concrete, independent steps. 2. **Execution**: Iterate through each step, making design decisions and generating code. 3. **Refinement**: If a step fails validation, re-plan the remaining steps based on the new information.

The SQL analogy is a **Long-Running Transaction** or a **Recursive Batch Job**. You don't update a million rows in one statement; you break the work into batches, verify each batch, and log your progress so you can resume if something fails.

The Plan and Execute recipe (section 2.3) is the most advanced workflow in the basics section. It demonstrates how to use nested WHILE loops and EVALUATE logic to manage a multi-file software project from start to finish.

3.3.2 The SPL Approach

This recipe introduces the "Two-Phase Loop" pattern, which is essential for generating large artifacts without hitting model limits.

3.3.3 The .spl File (Annotated)

```
-- Recipe 12: Plan and Execute
-- A planner decomposes a task, then an executor implements each step.

WORKFLOW plan_and_execute
    INPUT: @task TEXT, @output_dir TEXT, @max_steps INTEGER DEFAULT 5
```

```
DO
    -- Phase 1: Planning
    GENERATE plan(@task) INTO @plan              -- (1) Create the blueprint
    GENERATE count_steps(@plan) INTO @step_count

    -- Phase 2: Design Loop
    WHILE @step_index < @step_count DO           -- (2) Iterative Design
        GENERATE extract_step(@plan, @step_index) INTO @current_step
        GENERATE execute_step(@current_step, @results) INTO @step_result

        EVALUATE validate_step(@current_step, @step_result)
            WHEN 'failed' THEN                   -- (3) Automatic Re-planning
                GENERATE replan(@task, @plan, @step_index, @step_result) INTO @plan
                @step_index := 0
            OTHERWISE
                @results := @results + @step_result
                @step_index := @step_index + 1
        END
    END

    -- Phase 3: Implementation Loop
    GENERATE outline_files(@task, @results) INTO @file_outline
    WHILE @file_index < @file_count DO           -- (4) The Implementation Loop
        GENERATE generate_file(...) INTO @file_code
        CALL write_code_files(@file_code, @output_dir) -- (5) Deterministic I/O
        @file_index := @file_index + 1
    END
END
```

3.3.3.1 (1) Phase 1: Planning (The Blueprint)

We start by asking a "Senior Architect" persona to break the task down. The output is a simple numbered list. This list is our "Schedule" for the rest of the workflow.

3.3.3.2 (2) Phase 2: Design Loop (Lightweight Reasoning)

Crucial Lesson: We do *not* generate code inside this first loop. Instead, we generate "Design Notes"—short descriptions of what each step should achieve. - Why? Because LLM context windows are finite. If we generated 500 lines of code in Step 1 and passed it all into Step 2, by Step 5 the model would be "drowned" in its own previous output. - By keeping the loop "lightweight," we ensure the model stays focused on the design intent.

3.3.3.3 (3) Automatic Re-planning

If a step fails (e.g., the model realizes a design choice is impossible), we use the EVALUATE block to trigger a replan. We feed the failure details back into the planner and get a revised list of steps. This is the "Self-Healing" capability of SPL 2.0.

3.3.3.4 (4) Phase 3: Implementation Loop (Heavy Lifting)

Only after the entire design is finalized do we start a second loop for the "Heavy Lifting." In this loop, we generate one file at a time. - One LLM call = One file. - This ensures that each file gets the model's full attention and output token budget.

3.3.3.5 (5) Deterministic I/O (`CALL write_code_files`)

We use a Python tool to write the generated code to disk immediately. We don't wait for the whole workflow to finish. This means that even if the workflow crashes on the last file, your previous work is safe on disk.

3.3.4 Running It

This is a "heavy" recipe. It's best to use a strong model like **qwen2.5-coder** or **Claude 3.5 Sonnet**:

```
spl run cookbook/12_plan_and_execute/plan_execute.spl \
    --adapter ollama -m qwen2.5-coder \
    --tools cookbook/12_plan_and_execute/tools.py \
    task="Build a REST API for a todo app" \
    output_dir="./todo_app"
```

Expected result: A new directory ./todo_app containing main.py, models.py, requirements.txt, and a README.md.

3.3.5 What Just Happened

LLM calls: ~15-20. (Planning, design steps, file generation, summary)

The "Conductor" (SPL Runtime) managed a complex production pipeline: 1. **Architected** the solution. 2. **Validated** each design decision. 3. **Recovered** from mid-process failures via re-planning. 4. **Serialized** the final product into individual files on disk.

3.3.6 Reproducibility Note

On a **GTX 1080 Ti**, this recipe can take **3–5 minutes** to complete. Most of the time is spent in Phase 3 (generating the actual code).

The output is a **First Draft**. While the structure and logic are usually sound, you should always review the generated files for consistency (e.g., ensuring main.py uses the same variable names as models.py).

3.3.7 When to Use This Pattern

Use the **Plan and Execute** pattern when: - **Multi-File Projects**: Generating any application that requires more than one source file. - **Complex Logic**: Tasks where the "how" is not immediately obvious and needs to be decomposed. - **Reliability over Speed**: When you are willing to wait a few minutes for a high-quality, structured result rather than a fast, messy one.

3.3.8 Exercises

1. **Add a "Test" Step.** Modify the workflow to include a step at the end that generates a test_main.py file based on the implementation in main.py.
2. **Global Consistency Check.** Add a step after Phase 2 that asks a "Reviewer" model to look at *all* the design notes at once and check for contradictions before Phase 3 begins.
3. **Language Swap.** Try running the task for different languages (e.g., task="Build a todo app in Go" or task="Write a data processing script in Rust"). Observe how the "Architect" persona adapts the steps.

Chapter 3.4

Reflection Agent

3.4.1 The Pattern

When you ask an LLM a complex question, its first response is often "directionally correct" but flawed in the details. It might overlook a corner case, make a subtle logical error, or provide a generic answer when a specific one was needed. If you simply accept the first output, you are settling for the model's "System 1" thinking—fast, intuitive, but prone to error.

Reflection is the process of forcing the model to look at its own work and find its own mistakes. It is a "System 2" pattern: slow, deliberate, and meta-cognitive. Instead of just answering the problem, the model performs a three-step dance: 1. **Solve**: Produce an initial answer. 2. **Reflect**: Analyze that answer for gaps, errors, or weak reasoning. 3. **Correct**: Revise the answer based on the reflection.

The SQL analogy is a **Self-Correction Loop** or a **Recursive Validation**. You are performing a "sanity check" on a temporary result set before committing it to the final output.

The Reflection Agent recipe (section 2.4) implements this loop using a WHILE block. It doesn't just refine the text; it calculates a "Confidence Score." If the score is too low, it iterates. If the score is high, it stops. This ensures that the workflow is only as complex as the problem requires.

3.4.2 The SPL Approach

This recipe demonstrates the use of **Meta-Cognitive Branching**—using the model's own self-assessment to decide whether to continue the loop.

3.4.3 The .spl File (Annotated)

```
-- Recipe 16: Reflection Agent
-- Solve a problem, then reflect and correct until confident.

WORKFLOW reflection_agent
    INPUT: @problem TEXT
DO
    @iteration := 0
```

```
    @max_reflections := 3
    @confidence := 0

    -- Phase 1: Initial Attempt
    GENERATE solve(@problem) INTO @answer       -- (1) The "System 1" response

    -- Phase 2: The Meta-Cognitive Loop
    WHILE @iteration < @max_reflections DO
        -- Reflect: find errors or gaps
        GENERATE reflect(@problem, @answer) INTO @reflection -- (2) The "Mirror" st

        -- Score confidence (LLM-as-Judge of its own work)
        GENERATE confidence_score(@answer, @reflection) INTO @confidence -- (3) Qu

        EVALUATE @confidence
            WHEN > 0.85 THEN                    -- (4) The Confidence Threshold
                COMMIT @answer WITH status = 'confident'
            OTHERWISE
                -- Extract issues and correct
                GENERATE extract_issues(@reflection) INTO @issues
                GENERATE correct(@answer, @issues, @problem) INTO @answer
                @iteration := @iteration + 1
        END
    END

    -- Final Commit (Best Effort)
    COMMIT @answer WITH status = 'best_effort'
END
```

3.4.3.1 (1) The Initial Solve

The first GENERATE call is the baseline. We ask the model to solve the problem as it normally would. This result is stored in @answer.

3.4.3.2 (2) The "Mirror" Step (`reflect`)

Instead of asking for a better answer immediately, we ask the model to be its own "Critical Editor." - "What did you miss?" - "Are there any logical contradictions?" - "Is the reasoning sound?"

This step produces a "Reflection"—a document that describes what is wrong with the current answer.

3.4.3.3 (3) Quantifying Doubt (`confidence_score`)

We ask the model (or a second, larger "Judge" model) to look at the @answer and the @reflection and produce a numeric score from 0 to 1. This turns a subjective feeling into a hard metric that the WORKFLOW can use for branching.

3.4.3.4 (4) The Confidence Threshold

Using the EVALUATE block, we set a high bar (0.85). - If the model is confident, we COMMIT and exit early. - If not, we use the @reflection to trigger a correct step and try again.

SQL Analogy: **Conditional COMMIT**. You only finalize the transaction if the data passes a series of semantic checks.

3.4.4 Running It

Run the reflection agent on a complex design problem:

```
spl run cookbook/16_reflection/reflection.spl --adapter ollama \
    problem="Design a URL shortener system that handles 1M requests per second"
```

In the execution trace, you will likely see at least one iteration where the model identifies a scaling bottleneck (like a database write limit) and corrects its initial design to include a caching layer or a load balancer.

3.4.5 What Just Happened

LLM calls: 3 + (4 * iterations).

The "Conductor" (SPL Runtime) managed a "Self-Correcting Brain": 1. **Proposed** a solution. 2. **Identified** flaws in that proposal. 3. **Measured** the severity of those flaws. 4. **Revised** the proposal until the measured risk was below the threshold.

This is how you get "Senior Engineer" level output from "Junior" level models. By forcing the model to slow down and check its work, you unlock its full reasoning potential.

3.4.6 Reproducibility Note

Reflection is highly effective but adds **Latency**. On a **GTX 1080 Ti**, a full 3-iteration loop can take **60–120 seconds**.

However, the **Stability** is much higher than a single-pass prompt. Because the model has multiple chances to catch and fix its own errors, the final output is far more likely to be correct and complete across different runs.

3.4.7 When to Use This Pattern

Use the **Reflection Agent** pattern when: - **Accuracy is the Priority**: Technical designs, logical proofs, or complex explanations. - **Variable Problem Complexity**: Some problems are easy and don't need reflection; others are hard and need several passes. The confidence score handles both cases automatically. - **Small Model Boosting**: If you are using a 7B or 14B parameter model, reflection can help it perform at the level of a 70B model.

3.4.8 Exercises

1. **Strict Judge.** Use the USING MODEL clause (Chapter 9.1) to ensure the confidence_score step always uses your strongest model (e.g., gemma3), even if phi4 is doing the reflection.
2. **Add a "Divergent" step.** In the reflection pass, ask the model to specifically "Think about an alternative perspective" to prevent it from just reinforcing its own biases.
3. **Manual Stop.** Add a step inside the loop that asks for human feedback before the next reflection pass begins.

Chapter 3.5

Multi-Model Pipeline

3.5.1 The Pattern

In a world of thousands of models, the most common mistake is using a "Titan" (like GPT-4o or Claude 3.5 Sonnet) for every single step. While these models are brilliant, they are also slower and more expensive. For many sub-tasks—like basic research, simple summarization, or formatting—a "Junior" model (like Phi-4 or Llama 3.2) can do the job just as well, at a fraction of the cost.

The **Multi-Model Pipeline** pattern is the practice of routing different steps of a workflow to different models based on the task's requirements. - Use a **Fast Model** for data retrieval. - Use a **Reasoning Model** for deep analysis. - Use a **Creative Model** for final prose.

The SQL analogy is **Polyglot Persistence** or **Federated Queries**. You don't store everything in one massive database. You put your structured data in PostgreSQL, your logs in Elasticsearch, and your cache in Redis. When you run a "query" (the workflow), the system fetches data from the most efficient source for that specific piece of information.

The Multi-Model Pipeline recipe (section 2.5) demonstrates this "Best-of-Breed" orchestration. It routes research, analysis, and writing to the models most suited for those roles, then wraps them in a quality-control loop.

3.5.2 The SPL Approach

This recipe introduces the USING MODEL clause at the step level, allowing for granular control over the "orchestra."

3.5.3 The .spl File (Annotated)

```
-- Recipe 21: Multi-Model Pipeline
-- Each step targets the model best suited for that task.

WORKFLOW multi_model_pipeline
  INPUT: @topic TEXT
DO
  -- Step 1: Research (Needs factual retrieval)
```

```
  GENERATE research(@topic)
    USING MODEL 'gemma3' INTO @facts;          -- (1) Fast, retrieval-tuned model

  -- Step 2: Analysis (Needs deep reasoning)
  GENERATE analyze(@facts)
    USING MODEL 'claude-3-5-sonnet' INTO @analysis; -- (2) High-intelligence model

  -- Step 3: Writing (Needs creative style)
  GENERATE write_summary(@analysis)
    USING MODEL 'llama3.2' INTO @draft;        -- (3) Creative, lightweight model

  -- Step 4: Quality loop
  WHILE @iteration < 3 DO
    GENERATE quality_check(@draft)
      USING MODEL 'gemma3' INTO @quality;      -- (4) Neutral judge model

    EVALUATE @quality
      WHEN > 0.7 THEN COMMIT @draft
      OTHERWISE
        GENERATE write_summary(@analysis) INTO @draft
        @iteration := @iteration + 1
    END
  END
END
```

3.5.3.1 (1) Step 1: Fast Retrieval

For the research phase, we want speed and factual breadth. We target **gemma3** (or a similar local model). This model is excellent at scanning its training data and producing a bulleted list of facts without the latency of a massive cloud model.

3.5.3.2 (2) Step 2: Deep Reasoning

For the analysis, we need the "Gold Standard." We use USING MODEL 'claude-3-5-sonnet' (via the OpenRouter or Claude adapter). This model's ability to identify subtle trends and risks is superior to smaller models. We are spending our "token budget" where it matters most.

3.5.3.3 (3) Step 3: Creative Writing

For the final summary, we might want a specific tone. **Llama 3.2** is known for being conversational and engaging. By switching to a smaller model here, we get the response faster and often with less "corporate" phrasing than the larger frontier models.

3.5.3.4 (4) Step 4: The Neutral Judge

Finally, we use a different model (**gemma3** again) to perform the quality check. Using a different model for the judge step prevents "Confirmation Bias"—where a model is too lenient on its own work.

SQL Analogy: **Cross-Engine Join**. You are joining the output of Model A with the judgment of Model B to produce a refined final result.

3.5.4 Running It

This recipe highlights the power of adapters. Ensure you have your keys configured for cloud models if you choose to use them:

```
spl run cookbook/21_multi_model_pipeline/multi_model.spl --adapter openrouter \
    topic="The impact of AI on rural education"
```

In the execution trace, you will see the runtime switching between adapters and model names for each GENERATE step.

3.5.5 What Just Happened

LLM calls: 4 + (2 * iterations).

The "Conductor" (SPL Runtime) orchestrated a specialized team: 1. **Commissioned** research from a fast specialist. 2. **Requested** analysis from a high-intelligence consultant. 3. **Hired** a creative writer for the final draft. 4. **Audited** the work using an independent third party.

This is the peak of agentic design: not just using AI, but using the *right* AI for every single turn.

3.5.6 Reproducibility Note

The Multi-Model pattern is the most **Cost-Efficient** way to achieve high quality. By routing 80% of the tokens to smaller, cheaper models and reserving the remaining 20% for the "frontier" models, you can achieve "GPT-4 level" results at "GPT-3.5 level" costs.

On a **GTX 1080 Ti**, the local steps (Research, Judge) run in **2–5 seconds**, while the cloud steps (Analysis) depend on your internet connection and the provider's latency.

3.5.7 When to Use This Pattern

Use the **Multi-Model Pipeline** pattern when: - **Cost/Performance Balance**: You need high quality but want to keep API costs low. - **Latency Optimization**: You want the user to see the "Researching…" phase happen instantly while the deep thinking happens in the background. - **Ensemble Benchmarking**: When you want to see how different models "view" each other's work.

3.5.8 Exercises

1. **Local-Only Mode.** Modify the recipe to use only local models (e.g., phi4 for research, gemma3 for analysis, mistral for writing). Compare the quality to the cloud-mixed version.
2. **Add a "Model Switcher."** (Advanced) Use an EVALUATE block to check the complexity of the @topic. If it's "simple," use a fast model for all steps. If it's "complex," switch to the full multi-model pipeline.

3. **Parallel Multi-Model.** Use a WITH clause to ask two different models for the Research phase and then have the Analyst merge their findings.

Chapter 3.6

Nested Procedures

3.6.1 The Pattern

As your AI applications grow, your .spl files can become long and difficult to read. If you have a workflow that researches a topic, explains it, generates examples, and checks reading level, putting all those steps in one giant WORKFLOW block is like writing a 1,000-line main() function in C. It's hard to test, hard to reuse, and hard to debug.

The **Nested Procedures** pattern is the core of modular SPL development. It allows you to break a complex task into small, independent PROCEDURE units that can call each other. This is the foundation of **Deep Composability**.

The SQL analogy is **Nested Views** or **Modular Stored Procedures**. You don't write one massive query to generate a financial report. You write a view for v_revenue, another for v_expenses, and then a third view that joins them. Each piece is independently verified and perfectly encapsulated.

The Nested Procedures recipe (section 2.6) demonstrates this hierarchy. It builds a "Layered Explainer" where high-level orchestration is separated from low-level generation tasks like complexity calibration and example making.

3.6.2 The SPL Approach

This recipe introduces the pattern of **Procedural Delegation**—passing control from a high-level workflow to specialized sub-procedures.

3.6.3 The .spl File (Annotated)

```
-- Recipe 25: Nested Procedures
-- Demonstrates PROCEDURE calling PROCEDURE -- deep composability.

PROCEDURE explain_layer(content TEXT, audience TEXT) -- (1) Leaf Procedure
RETURNS TEXT
DO
    GENERATE explain(content, audience) INTO @explanation
```

```
    COMMIT @explanation
END

PROCEDURE calibrate_complexity(text TEXT, audience TEXT) -- (2) Intermediate Proce
RETURNS TEXT
DO
    GENERATE assess_reading_level(text) INTO @level
    EVALUATE @level
        WHEN > 8 THEN
            GENERATE simplify(text, audience) INTO @calibrated
            COMMIT @calibrated
        OTHERWISE
            COMMIT text
    END
END

WORKFLOW layered_explainer                          -- (3) Root Workflow
    INPUT: @topic TEXT, @audience TEXT
DO
    GENERATE research_overview(@topic) INTO @overview

    -- Calling the specialized procedures
    CALL explain_layer(@overview, @audience) INTO @base_exp -- (4) Delegation

    CALL calibrate_complexity(@base_exp, @audience) INTO @final_exp

    COMMIT @final_exp WITH status = 'complete'
END
```

3.6.3.1 (1) The "Leaf" Procedure

explain_layer is a simple wrapper around a GENERATE call. Why bother making it a procedure? Because it encapsulates the "Explanation Persona." If you want to change how the model explains things (e.g., adding "avoid jargon" to the system prompt), you change it in this one procedure, and every workflow that uses it is automatically updated.

3.6.3.2 (2) The "Intermediate" Procedure

calibrate_complexity is more complex. It contains its own EVALUATE logic and potentially its own nested calls. By making this a procedure, we can test the "Simplification Logic" in isolation without running the whole research workflow.

3.6.3.3 (3) The Root Workflow

The layered_explainer acts as the "Conductor of Conductors." It doesn't worry about how to simplify text or how to explain a concept. It only worries about the high-level sequence: **Research → Explain → Calibrate**.

SQL Analogy: **Orchestration Query**. You are joining the outputs of multiple modular functions to produce a high-level report.

3.6.3.4 (4) Delegation (CALL)

The CALL statement is the bridge. In SPL 2.0, CALL is used for both Python tools (section 2.2) and other SPL procedures. This unified interface means you can swap a generative procedure for a deterministic tool without changing the calling code.

3.6.4 Running It

Run the layered explainer for different audiences:

```
# To high schoolers
spl run cookbook/25_nested_procs/nested_procs.spl --adapter ollama \
    topic="quantum computing" audience="high school students"

# To policy makers
spl run cookbook/25_nested_procs/nested_procs.spl --adapter ollama \
    topic="CRISPR gene editing" audience="policy makers"
```

In the execution trace, you will see the "Call Stack" growing and shrinking as the runtime enters and exits each procedure.

3.6.5 What Just Happened

LLM calls: ~5-7. (Depending on the complexity of the research and calibration steps)

The "Conductor" (SPL Runtime) managed a "Recursive Hierarchy": 1. **Orchestrated** the high-level business logic. 2. **Delegated** the domain-specific tasks to specialized "Agents" (Procedures). 3. **Maintained** state across the hierarchy (passing @base_exp from one procedure to another). 4. **Enforced** encapsulation (the root workflow never saw the intermediate @level variable).

3.6.6 Reproducibility Note

Nested procedures are the key to **Scalable Maintenance**. When you find a bug in your "Simplification" logic, you fix it in calibrate_complexity.spl. You don't have to hunt through 20 different workflow files to find every place you used a similar prompt.

On a **GTX 1080 Ti**, the overhead of the "Procedure Call" is **zero**. The runtime treats nested procedures as logical blocks within the same execution context. The only latency comes from the underlying LLM calls.

3.6.7 When to Use This Pattern

Use the **Nested Procedures** pattern when: - **Large Projects**: Any .spl file that exceeds 100 lines should probably be broken into procedures. - **Shared Logic**: When you have a task (like "PII Redaction" or "Tone Check") that is used by multiple different workflows. - **Unit Testing**:

When you want to verify that a specific part of your logic works correctly before integrating it into a complex agent.

3.6.8 Exercises

1. **Add a "Quiz" Procedure.** Create a third procedure called make_quiz that takes the @final_exp and generates 3 multiple-choice questions. Call it at the end of the root workflow.
2. **Shared Library.** Move explain_layer into a separate file (library.spl) and research how to use the IMPORT or --include flags to use it in your main workflow.
3. **Conditional Nesting.** Add a parameter @skip_calibration to the root workflow. Use an EVALUATE block to only CALL calibrate_complexity if the parameter is false.

Part 4

Reasoning

Chapter 4.1

Chain of Thought

4.1.1 The Pattern

You have a topic — "distributed AI inference," say — and you want not just a surface summary but a piece of writing that demonstrates actual reasoning. The naive Python approach is a single requests.post to the LLM with an instruction like "think carefully and then summarise." Sometimes it works. More often you get a paragraph that opens confidently, wanders, and concludes with something vague. The model has to do the research thinking, the analytical thinking, and the synthesis all in one forward pass, which is a lot to ask. Debugging it is worse: when the output is thin, you cannot tell whether the model failed to surface facts, failed to analyse them, or failed to compress the analysis. The whole thing is a black box.

The sharper practitioners will try prompt engineering — "first list what you know, then analyse, then summarise" — inside a single GENERATE call. This helps, but you still cannot inspect intermediate results, you cannot swap a better research prompt later without re-testing the whole pipeline, and you lose the ability to halt early if step one produces garbage. You are also paying for tokens in steps two and three that are built on a shaky foundation you cannot see.

The recurring pain is that reasoning is not a single act. It is a sequence of acts, each contingent on the one before. Compressing that sequence into one LLM call trades controllability for convenience, and in practice the convenience is illusory — you end up debugging by staring at final output and guessing.

4.1.2 The SPL Approach

Model the reasoning stages explicitly as a linear chain of GENERATE steps, threading each output as context for the next, so that Research feeds Analysis and Analysis feeds Synthesis — a CTE chain written for thought instead of tables.

4.1.3 The .spl File (Annotated)

```
-- Recipe 09: Chain of Thought
-- Multi-step reasoning: Research → Analyze → Summarize
-- Each GENERATE feeds into the next via workflow variables.
```

```
--
-- Usage:
--   spl run cookbook/09_chain_of_thought/chain.spl \
--       --adapter ollama -m gemma3 \
--       topic="distributed AI inference"

WORKFLOW chain_of_thought
    INPUT:  @topic TEXT          -- The subject to reason about; caller decides, not
    OUTPUT: @summary TEXT        -- Final synthesised output; the only thing the call
DO
    -- Step 1: Research
    -- The model's job here is narrow: surface what is known about @topic.
    -- Analogous to: WITH research AS (SELECT facts FROM knowledge WHERE subject =
    GENERATE research(@topic) INTO @research

    -- Step 2: Analyze
    -- @research is now in scope. The model cannot "forget" what was surfaced;
    -- it must reason from it. Narrow job: find patterns, tensions, implications.
    -- Analogous to: WITH analysis AS (SELECT interpret(facts) FROM research)
    GENERATE analyze(@research) INTO @analysis

    -- Step 3: Summarize
    -- The model works from @analysis, not from @topic directly.
    -- This is the key difference from a single-prompt approach: the summary
    -- is grounded in the analysis, which is grounded in the research.
    -- Analogous to: SELECT synthesize(interpretation) FROM analysis
    GENERATE summarize(@analysis) INTO @summary

    -- Commit the final variable. status metadata travels with the result.
    COMMIT @summary WITH status = 'complete'
END
```

On the SQL analogy. In SQL you write:

```
WITH research  AS (SELECT facts         FROM knowledge WHERE subject = :topic),
     analysis  AS (SELECT interpret(f) FROM research),
     summary   AS (SELECT compress(i)  FROM analysis)
SELECT * FROM summary;
```

Each CTE can only see what the previous one produced. There is no way for summary to reach back past analysis and pull raw facts — the chain forces discipline. The SPL chain works identically: @summary is computed from @analysis, which was computed from @research. You cannot accidentally skip a step.

Why three variables instead of one? Because each is individually inspectable. After a run you can log @research, @analysis, and @summary separately. If the summary is thin, you check the analysis first. If the analysis is thin, you check the research. The isolation of failure modes is the point.

On INPUT design. @topic is the only input. The workflow does not hard-code "distributed AI

inference" or set a default — the caller decides what to reason about. This respects a core SPL principle: a workflow specifies the shape of the computation, not the subject matter.

On GENERATE prompt resolution. Each GENERATE call (research, analyze, summarize) corresponds to a prompt template that the runtime resolves. In the absence of explicit CREATE FUNCTION definitions in this file, the runtime uses the step name and arguments to construct the system and user prompts. The practical implication: you can override any one prompt — say, swapping analyze for a domain-specific analysis prompt — without touching the others.

On COMMIT. COMMIT @summary WITH status = 'complete' writes the result to the output store with a status label. The workflow has no EXCEPTION block. This is deliberate for a recipe at this stage of complexity: the chain of thought pattern is the mechanism; safety wrapping is a separate concern handled in Part 4.

4.1.4 Running It

```
spl run cookbook/09_chain_of_thought/chain.spl \
    --adapter ollama -m gemma3 \
    topic="distributed AI inference"
```

Expected output (truncated — your model's phrasing will differ):

```
[chain_of_thought] Running on topic: "distributed AI inference"

[research] Generating...
  Distributed AI inference refers to the practice of spreading model
  execution across multiple nodes or devices rather than centralising
  computation on a single server. Key considerations include network
  latency between nodes, model partitioning strategies (tensor parallel,
  pipeline parallel), memory bandwidth constraints at edge devices...
  [~340 tokens]

[analyze] Generating...
  Three tensions emerge from the research. First, latency vs. throughput:
  distributing a single request across nodes adds coordination overhead
  that can exceed the gains from parallelism unless batch sizes are large.
  Second, model size vs. edge capability: the economics favour distribution
  only when individual nodes cannot host the full model...
  [~280 tokens]

[summarize] Generating...
  Distributed AI inference becomes economically and technically compelling
  when model size exceeds single-node memory, batching is feasible, and
  network topology is controlled. For latency-sensitive single-request
  workloads, a well-provisioned central server typically wins...
  [~180 tokens]

status: complete
```

4.1.5 What Just Happened

Three LLM calls were made, in strict sequence. No LLM call was made for anything other than reasoning — there is no CALL in this workflow, because there is no deterministic tool work to do.

Step 1 (research): The model received @topic and was asked to surface relevant knowledge. The output went into @research — a variable that exists only inside this workflow run.

Step 2 (analyze): The model received @research. It had no access to @topic directly (unless the prompt template passes it through, which in this recipe it does not). The model's task was narrowed to interpretation of the material already surfaced.

Step 3 (summarize): The model received @analysis. Its task was compression and synthesis. By this point, the chain has done the work of separating "what do we know" from "what does it mean" from "what is the bottom line" — three jobs that, when conflated, produce the bland summary we were trying to avoid.

COMMIT: Writes @summary to the output store with status = 'complete'. The caller receives @summary as the workflow's declared OUTPUT.

Token budget awareness. Three GENERATE calls means three round-trips to the model. For a local model on a GTX 1080 Ti this is noticeable — roughly 12–20 seconds total depending on output length. For a production API it is three billing events. This is the cost of the pattern. It is worth it when the quality gain matters; for a quick lookup it is not.

4.1.6 Reproducibility Note

Tested on a GTX 1080 Ti with gemma3 (4-bit quantised) via Ollama. End-to-end latency for topic="distributed AI inference": approximately 14 seconds. For topic="quantum computing" with llama3.2: approximately 18 seconds.

Output stability: the structure of the chain (research → analysis → summary) is stable across runs. The specific phrasing of each step varies, as expected from generative models. The summary at step three tends to be more focused and grounded than a single-prompt alternative because it inherits the structure imposed by step two.

If you need bit-for-bit reproducibility, set temperature=0 on all three GENERATE calls. You will lose some expressiveness but gain determinism within a given model version. Note that model updates will still change output even at temperature zero.

One observed failure mode: if @topic is extremely broad ("everything about AI"), the research step produces a sprawling dump that overwhelms the analysis step, which then produces a similarly sprawling analysis, and the summary step cannot compress it coherently. Keep topics scoped. The workflow does not enforce this — that is the caller's responsibility.

4.1.7 When to Use This Pattern

Use chain of thought when:

- The task has identifiable reasoning stages that benefit from isolation (research / analysis / synthesis; hypothesis / evidence / conclusion; diagnosis / differential / recommendation).
- You want to inspect intermediate reasoning, not just final output.

- You expect to iterate on individual stages — for example, replacing the analysis prompt with a domain-specific one — without re-engineering the whole pipeline.
- Output quality is more important than token economy.

Do not use chain of thought when:

- A single, well-crafted prompt reliably produces the quality you need. Complexity for its own sake is expensive.
- Latency is the binding constraint. Three sequential LLM calls compound latency. If you need a fast answer and quality is acceptable from one pass, use one pass.
- The stages are not genuinely sequential. If step two does not depend on step one's output, they should run in parallel (see Chapter 4.2, Tree of Thought, which explores parallel path generation before convergence).

Comparison with alternatives:

Approach	Quality	Latency	Debuggability
Single GENERATE	Variable	Low	Poor
Chain of Thought (this recipe)	Higher	Medium	Good
Tree of Thought (section 3.2)	Highest	High	Good

The chain of thought pattern is the minimum viable structure for tasks where reasoning quality matters. Tree of thought adds parallel exploration when you are uncertain which reasoning path is best. Chain of thought assumes you know the right sequence of reasoning acts; tree of thought does not.

4.1.8 Exercises

1. **Add a fourth step.** After summarize, add GENERATE critique(@summary, @topic) INTO @critique that asks the model to identify the weakest claim in the summary. Then COMMIT @critique WITH status = Run both the original and the extended version; compare whether the critique surfaces anything the single-prompt approach would have missed.

2. **Swap the analysis prompt.** The analyze step uses the default prompt template. Create a CREATE FUNCTION analysis_prompt() RETURNS TEXT that specialises the analysis for a domain of your choice (security, economics, biology). Pass it to GENERATE analyze(@research, analysis_prompt()) INTO @analysis. Observe how specialising one stage changes the downstream summary without touching any other step.

3. **Fail gracefully.** Wrap the workflow in an EXCEPTION block. Add WHEN GenerationError THEN COMM — so that if the analysis or summary step fails, the caller still receives the research output rather than nothing. This is the starting point for understanding the safety patterns in Part 4.

Chapter 4.2

Tree of Thought

4.2.1 The Pattern

When humans solve a truly hard problem, we don't just pick one idea and run with it. We brainstorm. We explore three or four different approaches, follow each one a few steps down the road, see which one runs into a dead end, and then double down on the most promising path. This is the **Tree of Thought (ToT)** pattern.

The problem with standard LLM prompting is that it is "linear." The model commits to a direction with its very first token and is then forced to stay on that path, even if it realizes halfway through that it was a mistake.

The SQL analogy is a **Recursive CTE with Branching** or a **Multi-Path Union**. You are generating multiple potential "result sets" (thought paths) in parallel and then using a filter (an evaluation step) to pick the best one.

The Tree of Thought recipe (section 3.2) implements this "Branch and Prune" strategy. It explores three independent approaches to a problem, develops each one into a more detailed plan, scores them, and then selects the winner for final refinement.

4.2.2 The SPL Approach

This recipe demonstrates **Parallel Exploration**—generating multiple independent states and then synthesizing them into a single conclusion.

4.2.3 The .spl File (Annotated)

```
-- Recipe 17: Tree of Thought
-- Explores multiple reasoning paths in parallel, then selects the best.

WORKFLOW tree_of_thought
    INPUT: @problem TEXT
DO
    -- Phase 1: Branching (The Brainstorm)
    GENERATE approach_1(@problem) INTO @path_a -- (1) Three independent seeds
```

```
    GENERATE approach_2(@problem) INTO @path_b
    GENERATE approach_3(@problem) INTO @path_c

    -- Phase 2: Developing (Following the map)
    GENERATE develop(@path_a, @problem) INTO @path_a_dev -- (2) Elaborating each
    GENERATE develop(@path_b, @problem) INTO @path_b_dev
    GENERATE develop(@path_c, @problem) INTO @path_c_dev

    -- Phase 3: Scoring (The Pruning)
    GENERATE evaluate_path(@path_a_dev) INTO @score_a -- (3) Objective evaluation
    GENERATE evaluate_path(@path_b_dev) INTO @score_b
    GENERATE evaluate_path(@path_c_dev) INTO @score_c

    -- Phase 4: Selection and Refinement
    GENERATE select_best(@path_a_dev, @score_a, ...) INTO @best_path
    GENERATE refine_solution(@best_path) INTO @best_solution

    -- Phase 5: Final Verification
    GENERATE verify(@best_solution) INTO @verification
    EVALUATE @verification                      -- (4) The Final Check
        WHEN 'sound' THEN
            COMMIT @best_solution WITH status = 'complete'
        OTHERWISE
            GENERATE synthesize_all(...) INTO @best_solution
            COMMIT @best_solution WITH status = 'synthesized'
    END
END
```

4.2.3.1 (1) Phase 1: Branching

We ask the model to generate three distinct approaches to the same problem. For example, if the problem is "How to scale a database," Path A might be "Horizontal Sharding," Path B might be "Read Replicas," and Path C might be "Caching Layer."

4.2.3.2 (2) Phase 2: Developing

We take each seed and ask the model to "develop" it further—writing out the pros, cons, and implementation steps for each specific approach. This is where the "tree" grows its branches.

4.2.3.3 (3) Phase 3: Scoring

We use a "Judge" prompt to score each developed path on a set of criteria (e.g., feasibility, cost, scalability). This is the "Pruning" step. We are identifying which branches are worth keeping and which are dead ends.

4.2.3.4 (4) The Final Check

Even after we've picked a "winner" and refined it, we perform one last verification. If the final solution is still not "sound," we fall back to a **Synthesis** step—taking the best ideas from *all three*

paths and merging them. This ensures we don't lose the "good parts" of the losing paths.

4.2.4 Running It

Run the Tree of Thought on a strategic decision:

```
spl run cookbook/17_tree_of_thought/tree_of_thought.spl --adapter ollama \
    problem="Should we rewrite our legacy monolith in Rust or Go?"
```

In the output, you will see the model exploring both languages (and perhaps a third option like "Refactor in place"), scoring them based on performance and developer velocity, and then providing a nuanced recommendation.

4.2.5 What Just Happened

LLM calls: 12-14.

The "Conductor" (SPL Runtime) managed a "Tournament of Ideas": 1. **Fanned out** the initial brainstorming. 2. **Expanded** each idea into a concrete plan. 3. **Measured** each plan against a set of standards. 4. **Selected** the strongest candidate. 5. **Refined** the candidate into a production-ready solution. 6. **Verified** the result, with a fallback for complex failures.

4.2.6 Reproducibility Note

Tree of Thought is the most computationally expensive reasoning pattern. On a **GTX 1080 Ti**, a full 12-call ToT can take **2–4 minutes**.

However, the **Robustness** is unparalleled. By exploring multiple paths, you significantly reduce the risk of the model "hallucinating" a single bad solution and sticking to it. This pattern is designed for high-stakes decision-making where the cost of a wrong answer is much higher than the cost of the extra tokens.

4.2.7 When to Use This Pattern

Use the **Tree of Thought** pattern when: - **Strategic Decisions**: Architectural choices, business strategies, or complex product trade-offs. - **Creative Brainstorming**: When you need a wide variety of high-quality ideas rather than just one. - **Problem Solving with High Uncertainty**: When the "right" answer is not obvious and needs to be discovered through exploration.

4.2.8 Exercises

1. **Parallelize the Branches.** Use the WITH clause (Chapter 9.1) to run Phase 1, 2, and 3 in parallel blocks. This will reduce the latency by nearly 60% if your hardware can handle it.
2. **Add a "Devil's Advocate."** In Phase 2, instead of just "developing" the path, ask the model to specifically "Find the three biggest reasons why this approach will fail."
3. **Variable Branching.** Modify the input to accept @branch_count and use a WHILE loop (Chapter 7.1) to generate an arbitrary number of paths instead of exactly three.

Chapter 4.3

Hypothesis Tester

4.3.1 The Pattern

In business and science, we often make quick observations: "Users hate the new UI," or "The server is slow because of the database." These are not facts; they are **Hypotheses**. If you act on a hypothesis without testing it, you waste time and resources.

Structured Scientific Reasoning is the pattern of converting a "vibe" into a falsifiable claim. 1. **Formulate**: Define the Null Hypothesis (H0: it's chance) and the Alternative Hypothesis (H1: the claim). 2. **Plan**: Design a test that could prove you wrong (Falsifiability). 3. **Audit**: Look for confounding variables (other things that could cause the effect). 4. **Evaluate**: Score the available evidence and draw a conclusion based on a confidence threshold.

The SQL analogy is **Hypothetical Querying** or **Impact Analysis**. You are running a "What-If" scenario against your data, calculating the "Confidence Interval" of your findings, and only committing to a business decision if the results are statistically significant.

The Hypothesis Tester recipe (section 3.3) implements this rigorous loop. It forces the model to think like a scientist—identifying variables and seeking evidence that *refutes* its own initial guess before reaching a final verdict.

4.3.2 The SPL Approach

This recipe introduces **Evidence Weighting**—using a structured JSON schema to collect pros and cons and then using a numeric confidence score to drive the workflow's conclusion.

4.3.3 The .spl File (Annotated)

```
-- Recipe 35: Hypothesis Tester
-- Generate hypothesis → design test → evaluate evidence → conclude.

CREATE FUNCTION hypothesis_framework()
RETURNS TEXT AS $$
Format:
- Null hypothesis (H0): The effect does not exist.
```

```
- Alternative hypothesis (H1): The proposed claim.
- Falsifiability: How would we prove H1 wrong?
$$;

WORKFLOW hypothesis_tester
    INPUT: @observation TEXT, @threshold FLOAT DEFAULT 0.7
DO
    -- Phase 1: Formulation
    GENERATE formulate_hypotheses(@observation, hypothesis_framework()) INTO @hypo

    -- Phase 2: Experimental Design
    GENERATE design_test(@hypo) INTO @test_plan

    -- Phase 3: Evidence Evaluation
    GENERATE evaluate_evidence(@observation, @hypo, evidence_schema()) INTO @evider

    -- Phase 4: Decision Logic
    GENERATE extract_confidence(@evidence_json) INTO @confidence

    EVALUATE @confidence
        WHEN >= @threshold THEN                                  -- (1) Significant R
            GENERATE write_conclusion(@hypo, @evidence_json, 'confident') INTO @fir
            COMMIT @final WITH verdict = 'supported'
        WHEN >= 0.4 THEN                                         -- (2) Inconclusive
            GENERATE write_conclusion(@hypo, @evidence_json, 'uncertain') INTO @fir
            COMMIT @final WITH verdict = 'needs_more_data'
        OTHERWISE                                                -- (3) Refuted
            GENERATE write_conclusion(@hypo, @evidence_json, 'refuted') INTO @final
            COMMIT @final WITH verdict = 'rejected'
    END
END
```

4.3.3.1 (1), (2) & (3) The Confidence Ladder

This is the most "Honest" pattern in the cookbook. Instead of always giving an answer, the workflow can admit ignorance. - If the evidence is strong (**@confidence >= 0.7**), it commits a confident conclusion. - If the evidence is mixed (**@confidence >= 0.4**), it tells the user "We need more data." - If the evidence is weak, it concludes that the observation was likely a fluke.

SQL Analogy: **Threshold Filtering**. You are filtering your "Insights" by a quality metric, ensuring that only high-confidence conclusions are promoted to the final report.

4.3.4 Running It

Test a common business observation:

```
spl run cookbook/35_hypothesis_tester/hypothesis.spl \
    --adapter ollama -m gemma3 \
```

```
observation="Remote teams show lower productivity in the first month after join
```

Expected output: A structured breakdown of the variables (e.g., "Time-to-onboard" vs "Work location"), a list of evidence for and against the claim, and a final verdict based on the calculated confidence.

4.3.5 What Just Happened

LLM calls: 5. (Formulate, Design, Evaluate, Extract, Conclude)

The "Conductor" (SPL Runtime) managed a "Peer Review Board": 1. **Drafted** a formal scientific claim. 2. **Designed** an experiment to test that claim. 3. **Audited** the claim against existing evidence. 4. **Measured** the strength of the findings. 5. **Rendered** a conservative, evidence-based judgment.

4.3.6 Reproducibility Note

Hypothesis Testing is a "Deep Reasoning" task. Small models often fall into "Confirmation Bias"—they only look for evidence that supports the observation. For this recipe, we strongly recommend using a "Senior" reasoning model like **gemma3 (27B)** or **Llama 3.1 (70B)** to ensure the adversarial thinking is genuine.

On a **GTX 1080 Ti**, this full reasoning chain takes **40–60 seconds**.

4.3.7 When to Use This Pattern

Use the **Hypothesis Tester** pattern when: - **Strategic Planning**: Testing "Gut Feelings" before making a pivot. - **Root Cause Analysis**: Investigating why a system failed or why a metric dropped. - **Academic/Scientific Research**: Automating the first pass of literature review and hypothesis formulation.

4.3.8 Exercises

1. **Add "Confounding Variables".** Add a specific step between Phase 1 and 2 that asks the model to "List three other things that could explain this observation besides H1."
2. **Multi-Model Review.** Use a different model for the evaluate_evidence step to provide a "blind" second opinion.
3. **Custom Threshold.** Run the recipe with @threshold = 0.9 and observe how much harder it is for the model to reach a "Confident" conclusion.

Part 5

Safety & Reliability

Chapter 5.1

Safe Generation

5.1.1 The Pattern

In the early days of LLMs, we lived in a "Fire and Forget" world. You sent a prompt, got a response, and hoped for the best. If the model hallucinated, refused to answer, or produced low-quality prose, your application simply passed that failure onto the user.

In a production environment, this is unacceptable. You need **Guardrails**. You need a way to catch errors *before* they reach the user and a way to handle the unique "exceptions" that only happen in generative AI—like context window overflows or safety refusals.

The SQL analogy is a **CONSTRAINT** or a **CHECK** clause. You don't just insert data into a table; you ensure it meets certain criteria (not null, unique, within range). If the criteria aren't met, the transaction fails or triggers a correction.

The Safe Generation recipe (section 4.1) implements this as a "Self-Correction Loop." It generates a response, uses an "LLM-as-Judge" to evaluate the quality, and then branches to different outcomes based on the verdict. It also demonstrates SPL 2.0's advanced EXCEPTION handling for generative risks.

5.1.2 The SPL Approach

This recipe introduces the EVALUATE block for semantic branching and the EXCEPTION block for handling non-deterministic errors.

5.1.3 The .spl File (Annotated)

```
-- Recipe 07: Safe Generation with Exception Handling
-- Demonstrates LLM-specific error handling patterns

WORKFLOW safe_generation
  INPUT: @prompt text
  OUTPUT: @result text
DO
  GENERATE response(@prompt) INTO @result        -- (1) Initial generation
```

```
  -- Quality check via semantic evaluation (LLM-as-judge)
  GENERATE quality_assess(@result) INTO @quality -- (2) The "Judge" step

  EVALUATE @quality                                -- (3) Semantic Branching
    WHEN 'high_quality' THEN
      COMMIT @result WITH status = 'high_quality'
    WHEN 'acceptable' THEN                         -- (4) Refinement loop
      GENERATE improved(@result, @prompt) INTO @result
      COMMIT @result WITH status = 'refined'
    OTHERWISE
      COMMIT @result WITH status = 'best_effort'
  END

EXCEPTION                                          -- (5) Generative Exceptions
  WHEN HallucinationDetected THEN
    GENERATE response(@prompt) INTO @result        -- (6) Strategic recovery
    COMMIT @result WITH status = 'conservative'

  WHEN ContextLengthExceeded THEN
    RETRY WITH temperature = 0.1                   -- (7) Automated retry

  WHEN RefusalToAnswer THEN
    COMMIT @prompt WITH status = 'refused'
END
```

5.1.3.1 (1) & (2) Generation and Assessment

We start with a standard GENERATE call. But instead of returning the result immediately, we pass it to a second GENERATE step called quality_assess. This is the "LLM-as-Judge" pattern: using a model (often a larger, more capable one) to grade the work of another model.

5.1.3.2 (3) EVALUATE (Semantic Branching)

In traditional programming, you branch on booleans or integers (if $x > 5$). In SPL, you branch on **Semantic Labels**. The EVALUATE block looks at the string returned by @quality and matches it against your WHEN clauses.

SQL Analogy: A **CASE** statement. It maps inputs to specific outputs or actions based on a set of conditions.

5.1.3.3 (4) Refinement

If the judge returns 'acceptable', we don't give up. We trigger a third generation—improved(@result, @prompt)—asking the model to fix the specific issues identified. This "Multi-Pass" approach significantly increases the reliability of the final output.

5.1.3.4 (5) `EXCEPTION`

This is where SPL 2.0 handles the "known unknowns" of AI. These aren't syntax errors; they are **runtime behaviors** of the model or the adapter.

5.1.3.5 (6) `HallucinationDetected`

This is a semantic exception. If the runtime detects (via internal consistency checks or external fact-checkers) that the model is making things up, it triggers this block. Here, we try one more time, perhaps with a more "conservative" prompt.

5.1.3.6 (7) `RETRY WITH ...`

Some errors are transient or can be fixed by changing parameters. If the context length is exceeded, we might RETRY with a lower temperature to encourage the model to be more concise.

5.1.4 Running It

Run the safe generation with a potentially complex prompt:

```
spl run cookbook/07_safe_generation/safe_generation.spl --adapter ollama \
    prompt="What are the trade-offs of microservices?"
```

Observe the output status. If the first pass is good, you'll see status='high_quality'. If the model needed a second try, you'll see status='refined'.

5.1.5 What Just Happened

LLM calls: 2 or 3. (Depending on the quality pass)

The "Conductor" (SPL Runtime) ensured the safety of the transaction: 1. Proposed a response. 2. Validated the proposal against a quality standard. 3. Executed a "Compensating Transaction" (improvement) if the quality was marginal. 4. Committed the result with a clear metadata tag (status).

This pattern transforms an LLM from a "black box" into a **controlled component** within a larger system.

5.1.6 Reproducibility Note

The "Judge" model needs to be highly stable. If your judge is too small (e.g., a 1B parameter model), its evaluations might be inconsistent. For this recipe, we recommend using a "Senior" model like **gemma3 (27B)** or **Claude 3.5 Sonnet** as the evaluator, even if a "Junior" model (like **phi4**) is doing the initial generation.

5.1.7 When to Use This Pattern

Use the **Safe Generation** pattern when: - **Quality is Non-Negotiable**: Customer-facing content, documentation, or legal summaries. - **Handling Refusals**: When you need to gracefully handle

cases where the model's safety filters block a legitimate request. - **Self-Correction**: When the task is complex enough that the model rarely gets it perfect on the first try.

5.1.8 Exercises

1. **Strict Guardrails.** Modify the EVALUATE block so that OTHERWISE results in a REJECT or an error message rather than a best_effort commit.
2. **Add a "Banned Word" check.** Use a Python tool (Chapter 3.2) to check for specific words in @result. If found, trigger a custom exception.
3. **Two-Model Judge.** Modify the recipe to use two different models as judges and only commit if they *both* agree the quality is high.

Chapter 5.2

Guardrails Pipeline

5.2.1 The Pattern

In a production AI system, "hoping" the model stays safe is not a strategy. You need a rigorous, multi-layered defense that catches harmful content, detects private data (PII), and ensures the output is both relevant and accurate.

A single "Safety Prompt" is rarely enough. You need **Guardrails**—a series of "Gates" that every request must pass through before it is allowed to generate a response.

The SQL analogy is **Multi-Layered Security and Validation**. You have a firewall (the keyword pre-screen), Row-Level Security (the input classifier), Data Masking (PII redaction), and a final Audit (output validation). If any gate fails, the transaction is rolled back or modified before it reaches the user.

The Guardrails Pipeline recipe (section 4.2) is the gold standard for safe AI. It implements four distinct gates, mixing deterministic code (for speed and cost) with probabilistic LLMs (for nuance).

5.2.2 The SPL Approach

This recipe demonstrates the **80/20 Rule** in action: using CALL for 80% of the safety work (keywords, regex) and GENERATE for the remaining 20% (nuanced context).

5.2.3 The .spl File (Annotated)

```
-- Recipe 18: Guardrails Pipeline
-- Input validation → safe generation → output validation.

WORKFLOW guardrails_pipeline
    INPUT: @user_input TEXT
DO
    -- Gate 1a: Keyword pre-screen (Deterministic)
    CALL classify_input_keywords(@user_input) INTO @keyword_class -- (1) The Firew

    EVALUATE @keyword_class
```

```
        WHEN STARTSWITH 'harmful' THEN
            COMMIT 'I cannot help with that.' WITH status = 'blocked_harmful'
        WHEN STARTSWITH 'off_topic' THEN
            COMMIT 'Outside my scope.' WITH status = 'blocked_off_topic'
    END

    -- Gate 1b: LLM input classification (Probabilistic)
    GENERATE classify_input(@user_input) INTO @input_class -- (2) The Nuanced Check

    EVALUATE @input_class
        WHEN 'harmful' THEN COMMIT '...' WITH status = 'blocked_harmful'
        WHEN 'off_topic' THEN COMMIT '...' WITH status = 'blocked_off_topic'
    END

    -- Gate 2: PII detection and redaction (Deterministic)
    CALL detect_pii(@user_input) INTO @pii_report -- (3) Data Masking
    EVALUATE @pii_report
        WHEN STARTSWITH 'pii_found' THEN
            CALL redact_pii(@user_input) INTO @clean_input
        OTHERWISE
            SET @clean_input = @user_input
    END

    -- Gate 3: Safe Generation
    GENERATE safe_response(@clean_input) INTO @raw_response

    -- Gate 4: Output validation
    GENERATE validate_output(@raw_response) INTO @output_check -- (4) The Final Au
    EVALUATE @output_check
        WHEN 'safe' THEN SET @safe_response = @raw_response
        WHEN 'contains_pii' THEN CALL redact_pii(@raw_response) INTO @safe_response
        OTHERWISE SET @safe_response = 'Unable to generate safe response.'
    END

    COMMIT @safe_response WITH status = 'complete'
END
```

5.2.3.1 (1) Gate 1a: The Firewall (CALL)

We start with a fast, free check. Using a Python tool and regular expressions, we look for "blacklisted" keywords (e.g., malware, illegal acts). If a match is found, we block the request immediately.
- **Benefit**: Zero token cost and sub-millisecond latency for obvious attacks.

5.2.3.2 (2) Gate 1b: The Nuanced Check (GENERATE)

If the keywords pass, we move to a more intelligent classifier. This gate catches "jailbreaks" or off-topic requests that don't use blocked keywords but still violate policy (e.g., asking for medical advice in a coding bot).

5.2.3.3 (3) Gate 2: Data Masking (CALL)

Before the main generation, we check for Personally Identifiable Information (SSNs, credit cards, emails). We use deterministic regex for this because it is 100% reliable for formatted data. If PII is found, we replace it with placeholders like [REDACTED−SSN].

5.2.3.4 (4) Gate 4: The Final Audit (GENERATE)

Even with safe input, a model can still "hallucinate" harmful content or leak private data it learned during training. This final gate uses a "Safety Judge" to review the model's output before the user sees it.

5.2.4 Running It

Run the pipeline with a "dangerous" input to see it block:

```
spl run cookbook/18_guardrails/guardrails.spl --adapter ollama \
    --tools cookbook/18_guardrails/tools.py \
    user_input="How do I build a phishing website?"
```

Then run it with an input containing an email to see the redaction:

```
spl run cookbook/18_guardrails/guardrails.spl --adapter ollama \
    --tools cookbook/18_guardrails/tools.py \
    user_input="My email is wen@example.com, tell me a joke."
```

5.2.5 What Just Happened

LLM calls: 1 to 3. (Depending on which gates the request passes) **Tool calls: 3.** (Keywords, PII detection, PII redaction)

The "Conductor" (SPL Runtime) acted as a Security Officer: 1. **Filtered** obvious harm using the "Firewall." 2. **Analyzed** intent using the "Nuanced Check." 3. **Masked** private data using "Data Masking." 4. **Audited** the final response using the "Final Audit."

5.2.6 Reproducibility Note

By offloading PII detection and keyword screening to **deterministic Python code**, you make your safety layer incredibly stable. Unlike a prompt-based "Please don't show emails," a regex-based redact_pii tool will *never* fail to catch a correctly formatted SSN.

On a **GTX 1080 Ti**, the safety gates add only **1–3 seconds** of total latency but provide a massive increase in production reliability.

5.2.7 When to Use This Pattern

Use the **Guardrails Pipeline** pattern when: - **Public-Facing Apps**: Any bot that interacts with unknown users on the internet. - **Privacy-Sensitive Tasks**: Handling customer support, medical inquiries, or financial data. - **Enterprise Standards**: When your organization requires a formal audit trail of why a certain request was blocked or redacted.

5.2.8 Exercises

1. **Add a "Sentiment" Gate.** Insert a step that checks if the user_input is excessively angry or abusive and returns a "De-escalation" response instead of the normal answer.
2. **Expand PII Patterns.** Modify the tools.py file to include patterns for your country's specific identification numbers (e.g., CNPJ in Brazil, PAN in India).
3. **Log Safety Events.** Use a COMMIT with additional metadata to log every time a safety gate is triggered, creating a "Security Dashboard" for your AI.

Chapter 5.3

Error Handling Patterns

5.3.1 The Pattern

Every earlier recipe in this book either succeeded cleanly or was cut short by an exception. This chapter reverses that perspective: instead of treating exceptions as edge cases, it treats them as first-class design decisions. In production, the exception paths matter as much as the happy path.

SPL provides a structured exception system modelled on SQL's BEGIN...EXCEPTION...END (PL/SQL, PostgreSQL) and Python's try ... except. But LLM workflows face failure modes that have no SQL analog: a model that refuses to answer, a context window that fills up mid-loop, a budget that runs out at step 4 of 6, a generation that produces plausible-sounding but invalid output. This recipe maps the full exception landscape and shows how to handle each type.

5.3.2 Who This Chapter Is For

If you are a SQL practitioner: you already understand transaction semantics — a failed constraint rolls back the INSERT, and you handle it explicitly. SPL exceptions are the same idea applied to LLM calls. The vocabulary differs but the principle is identical: name the failure, handle it specifically, commit a meaningful result even when the happy path fails.

If you are building production workflows: this chapter is the difference between a demo and a system. Every workflow you deploy should have an EXCEPTION block that specifies, for each likely failure type, what the system should do and what it should commit as evidence of the failure.

5.3.3 The SPL Exception System

Every WORKFLOW can include an EXCEPTION block following the DO body:

```
WORKFLOW my_workflow
    INPUT: @input TEXT
    OUTPUT: @result TEXT
DO
    -- happy path steps
    GENERATE process(@input) INTO @result
    COMMIT @result WITH status = 'complete'
```

```
EXCEPTION
    WHEN GenerationError THEN
        COMMIT 'Generation failed.' WITH status = 'error', input = @input
    WHEN MaxIterationsReached THEN
        COMMIT @result WITH status = 'partial'
    WHEN BudgetExceeded THEN
        COMMIT @result WITH status = 'budget_limit'
END
```

The EXCEPTION block catches named exception types and routes each to a specific recovery action. A COMMIT inside an exception handler is a partial commit: it records what was produced before the failure, along with diagnostic metadata. This is the most important pattern in this chapter — **always commit something meaningful from every exception handler**.

5.3.4 Exception Type Reference

Exception Type	Trigger	Recommended Response
GenerationError	Model call failed (network, quota, refusal, malformed response)	Commit last valid state with status = 'error'
MaxIterationsReached	WHILE loop exceeded its iteration limit	Commit best-effort result with status = 'partial'
BudgetExceeded	Token budget exhausted mid-workflow	Commit what was generated so far; do not attempt more LLM calls
ContextLengthExceeded	Input to a GENERATE step exceeds the model's context window	Truncate and retry with a shorter context, or commit partial result
HallucinationDetected	LLM-as-judge flagged the output as containing hallucinated content	Retry with stricter instructions, or escalate with status = 'flagged'
ValidationError	A CALL to a validation tool returned a failure	Retry with corrected input, or commit with status = 'invalid'
TimeoutError	A GENERATE step exceeded the configured timeout	Commit partial result; log for investigation

5.3.5 Pattern 1: Graceful Degradation

The most common production pattern: if the full workflow fails, return the best partial result with honest metadata.

```
CREATE FUNCTION summarize_doc(doc TEXT) RETURNS TEXT AS $$
Summarize the following document in 3-5 sentences:
```

```
{doc}
$$;

CREATE FUNCTION fallback_extract(doc TEXT) RETURNS TEXT AS $$
Extract the first paragraph of the following document as a fallback summary:
{doc}
$$;

WORKFLOW robust_summarizer
    INPUT:
        @document TEXT,
        @max_length INTEGER DEFAULT 500
    OUTPUT: @summary TEXT
DO
    GENERATE summarize_doc(@document) INTO @summary
    COMMIT @summary WITH status = 'complete'

EXCEPTION
    WHEN ContextLengthExceeded THEN
        -- Document too long for the model's context window.
        -- Fall back to extracting just the first paragraph.
        GENERATE fallback_extract(@document) INTO @summary
        COMMIT @summary WITH status = 'truncated', note = 'document exceeded contex

    WHEN GenerationError THEN
        -- Model call failed entirely.
        -- Commit an empty result with diagnostic information rather than crashing.
        COMMIT '' WITH status = 'error', document_length = @max_length

    WHEN BudgetExceeded THEN
        -- We ran out of tokens. Commit whatever we have.
        COMMIT @summary WITH status = 'budget_limit'
END
```

Key principle: Every exception handler commits a result. The consumer of the workflow always gets a committed output — even if that output is an empty string with status = 'error'. Silent failures are worse than honest failures.

5.3.6 Pattern 2: Retry with Backoff

For transient failures (network timeouts, rate limits), retry the failing step with progressively adjusted instructions.

```
CREATE FUNCTION generate_with_retry_hint(input TEXT, attempt TEXT) RETURNS TEXT AS
Attempt {attempt}: Process the following input.
If previous attempts failed due to output length, produce a shorter response.

Input: {input}
```

```
$$;

WORKFLOW retry_workflow
    INPUT: @input TEXT
    OUTPUT: @result TEXT
DO
    @attempt := 1
    @max_attempts := 3

    WHILE @attempt <= @max_attempts DO
        GENERATE generate_with_retry_hint(@input, @attempt) INTO @result

        EVALUATE @result
            WHEN 'error' THEN
                @attempt := @attempt + 1
            OTHERWISE
                COMMIT @result WITH status = 'complete', attempts = @attempt
        END
    END

    COMMIT @result WITH status = 'max_retries', attempts = @attempt

EXCEPTION
    WHEN GenerationError THEN
        COMMIT '' WITH status = 'generation_failed', attempts = @attempt
END
```

Important limitation: SPL does not provide automatic retry with exponential backoff — that is implemented at the adapter level for transient infrastructure failures. The pattern above is for *application-level* retry where you want to modify the prompt or reduce scope on each attempt.

5.3.7 Pattern 3: Multi-Step Error Isolation

In long workflows, a failure in step 3 should not discard the results of steps 1 and 2. Checkpoint intermediate results.

```
WORKFLOW pipeline_with_checkpoints
    INPUT:
        @topic TEXT,
        @output_dir TEXT DEFAULT ''
    OUTPUT: @report TEXT
DO
    -- Step 1: Research
    GENERATE research(@topic) INTO @research_notes
    CALL write_checkpoint(@output_dir, 'research.txt', @research_notes) INTO @_

    -- Step 2: Analysis (builds on research)
    GENERATE analyze(@topic, @research_notes) INTO @analysis
```

```
    CALL write_checkpoint(@output_dir, 'analysis.txt', @analysis) INTO @_

    -- Step 3: Report (builds on both)
    GENERATE write_report(@topic, @research_notes, @analysis) INTO @report
    COMMIT @report WITH status = 'complete'

EXCEPTION
    WHEN GenerationError THEN
        -- Commit whatever was last successfully generated.
        -- The checkpoint files preserve earlier steps for manual recovery.
        COMMIT @analysis WITH
            status = 'partial',
            completed_through = 'analysis',
            checkpoint_dir = @output_dir
END
```

The write_checkpoint CALL uses a tool function to persist intermediate state. If the workflow fails at step 3, steps 1 and 2 are preserved on disk. Manual recovery is possible without re-running the expensive early steps.

5.3.8 Pattern 4: LLM-as-Judge for Output Validation

Use a second LLM call to validate the output of the first, and escalate if it fails.

```
CREATE FUNCTION generate_content(task TEXT) RETURNS TEXT AS $$
Complete the following task:
{task}
$$;

CREATE FUNCTION validate_output(task TEXT, output TEXT) RETURNS TEXT AS $$
Does the following output correctly complete the task?

Task: {task}
Output: {output}

Reply with exactly one word: "valid" or "invalid".
$$;

WORKFLOW validated_generation
    INPUT:
        @task TEXT,
        @max_validation_attempts INTEGER DEFAULT 2
    OUTPUT: @output TEXT
DO
    @attempt := 0

    WHILE @attempt < @max_validation_attempts DO
        GENERATE generate_content(@task) INTO @output
```

```
        GENERATE validate_output(@task, @output) INTO @verdict

        EVALUATE @verdict
            WHEN 'valid' THEN
                COMMIT @output WITH status = 'complete', attempts = @attempt
            OTHERWISE
                @attempt := @attempt + 1
        END
    END

    -- Exceeded validation attempts -- commit best effort
    COMMIT @output WITH status = 'unvalidated', attempts = @attempt

EXCEPTION
    WHEN MaxIterationsReached THEN
        COMMIT @output WITH status = 'max_retries'
    WHEN BudgetExceeded THEN
        COMMIT @output WITH status = 'budget_limit'
END
```

Design note: The validator here is an LLM call, which means it is probabilistic. A "valid"/"invalid" verdict from an LLM is not a guarantee. For applications where correctness is critical, supplement LLM validation with a deterministic CALL to a schema validator, unit test, or rule engine.

5.3.9 The SQL Analogy

SQL mechanism	SPL equivalent	Purpose
BEGIN...EXCEPTION...END	EXCEPTION block	Catch named failures
ROLLBACK TO SAVEPOINT	COMMIT partial WITH status = 'partial'	Preserve progress up to the failure
CONSTRAINT / CHECK	EVALUATE @verdict WHEN 'invalid'	Validate output quality
RETRY (in some DBs)	WHILE @attempt < @max DO ... @attempt := @attempt + 1	Retry on transient failure
Audit log	COMMIT WITH status = 'error', error_data	Record the failure for investigation

5.3.10 Running It

```
# Safe generation with all exception types visible
spl run cookbook/07_safe_generation/safe_generation.spl \
    --adapter ollama -m gemma3 \
    topic="Explain quantum entanglement in simple terms"

# Force a budget limit to see the BudgetExceeded handler
spl run cookbook/07_safe_generation/safe_generation.spl \
    --adapter ollama -m gemma3 \
```

```
--budget 50 \
topic="Write a 5000-word essay on the history of computing"
```

5.3.11 Reproducibility Note

Exception handling paths are inherently harder to reproduce than happy paths, because they depend on failure conditions that may be intermittent (rate limits, network timeouts) or model-specific (refusals, context limits). Test exception handlers by simulating failure conditions explicitly: use --budget to trigger BudgetExceeded, use an extremely long document to trigger ContextLengthExceeded, or use a deliberately nonsensical prompt to observe GenerationError recovery.

5.3.12 When to Use These Patterns

Every workflow you deploy to production should have:

1. **A GenerationError handler** — network failures and model refusals are not rare.
2. **A BudgetExceeded handler** — token budgets are real in production; unexpected long documents happen.
3. **Meaningful committed output in every handler** — consumers of the workflow need to know what happened.

Add MaxIterationsReached and ContextLengthExceeded handlers for any workflow with a WHILE loop or that processes user-supplied document content.

The LLM-as-judge validation pattern (Pattern 4) should be added to any workflow where output quality is a hard requirement — not just a preference.

5.3.13 Exercises

1. **Add exception handling to an existing recipe.** Pick any recipe from Part 2 (Agentic Patterns) that does not have an EXCEPTION block. Add handlers for GenerationError, MaxIterationsReached, and BudgetExceeded. Decide what each handler should commit and why.

2. **Simulate a budget limit.** Run any multi-step recipe with --budget 100 — a very small token budget. Observe what the BudgetExceeded handler commits. Is it useful? Would a real consumer of this workflow be able to recover from it?

3. **Write a validator.** For section 1.2.3 (Plan and Execute), add a validation step using a second GENERATE call that checks whether the generated plan is well-formed (numbered steps, at least 3 steps, no duplicate step descriptions). Route invalid plans back to the planner for a retry, up to 2 retries.

Chapter 5.4

Human-in-the-Loop Approval Gate

5.4.1 The Pattern

You have an agentic workflow that produces consequential output: a financial report, a customer-facing email, a database mutation, a code deployment. You want the speed of automation but not the risk of fully unsupervised execution. The solution is a human approval gate — a deliberate pause where the workflow surfaces its intermediate result, waits for a human decision, and only proceeds (or aborts) based on that decision.

This is not a sign of weakness in your workflow. It is a design choice. Production agentic systems in regulated industries (finance, healthcare, legal) often *require* human-in-the-loop steps for compliance. Even in unregulated settings, approval gates are the mechanism by which humans maintain meaningful control over automated processes that act on their behalf.

The Python equivalent embeds the approval logic in the application layer — a webhook, a message queue, a polling loop. The result is that the approval logic is separated from the workflow definition, making it hard to read, audit, or modify. SPL keeps the approval gate *inside* the workflow as a named step, visible to anyone reviewing the .spl file.

5.4.2 The SPL Approach

SPL expresses the approval gate through a CALL to a tool that blocks until human input is received, combined with conditional logic on the response. The tool handles the mechanics (sending a notification, presenting the draft, waiting for a response); the SPL file handles the workflow logic (what to do when approved, what to do when rejected).

The key insight: the approval step is a **deterministic operation** (CALL, not GENERATE). It calls a Python tool that returns a structured decision. The workflow branches on that decision using EVALUATE. No LLM is involved in the approval step itself — the human *is* the judge.

5.4.3 The `.spl` File (Annotated)

```
-- human_approval_gate.spl
-- Pattern: block workflow at approval checkpoint, branch on human decision
```

```
CREATE FUNCTION draft_report(topic TEXT, data TEXT) RETURNS TEXT AS $
You are a professional analyst. Write a concise executive summary report on the fol

Topic: {topic}
Source data:
{data}

Format your report with:
- An executive summary (2-3 sentences)
- Key findings (3-5 bullet points)
- Recommended action (1 paragraph)

Write in formal business language. Be specific. Avoid vague generalities.
$;

WORKFLOW approve_and_send_report
  INPUT:
    @topic TEXT DEFAULT 'Q1 Performance Review',
    @data  TEXT DEFAULT 'See attached data file',
    @approver_email TEXT DEFAULT 'manager@example.com'
  OUTPUT:
    @final_report TEXT,
    @disposition  TEXT   -- 'approved' | 'rejected' | 'revised'
DO
  -- Step 1: Generate the draft
  GENERATE draft_report(@topic, @data) INTO @draft

  -- Step 2: Human approval gate
  -- tool.request_approval() sends @draft to @approver_email,
  -- blocks until they respond, and returns a structured decision:
  -- { "decision": "approve"|"reject"|"revise", "comments": "..." }
  CALL tool.request_approval(
    content    = @draft,
    recipient  = @approver_email,
    subject    = 'Approval required: ' || @topic,
    timeout_h  = 24
  ) INTO @approval

  -- Step 3: Branch on the decision
  EVALUATE @approval.decision
    WHEN 'approve' THEN
      @final_report := @draft
      @disposition  := 'approved'
      CALL tool.send_email(
        to      = @approver_email,
        subject = 'Report sent: ' || @topic,
        body    = @final_report
      ) INTO @_
```

```
      COMMIT @final_report WITH status = 'sent'

    WHEN 'revise' THEN
      -- Incorporate reviewer comments and generate a revised draft
      GENERATE draft_report(
        @topic,
        @data || '\n\nReviewer comments: ' || @approval.comments
      ) INTO @revised
      @final_report := @revised
      @disposition  := 'revised'
      COMMIT @final_report WITH status = 'revised_pending_send'

    WHEN 'reject' THEN
      @final_report := ''
      @disposition  := 'rejected'
      COMMIT @final_report WITH status = 'rejected'

  END

EXCEPTION
  WHEN TimeoutError THEN
    -- Approval not received within 24 hours
    @disposition := 'timed_out'
    CALL tool.notify(@approver_email, 'Approval request expired: ' || @topic) INTO
    COMMIT '' WITH status = 'approval_timed_out'
END
```

Line-by-line highlights:

- CREATE FUNCTION draft_report — explicit prompt template; the LLM gets a structured role and output format, not an open-ended instruction
- CALL tool.request_approval(...) — the approval gate is a deterministic CALL, not GENERATE; the human provides the probabilistic element
- EVALUATE @approval.decision WHEN ... — branches cleanly on three outcomes; the WHEN 'revise' branch incorporates reviewer comments into a second generation pass
- EXCEPTION WHEN TimeoutError — production-grade: if no response within 24 hours, notify and abort rather than hanging indefinitely

5.4.4 The SQL Analogy

The approval gate is a stored procedure that includes a WAITFOR step — an explicit synchronization point that pauses execution until an external condition is met. SQL Server's WAITFOR DELAY is the closest syntactic analogue, though the semantics here are event-driven rather than time-driven.

The EVALUATE @approval.decision WHEN ... block is a CASE statement on the procedure's output. The three branches (approve, revise, reject) correspond exactly to three different code paths in a SQL IF/ELSIF/ELSE construct.

The EXCEPTION WHEN TimeoutError block is the stored procedure's error handler — the same pattern as BEGIN ... EXCEPTION WHEN others THEN ... END in PL/SQL.

5.4.5 Running It

```
# Run with a local approval tool (development mode: auto-approves after 5 seconds)
spl run human_approval_gate.spl \
  --adapter ollama -m gemma3 \
  --tools approval_tools.py \
  --input topic="Q1 Performance Review" \
  --input data="Revenue: $2.4M (+12% YoY). Churn: 3.2% (-0.8pp). NPS: 47 (+5)." \
  --input approver_email="manager@example.com"
```

Expected output (approved path):

```
[draft_report] Generated 312-word executive summary
[request_approval] Sent approval request to manager@example.com
[request_approval] Response received: decision=approve, comments=""
[send_email] Report sent to manager@example.com
Status: sent
```

5.4.6 What Just Happened

1. draft_report was called with the topic and data; the LLM produced a 312-word executive summary
2. tool.request_approval sent the draft to the approver and blocked — no LLM tokens consumed during the wait
3. The approver responded with decision=approve
4. The workflow took the approval branch, sent the email, and committed with status sent

The EVALUATE/EXCEPTION structure means that all three outcomes (approve, revise, reject) and the timeout case are explicitly handled. There is no "what if the human rejects it?" case that falls through to undefined behavior.

5.4.7 Reproducibility Note

- **Hardware**: GTX 1080 Ti, 11 GB VRAM
- **Model**: Gemma 3 via Ollama
- **Draft generation latency**: 18–25 seconds (varies with report length)
- **Approval tool**: in development mode, tool.request_approval returns immediately with decision=approve; in production mode, it sends an actual email and blocks on response
- **Revision path**: adds one additional GENERATE call; total latency approximately doubles

5.4.8 When to Use This Pattern

Use it when: - The workflow output has real-world consequences (sending communications, writing files, calling external APIs) - Compliance or audit requirements mandate a human sign-off step

- The generation quality is not yet reliably high enough for unsupervised deployment - You want to train a human reviewer loop that gradually reduces approval frequency as confidence grows

Do not use it when: - The workflow output is purely informational and never acted on automatically - The approval latency (hours) would defeat the purpose of automation - The volume of executions makes human review impractical (consider automated quality gates instead — see section 1.4.2)

Anti-pattern to avoid: Treating the approval gate as an afterthought. If the approval step is not in the .spl file, it is invisible to anyone reviewing the workflow. Human-in-the-loop logic belongs in the declarative artifact, not in the application layer.

5.4.9 Exercises

1. Modify the revise branch to limit revision attempts: if the reviewer rejects a second revised draft, automatically escalate to a second approver rather than generating a third draft.
2. Add an audit log: call tool.write_log(disposition, timestamp, reviewer_comments) at the end of each branch so every workflow execution produces an immutable audit trail.
3. Replace tool.request_approval with a Slack webhook: implement approval_tools.py so that approval requests are sent as Slack messages with approve/reject buttons, and the tool blocks on the Slack API response.

Part 6

Memory & Retrieval

Chapter 6.1

RAG Query

6.1.1 The Pattern

LLMs are brilliant, but they have a "Knowledge Cutoff." They only know what they were trained on. If you ask a model about a private document, a recent news event, or your own internal company data, it will either admit ignorance or, worse, hallucinate a plausible-sounding lie.

RAG (Retrieval-Augmented Generation) is the industry-standard solution. Instead of relying on the model's memory, you provide it with the relevant "background reading" alongside your question. You fetch the right information, inject it into the prompt, and ask the model to synthesize an answer based *only* on that context.

The SQL analogy is a **JOIN against an external table** or a **Subquery**:

```
SELECT answer
FROM (
    SELECT content FROM documents
    ORDER BY similarity(content, :question) DESC
    LIMIT 3
) AS background
WHERE question = :question;
```

You aren't querying the model; you are querying a "table" of your own data and joining it with the model's reasoning capabilities.

The RAG Query recipe (section 5.1) demonstrates the simplest possible RAG implementation in SPL. It uses the built-in rag.query function to perform a semantic search against a local vector database and feeds the results directly into the prompt.

6.1.2 The SPL Approach

This recipe introduces the rag namespace, which provides high-level access to the SPL runtime's integrated vector store.

6.1.3 The .spl File (Annotated)

```
-- Recipe 08: RAG Query
-- Retrieval-augmented generation over indexed documents.

PROMPT rag_answer                                   -- (1) name the prompt block
SELECT
    system_role('You are a knowledgeable assistant. Use the provided context to ans
                                                    -- (2) context-aware instruction
    rag.query(context.question, top_k=3) AS background,
                                                    -- (3) The Semantic Join
    context.question AS question                    -- (4) The user's query
GENERATE answer(question)                           -- (5) Grounded generation
```

6.1.3.1 (1) `PROMPT rag_answer`

Even though RAG sounds complex, the execution flow is still a single model call. The "retrieval" happens during the SELECT phase, just like a SQL join happens before the final result set is produced.

6.1.3.2 (2) `system_role(...)`

It is crucial to tell the model that it *has* context and that it should use it. This prevents the model from ignoring the retrieved data in favor of its own pre-trained weights.

6.1.3.3 (3) `rag.query(context.question, top_k=3) AS background`

This is the heart of RAG in SPL. - **rag.query**: A built-in function that takes a string, converts it into a vector (an embedding), and searches the local FAISS index for the 3 most similar chunks of text. - **top_k=3**: We only want the most relevant information. Too much context can "drown" the model or exceed its token limit.

SQL Analogy: A **Vector Similarity Search**. You are ordering your "documents" table by how closely they match the meaning of the question.

6.1.3.4 (4) & (5) `question` and `GENERATE`

We bind the question and the retrieved background to the context row. When GENERATE fires, the model sees the question *and* the relevant evidence side-by-side. The resulting answer is "grounded" in your data.

6.1.4 Running It

RAG requires a one-time setup to index your data:

```
# 1. Install dependencies
pip install numpy faiss-cpu sentence-transformers

# 2. Add a document to the index
# This chunks the file into paragraphs and stores their embeddings locally.
spl rag add /path/to/your/document.md
```

Now, run the query:

```
spl run cookbook/08_rag_query/rag_query.spl --adapter ollama \
    question="What is the main argument of this document?"
```

6.1.5 What Just Happened

LLM calls: 1. Vector Search calls: 1.

The "Conductor" (SPL Runtime) managed the retrieval: 1. Calculated the embedding for the question. 2. Searched the local `.spl/vectors.faiss` file for matches. 3. Fetched the corresponding text from the `.spl/vectors_meta.db` database. 4. Injected the found text into the background column. 5. Passed the complete, grounded context to the model for the final answer.

6.1.6 Reproducibility Note

RAG adds a layer of complexity to reproducibility: the **Embedding Model**. SPL defaults to `all-MiniLM-L6-v2`, a small but efficient model that runs on your CPU. If you change your embedding model, your search results will change, which will change the model's answer.

On a **GTX 1080 Ti**, the vector search takes **less than 50ms** for an index of thousands of paragraphs. The bottleneck remains the LLM generation time.

6.1.7 When to Use This Pattern

Use the **RAG Query** pattern when: - **Private Data**: You need to query documents that were never part of a public training set. - **Up-to-Date Info**: You need the model to know about events that happened after its knowledge cutoff. - **Fact-Checking**: You want to reduce hallucinations by forcing the model to cite its sources from the provided context.

6.1.8 Exercises

1. **Inspect the Retrieval.** Run `spl rag query "your question" --top-k 5` from the command line. This shows you exactly what the model will see *before* it generates an answer. This is the "EXPLAIN" plan for your RAG query.
2. **Filter by Metadata.** (Advanced) Research how to add metadata to your documents (e.g., author, date) and filter your `rag.query` to only search specific files.
3. **No-Grounding Test.** Run the recipe *before* you index any documents. Observe how the model handles the empty context. Does it admit it doesn't know, or does it try to guess? This is a great way to test the "honesty" of a model.

Chapter 6.2

Memory Conversation

6.2.1 The Pattern

Most LLM interactions are "stateless." The model doesn't remember who you are, what you said five minutes ago, or what your preferences are. Every time you start a new session, you have to re-explain everything — a frustrating Groundhog Day experience.

Memory-Augmented Conversation solves this by giving the model a long-term memory. Before each turn, the system loads facts about the user from a persistent store and injects them into the prompt. It also extracts *new* facts from the current turn and writes them back for next time.

The SQL analogy: a **User Profile Table**. Every request fetches the user's profile, personalizes the response, and writes any updates back. The difference here is that the profile is written in natural language — a concise bullet list — not a relational schema.

6.2.2 The SPL Approach

section 5.2 introduces two ideas that work together:

1. **STORAGE-typed INPUT** — memory is declared as a first-class workflow parameter, not a hidden global. The caller can override the path; the default is ~/.spl/memory.db.
2. **Fact Extraction and Merging** — a two-step LLM pattern: first *identify* new facts in the user's message, then *merge* them into the existing profile. This keeps the profile clean and non-redundant.

6.2.3 The .spl File (Annotated)

```
-- Recipe 19: Memory-Augmented Conversation

-- Extract any personal facts the user shared. Return the exact string 'no_new_fac
-- if nothing new was shared, otherwise return a bullet list of facts.
CREATE FUNCTION extract_facts(user_input TEXT) RETURNS TEXT
AS $$

You are a fact extractor. Read the user message and extract any personal facts they
```

```
about themselves (name, job, preferences, location, etc.).

If no personal facts are present, reply with exactly: no_new_facts
Otherwise reply with a concise bullet list, one fact per line, e.g.:
- Name: Alice
- Role: data scientist
- Preference: Python over R

User message: "{user_input}"
$$;

-- Merge new facts into an existing profile. Return the updated profile as a bullet
CREATE FUNCTION merge_profile(existing_profile TEXT, new_facts TEXT) RETURNS TEXT
AS $$

You are maintaining a user profile as a concise bullet list.
If the existing profile is empty or blank, start a new profile from the new facts o
Otherwise merge the new facts in, keeping all prior facts unless directly contradic
Do NOT invent or assume any facts not explicitly stated.
Return ONLY the bullet list, no preamble.

Existing profile:
{existing_profile}

New facts to merge:
{new_facts}
$$;

-- Reply using full context: known profile + conversation history + current message
CREATE FUNCTION contextual_reply(user_input TEXT, profile TEXT, history TEXT) RETUR
AS $$

Known facts about the user: {profile}

Recent conversation:
{history}

Based solely on the known facts above, answer in one sentence: "{user_input}"
If the fact is not listed above, respond: "I don't have that in memory."
$$;

WORKFLOW memory_conversation
    INPUT:
        @user_input TEXT,
        @memory     STORAGE(sqlite, '~/.spl/memory.db')   -- (1) Memory as a param
    OUTPUT: @response TEXT
DO
    LOGGING f'Memory conversation | input: {@user_input}' LEVEL INFO
```

```
    -- Load existing user profile and chat history from the injected storage backen
    @profile      := @memory['chat_user_profile']          -- (2) Read from storage
    @chat_history := @memory['chat_history']

    -- Extract any new personal facts the user shared
    GENERATE extract_facts(@user_input) INTO @new_facts    -- (3) Fact extraction

    -- Update profile only when new facts were actually found (exact string check)
    EVALUATE @new_facts
        WHEN = 'no_new_facts' THEN
            LOGGING 'No new facts - profile unchanged' LEVEL DEBUG
        ELSE
            LOGGING 'New facts detected - merging profile' LEVEL DEBUG
            GENERATE merge_profile(@profile, @new_facts) INTO @profile
-- (4) Merge
            @memory['chat_user_profile'] := @profile       -- (5) Write back
    END

    -- Generate response with full context: profile + history + current input
    GENERATE contextual_reply(@user_input, @profile, @chat_history) INTO @response

    -- Persist updated chat history (keep last 10 turns, deterministic -
 no LLM cost)
    @chat_history := @chat_history || '\nUser: ' || @user_input || '\nAssistant: '
    CALL trim_turns(@chat_history, '10') INTO @chat_history  -- (6) Deterministic t
    @memory['chat_history'] := @chat_history

    LOGGING 'Response ready' LEVEL INFO
    COMMIT @response WITH status = 'complete'

EXCEPTION
    WHEN BudgetExceeded THEN
        COMMIT 'I remember you! But I ran out of budget for this response.' WITH st
END
```

6.2.3.1 (1) Memory as a First-Class Parameter

```
@memory STORAGE(sqlite, '~/.spl/memory.db')
```

Memory is not a hidden global or a hardcoded path — it is a typed INPUT parameter. STORAGE(sqlite, '~/.spl/memory.db') is the default, aligned with the ~/.spl/ workspace that spl init creates. The caller can override it at run time:

```
# Use a different memory file for a specific project
spl run ... memory=~/projects/myapp/memory.db
```

This makes memory **portable and testable**. Point it at a fresh .db in CI, a shared file for a team, or a per-user path in a multi-tenant deployment — without changing a line of SPL.

6.2.3.2 (2) Reading from Storage

```
@profile        := @memory['chat_user_profile']
@chat_history := @memory['chat_history']
```

@memory['key'] dispatches to the storage backend's get(key) method. Returns an empty string if the key doesn't exist yet — no error, no null checks needed. On the first-ever run, both variables start empty and the workflow handles that gracefully.

6.2.3.3 (3) Fact Extraction

The extract_facts function is the "listener." It reads the user's message and returns either no_new_facts (exact string) or a bullet list of newly shared information. The exact-string contract is the bridge between probabilistic LLM output and deterministic SPL branching — EVALUATE ... WHEN = 'no_new_facts' is a zero-token string comparison.

6.2.3.4 (4) Profile Merging

merge_profile receives the *old* profile and the *new* facts and produces a single updated bullet list. The merge step prevents the profile from becoming a growing pile of sentences. It also handles contradictions — if the user says "I moved to Berlin" after previously saying "I live in Paris," the merged profile keeps only Berlin.

6.2.3.5 (5) Writing Back to Storage

```
@memory['chat_user_profile'] := @profile
```

@memory['key'] := value dispatches to set(key, value). The write happens only inside the ELSE branch — only when new facts were actually found. No unnecessary writes.

6.2.3.6 (6) Deterministic History Trimming

```
CALL trim_turns(@chat_history, '10') INTO @chat_history
```

trim_turns is a **deterministic built-in** — it keeps the last 10 User:/Assistant: turn pairs by string manipulation. CALL costs zero tokens. Compare this to GENERATE trim_history(...) which would cost tokens on every single turn just to do string truncation. The SPL principle: *if you can write it as code, write it as code.*

6.2.4 Running It

```
# Turn 1: Introduce yourself
spl run cookbook/19_memory_conversation/memory_chat.spl --adapter ollama \
    user_input="My name is Alice and I am a data scientist"

# Turn 2: Test the memory
spl run cookbook/19_memory_conversation/memory_chat.spl --adapter ollama \
    user_input="Who is Alice"

# Turn 3: Unknown person — should say "I don't have that in memory"
```

```
spl run cookbook/19_memory_conversation/memory_chat.spl --adapter ollama \
    user_input="Who is Bob"
```

Inspect and manage memory with the spl memory commands:

```
spl memory list                          # Show all stored keys
spl memory get chat_user_profile         # Inspect the profile
spl memory delete chat_history           # Clear conversation history
spl memory delete chat_user_profile      # Reset the profile
```

6.2.5 What Just Happened

LLM calls: 2 (no new facts) or 3 (new facts found).

- Turn 1 ("My name is Alice"): 3 calls — extract (finds facts), merge, reply
- Turn 2 ("Who is Alice"): 2 calls — extract (no new facts, skips merge), reply

The EVALUATE branch eliminates the merge call when nothing new was shared. On a long-running assistant where most turns are questions rather than introductions, this saves a meaningful fraction of token spend.

Zero-token operations: - @memory['key'] reads (SQLite lookup) - @memory['key'] := value writes (SQLite upsert) - CALL trim_turns(...) (string manipulation)

The SPL runtime makes the efficient choice the natural choice.

6.2.6 The Prompt Design Lesson

The contextual_reply function took several iterations to get right. Early versions suffered from two failure modes:

1. **History poisoning** — if the model hallucinated an answer and it got saved to @chat_history, every subsequent turn reinforced the same hallucination (the model treated its own prior output as a factual source).

2. **Profile ignored** — rules like "if the answer is not in the profile, say I don't know" were applied by small models *even when the answer was in the profile.*

The final prompt resolves both by: - Putting the known facts on the **very first line** — small models weight early context heavily - Giving a single, concrete instruction: "answer from the known facts; if not listed, say so" - Avoiding long rule lists that small models misapply

```
Known facts about the user: {profile}

Recent conversation:
{history}

Based solely on the known facts above, answer in one sentence: "{user_input}"
If the fact is not listed above, respond: "I don't have that in memory."
```

Minimal rules. Concrete instruction. Profile first.

6.2.7 Model Notes

Local models vary significantly in how reliably they follow the "answer only from provided facts" constraint:

Model	Behavior
phi4	Follows the constraint reliably; tested and recommended
gemma3	Prone to using pre-trained knowledge even when instructed not to
llama3.2, mistral	Generally reliable; good alternatives to phi4

If you observe hallucinated answers, check whether the history has been poisoned by a prior wrong response. Reset with spl memory delete chat_history and retry.

6.2.8 When to Use This Pattern

Use the **Memory Conversation** pattern when: - **Personal Assistants** — bots that help with scheduling, coding, or writing over long periods - **Educational Tutors** — a bot that remembers which concepts a student has mastered and where they are struggling - **Customer Support** — a bot that knows a user's purchase history without being told every time

6.2.9 Exercises

1. **Inspect the Memory.** After a few turns, run spl memory get chat_user_profile to see the raw profile the model has built. Notice how facts accumulate and merge across turns.
2. **Override the Storage Path.** Run the workflow with a project-specific memory file: memory=~/myproject/memory.db. Confirm that spl memory list is now empty (pointing at ~/.spl/memory.db by default) while the project file has data.
3. **Add a "Mood" Memory.** Extend the workflow with a third memory key chat_user_mood. Add a detect_mood function that classifies the user's emotional tone and an update_mood branch (similar to profile merging) that persists it. Have contextual_reply adjust its tone based on the stored mood.
4. **Reset Logic.** Add a special-case branch: if @user_input equals "forget everything", delete both memory keys and commit a "Memory cleared." response — without making any LLM calls.

Chapter 6.3

Chat Over Your PDF

6.3.1 The Pattern

You have a PDF — a research paper, a contract, a technical specification, a financial report. You want to ask questions about it in natural language and get answers grounded in the document's actual content, not the model's training data.

This is the most commonly requested recipe in the SPL repertoire. "Chat over your documents" is the killer use case for RAG (Retrieval-Augmented Generation) in enterprise settings — it is what most teams mean when they say they want to "do something with AI and our documents."

The naive approach — paste the PDF text into a prompt — fails at scale. PDFs are long. Context windows are finite. A 50-page contract does not fit in a single prompt. Even when it technically fits, the model's attention disperses across the full document and precision drops. The right architecture is *retrieve-then-generate*: find the relevant passages first, then generate an answer from those passages only.

What makes this recipe different from section 2.1.1 (RAG Query) is the *conversation loop*. section 2.1.1 answers a single question. This recipe maintains a conversation — tracking what has been asked and answered, so follow-up questions like "What does that mean in practice?" resolve correctly.

6.3.2 The SPL Approach

Two components work together: a PDF ingestion step (CALL to a tool connector) and a conversational RAG loop (WORKFLOW with memory). The PDF is ingested once at the start of the session; the conversation loop runs until the user ends the session.

The tool.pdf_to_md() connector handles the ingestion — no new SPL keywords, just the existing CALL construct routing to a configured tool backend (pymupdf by default). The memory layer stores conversation history so each new question has context from prior exchanges.

6.3.3 The `.spl` File (Annotated)

```
-- chat_over_pdf.spl
```

```
-- Pattern: document-grounded conversational Q&A with memory

CREATE FUNCTION answer_question(question TEXT, context TEXT, history TEXT) RETURNS
You are a precise research assistant. Answer the user's question using ONLY the inf

Document context:
{context}

Conversation history (for follow-up resolution):
{history}

User question: {question}

Instructions:
- If the answer is in the document, quote the relevant passage and explain it
- If the answer is NOT in the document, say explicitly: "This information is not in
- Do not speculate beyond what the document states
- Keep your answer concise (under 200 words unless the question requires more)
$;

CREATE FUNCTION extract_search_terms(question TEXT, history TEXT) RETURNS TEXT AS $
Given this conversation history and new question, extract the key search terms need

Conversation history: {history}
New question: {question}

Return only a list of 3-5 search terms or phrases, one per line. Focus on nouns and
$;

WORKFLOW chat_over_pdf
  INPUT:
    @pdf_path      TEXT DEFAULT './document.pdf',
    @max_turns     INT  DEFAULT 20,
    @top_k_chunks  INT  DEFAULT 5
  OUTPUT:
    @turns_completed INT
DO
  -- Step 1: Ingest the PDF once
  CALL tool.pdf_to_md(@pdf_path) INTO @document_text

  -- Step 2: Chunk and index the document for retrieval
  CALL memory.index_document(@document_text, chunk_size=500, overlap=50) INTO @doc

  -- Step 3: Initialize conversation state
  @history       := ''
  @turns_completed := 0

  -- Step 4: Conversation loop
```

```
  WHILE @turns_completed < @max_turns DO

    -- Get user question (blocks until input received)
    CALL tool.get_user_input(prompt='Ask a question (or type "quit" to exit): ') IN

    -- Exit condition
    EVALUATE @question
      WHEN 'quit' THEN
        COMMIT @turns_completed WITH status = 'session_ended'
        BREAK
    END

    -- Step 4a: Extract search terms, resolving any pronouns via history
    GENERATE extract_search_terms(@question, @history) INTO @search_terms

    -- Step 4b: Retrieve relevant chunks from the indexed document
    CALL memory.retrieve(@doc_index, @search_terms, top_k=@top_k_chunks) INTO @con

    -- Step 4c: Generate a grounded answer
    GENERATE answer_question(@question, @context, @history) INTO @answer

    -- Step 4d: Display and update history
    CALL tool.display(@answer) INTO @_
    @history := @history || 'Q: ' || @question || '\nA: ' || @answer || '\n\n'
    @turns_completed := @turns_completed + 1

  END

  COMMIT @turns_completed WITH status = 'completed'

EXCEPTION
  WHEN ContextLengthExceeded THEN
    -- History grew too long; summarize it and continue
    GENERATE summarize_history(@history) INTO @history_summary
    @history := '[Earlier conversation summarized] ' || @history_summary
    RETRY

  WHEN FileNotFoundError THEN
    COMMIT 0 WITH status = 'pdf_not_found', error = 'PDF file not found: ' || @pdf_
END
```

Line-by-line highlights:

- CALL tool.pdf_to_md(@pdf_path) — PDF ingestion via tool connector; no new syntax, just CALL routing to pymupdf (configurable: --connector pdf=pdfplumber)
- CALL memory.index_document(...) — chunks the document and builds an in-memory retrieval index; chunk size and overlap are tunable INPUT params
- GENERATE extract_search_terms(...) — resolves pronouns and extracts search terms *be-*

fore retrieval; this step is what makes follow-up questions ("What does *that* mean?") work correctly
- CALL memory.retrieve(...) — deterministic retrieval (CALL, not GENERATE); top-k chunks returned as structured context
- @history accumulation — running conversation history passed to each answer_question call; the model uses it to resolve follow-up references
- EXCEPTION WHEN ContextLengthExceeded — when history grows beyond the model's context window, summarize it in-place and retry; the workflow does not crash

6.3.4 The SQL Analogy

This recipe is a stored procedure with a cursor loop. The WHILE @turns_completed < @max_turns DO loop is a WHILE loop over a cursor. Each iteration: 1. Fetches a new input row (the user's question) 2. Runs a subquery to retrieve relevant document chunks (memory.retrieve = a filtered SELECT) 3. Calls a function to generate the answer 4. Appends to an accumulator variable (conversation history)

The EXCEPTION WHEN ContextLengthExceeded block is the cursor loop's error handler — equivalent to catching ORA−01555 (snapshot too old) and refreshing the cursor.

The memory.index_document call is equivalent to building an index before querying a large table — you pay the indexing cost once, then retrieve cheaply per query.

6.3.5 Running It

```
# Chat over a PDF document
spl run chat_over_pdf.spl \
  --adapter ollama -m gemma3 \
  --tools pdf_tools.py memory_tools.py \
  --input pdf_path="./research_paper.pdf" \
  --input top_k_chunks=5
```

Expected session:

```
[pdf_to_md] Ingested 42 pages (18,400 words)
[index_document] Built retrieval index: 148 chunks

Ask a question (or type "quit" to exit): What is the main finding?
[extract_search_terms] Terms: main finding, conclusion, result, key result
[retrieve] 5 chunks retrieved (avg relevance: 0.84)
[answer_question] The paper's main finding is that...

Ask a question (or type "quit" to exit): How did they measure that?
[extract_search_terms] Terms: measurement method, metric, evaluation (resolved "tha
main finding)
[retrieve] 5 chunks retrieved (avg relevance: 0.79)
[answer_question] The measurement methodology described in Section 3...

Ask a question (or type "quit" to exit): quit
```

```
Session ended. 2 turns completed.
```

6.3.6 What Just Happened

1. The PDF was ingested with tool.pdf_to_md and chunked into 148 overlapping segments
2. On each turn, extract_search_terms resolved the question (including pronoun references) into retrieval terms
3. memory.retrieve fetched the 5 most relevant chunks — a tiny fraction of the full document
4. answer_question generated a grounded response from those chunks and the conversation history
5. The "How did they measure that?" follow-up resolved correctly because extract_search_terms used the history to resolve "that" before retrieval

The architecture ensures the model never sees more than top_k_chunks × chunk_size tokens of document context per question — typically 2,000–3,000 tokens, well within any model's comfortable working range.

6.3.7 Reproducibility Note

- **Hardware**: GTX 1080 Ti, 11 GB VRAM
- **Model**: Gemma 3 via Ollama
- **PDF**: 42-page research paper (18,400 words)
- **Ingestion latency**: 3–5 seconds (pymupdf; CPU-bound)
- **Indexing latency**: 8–12 seconds (embedding 148 chunks)
- **Per-question latency**: 12–18 seconds (search terms + retrieval + answer generation)
- **Context window consumption**: ~2,500 tokens per turn (context + history); grows slowly as history accumulates

6.3.8 When to Use This Pattern

Use it when: - The document is too long for a single prompt (most real-world PDFs qualify) - You need multi-turn conversation with follow-up resolution - Precision is critical — you want answers grounded in document text, not model priors

Use section 2.1.1 (RAG Query) instead when: - You have a single question, not a conversation - You are querying across many documents (section 2.1.1's corpus indexing model scales better)

Anti-patterns to avoid: - Ingesting the full PDF text into the prompt instead of using retrieval — works for short documents, fails for anything over ~10 pages - Skipping the extract_search_terms step — direct embedding of the user question against the document works for simple questions but fails for pronoun references and multi-part follow-ups - Forgetting the ContextLengthExceeded handler — long conversations will eventually hit the model's context limit; without the handler, the workflow crashes on turn N with no recovery

6.3.9 Exercises

1. Add a citation mode: modify answer_question to always include the document page number or section heading from which it sourced the answer. Modify memory.retrieve to return chunk

metadata (page number, section).

2. Add multi-document support: accept a list of PDF paths as input and index all documents before the conversation begins. The retrieval step should label which document each chunk came from.
3. Implement conversation summarization proactively: instead of waiting for ContextLengthExceeded, summarize the history every 5 turns to keep the context window usage flat regardless of conversation length.

Part 7

Multi-Agent Systems

Chapter 7.1

Debate Arena

7.1.1 The Pattern

When a single model answers a question, you get a single perspective. That perspective is shaped by the model's training data, its safety filters, and its inherent biases. If you are exploring a complex, nuanced topic—like "Should AI be open-sourced?"—a single response is rarely enough. You need to see the "pro" and the "con" argued with equal vigor.

The **Debate Arena** pattern uses multiple LLM instances (or personas) to argue opposing sides of a motion. By forcing the models to rebut each other's points over several rounds, you expose the depth of the topic and the weaknesses in each argument.

The SQL analogy is a **Self-Join with State** or a **Recursive CTE**. You are taking the "output" of one row (the pro argument) and using it as the "input" for the next row (the con rebuttal), repeating the process until a termination condition is met.

The Debate Arena recipe (section 6.1) implements a multi-round debate. Two personas trade opening statements and rebuttals, while a third "Judge" persona monitors the history and declares a winner. This is a powerful technique for stress-testing ideas and discovering edge cases.

7.1.2 The SPL Approach

This recipe introduces the WHILE loop for iterative reasoning and the pattern of "State Accumulation" (building a history string over multiple steps).

7.1.3 The .spl File (Annotated)

```
-- Recipe 11: Debate Arena
-- Two personas argue opposing sides, then a judge picks the winner.

WORKFLOW debate_arena
    INPUT:
        @topic      TEXT,
        @max_rounds INTEGER DEFAULT 3
    OUTPUT: @verdict TEXT
```

```
DO
    @round := 0
    @pro_history := ''                          -- (1) State initialization
    @con_history := ''

    LOGGING f'Debate started | topic: {@topic} | rounds: {@max_rounds}' LEVEL INFO

    -- Opening statements
    GENERATE pro_argument(@topic, 'opening statement') INTO @pro
    GENERATE con_argument(@topic, 'opening statement') INTO @con

    @pro_history := @pro                        -- (2) Seeding the history
    @con_history := @con

    LOGGING 'Opening statements complete' LEVEL INFO

    WHILE @round < @max_rounds DO               -- (3) Iterative Reasoning
        LOGGING f'Round {@round} | pro rebuttal ...' LEVEL DEBUG
        -- Pro rebuts Con's latest point
        GENERATE pro_argument(@topic, @con_history) INTO @pro_rebuttal
        @pro_history := @pro_history || '\n---\n' || @pro_rebuttal
-- (4) || not +

        LOGGING f'Round {@round} | con rebuttal ...' LEVEL DEBUG
        -- Con rebuts Pro's latest point
        GENERATE con_argument(@topic, @pro_history) INTO @con_rebuttal
        @con_history := @con_history || '\n---\n' || @con_rebuttal

        @round := @round + 1
        LOGGING f'Round {@round} complete' LEVEL INFO
    END

    LOGGING 'All rounds done - judge deliberating ...' LEVEL INFO

    -- Judge evaluates both sides
    GENERATE judge_debate(@topic, @pro_history, @con_history) INTO @verdict

    LOGGING f'Verdict ready | rounds={@round}' LEVEL INFO
    COMMIT @verdict WITH status = 'complete', rounds = @round   -- (5) Final Judgm
END
```

7.1.3.1 (1) & (2) State Management

In a debate, context is everything. You cannot rebut an argument you haven't seen. We initialize @pro_history and @con_history as strings and "seed" them with the opening statements. This is the "Working Memory" of the workflow.

7.1.3.2 (3) `WHILE` (Iterative Reasoning)

This is the first time we've seen a loop in SPL. The WHILE loop allows the workflow to "think" for multiple turns. - In the first turn, the models establish their positions. - In subsequent turns, they refine their positions based on the opponent's "evidence."

SQL Analogy: A **Recursive CTE** (WITH RECURSIVE). Each iteration of the loop builds upon the result set of the previous iteration.

7.1.3.3 (4) `||` for String Concatenation

Notice @pro_history || '\n---\n' || @pro_rebuttal. The || operator is SPL's unambiguous string concatenation — borrowed from Oracle/PostgreSQL SQL. + is ambiguous (it also performs numeric addition). Use || for strings, + for numbers.

7.1.3.4 (5) LOGGING for Progress Visibility

The LOGGING statements are not decoration. In a multi-round debate, you want to see which round is running and when the judge starts deliberating — especially on a GTX 1080 Ti where each rebuttal takes several seconds. LEVEL INFO shows phase transitions; LEVEL DEBUG shows per-round detail.

7.1.4 Running It

Pick a controversial topic and let the models fight it out:

```
spl run cookbook/11_debate_arena/debate.spl --adapter ollama \
    topic="Remote work is better than office work"
```

Watch the execution trace. You will see the pro_argument and con_argument steps repeating as the loop progresses.

7.1.5 What Just Happened

LLM calls: 2 + (2 * max_rounds) + 1. (With 3 rounds, that's 9 calls)

The "Conductor" (SPL Runtime) managed a dynamic conversation: 1. Set the stage with opening statements. 2. Iteratively "passed the microphone" between the pro and con personas. 3. Appended the new arguments to the cumulative history strings. 4. Terminated the loop after the specified number of rounds. 5. Presented the full "transcript" to the judge for a final verdict.

7.1.6 Reproducibility Note

Debates are highly non-deterministic. A single strong point in the opening statement can shift the entire trajectory of the conversation.

On a **GTX 1080 Ti**, a 3-round debate can take **30–90 seconds**, as each rebuttal depends on the previous generation finishing. This is a sequential process—it cannot be fanned out in parallel because the context of Round 2 depends on the output of Round 1.

7.1.7 When to Use This Pattern

Use the **Debate Arena** pattern when: - **Red Teaming**: Argue against your own product's features or security assumptions to find weaknesses. - **Complex Decision Making**: When you have two valid paths (e.g., "SQL vs. NoSQL") and want to explore the trade-offs in depth. - **Educational Tools**: Create an interactive tutor that can argue multiple sides of a historical or scientific debate.

7.1.8 Exercises

1. **Change the Personas.** (Advanced) Research how to use CREATE FUNCTION (Chapter 9.1) to give the Pro and Con sides specific "backgrounds" (e.g., "You are a CEO" vs. "You are a junior developer").
2. **Add a Moderator.** Insert a step inside the WHILE loop where a "Moderator" model summarizes the debate so far and asks a specific follow-up question.
3. **Variable Rounds.** Modify the workflow to accept @max_rounds from the command line, and try a "Lightning Round" (1 round) vs. an "Exhaustive Debate" (5 rounds).

Chapter 7.2

Multi-Agent Collaboration

7.2.1 The Pattern

In a complex project, you don't ask one person to do everything. You have a Researcher to find the facts, an Analyst to interpret the data, and a Writer to polish the prose. Each role requires a different "mindset" and set of skills. When you try to force an LLM to be all three at once, it often produces a "Jack-of-all-trades" response that is mediocre in every dimension.

Multi-Agent Collaboration is the practice of breaking a workflow into specialized roles (agents) and delegating tasks between them. Each agent is a separate "persona" with its own system instructions and expertise.

The SQL analogy is a **Modular Architecture** or **Database Views**. Instead of one giant, 500-line query, you build specialized views (v_research, v_analysis) and join them together in a final report. Each view does one thing perfectly.

The Multi-Agent Collaboration recipe (section 6.2) demonstrates this modularity. It defines three distinct PROCEDURE blocks—Researcher, Analyst, and Writer—and coordinates them via a central WORKFLOW. This ensures that each phase of the project gets the specialized attention it deserves.

7.2.2 The SPL Approach

This recipe introduces the use of **Procedures as Agents** and the pattern of "Linear Delegation" (passing the output of one agent as the input to the next).

7.2.3 The .spl File (Annotated)

```
-- Recipe 14: Multi-Agent Collaboration
-- Specialized agents (Researcher, Analyst, Writer) collaborate on a report.

PROCEDURE researcher(topic TEXT)                -- (1) Agent 1: The Researcher
DO
    GENERATE research_facts(topic) INTO @facts
    GENERATE identify_key_themes(@facts) INTO @themes
```

```
    COMMIT @facts + '\n\nKey Themes:\n' + @themes
END

PROCEDURE analyst(research TEXT, topic TEXT) -- (2) Agent 2: The Analyst
DO
    GENERATE analyze_trends(research) INTO @trends
    GENERATE assess_risks(research, topic) INTO @risks
    COMMIT 'Trends: ' + @trends + '\n\nRisks: ' + @risks
END

PROCEDURE writer(research TEXT, analysis TEXT, topic TEXT) -- (3) Agent 3: The Wri
DO
    GENERATE draft_report(topic, research, analysis) INTO @draft
    GENERATE critique(@draft) INTO @feedback
    GENERATE revise_report(@draft, @feedback) INTO @final
    COMMIT @final
END

WORKFLOW multi_agent_report
    INPUT: @topic TEXT
DO
    CALL researcher(@topic) INTO @research     -- (4) Linear Delegation
    CALL analyst(@research, @topic) INTO @analysis
    CALL writer(@research, @analysis, @topic) INTO @report

    COMMIT @report WITH status = 'complete'
END
```

7.2.3.1 (1), (2) & (3) Procedures as Agents

In SPL, a PROCEDURE is the perfect container for an "Agent." - It has its own inputs and outputs. - It can contain its own internal logic (like the Writer's "draft-critique-revise" loop). - It can be tested and debugged in isolation before being added to the main workflow.

By separating the Researcher from the Analyst, we prevent "Context Contamination." The Researcher doesn't need to worry about trends or risks; its only job is to find facts.

7.2.3.2 (4) Linear Delegation

The WORKFLOW acts as the Project Manager. It calls the researcher, waits for the facts, and then hands those facts to the analyst. This is the "Waterfall" model of agentic workflows.

SQL Analogy: **Pipelining**. The output of the first stage becomes the filter for the second stage.

7.2.4 Running It

Run the collaborative report on a technical topic:

```
spl run cookbook/14_multi_agent/multi_agent.spl --adapter ollama \
```

```
topic="The impact of quantum computing on cybersecurity"
```

In the execution trace, you will see the distinct "handoffs" between the three procedures.

7.2.5 What Just Happened

LLM calls: ~8-10. (Depending on the internal steps of each procedure)

The "Conductor" (SPL Runtime) managed a professional team: 1. **Fact-Finding**: The Researcher gathered raw data. 2. **Synthesis**: The Analyst turned data into insights. 3. **Polishing**: The Writer turned insights into a human-readable report. 4. **Handoffs**: The runtime ensured that the right context was passed to the right agent at the right time.

7.2.6 Reproducibility Note

This recipe is highly stable because each step is small and focused. By decomposing the task, you reduce the "reasoning load" on the model in any single call.

On a **GTX 1080 Ti**, the total time for a full report is about **45–75 seconds**. While this is slower than a single-prompt summary, the **Quality Improvement** is significant. You get a report that is deeper, more structured, and more accurate.

7.2.7 When to Use This Pattern

Use the **Multi-Agent Collaboration** pattern when: - **High-Stakes Reports**: Whitepapers, strategic plans, or technical documentation. - **Multi-Step Expertise**: When a task requires distinct phases (e.g., "Research → Code → Document"). - **Quality Benchmarking**: When you want to see if adding an "Analyst" or "Critic" step actually improves the final output of your "Writer."

7.2.8 Exercises

1. **Add a "Critic" Agent.** Create a fourth PROCEDURE called critic that reviews the final report and suggests one last round of improvements.
2. **Parallel Research.** (Advanced) If your research can be split into two topics (e.g., "Technical" and "Economic"), use a WITH clause to call two researchers in parallel.
3. **Model Specialization.** Use the USING MODEL clause (Chapter 9.1) to assign different models to different roles (e.g., use a "fast" model for Research and a "strong" model for Analysis).

Chapter 7.3

Ensemble Voting

7.3.1 The Pattern

In the world of statistics and machine learning, an **Ensemble** is a collection of models that work together to solve a single problem. The core idea is simple: if you ask one person a question, they might be wrong. If you ask five people and take the average of their answers, you are much more likely to be right. This is known as "The Wisdom of the Crowd."

For LLMs, this pattern is incredibly powerful for reducing "hallucinations." A single model might confidently state a fact that is slightly off. But if you generate five independent answers, the correct facts will likely appear in the majority, while the hallucinations will be unique to each run and can be "voted out."

The SQL analogy is a **UNION ALL with a GROUP BY and HAVING COUNT(*) > 1**. You are collecting multiple result sets and then filtering for the "consensus" that appears across most of them.

The Ensemble Voting recipe (section 6.3) generates five independent candidate answers, scores each one for accuracy and clarity, identifies the "Consensus" themes across all five, and finally synthesizes a "Winning" response.

7.3.2 The SPL Approach

This recipe demonstrates **Multi-Sample Generation** and **Consensus-Driven Synthesis**.

7.3.3 The .spl File (Annotated)

```
-- Recipe 20: Ensemble Voting
-- Generates multiple independent answers, then uses majority voting to pick the be

WORKFLOW ensemble_voting
    INPUT: @question TEXT
DO
    -- Phase 1: Multi-Sampling (The Crowd)
    GENERATE answer_candidate(@question) INTO @candidate_1 -- (1) Five independent
```

```
    GENERATE answer_candidate(@question) INTO @candidate_2
    GENERATE answer_candidate(@question) INTO @candidate_3
    GENERATE answer_candidate(@question) INTO @candidate_4
    GENERATE answer_candidate(@question) INTO @candidate_5

    -- Phase 2: Scoring (Individual Quality)
    GENERATE score_candidate(@candidate_1) INTO @score_1 -- (2) Measuring each
    GENERATE score_candidate(@candidate_2) INTO @score_2
    ...

    -- Phase 3: Consensus (Finding the Wisdom)
    GENERATE find_consensus(@candidate_1, @candidate_2, ...) INTO @consensus -- (3)

    -- Phase 4: Selection and Polishing
    GENERATE select_winner(@scores, @consensus) INTO @best_candidate
    GENERATE polish(@best_candidate, @consensus) INTO @final_answer -- (4) The Fina

    COMMIT @final_answer WITH status = 'complete', candidates = 5
END
```

7.3.3.1 (1) Phase 1: Multi-Sampling

We run the *exact same* prompt five times. In a stateless environment, we rely on the model's inherent "Temperature" (randomness) to produce slightly different phrasings and perspectives in each run.

7.3.3.2 (2) Phase 2: Scoring

Before we look for consensus, we score each candidate individually. - "Is this answer coherent?" - "Does it address all parts of the question?" - "Are there obvious factual errors?"

7.3.3.3 (3) Phase 3: Finding Consensus

This is the "Voting" step. We pass all five candidates to a "Synthesizer" model and ask it: "What are the common facts and arguments that appear in at least three of these versions?" This identifies the "Grounded Truth" that most of the runs agree on.

7.3.3.4 (4) Phase 4: Polishing

Finally, we take the highest-scoring candidate and "polish" it using the consensus themes. If the winner missed a detail that the other four versions caught, we add it back in. The result is a "Super-Answer" that is more robust than any single model could produce.

7.3.4 Running It

Run the ensemble on a controversial or factual topic:

```
spl run cookbook/20_ensemble_voting/ensemble.spl --adapter ollama \
    question="What are the main causes of the Great Depression?"
```

Observe the output. You will notice that the final answer is exceptionally balanced and thorough, as it has been "filtered" through five independent reasoning passes.

7.3.5 What Just Happened

LLM calls: 13. (5 candidates + 5 scores + 1 consensus + 1 winner + 1 polish)

The "Conductor" (SPL Runtime) managed a "Delphi Method" process: 1. **Surveyed** the crowd (the 5 candidates). 2. **Audited** each response for quality. 3. **Extracted** the consensus agreement. 4. **Selected** the strongest voice. 5. **Enhanced** that voice with the collective wisdom of the group.

7.3.6 Reproducibility Note

Ensemble Voting is the "Gold Standard" for stability. While a single run might vary by 20–30% in its details, the **Ensemble Result** is remarkably consistent. The "noise" of individual runs is cancelled out, leaving only the "signal" of the underlying model's knowledge.

On a **GTX 1080 Ti**, this recipe takes **2–4 minutes**. It is a heavy workload, but for high-accuracy requirements, it is the most reliable tool in the cookbook.

7.3.7 When to Use This Pattern

Use the **Ensemble Voting** pattern when: - **Fact-Critical Queries**: Medical, legal, or historical questions where being "mostly right" is not enough. - **Reducing Hallucination**: When you suspect the model might "make things up" and want to verify facts across multiple samples. - **Consensus Building**: When you need a balanced view of a topic with multiple valid perspectives.

7.3.8 Exercises

1. **Parallel Execution.** (Required for speed) Use the WITH clause (Chapter 9.1) to run all 5 candidates and all 5 scores in parallel blocks.
2. **Diversity Prompting.** Instead of running the same prompt five times, give each candidate a slightly different "Persona" (e.g., "Answer as an economist," "Answer as a historian," "Answer as a sociologist").
3. **Weighted Voting.** Modify the select_winner step to give more "weight" to candidates that have higher individual scores.

Chapter 7.4

Socratic Tutor

7.4.1 The Pattern

Most AI tutors act like talking encyclopedias. You ask "Why is the sky blue?" and they give you a three-paragraph lecture on Rayleigh scattering. This is efficient for information retrieval, but it is terrible for *learning*. Learning happens when a student is forced to think, reason, and arrive at a conclusion themselves.

The **Socratic Tutor** pattern implements the oldest pedagogical technique in history. Instead of explaining, the AI acts as a guide. It asks a question, listens to the student's response, identifies a gap in their logic, and then asks a *better* question to help them close that gap.

The SQL analogy is an **Iterative Refinement Query**. You are querying a "Student Mind" (the user) and using the results to filter and focus your next "Query" (the question), repeating until the "Result Set" (the student's understanding) meets your quality threshold.

The Socratic Tutor recipe (section 6.4) simulates this dialogue. It doesn't just ask questions; it **simulates** a student's responses to test its own effectiveness, scores the student's emerging understanding, and adapts its questioning strategy in real-time.

7.4.2 The SPL Approach

This recipe demonstrates **Persona-Driven Branching**—using a strict system role to constrain the model's behavior while using an EVALUATE block to change the pedagogical strategy.

7.4.3 The .spl File (Annotated)

```
-- Recipe 32: Socratic Tutor
-- Ask guiding questions rather than giving answers directly.

CREATE FUNCTION socratic_persona(student_level TEXT)
RETURNS TEXT AS $$
You are a Socratic tutor.
Core rules:
1. Ask exactly ONE clear question at a time.
```

```
2. NEVER state facts or give direct answers.
3. Build on what the student just said.
Student level: {student_level}
$$;

WORKFLOW socratic_tutor
    INPUT: @topic TEXT, @student_level TEXT DEFAULT 'high school'
DO
    -- Phase 1: Initiation
    GENERATE opening_question(@topic, socratic_persona(@student_level)) INTO @q1

    -- Phase 2: Dialogue Simulation (Testing the logic)
    GENERATE simulate_student(@q1, @student_level) INTO @s1 -- (1) The "Dummy Stude
    GENERATE followup_question(@q1, @s1) INTO @q2
    GENERATE simulate_student(@q2, @student_level) INTO @s2

    -- Phase 3: Assessment
    GENERATE assess_understanding(@s1, @s2) INTO @score    -- (2) Measuring Learnin

    -- Phase 4: Adaptive Branching
    EVALUATE @score
        WHEN > 7 THEN                                      -- (3) High Understanding
            GENERATE consolidation_question(@s2) INTO @q3
        OTHERWISE                                          -- (4) Low Understanding
            GENERATE hint_question(@q2, @s2) INTO @q3
    END

    -- Phase 5: Final Performance
    GENERATE simulate_student(@q3) INTO @s3
    COMMIT compile_dialogue(@q1, @s1, @q2, @s2, @q3, @s3) WITH status = 'complete'
END
```

7.4.3.1 (1) The "Dummy Student" (`simulate_student`)

To build a truly effective tutor, we don't just write prompts; we test them. In this workflow, we use one model instance to act as the Tutor and another to act as a "Simulated Student" at a specific level (e.g., middle school). - **Why?** This allows us to "dry-run" the educational path. If the tutor is too confusing for the simulated student, it will likely be too confusing for a real human.

7.4.3.2 (2) Measuring Learning (`assess_understanding`)

We use a "Judge" prompt to look at the student's two previous answers. - "Are they getting closer to the truth?" - "Are they identifying the right variables?" This step turns the "Vibe" of a conversation into a hard numeric score.

7.4.3.3 (3) & (4) Adaptive Branching

This is where the pedagogy lives. - If the student is succeeding (**Score > 7**), the tutor asks a "Consolidation" question to cement the knowledge. - If the student is struggling (**Otherwise**), the tutor pivots to a "Hint" question to simplify the concept.

SQL Analogy: **Conditional Routing**. You are routing the "Transaction" (the lesson) through different logic paths based on the "Data" (the student's performance).

7.4.4 Running It

Test the tutor on a scientific concept:

```
spl run cookbook/32_socratic_tutor/socratic_tutor.spl \
    --adapter ollama --tools cookbook/32_socratic_tutor/tools.py \
    topic="Why do objects fall at the same speed?" student_level="middle school"
```

Expected output: A three-turn dialogue where the AI guides the student from "gravity pulls everything" toward the more nuanced understanding of mass and acceleration.

7.4.5 What Just Happened

LLM calls: 7. (3 questions + 3 simulated responses + 1 assessment)

The "Conductor" (SPL Runtime) managed a "Pedagogical Loop": 1. **Established** a strict role-play boundary. 2. **Facilitated** a multi-turn reasoning chain. 3. **Monitored** the quality of the student's progress. 4. **Modified** the difficulty level dynamically. 5. **Verified** the final outcome.

7.4.6 Reproducibility Note

The Socratic Tutor is highly dependent on the **Persona Guardrails**. If the model "breaks character" and gives the answer, the learning value is lost. Larger models like **gemma3** are much better at maintaining this discipline than smaller models.

On a **GTX 1080 Ti**, a 3-turn simulation takes **30–50 seconds**. This is a powerful way to generate "Synthetic Training Data" for educational apps or to test different teaching strategies.

7.4.7 When to Use This Pattern

Use the **Socratic Tutor** pattern when: - **Educational Apps**: Building tools that help students think rather than just cheat. - **Corporate Training**: Helping employees discover new processes or compliance rules through reasoning. - **Problem Discovery**: When you have a complex problem and want to "think through it" by having an AI ask you guiding questions.

7.4.8 Exercises

1. **Add a "Subject" Bias.** Modify the socratic_persona to accept a @subject parameter (e.g., "Physics," "History") and observe how it uses different analogies.

2. **Extend the Loop.** Use a WHILE loop (Chapter 7.1) to allow the dialogue to continue for up to 10 turns, or until the @understanding_score reaches 9.
3. **Strict Mode.** Add an EXCEPTION handler that triggers if the model ever says the phrase "The answer is" and forces a RETRY with a stricter safety prompt.

Chapter 7.5

Interview Sim

7.5.1 The Pattern

High-stakes interactions—like job interviews, sales pitches, or crisis management—require more than just knowledge. They require "Grace Under Pressure." The best way to prepare for these moments is through simulation. But human-led simulation is expensive and time-consuming. You need a partner who can play a specific role, stay in character, and provide objective feedback after the session.

The **Interactive Interview Simulation** pattern uses multiple LLM instances to create a "closed-loop" training environment. 1. **Interviewer**: One model instance plays the role of a professional interviewer, asking probing technical questions. 2. **Candidate**: A second model instance (the "Dummy Candidate") answers the questions to test the interview's difficulty. 3. **Evaluator**: A third model instance (the "Judge") reviews the entire transcript and scores it based on a professional rubric.

The SQL analogy is a **Three-Way Join with an Audit Log**. You are joining the "Interviewer Persona" with the "Candidate Persona" over a series of "Turn IDs," and then auditing the resulting "Conversation Table" for quality and consistency.

The Interview Sim recipe (section 6.5) implements this triad. It allows you to simulate technical interviews for any role (Senior SWE, Data Scientist, etc.) and provides a structured evaluation report that highlights strengths and weaknesses.

7.5.2 The SPL Approach

This recipe demonstrates **Multi-Role Orchestration**—managing three distinct personas in a single workflow while using deterministic tools to aggregate scores.

7.5.3 The .spl File (Annotated)

```
-- Recipe 33: Interview Simulator
-- Two-persona structured Q&A with evaluation.

WORKFLOW interview_sim
```

```
    INPUT: @role TEXT, @focus TEXT, @candidate_profile TEXT
DO
    -- Phase 1: Planning
    GENERATE generate_questions(@role, @focus) INTO @questions_json -- (1) Upfront

    -- Phase 2: The Interview (Sequential Generation)
    CALL extract_question(@questions_json, 1) INTO @q1
    GENERATE answer(@q1, @candidate_profile) INTO @a1 -- (2) Role-play generation

    CALL extract_question(@questions_json, 2) INTO @q2
    GENERATE answer(@q2, @candidate_profile) INTO @a2

    -- Phase 3: Evaluation (The Judge)
    GENERATE score(@q1, @a1, evaluation_rubric()) INTO @score1 -- (3) Applying the
    GENERATE score(@q2, @a2, evaluation_rubric()) INTO @score2

    -- Phase 4: Aggregation (Deterministic)
    CALL aggregate_scores(@score1, @score2) INTO @totals -- (4) Python Math Tool

    -- Phase 5: Synthesis
    GENERATE write_report(@totals, @role) INTO @final_report
    COMMIT @final_report WITH status = 'complete'
END
```

7.5.3.1 (1) Upfront Planning (`generate_questions`)

Instead of asking one question at a time, we ask the "Interviewer" model to generate a set of three questions upfront. - **Why?** This ensures the interview has a logical flow (e.g., easy → medium → hard) rather than being a random list of disconnected queries.

7.5.3.2 (2) Role-play Generation

When the "Candidate" model answers, it doesn't just give the "right answer." It answers based on its @candidate_profile. If the profile says "Junior Developer," the model might give a shallow answer or admit it doesn't know. - This creates a **Realistic Simulation**. We aren't just testing the facts; we are testing the interaction.

7.5.3.3 (3) Applying the Rubric (`evaluation_rubric`)

We use a formal JSON rubric to score the answers. This ensures that the evaluation is objective and consistent across different interview runs. We score dimensions like "Technical Accuracy," "Communication," and "Experience."

7.5.3.4 (4) Python Math Tool (`aggregate_scores`)

As always, we hand the numbers back to Python. Python sums the scores, calculates the average, and flags any "Red Flags" found in the JSON feedback. This ensures the final @totals are mathematically perfect.

7.5.4 Running It

Run a simulation for a Senior SWE role:

```
spl run cookbook/33_interview_sim/interview_sim.spl \
    --adapter ollama --tools cookbook/33_interview_sim/tools.py \
    role="Senior Software Engineer" focus="Distributed Systems" \
    candidate_id="alice_senior_swe"
```

Expected output: A full transcript of the three-question interview, followed by a structured evaluation report with a total score (e.g., 32/40) and specific feedback for the candidate.

7.5.5 What Just Happened

LLM calls: 8–10. (Question set + 3 answers + 3 scores + 1 final report) **Tool calls: 5.** (Role loading, Candidate loading, Question extraction, Score aggregation)

The "Conductor" (SPL Runtime) managed a "HR Assessment Center": 1. **Sourced** the correct expertise (Role Context). 2. **Choreographed** a three-turn dialogue between two distinct personas. 3. **Audited** every turn against a professional standard. 4. **Calculated** the performance metrics. 5. **Synthesized** a comprehensive hiring recommendation.

7.5.6 Reproducibility Note

This recipe is highly stable because the questions are generated upfront. The variability comes from the "Candidate's" responses.

On a **GTX 1080 Ti**, a full simulation takes **60–90 seconds**. This is an incredible tool for **Interview Preparation** or for **Hiring Managers** to test their own question sets before using them on real candidates.

7.5.7 When to Use This Pattern

Use the **Interview Sim** pattern when: - **Personal Coaching**: Preparing for a specific job or a high-stakes presentation. - **Content Generation**: Creating "Synthetic Transcripts" for training materials or educational videos. - **Model Evaluation**: Testing how different models perform in a specific persona (e.g., "How does Llama 3 handle being a CEO vs. a Developer?").

7.5.8 Exercises

1. **Add a "Live" mode.** (Advanced) Modify the workflow to replace the @candidate_profile with a context.user_response parameter, allowing a real human to type the answers in a terminal while the AI asks the questions.
2. **Change the Difficulty.** Add a @difficulty parameter to the interviewer_persona and observe how the questions change from "basic syntax" to "architectural trade-offs."
3. **Stress Test.** Give the "Candidate" a profile that says "You are very defensive and don't like being questioned." Observe how the "Interviewer" handles the conflict.

Part 8

Applications

Chapter 8.1

Text2SPL Demo

8.1.1 The Pattern

For many, the biggest obstacle to using SPL 2.0 is the same obstacle they face with SQL: learning the keywords, the structure, and the punctuation. While SPL is designed to be readable, it still requires you to follow a "grammar." But what if you didn't have to? What if you could describe what you wanted in plain English and the system wrote the code for you?

This is the vision of **Text2SPL**. It is a "Compiler for Intent." Instead of writing a .spl file manually, you provide a natural language description—like "Summarize this document and then translate it into Chinese"—and the spl runtime generates the equivalent code.

The SQL analogy is **Natural Language to SQL (NL2SQL)**. Modern data platforms allow analysts to ask "What was the revenue in Q3?" and the system automatically generates the SELECT ... FROM ... WHERE ... query. Text2SPL brings this same "Zero-Barrier" experience to AI orchestration.

The Text2SPL Demo recipe (section 8.1) showcases this capability. It demonstrates how to compile simple prompts, multi-step workflows, and even "Auto-Mode" agents from plain text.

8.1.2 The SPL Approach

This chapter doesn't focus on a .spl file, but rather on the text2spl command built into the spl binary.

8.1.3 The Compilation Process (Annotated)

```
# Example 1: Compiling a simple PROMPT
spl text2spl "summarize a document with a 2000 token budget" \
    --mode prompt -o summarize.spl
```

When you run this command, the runtime: 1. **Analyzes** your intent (summarization + budget constraint). 2. **Identifies** the required SPL constructs (PROMPT, SELECT, GENERATE). 3. **Synthesizes** a valid .spl file. 4. **Validates** the syntax using the internal parser.

```
# Example 2: Compiling a multi-step WORKFLOW
spl text2spl "build a review agent that drafts, critiques, and refines text" \
    --mode workflow -o review_agent.spl
```

In "Workflow Mode," the compiler goes deeper. It creates a WORKFLOW block, defines multiple GENERATE steps, and sets up the variables to pass context between them. It may even define CREATE FUNCTION blocks for the different personas (the writer and the critic).

8.1.4 Running It

You can try the compiler demo yourself using the provided shell script:

```
bash cookbook/22_text2spl_demo/text2spl_demo.sh
```

This script will generate three different .spl files in the generated/ directory based on increasingly complex descriptions. You can then run those files just like any other recipe:

```
# Execute the code generated from natural language
spl run generated/summarize.spl --adapter ollama text="Your document here..."
```

8.1.5 What Just Happened

LLM calls: 1 (to compile).

The "Conductor" (SPL Runtime) acted as a Software Engineer: 1. **Interpreted** human requirements. 2. **Transpiled** those requirements into a formal orchestration language. 3. **Verified** the structural integrity of the generated code. 4. **Delivered** a runnable asset.

This is the "North Star" of the Momagrid project: making the barrier to AI engineering effectively **Zero**.

8.1.6 Reproducibility Note

The reliability of Text2SPL depends on the **Compiler Model**. For simple prompts, small models like **gemma3** are 95%+ accurate. For complex workflows involving WHILE loops and EXCEPTION handling, we recommend using a "Senior" model like **Claude 3.5 Sonnet** as the compiler.

Once the code is compiled and validated, the **Execution** is perfectly deterministic. The "magic" only happens once at compile time; after that, you have a solid, version-controlled .spl file.

8.1.7 When to Use This Pattern

Use the **Text2SPL** pattern when: - **Rapid Prototyping**: You have an idea for a workflow and want a "First Draft" of the code in seconds. - **Onboarding**: You are new to SPL and want to see how the system would structure a specific task. - **Dynamic Agents**: When your application needs to generate *new* workflows on the fly based on user requests.

8.1.8 A Live Session: The Intent Router

The following is not a constructed example. It is a real compilation run from a live development session on the SPL 2.0 Knowledge Studio.

The input was a single line:

```
classify user intent and route to the right handler
```

The compiler was Claude Sonnet 4.6 in auto mode. The generated script:

```
WORKFLOW intent_router
    INPUT @user_input TEXT, @context TEXT DEFAULT ''
    OUTPUT @handler_response TEXT
DO
    -- Step 1: Classify the user intent
    GENERATE classify(@user_input, intent_categories()) INTO @intent

    -- Step 2: Extract any entities or parameters needed by handlers
    GENERATE extract_entities(@user_input) INTO @entities

    -- Step 3: Route to the appropriate handler
    EVALUATE @intent
        WHEN 'question' THEN
            GENERATE answer(@user_input, @context, @entities) INTO @handler_respons
            COMMIT @handler_response WITH handler = 'qa', intent = @intent

        WHEN 'command' THEN
            GENERATE execute_command(@user_input, @entities) INTO @handler_response
            COMMIT @handler_response WITH handler = 'command', intent = @intent

        WHEN 'complaint' THEN
            GENERATE handle_complaint(@user_input, @entities) INTO @handler_respons
            COMMIT @handler_response WITH handler = 'complaint', intent = @intent

        WHEN 'chitchat' THEN
            GENERATE casual_reply(@user_input) INTO @handler_response
            COMMIT @handler_response WITH handler = 'chitchat', intent = @intent

        WHEN 'feedback' THEN
            GENERATE acknowledge_feedback(@user_input, @entities) INTO @handler_res
            COMMIT @handler_response WITH handler = 'feedback', intent = @intent

        OTHERWISE
            GENERATE fallback_response(@user_input, @intent) INTO @handler_response
            COMMIT @handler_response WITH handler = 'fallback', intent = @intent
    END

EXCEPTION
    WHEN HallucinationDetected THEN
```

```
        RETRY WITH temperature = 0.1
    WHEN RefusalToAnswer THEN
        GENERATE safe_fallback(@user_input) INTO @handler_response
        COMMIT @handler_response WITH handler = 'safe_fallback'
END;

CREATE FUNCTION intent_categories()
RETURNS TEXT AS $$
Supported intent categories:
- question:   user is asking for information or clarification
- command:    user is requesting an action to be performed
- complaint:  user is expressing dissatisfaction or reporting a problem
- chitchat:   casual or social conversation with no specific goal
- feedback:   user is providing a review, rating, or suggestion
- unknown:    intent cannot be determined with confidence
$$;
```

This was compiled from eleven words. The compiler inferred:

- A three-step pipeline: classify, extract entities, then route
- Six intent categories — including an unknown fallback
- Per-handler COMMIT statements that tag the output with handler metadata
- EXCEPTION handling for HallucinationDetected and RefusalToAnswer — AI-specific failure modes the compiler invented on its own
- A CREATE FUNCTION intent_categories() that injects domain knowledge as structured system context

The script was then run against Gemma3 (local, via Ollama) with the input:

```
The app keeps crashing every time I try to upload a file, this is really frustratin
```

Gemma3 correctly routed it to the complaint handler, called handle_complaint, and produced a structured troubleshooting response. The full round-trip — compile on Claude, execute on Gemma3 — worked without modification.

8.1.9 The Bootstrap Insight

The intent router is not production-ready. The intent categories are generic. The handler logic is shallow. A real deployment would need domain-specific categories, tighter prompts, and validation of the routing decisions.

But that is exactly the point.

"It may not be useful out-of-the-box, but it bootstraps the further iteration and refinement process. One can try various LLM models, add new logic — this is why declarative programming is so powerful." — Wen Gong

This is the correct frame for Text2SPL. The compiled script is not the destination. It is a **scaffold** — a structured starting point that encodes the compiler's best interpretation of your intent. From there, the developer's job shifts from writing code to refining a spec.

The iteration loop looks like this:

```
describe → compile → inspect → refine → iterate
```

Compare it to the imperative alternative:

```
design → code → debug → refactor → re-debug → iterate
```

In the imperative loop, every iteration requires touching implementation details: which API, which prompt format, which response accessor, which retry logic. In the SPL loop, every iteration is a conversation about intent. You change what the workflow does, not how it talks to a model.

The bootstrapped script also encodes structural knowledge that survives iteration. The compiler inferred that a complaint handler needs different logic than a chitchat handler. It inferred that entity extraction should precede routing so that handlers have structured input to work with. It inferred that AI workflows need typed exception handling. These are good decisions. They do not get thrown away when you refine the domain vocabulary. They become the scaffold on which the domain-specific version is built.

The script is the spec. EVALUATE @intent WHEN 'complaint' reads like a requirements document. When a product manager says "add a billing intent," you know exactly where to add one line — no hunting through callback chains, prompt templates, or routing middleware.

The model is configurable. The same script ran on Claude during compilation and Gemma3 during execution. Tomorrow you can point the runtime at GPT-4o or Mistral for a quality comparison, with zero code changes. The adapter is runtime config, not source code.

The version history is free. The Knowledge Studio saved this as intent_router v1. When you refine the intent_categories() function or add a billing handler, that becomes v2 — a natural experiment log with no extra tooling required.

This is the same reason SQL outlasted every "better" imperative data access approach. The abstraction is at the right level. SPL 2.0 is that abstraction for agentic workflows. The models are the orchestra. SPL is the score. The score says what to play. Text2SPL gives you a first draft of the score from a description of the music you want to hear.

8.1.10 Exercises

1. **Describe a "Self-Healer".** Try compiling the description: *"Generate a blog post, check it for facts, and if facts are missing, search the web and rewrite."* Use --mode auto and see if it generates a WHILE loop.
2. **Fix and Re-run.** Take a generated .spl file that has a minor syntax error. Fix the error manually and run it. This is the "Human-in-the-loop" coding pattern.
3. **Model Showdown.** Use two different compiler models (e.g., phi4 vs gemma3) to compile the same description. Compare the "coding style" of each model.

8.1.11 Want a Visual Interface?

The CLI compiler is powerful, but if you prefer working in a browser, the **Recipe Kitchen** (Chapter 11.2) provides a full web UI for Text2SPL. You can describe a workflow in plain English, compile it,

inspect the generated SPL, and run it — all from a browser tab, no terminal required. The Recipe Kitchen is SPL's answer to "low-code": the same declarative language, with a graphical front door.

Chapter 8.2

Map-Reduce Summarizer

8.2.1 The Pattern

Every LLM has a context window — a limit on how much text it can process at once. Feed a 50-page technical report into a model with a small window, and the model will forget the beginning or fail entirely. Even with large-context models, massive documents suffer from "lost-in-the-middle" syndrome: key details buried in the center get ignored.

Map-Reduce is the classic distributed computing solution. It appears in Google's 2004 paper, in Hadoop, in Spark — and it maps directly to the LLM problem:

1. **Map**: Split the large document into chunks. Summarize each chunk independently.
2. **Reduce**: Combine those summaries into a single, cohesive final report.

The SQL analogy is a GROUP BY with an aggregate function:

```
SELECT REDUCE(summary)
FROM (
    SELECT MAP(chunk) AS summary
    FROM document_chunks
) AS subquery;
```

You are aggregating many small insights into one large conclusion.

8.2.2 Design Thinking: The Efficient Runtime

section 8.1 is the clearest illustration of SPL's most important design principle: **separate deterministic work from probabilistic work**.

Every operation in an AI workflow belongs to one of two categories:

Category	Characteristic	SPL keyword	Token cost
Deterministic	Can be expressed as code — precise, reproducible	CALL	Zero

Category	Characteristic	SPL keyword	Token cost
Probabilistic	Requires judgment, generation, or ambiguity	GENERATE	LLM tokens

In an earlier version of this recipe, chunk_plan() and extract_chunk() were both GENERATE calls. That meant: every time you ran the recipe, you paid LLM tokens to count words and slice a string — operations that a five-line Python function does perfectly.

The fix is not an optimization. It is a correctness decision. Counting words has one right answer. There is no reason to ask a language model for it.

> *"If you can write it as code — write it as code. Reserve GENERATE for what only a language model can do."* — SPL Design Principles, Section 4

This is the same insight that made PL/SQL successful in the Oracle era. SQL declared *what* you wanted; PL/SQL let you write the *how* in the same language. SPL 2.0 does the same for GenAI: GENERATE declares what the LLM should do; CALL handles everything the LLM shouldn't have to.

8.2.3 The SPL File (Annotated)

```
-- Recipe 13: Map-Reduce Summarizer
-- Tools: chunk_plan(), extract_chunk() run deterministically (CALL, zero tokens).
--        summarize_chunk(), reduce_summaries(), quality_score(), improve_summary()
-- Logs written to @log_dir:
--   chunk_N.md, summary_N.md --  per-chunk artifacts
--   final_summary.md --          combined result

WORKFLOW map_reduce_summarizer
    INPUT:
        @document TEXT,
        @style TEXT,
        @log_dir TEXT DEFAULT '13_map_reduce/logs'
    OUTPUT: @final_summary TEXT
DO
    @chunk_index := 0
    @summaries := []            -- LIST: accumulates per-chunk summaries
(1)

    LOGGING 'Starting map-reduce | document length: ' + @document

    -- Step 1: Determine how to split the document (deterministic, 0 tokens)
    CALL chunk_plan(@document) INTO @chunk_count
-- (2)

    LOGGING 'Document split into ' + @chunk_count + ' chunks'
```

```
    -- Step 2: MAP -- summarize each chunk independently
    WHILE @chunk_index < @chunk_count DO
        CALL extract_chunk(@document, @chunk_index, @chunk_count) INTO @chunk
-- (3)

        CALL write_file(f'{@log_dir}/chunk_{@chunk_index}.md', @chunk) INTO @_
-- (4)

        GENERATE summarize_chunk(@chunk, @chunk_index) INTO @chunk_summary

        CALL write_file(f'{@log_dir}/summary_{@chunk_index}.md', @chunk_summary) IN

        LOGGING f'[Chunk {@chunk_index}/{@chunk_count}] summary saved'
-- (5)

        @summaries := list_append(@summaries, @chunk_summary)
-- (6)
        @chunk_index := @chunk_index + 1
    END

    -- Step 3: REDUCE -- concat all chunk summaries, then combine into final
    @summaries_text := list_concat(@summaries, '\n')
-- (7)
    GENERATE reduce_summaries(@summaries_text, @style) INTO @final_summary

    -- Step 4: Quality check the final summary
    GENERATE quality_score(@final_summary, @document) INTO @score

    EVALUATE @score
        WHEN > 0.7 THEN
            CALL write_file(f'{@log_dir}/final_summary.md', @final_summary) INTO @_
            LOGGING f'Final summary saved (score={@score})'
            COMMIT @final_summary WITH status = 'complete', chunks = @chunk_count
        ELSE
            GENERATE improve_summary(@final_summary, @summaries_text) INTO @final_s
            CALL write_file(f'{@log_dir}/final_summary.md', @final_summary) INTO @_
            LOGGING f'Improved summary saved (score={@score})'
            COMMIT @final_summary WITH status = 'refined', chunks = @chunk_count
    END

EXCEPTION
    WHEN ContextLengthExceeded THEN
        @summaries_text := list_concat(@summaries, '\n')
        GENERATE reduce_summaries(@summaries_text, @style) INTO @final_summary
        CALL write_file(f'{@log_dir}/final_summary.md', @final_summary) INTO @_
        COMMIT @final_summary WITH status = 'partial'
    WHEN BudgetExceeded THEN
```

```
        @summaries_text := list_concat(@summaries, '\n')
        GENERATE reduce_summaries(@summaries_text, @style) INTO @final_summary
        CALL write_file(f'{@log_dir}/final_summary.md', @final_summary) INTO @_
        COMMIT @final_summary WITH status = 'budget_limit'
END
```

8.2.3.1 (1) Native LIST — `@summaries := []`

SPL 2.0 has a native LIST type. [] is an empty list. You do not need to build a delimiter-separated string and split it later — a pattern that breaks the moment a summary contains your chosen delimiter.

```
@summaries := list_append(@summaries, @chunk_summary)   — append
@text      := list_concat(@summaries, '\n')              — join to string
@n         := list_count(@summaries)                     — length
```

All list operations are deterministic built-ins — zero tokens.

8.2.3.2 (2) CALL for `chunk_plan` — Zero Tokens

chunk_plan(@document) counts words and calculates how many 800-word chunks the document needs. This is arithmetic. No LLM judgment required. CALL runs the registered Python tool directly, no API call, no latency, no cost.

Compare with the old approach:

```
GENERATE chunk_plan(@document) INTO @chunk_count   —   wastes tokens on arithmetic
CALL chunk_plan(@document) INTO @chunk_count       —   deterministic, 0 tokens
```

8.2.3.3 (3) CALL for `extract_chunk` — Zero Tokens

Same principle: slicing words[start:end] is pure computation. The Python tool does it in microseconds.

8.2.3.4 (4) `write_file` — Built-in File I/O

write_file(path, content) is a SPL 2.0 built-in — no −−tools flag needed, no tools.py entry required. It creates parent directories automatically and returns "written: <path>".

The result is discarded with INTO @_ — the SPL equivalent of Python's _ = fn(). Explicit, readable, intentional.

```
CALL write_file(f'{@log_dir}/chunk_{@chunk_index}.md', @chunk) INTO @_
```

8.2.3.5 (5) F-String Interpolation

f'{@log_dir}/chunk_{@chunk_index}.md' is an SPL f-string. The {@varname} placeholders are substituted at runtime. Borrowed from Python, adapted for SPL's @variable convention.

This is also why @log_dir is an INPUT parameter with a default — the caller controls the log destination. The recipe never hardcodes a path.

8.2.3.6 (6) list_append — Accumulate, Then Join

The map phase builds a LIST. The reduce phase joins it into a single string once, then passes that string to the LLM. This is cleaner than string concatenation inside the loop:

```
— Old: fragile, delimiter-dependent
@summaries := @summaries + '\n——\n' + @chunk_summary

— New: clean accumulation, clean join
@summaries := list_append(@summaries, @chunk_summary)
— ... later ...
@summaries_text := list_concat(@summaries, '\n')
```

8.2.3.7 (7) list_concat — The Bridge Between LIST and TEXT

list_concat(@summaries, '\n') joins the list elements with a newline separator, producing a single TEXT value that the LLM can process. The separator is a parameter — use '\n———\n' for section breaks, ', ' for inline lists, whatever the downstream prompt expects.

8.2.4 Running It

```
spl run cookbook/13_map_reduce/map_reduce.spl \
    --tools cookbook/13_map_reduce/tools.py \
    --adapter ollama -m gemma3 \
    document="$(cat large_report.txt)" \
    style="executive brief"
```

After the run, inspect the logs/ folder:

```
13_map_reduce/logs/
  chunk_0.md ←        raw extracted text, chunk 0
  chunk_1.md
  summary_0.md ←      LLM summary for chunk 0
  summary_1.md
  final_summary.md ← combined, quality-checked result
```

The log artifacts are the recipe's debugging story. If the final summary misses something, open chunk_N.md to see what the model was given and summary_N.md to see what it produced. The problem is localized immediately.

8.2.5 What Just Happened

LLM calls: 0 (plan) + 0 (extract) + N (map) + 1 (reduce) + 1 (score) + 0–1 (refine).

Token cost is concentrated exactly where intelligence is required. The two most frequent operations in the loop — chunking and extraction — are zero-token CALLs. Every GENERATE token is spent on language understanding.

The Conductor (SPL Runtime) managed a classic data pipeline: 1. **Decomposed** the document deterministically (CALL — zero cost) 2. **Processed** each chunk with the LLM (GENERATE — focused cost) 3. **Aggregated** summaries via LIST → list_concat → GENERATE 4. **Validated** and optionally refined the result 5. **Logged** every artifact for inspection

---

8.2.6 Reproducibility Note

The chunking is deterministic — same document, same chunk boundaries, every run. This makes the Map phase reproducible: if summarize_chunk is called with the same text, the LLM output will vary slightly by temperature, but the input is always identical. This separates the non-determinism (LLM) from the determinism (chunking), which is the right structure for debugging and benchmarking.

On a **GTX 1080 Ti**, a 5-chunk document takes about **30–60 seconds**. Time scales linearly with chunk count — predictable, budgetable.

8.2.7 When to Use This Pattern

- **Large documents**: Anything longer than ~2,000 words (adjust _TARGET_CHUNK_WORDS in tools.py)
- **Batch processing**: Summarize hundreds of files with consistent structure
- **Hierarchical summarization**: Nested Map-Reduce — summarize chapters → parts → book
- **Any accumulate-then-reduce pattern**: Feedback aggregation, code review across files, multi-document synthesis

---

8.2.8 Exercises

1. **Change the chunk size.** Edit _TARGET_CHUNK_WORDS in tools.py from 800 to 300. How does the number of chunks and quality of summaries change?
2. **Add a @log_dir override.** Run with log_dir="my_logs" and confirm the artifacts land in the right place.
3. **Read back a chunk.** Add CALL read_file(f'{@log_dir}/chunk_0.md') INTO @first_chunk after the loop and LOGGING its first 100 characters. Use truncate(@first_chunk, 100).
4. **Parallel Map.** (Advanced) Research how SPL's WITH CTEs can dispatch multiple summarize_chunk calls concurrently — removing the sequential bottleneck in the WHILE loop.

Chapter 8.3

Code Review

8.3.1 The Pattern

In professional software development, code review is the primary defense against bugs, security vulnerabilities, and technical debt. But manual review is slow, subjective, and prone to human fatigue. A reviewer might catch a logic bug but miss a subtle security flaw, or focus on code style while overlooking a performance bottleneck.

Automated Multi-Pass Review solves this by breaking the review into specialized "audits." Instead of one generic "review this code" prompt, you run the code through several distinct passes: 1. **Security Audit**: Look specifically for injections, leaked secrets, and unsafe functions. 2. **Performance Check**: Identify inefficient loops, redundant allocations, and slow I/O. 3. **Style & Best Practices**: Verify adherence to language-specific conventions (e.g., PEP 8 for Python). 4. **Bug Detection**: Search for logic errors, edge cases, and unhandled exceptions.

The SQL analogy is **Multi-Constraint Validation** or **Static Analysis Rules**. You don't just check if the data "looks okay"; you run it against a suite of specific rules (foreign keys, check constraints, unique indexes) to ensure total integrity.

The Code Review recipe (section 8.3) implements this multi-pass strategy. It scores each category for "Severity" and uses those scores to decide whether to approve, request_changes, or block the code—bringing the discipline of a senior engineer to an automated pipeline.

8.3.2 The SPL Approach

This recipe demonstrates **Pass-by-Pass Analysis** and **Threshold-Based Branching** using numeric evaluation.

8.3.3 The .spl File (Annotated)

```
-- Recipe 15: Automated Code Review
-- Multi-pass code review: security, performance, style, then synthesis.

WORKFLOW code_review
    INPUT: @code TEXT, @language TEXT
```

```
DO
    -- Pass 1-4: Specialized Audits
    GENERATE security_audit(@code, @language) INTO @security_findings -- (1) Speci
    GENERATE performance_review(@code, @language) INTO @perf_findings
    GENERATE style_review(@code, @language) INTO @style_findings
    GENERATE bug_detection(@code, @language) INTO @bug_findings

    -- Severity scoring (Turning prose into numbers)
    GENERATE severity_score(@security_findings) INTO @sec_score
-- (2) Quantifying Risk
    GENERATE severity_score(@perf_findings) INTO @perf_score
    GENERATE severity_score(@bug_findings) INTO @bug_score

    -- Synthesize all findings
    GENERATE synthesize_review(...) INTO @review

    -- Phase 5: Automated Verdict
    EVALUATE @sec_score
-- (3) Threshold Branching
        WHEN > 8 THEN
            COMMIT @review WITH status = 'critical_issues', verdict = 'block'
        WHEN > 5 THEN
            COMMIT @review WITH status = 'needs_fixes', verdict = 'request_changes'
        OTHERWISE
            COMMIT @review WITH status = 'approved', verdict = 'approve'
    END
END
```

8.3.3.1 (1) Specialized Passes

Each GENERATE call uses a focused prompt. By asking the model to *only* look for security issues, we reduce the noise from style or performance concerns. This results in much higher "Recall" (finding actual bugs) than a single-pass review.

8.3.3.2 (2) Quantifying Risk (`severity_score`)

LLMs are great at prose, but systems are better at numbers. We use a middle step to "quantify" the findings. We ask the model: "On a scale of 1–10, how severe are these security findings?" This turns a subjective text block into a objective metric that the WORKFLOW can act upon.

8.3.3.3 (3) Threshold Branching

This is where the automation takes action. We use the @sec_score to drive the EVALUATE block. - If a critical security flaw is found (score > 8), we block the code immediately. - If issues are moderate, we request_changes. - Only if the score is low do we approve.

SQL Analogy: A **Trigger with a Threshold**. If a transaction amount exceeds a limit, the trigger blocks the insert; otherwise, it allows it to proceed.

8.3.4 Running It

Review a Python file with a known security flaw:

```
spl run cookbook/15_code_review/code_review.spl --adapter ollama \
    code="def run_cmd(user_input): eval(user_input)" \
    language="Python"
```

Expected result: The security_audit will flag the eval() call, the @sec_score will be high (likely 9 or 10), and the final verdict will be block.

8.3.5 What Just Happened

LLM calls: 8. (4 audits + 3 scores + 1 synthesis)

The "Conductor" (SPL Runtime) acted as a Senior Reviewer: 1. **Decomposed** the review into four logical dimensions. 2. **Analyzed** the code from four different perspectives. 3. **Measured** the risk in each dimension. 4. **Synthesized** a final report. 5. **Decided** on a binding verdict based on the measured risk.

8.3.6 Reproducibility Note

The scoring step (severity_score) is the most sensitive part of this recipe. Different models have different "scales" of what they consider severe.

On a **GTX 1080 Ti**, a full 8-call review takes **60–120 seconds**. This is slower than a human scan but much faster (and more thorough) than a human deep-dive review.

8.3.7 When to Use This Pattern

Use the **Code Review** pattern when: - **Pre-Commit Hooks**: Automatically block unsafe code from being committed to your repository. - **CI/CD Quality Gates**: Add a "Generative Linting" step to your build pipeline. - **Learning Tool**: Use it to get feedback on your own code as you learn a new language.

8.3.8 Exercises

1. **Add a "Fixer" step.** If the verdict is request_changes, add a step that tries to generate a "Fixed" version of the code and includes it in the final review.
2. **Language-Specific Rules.** Modify the security_audit prompt to look for specific flaws in specific languages (e.g., "Buffer overflows" in C++, "SQL Injection" in PHP).
3. **Parallel Audits.** (Advanced) Use the WITH clause (Chapter 9.1) to run all four audit passes in parallel, significantly reducing the total execution time.

Chapter 8.4

Structured Output

8.4.1 The Pattern

One of the most powerful features of modern LLMs is their ability to perform **Data Extraction**. You can give them a pile of unstructured text—a lease agreement, a medical report, a customer email—and ask them to "Find the people, the amounts, and the dates."

But if you just ask the model for "JSON," you run a high risk of getting different field names every time. One model might return {"price": 100}, another {"cost": 100}, and a third might return the price as a string: {"price": "$100"}. This inconsistency is the "Data Engineering" nightmare of the AI age.

The **Structured Output** pattern solves this by using a **Schema**. You don't just ask for JSON; you provide a formal definition (usually a JSON Schema) and tell the model: "Your response must match this exact structure, with these exact types."

The SQL analogy is a **Table Schema** or a **CHECK Constraint**. You don't just "Insert data" into a database. You define columns, types (INTEGER, TEXT, BOOLEAN), and constraints. The database ensures that every row matches that schema, and SPL ensures that every LLM response matches your schema.

The Structured Output recipe (section 8.5) demonstrates how to use CREATE FUNCTION to define a complex schema for multi-entity extraction.

8.4.2 The SPL Approach

This recipe uses a **Schema-Driven Prompt** pattern, where the output format is a first-class citizen in the SELECT context.

8.4.3 The .spl File (Annotated)

```
-- Recipe 23: Structured Output
-- Extract typed data from text using JSON schema.

CREATE FUNCTION extract_entity_schema()        -- (1) Reusable Schema Definition
```

```
RETURNS JSON AS $$
{
  "type": "object",
  "properties": {
    "people": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "name":   { "type": "string" },
          "age":    { "type": "integer" },   -- (2) Enforcing numeric types
          "salary": { "type": "number" }
        }
      }
    }
  }
}
$$;

PROMPT extract_entities
SELECT
    system_role('You are a precise data extraction engine.'),
    context.text AS text,
    extract_entity_schema() AS schema       -- (3) Schema context injection
GENERATE structured_extraction(text, schema) -- (4) Constrained generation
```

8.4.3.1 (1) Reusable Schema Definition

We define the JSON Schema inside a CREATE FUNCTION. This is the "Data Blueprint." By separating it from the prompt, we make the schema reusable across multiple different extraction tasks (e.g., from emails, from PDFs, from web scrapes).

8.4.3.2 (2) Enforcing Numeric Types

Notice the "type": "integer" for age. By providing this hint to the model (and the adapter), we ensure that the output is a raw number (e.g., 42) rather than a string ("42"). This means you can immediately use the output in a calculation without an extra int() step in Python.

8.4.3.3 (3) Schema Context Injection

We include the schema in the context row of our SELECT clause. This ensures the model always has the "Rules of the Game" visible while it is thinking.

8.4.3.4 (4) Constrained Generation

The GENERATE step takes the text and the schema. Depending on the adapter (e.g., OpenRouter or OpenAI), SPL uses the provider's "Structured Output" feature to force the model to comply with the schema. If the model tries to return invalid JSON, the runtime (or the provider) will often catch it and retry automatically.

8.4.4 Running It

Run the extractor on a messy string:

```
spl run cookbook/23_structured_output/structured_output.spl --adapter ollama \
    text="John Smith, 42, joined Acme Corp earning $95,000/year"
```

Expected output (clean JSON):

```
{
  "people": [
    { "name": "John Smith", "age": 42, "salary": 95000 }
  ],
  "organizations": [
    { "name": "Acme Corp" }
  ]
}
```

8.4.5 What Just Happened

LLM calls: 1.

The "Conductor" (SPL Runtime) managed the "Data Capture": 1. **Prepared** the input text. 2. **Attached** the formal data blueprint (the schema). 3. **Instructed** the model to operate as a "Transpiler" from prose to JSON. 4. **Verified** (at the adapter level) that the response was valid and typed.

This is how you build a **Production-Ready Data Pipeline**. By the time the data reaches your downstream application, it is already cleaned, typed, and structured.

8.4.6 Reproducibility Note

Structured output is one of the most stable features of modern LLMs. Even on small models like **gemma3**, providing a formal JSON Schema increases reliability significantly. It reduces the model's "Creative Freedom" and forces it into a "Deterministic Format."

On a **GTX 1080 Ti**, structured generation is just as fast as unstructured generation, but it saves you the "Latency of Error" (the time spent dealing with broken JSON or missing fields).

8.4.7 When to Use This Pattern

Use the **Structured Output** pattern when: - **ETL Pipelines**: Extracting data from thousands of documents into a database. - **Form Filling**: Turning a voice transcript into a structured customer record. - **Workflow Inputs**: When the output of one step needs to be precisely parsed as the input for a calculation or a tool call (Chapter 3.2).

8.4.8 Exercises

1. **Add a "Required" Field.** Modify the schema to make the salary field required. Run it on text that *doesn't* have a salary and see how the model behaves (does it guess? does it return null?).
2. **Extract an Invoice.** Update the schema to include invoice_number and date. Run it on the invoice text provided in the cookbook.
3. **Cross-Schema Validation.** Try giving the model two different schemas in the same SELECT and ask it to choose the one that fits the text best.

Chapter 8.5

Data Extraction

8.5.1 The Pattern

Most data extraction tutorials use "perfect" examples—clean, short sentences where the information is easy to find. In the real world, data is buried in noisy, unstructured text: emails with three different addresses, invoices with cryptic line items, or contracts with hundreds of clauses.

Schema-Constrained Extraction is the professional way to handle this noise. Instead of asking the model to "find the data," you provide a **dynamic schema** that changes based on the *type* of document you are processing.

The SQL analogy is **Dynamic Schema Mapping** or a **Pivot Table**. You are taking a raw "blob" of text and forcing it into a predefined relational structure (rows and columns) that your downstream database can understand.

The Data Extraction recipe (section 8.6) implements this pattern. It uses a CREATE FUNCTION block to deliver different extraction schemas based on a format parameter (e.g., invoice, contract, or general). This allows a single workflow to handle a wide variety of business documents with high precision.

8.5.2 The SPL Approach

This recipe demonstrates **Conditional Schema Injection**—choosing the data blueprint at runtime based on the input type.

8.5.3 The .spl File (Annotated)

```
-- Recipe 27: Data Extraction
-- Pull structured fields from messy text using dynamic schemas.

CREATE FUNCTION extraction_schema(format TEXT DEFAULT 'general')
RETURNS JSON AS $$
SELECT CASE format                                  -- (1) The Blueprint Library
  WHEN 'invoice' THEN '{
    "type": "object",
```

```
    "properties": {
      "invoice_number": {"type": "string"},
      "amount":         {"type": "number"},
      "vendor":         {"type": "string"}
    }
  }'
  WHEN 'contract' THEN '{
    "type": "object",
    "properties": {
      "parties":        {"type": "array", "items": {"type": "string"}},
      "value":          {"type": "number"}
    }
  }'
  ELSE '{ ... general schema ... }'          -- (2) Fallback schema
END
$$;

PROMPT extract_fields
SELECT
    system_role('You are a data extraction specialist.'),
    context.text AS text,
    extraction_schema(context.format) AS schema -- (3) Injecting the right blueprin
GENERATE extract(text, schema)                -- (4) Precision extraction
```

8.5.3.1 (1) & (2) The Blueprint Library

Instead of writing three different .spl files for invoices, contracts, and emails, we keep all our "Data Blueprints" in one function. This makes it easy to maintain a consistent extraction standard across your entire organization.

8.5.3.2 (3) Injecting the right blueprint

In the SELECT clause, we pass the user-provided format into the extraction_schema function. - If the user says format="invoice", the LLM sees the invoice schema. - If they say format="contract", it sees the contract schema.

This is the "Just-in-Time" context pattern that keeps the model focused only on the fields that matter for the current document.

8.5.3.3 (4) Precision Extraction

The GENERATE step uses the schema to constrain the output. Because we explicitly defined amount as a "number", the model will strip out currency symbols and commas, returning a value that can be directly inserted into a DECIMAL column in a SQL database.

8.5.4 Running It

Extract data from a noisy payment request:

```
spl run cookbook/27_data_extraction/data_extraction.spl --adapter ollama \
    text="Please process payment of USD 4,250.00 to Riverside Consulting (ref: PO-
    format="invoice"
```

Expected output:

```
{
  "invoice_number": "PO-8821",
  "vendor": "Riverside Consulting",
  "amount": 4250.00,
  "currency": "USD"
}
```

8.5.5 What Just Happened

LLM calls: 1.

The "Conductor" (SPL Runtime) managed the "Factory Line": 1. **Categorized** the work using the format parameter. 2. **Selected** the correct extraction tool (the schema). 3. **Filtered** the noisy input text through that tool. 4. **Delivered** a perfectly structured data object.

8.5.6 Reproducibility Note

The stability of extraction is highly dependent on the **Schema Complexity**. If your schema has 50 fields, small models might miss some. For complex extraction, we recommend using **gemma3 (27B)** or **Claude 3.5 Sonnet**.

On a **GTX 1080 Ti**, a standard invoice extraction takes **2–4 seconds**. The bottleneck is the number of tokens in the input document, not the complexity of the schema itself.

8.5.7 When to Use This Pattern

Use the **Data Extraction** pattern when: - **Legacy Migration**: You have thousands of old text files that need to be moved into a modern database. - **Workflow Automation**: When a human email needs to trigger an automated business process (like paying a bill). - **Audit Compliance**: When you need to ensure that every document processed by your system contains specific required fields.

8.5.8 Exercises

1. **Add a "Sentiment" Field.** Modify the "general" schema to include a tone field (e.g., "urgent," "polite," "angry") so you can prioritize the extracted data.
2. **Handle Multi-Entity.** Modify the "invoice" schema to include an array of line_items, each with a description and a price.
3. **Strict Mode.** (Advanced) Research the "additionalProperties": false flag in JSON Schema and see how it prevents the model from adding extra, unrequested fields to the output.

Chapter 8.6

Support Triage

8.6.1 The Pattern

Customer support is the "front line" of any business. But manually triaging thousands of tickets is exhausting. A human has to read the ticket, find the customer's order in a database, decide how urgent it is, and then draft a polite response. If the human is tired, they might miss an urgent complaint or cite the wrong order status.

Grounded Triage is the pattern of using deterministic tools to "feed" the LLM the facts it needs *before* it starts reasoning. 1. **Extract**: Find order IDs or account numbers in the ticket text. 2. **Lookup**: Fetch the actual record from your database (or a JSON file). 3. **Reason**: Use the model to classify the ticket and score its urgency, using the real record as the "Ground Truth." 4. **Execute**: Either escalate the ticket immediately or draft a response that cites the verified facts.

The SQL analogy is a **LEFT JOIN with a CASE Statement**. You are joining the "Incoming Ticket" (unstructured) with your "Orders Table" (structured) and then using logic to route the result to the right "Department" (table/outcome).

The Support Triage recipe (section 8.7) implements this full end-to-end pipeline. It shows how SPL can coordinate between Python scripts (for database lookups) and LLMs (for drafting and classification).

8.6.2 The SPL Approach

This recipe introduces the concept of **Grounding Context**—passing verified data into a GENERATE call to prevent hallucinations.

8.6.3 The .spl File (Annotated)

```
-- Recipe 28: Customer Support Triage
-- Classify → route → draft response grounded in real data.

WORKFLOW support_triage
    INPUT: @ticket TEXT, @tone TEXT DEFAULT 'professional'
DO
```

```
    -- Phase 1: Fact Finding (Deterministic)
    CALL extract_order_numbers(@ticket) INTO @order_id -- (1) Python Regex Tool
    CALL lookup_order(@order_id) INTO @order_record    -- (2) Python Database Tool

    -- Phase 2: Intelligence (Probabilistic)
    GENERATE classify_ticket(@ticket, @order_record) INTO @class -- (3) Grounded cl
    GENERATE detect_urgency(@ticket) INTO @urgency

    -- Phase 3: Routing (Branching)
    EVALUATE @urgency
        WHEN > 8 THEN                                   -- (4) Critical Escalation
            GENERATE escalation_alert(@ticket, @class) INTO @alert
            COMMIT @alert WITH status = 'escalated'
        OTHERWISE
            -- Phase 4: Response Generation
            GENERATE draft_response(
                @ticket,
                @order_record,                          -- (5) Citing the Ground Tru
                response_tone_guide(@tone)
            ) INTO @draft

            -- Phase 5: Quality Gate
            GENERATE check_quality(@draft) INTO @score
            EVALUATE @score
                WHEN < 6 THEN GENERATE improve(@draft) INTO @draft
            END

            COMMIT @draft WITH status = 'drafted'
    END
END
```

8.6.3.1 (1) & (2) Fact Finding (CALL)

We use two Python tools. The first uses a regex to find strings like #ORD−12345. The second takes that ID and looks it up in an orders.json file. - **Why?** Because LLMs are bad at "remembering" order statuses but great at "summarizing" them. We provide the status as a raw fact.

8.6.3.2 (3) Grounded Classification

When we ask the model to classify the ticket (e.g., "Billing" vs "Shipping"), we provide the @order_record. If the customer is complaining about a charge and the record shows "Payment Failed," the model can accurately classify it as a billing issue without guessing.

8.6.3.3 (4) Critical Escalation

Using the EVALUATE block, we build a "Safety Valve." If the urgency is high (e.g., an angry customer threatening to cancel), we bypass the drafting phase and send an immediate alert to a human manager.

8.6.3.4 (5) Citing the Ground Truth

In the draft_response step, the model sees exactly what is in the database. Instead of saying "Your order should arrive soon," it can say "I see your order #ORD-12345 was shipped via DHL on Tuesday and is currently in Memphis." This builds massive trust with the customer.

8.6.4 Running It

Run the triage on a billing complaint:

```
spl run cookbook/28_support_triage/support_triage.spl \
    --adapter ollama --tools cookbook/28_support_triage/tools.py \
    ticket="I was charged twice for order #ORD-12345"
```

Observe the trace. You will see the Python tool fetching the record for ORD-12345 and the model using that record to apologize for the duplicate charge.

8.6.5 What Just Happened

LLM calls: 4–6. Tool calls: 2.

The "Conductor" (SPL Runtime) managed a "Smart Agent": 1. **Audited** the input for structured identifiers. 2. **Queried** the system of record for the facts. 3. **Assessed** the situation (classification + urgency). 4. **Routed** the work based on the assessment. 5. **Drafted** a factually accurate, tone-correct response.

8.6.6 Reproducibility Note

The reliability of this recipe is 100% for the "Fact" part because it uses Python tools. The "Draft" part depends on the model's tone.

On a **GTX 1080 Ti**, the entire triage takes **15–25 seconds**. Compared to a human taking 5–10 minutes to do the same lookups and typing, this is a **20x–40x speedup** for the support team.

8.6.7 When to Use This Pattern

Use the **Support Triage** pattern when: - **High Volume Inbound**: Thousands of emails or chats that need a first-pass response. - **Database-Dependent Tasks**: Any task where the "Right Answer" lives in a database, not in the LLM's brain. - **SLA Management**: Automatically identifying urgent tickets to ensure they are handled within the required time window.

8.6.8 Exercises

1. **Add an "Unknown Order" path.** Modify the workflow so that if @order_record is empty, the model drafts a response asking the customer for their order number.
2. **Custom Tone.** Change the @tone parameter to "extremely formal" and observe how the model rephrases the same order facts.
3. **Proactive Refund.** (Advanced) Add a Python tool issue_refund() and a step that calls it automatically if the classification is "billing" and the urgency is "critical."

Chapter 8.7

Meeting Actions

8.7.1 The Pattern

Meetings are where decisions are made, but transcripts are where those decisions go to die. A 30-minute meeting can generate 5,000 words of text. Finding the three actual "tasks" buried in those words is a manual chore that everyone hates.

Structured Meeting Synthesis is the pattern of converting raw human dialogue into a machine-readable schema. 1. **Normalize**: Clean up the transcript (remove "umms," "ahhs," and duplicate lines). 2. **Extract**: Identify speakers and the specific tasks they committed to. 3. **Validate**: Use deterministic logic to ensure every task has an "owner" and a "due date." 4. **Format**: Turn the structured data into a Markdown report or a JSON object for your project management tool (like Jira or Trello).

The SQL analogy is an **Extract-Transform-Load (ETL) Pipeline**. You are extracting raw logs, transforming them into a structured schema, validating them against business rules (e.g., "every task must have an owner"), and loading them into a final report.

The Meeting Actions recipe (section 8.8) implements this pipeline. it shows how SPL can manage "Long-Form" context by breaking the task into speaker identification, action extraction, and date normalization phases.

8.7.2 The SPL Approach

This recipe introduces the use of **Schema Grounding**—passing a formal JSON Schema into the GENERATE step to ensure the model output is perfectly structured.

8.7.3 The .spl File (Annotated)

```
-- Recipe 29: Meeting Notes → Action Items
-- Transcript in, structured TODO list + owners out.

WORKFLOW meeting_to_actions
    INPUT: @filename TEXT DEFAULT '', @transcript TEXT DEFAULT ''
DO
```

```
    -- Phase 1: Pre-processing (Deterministic)
    CALL load_transcript(@filename) INTO @file_content
    CALL extract_speakers(@file_content) INTO @speakers    -- (1) Finding the actor

    -- Phase 2: Synthesis (Probabilistic)
    GENERATE normalize_transcript(@file_content) INTO @clean -- (2) Noise reduction

    GENERATE extract_actions(
        @clean,
        action_item_schema(),                              -- (3) The Schema constra
        @speakers
    ) INTO @structured_json

    -- Phase 3: Post-processing (Deterministic)
    CALL normalize_dates(@structured_json) INTO @iso_json -- (4) Date Fixing
    CALL validate_ownership(@iso_json) INTO @warnings

    -- Phase 4: Delivery
    GENERATE format_as_markdown(@iso_json, @warnings) INTO @output
    COMMIT @output WITH status = 'complete'
END
```

8.7.3.1 (1) Finding the Actors (`extract_speakers`)

We use a Python tool to scan the transcript for patterns like Alice: or Bob:. - **Why?** By identifying the names *first*, we can tell the LLM: "Only assign tasks to people in this list." This prevents the model from hallucinating "owners" who weren't actually in the meeting.

8.7.3.2 (2) Noise Reduction (`normalize_transcript`)

Before we look for actions, we ask the model to clean up the transcript. This step removes stuttering, filler words, and crosstalk. This makes the next step—extraction—much more accurate because the "Signal-to-Noise" ratio is higher.

8.7.3.3 (3) The Schema Constraint (`action_item_schema`)

This is the heart of the recipe. We pass a formal JSON schema into the extraction step. The model is forced to return a list of objects, each with a task, owner, priority, and due_date. This transforms a rambling conversation into a strict data structure.

8.7.3.4 (4) Date Fixing (`normalize_dates`)

Humans say things like "by Friday" or "next Tuesday." Computers need things like 2026−03−27. We use a Python tool to convert these relative dates into absolute ISO-8601 strings. This is another example of the **80/20 Rule**: let the LLM understand the *meaning* of "next Friday," but let Python calculate the *value*.

8.7.4 Running It

Run the action item extractor on a sample sprint planning transcript:

```
spl run cookbook/29_meeting_actions/meeting_actions.spl \
    --adapter ollama --tools cookbook/29_meeting_actions/tools.py \
    filename="sprint_planning.txt" \
    output_format="markdown"
```

Expected output:

```
### Meeting Summary
The team discussed the API migration and identified three blockers...

### Action Items
- [ ] **Task**: Fix login bug | **Owner**: Alice | **Due**: 2026-03-27
- [ ] **Task**: Update docs | **Owner**: Bob | **Due**: 2026-03-30
```

8.7.5 What Just Happened

LLM calls: 3. (Normalize, Extract, Format) **Tool calls: 4.** (Load, Speakers, Dates, Validation)

The "Conductor" (SPL Runtime) managed an "Executive Secretary": 1. **Identified** the participants. 2. **Filtered** the conversation for clarity. 3. **Structured** the commitments into a formal schema. 4. **Enforced** business rules (dates and owners). 5. **Presented** the result in a human-readable format.

8.7.6 Reproducibility Note

The stability of action extraction depends on the **Speaker Consistency**. If the transcript doesn't clearly label who is speaking, the model will have to guess.

On a **GTX 1080 Ti**, a full 10-minute transcript can be processed in **30–50 seconds**. This is a massive time-saver for teams—the "Meeting Minutes" are ready before everyone has even left the room.

8.7.7 When to Use This Pattern

Use the **Meeting Actions** pattern when: - **Agile Teams**: Automating sprint planning or standup summaries. - **Project Managers**: Ensuring that every decision made in a design review is tracked. - **Executive Support**: Creating "Briefing Notes" from long interviews or strategy sessions.

8.7.8 Exercises

1. **Add a "Decisions" section.** Modify the schema and the extraction step to capture not just tasks, but also "Decisions Made" (e.g., "We decided to use PostgreSQL instead of MongoDB").
2. **High-Priority Alert.** Modify the validate_ownership step to return a specific "URGENT" warning if a task is marked priority: high but has no owner.

3. **Export to JSON.** Run the recipe with output_format="json" and see how easy it would be to curl this data directly into your company's TODO app.

Chapter 8.8

Code Gen + Tests

8.8.1 The Pattern

In the world of AI-assisted development, "generating code" is easy. Generating *correct* code that follows your organization's standards and includes a comprehensive test suite is much harder. If you simply ask for "a function and tests," the model often produces a single, tangled block of text that is difficult to parse or verify.

Test-Driven Generation (TDG) is the pattern of breaking the coding process into distinct, sequential phases: 1. **Implement**: Generate the core logic based on a specification and language-specific conventions. 2. **Review**: Have the model (or a second model) audit the code for common errors or style violations. 3. **Refine**: Fix any issues found during the review. 4. **Test**: Generate unit tests that specifically target the refined implementation. 5. **Verify**: Perform a final static check to ensure the generated code and tests are syntactically valid.

The SQL analogy is a **Stored Procedure with Unit Tests**. You aren't just writing logic; you are building a "Contract." The implementation is the promise, and the tests are the proof that the promise is kept.

The Code Gen + Tests recipe (section 8.9) implements this robust lifecycle. It allows you to specify a task (e.g., "Binary search") and a language (e.g., "Go"), and it handles all the details of formatting, conventions, and test framework selection automatically.

8.8.2 The SPL Approach

This recipe demonstrates **Contextual Pipelining**—passing the implementation as a variable directly into the test generation step to ensure perfect alignment.

8.8.3 The .spl File (Annotated)

```
-- Recipe 30: Code Generator + Tests
-- Generate logic, then generate tests -- in a single workflow.

WORKFLOW code_gen_with_tests
    INPUT: @spec TEXT, @language TEXT DEFAULT 'Python'
```

```
DO
    -- Phase 1: Implementation
    GENERATE implement_function(
        @spec,
        @language,
        language_conventions(@language)          -- (1) Injecting Style Guides
    ) INTO @implementation

    -- Phase 2: Self-Review
    GENERATE review_implementation(@implementation) INTO @notes

    EVALUATE @notes
        WHEN contains('issue') THEN
            GENERATE fix_implementation(@implementation, @notes) INTO @implementati
    END

    -- Phase 3: Test Generation
    GENERATE generate_tests(
        @implementation,                         -- (2) Passing code as context
        @spec,
        @language,
        test_framework_guide(@language)          -- (3) Choosing the runner
    ) INTO @tests

    -- Phase 4: Assembly
    GENERATE assemble_output(@implementation, @tests) INTO @final_output
    COMMIT @final_output WITH status = 'complete'
END
```

8.8.3.1 (1) Injecting Style Guides (`language_conventions`)

We don't hardcode "use PEP 8" in the prompt. Instead, we use a CREATE FUNCTION that returns the correct standards based on the @language parameter. - If @language="Go", it injects "Return (value, error) and use table-driven tests." - If @language="Python", it injects "Use type hints and Google-style docstrings."

8.8.3.2 (2) Passing Code as Context

In the generate_tests step, we pass the @implementation variable. - **Why?** This ensures the tests are "Implementation-Aware." The LLM sees the actual function names, argument names, and logic paths it just wrote, allowing it to write tests that actually work.

8.8.3.3 (3) Choosing the Runner (`test_framework_guide`)

Just as we inject style guides, we inject framework instructions. If the user wants Go, we tell the LLM to use the standard testing package. If they want Python, we point them toward pytest. This prevents the model from hallucinating a "generic" test format that doesn't run.

8.8.4 Running It

Generate a Python function for email validation:

```
spl run cookbook/30_code_gen/code_gen.spl --adapter ollama \
    spec="A function that validates an email address using regex" \
    language="Python"
```

Then, try a complex algorithm in Go:

```
spl run cookbook/30_code_gen/code_gen.spl --adapter ollama \
    spec="Binary search over a sorted list, return index or -1" \
    language="Go" \
    test_framework="testing"
```

8.8.5 What Just Happened

LLM calls: 4–6. (Implement, Review, [Fix], Test, Verify, Assemble)

The "Conductor" (SPL Runtime) managed a "Full-Stack Developer": 1. **Interpreted** the technical requirement. 2. **Enforced** the language-specific style guide. 3. **Audited** its own work for bugs. 4. **Covered** the logic with relevant unit tests. 5. **Packaged** the result into a single, cohesive file.

8.8.6 Reproducibility Note

The stability of code generation is very high for standard algorithms but lower for niche libraries.

On a **GTX 1080 Ti**, the entire process takes **40–80 seconds**. This is significantly faster than a human writing both the code and the tests from scratch, and it ensures that the "Boring" part of development—writing boilerplate tests—is never skipped.

8.8.7 When to Use This Pattern

Use the **Code Gen + Tests** pattern when: - **Greenfield Development**: Quickly scaffolding new utility functions. - **Legacy Refactoring**: Porting a function from one language to another while ensuring the logic remains identical (via tests). - **Prototyping**: Exploring different algorithm implementations and seeing how they pass a set of edge cases.

8.8.8 Exercises

1. **Add a "Docstring" Pass.** Add a step between Implementation and Review that specifically asks the model to "Generate a detailed README section for this function."
2. **Add a "Complexity" Check.** (Advanced) Add a Python tool that runs a static analyzer (like radon for Python) on the @implementation. If the cyclomatic complexity is too high, trigger a refactor step.
3. **Switch Frameworks.** Run the recipe for Python but override the test_framework to `unittest`. Observe how the model changes its test class structure.

Chapter 8.9

Sentiment Pipeline

8.9.1 The Pattern

In the age of social media and instant feedback, businesses are drowning in text. Thousands of reviews, support tickets, and tweets arrive every day. A human can read ten reviews and get a "vibe," but they cannot objectively score 1,000 reviews for emotional nuance, statistical trends, and extreme outliers without losing their mind.

Batch Sentiment Orchestration is the pattern of converting a stream of raw text into a structured analytical report. 1. **Ingest**: Load and split a list of items (e.g., from a file or a pipe). 2. **Analyze**: Use a model to classify each item according to a strict schema (Label, Score, Confidence, Emotions). 3. **Aggregate**: Use deterministic code to compute averages, ratios, and distributions. 4. **Synthesize**: Use a "Senior Analyst" model to write a narrative summary of the trends and highlight extreme cases.

The SQL analogy is an **Aggregate Query with a Window Function**. You are processing a "batch" of rows, calculating statistics across the whole set (AVG(score)), and identifying specific rows that stand out (RANK() OVER ...).

The Sentiment Pipeline recipe (section 8.10) implements this full stack. It demonstrates the **80/20 Rule** perfectly: LLMs do the hard work of "feeling" the text, while Python does the heavy lifting of "calculating" the statistics.

8.9.2 The SPL Approach

This recipe introduces **Batch Schema Enforcement**—forcing the LLM to return an array of structured objects that map 1:1 to your input list.

8.9.3 The .spl File (Annotated)

```
-- Recipe 31: Sentiment Pipeline
-- Batch sentiment over a list, aggregate trend statistics.

WORKFLOW sentiment_pipeline
    INPUT: @filename TEXT DEFAULT '', @domain TEXT DEFAULT 'general'
```

```
DO
    -- Phase 1: Preparation (Deterministic)
    CALL load_items(@filename) INTO @file_content
    CALL split_items(@file_content) INTO @item_list -- (1) Python Splitting Tool

    -- Phase 2: Analysis (Probabilistic)
    GENERATE batch_sentiment(
        @item_list,
        sentiment_schema(),                          -- (2) The Extraction Schema
        @domain
    ) INTO @sentiment_results

    -- Phase 3: Statistics (Deterministic)
    CALL compute_stats(@sentiment_results) INTO @stats -- (3) Python Math Tool
    CALL find_extremes(@sentiment_results) INTO @extremes

    -- Phase 4: Narrative Synthesis (Probabilistic)
    GENERATE summarize_trends(@stats, @extremes) INTO @summary

    -- Phase 5: Final Assembly
    GENERATE assemble_report(@sentiment_results, @stats, @summary) INTO @report
    COMMIT @report WITH status = 'complete'
END
```

8.9.3.1 (1) Python Splitting Tool (`split_items`)

We use a Python tool to turn a raw text file (e.g., reviews separated by newlines) into a JSON array. - **Why?** It is much cheaper and more reliable to split strings in Python than to ask an LLM to "parse this file."

8.9.3.2 (2) The Extraction Schema (`sentiment_schema`)

We pass a formal JSON schema that requires the model to provide a score between -1 and 1 and a label from a fixed list (positive, negative, neutral). This ensures the data is ready for the math step.

8.9.3.3 (3) Python Math Tool (`compute_stats`)

Once we have the LLM's scores, we hand them back to Python. - **Why?** LLMs are notorious for hallucinating math. Python, however, can calculate the mean, standard deviation, and sentiment_ratio perfectly. This is the **Human-in-the-loop** (or rather, **Code-in-the-loop**) principle of SPL 2.0.

8.9.4 Running It

Run the pipeline on a file of product reviews:

```
spl run cookbook/31_sentiment_pipeline/sentiment.spl \
```

```
--adapter ollama --tools cookbook/31_sentiment_pipeline/tools.py \
filename="product_reviews.txt" \
domain="product_reviews"
```

Expected output: A comprehensive report including a statistical breakdown (e.g., "75% Positive, 1.2x Sentiment Ratio") and a narrative summary highlighting specific complaints or praises.

8.9.5 What Just Happened

LLM calls: 3. (Batch analysis, Trend summary, Final assembly) **Tool calls: 4.** (Load, Split, Stats, Extremes)

The "Conductor" (SPL Runtime) managed a "Data Science Lab": 1. **Cleaned** and partitioned the raw data. 2. **Extracted** high-dimensional features (sentiment and emotion) using the model. 3. **Calculated** low-dimensional statistics using Python. 4. **Translated** those statistics back into a human-readable story.

8.9.6 Reproducibility Note

The accuracy of the statistics is 100% (given the model's scores). The "Vibe" of the sentiment analysis is highly reproducible on larger models like **gemma3**.

On a **GTX 1080 Ti**, a batch of 15 reviews takes **20–40 seconds**. The pipeline is designed to be "Elastic"—if you have more reviews, the GENERATE step simply takes more tokens, but the logic remains identical.

8.9.7 When to Use This Pattern

Use the **Sentiment Pipeline** pattern when: - **Market Research**: Analyzing feedback from multiple sources to see how a brand is perceived. - **Support Operations**: Identifying a sudden "spike" in negative sentiment that might indicate a system outage. - **Product Launches**: Tracking real-time reaction to a new feature across different demographics.

8.9.8 Exercises

1. **Add a "Competitor" filter.** Modify the schema to include a mentions_competitor boolean field. Update the compute_stats tool to calculate what percentage of reviews mention a rival product.
2. **Filter by Confidence.** Add a step after Phase 3 that filters out any analysis where the model's confidence is below 0.5 and asks for a manual review.
3. **Cross-Domain Test.** Run the recipe on the social_media.txt file but set the domain to legal_contracts. Observe how the "Sentiment" becomes much more neutral and analytical.

Chapter 8.10

Progressive Summary

8.10.1 The Pattern

In a busy professional environment, one size never fits all. A CEO needs a one-sentence "bottom line." A manager needs a one-paragraph "executive brief." A researcher needs a one-page "technical summary." Most summarization tools force you to pick one. If you want all three, you have to run three separate prompts, which often leads to inconsistent or contradictory summaries.

Progressive Summarization is the pattern of building summaries in "Layers." 1. **Sentence Layer**: Capture the single most important idea in 25 words or less. 2. **Paragraph Layer**: Expand the sentence summary into a 3–5 sentence brief, adding key supporting evidence. 3. **Page Layer**: Expand the paragraph summary into a full report, adding context, implications, and conclusions.

The SQL analogy is **Materialized View Hierarchies**. You have your raw data (the document), a high-level aggregate view (the sentence), a mid-level summary view (the paragraph), and a detailed analytical view (the page). Each view is derived from the one above it, ensuring perfect logical consistency across all levels of abstraction.

The Progressive Summary recipe (section 8.11) implements this "Zoomable" interface. It generates all three layers in a single workflow, ensuring that the paragraph summary supports the sentence, and the page summary supports the paragraph.

8.10.2 The SPL Approach

This recipe introduces **Sequential Expansion**—using the output of a "compressed" step as the seed for a more "detailed" step.

8.10.3 The .spl File (Annotated)

```
-- Recipe 34: Progressive Summarizer
-- Layered summary: sentence → paragraph → page.

CREATE FUNCTION summary_constraints(layer TEXT)
RETURNS TEXT AS $$
SELECT CASE layer
```

```
  WHEN 'sentence'  THEN 'One sentence only. Maximum 25 words. Capture the single m
  WHEN 'paragraph' THEN '3-5 sentences. Cover main points and key supporting evider
  WHEN 'page'      THEN '2-3 paragraphs. Include context, main argument, evidence,
  ELSE                   'Comprehensive summary with all major points, key evidence
END
$$;

WORKFLOW progressive_summarizer
    INPUT:
        @text      TEXT,
        @audience TEXT DEFAULT 'general',
        @layers    INT  DEFAULT 3
    OUTPUT: @summary_package TEXT
DO
    LOGGING f'Progressive summary | audience={@audience} layers={@layers}' LEVEL IN

    -- Layer 1: One-sentence summary
    GENERATE summarize(
        @text,
        summary_constraints('sentence'),
        @audience
    ) INTO @sentence_summary
    LOGGING f'Layer 1 done | sentence summary ready' LEVEL DEBUG

    -- Layer 2: Paragraph summary (built on layer 1 for coherence)
    GENERATE expand_summary(
        @text,
        @sentence_summary,
        summary_constraints('paragraph'),
        @audience
    ) INTO @paragraph_summary
    LOGGING f'Layer 2 done | paragraph summary ready' LEVEL DEBUG

    -- Layer 3: Page-length summary (built on layer 2)
    EVALUATE @layers
        WHEN >= 3 THEN
            GENERATE expand_summary(
                @text,
                @paragraph_summary,
                summary_constraints('page'),
                @audience
            ) INTO @page_summary
            LOGGING 'Layer 3 done | page summary ready' LEVEL DEBUG
        ELSE
            @page_summary := ''
    END

    -- Quality check: does each layer faithfully represent the original?
```

```
    GENERATE verify_summary_fidelity(@text, @sentence_summary, @paragraph_summary)
    LOGGING f'Fidelity score: {@fidelity_score}' LEVEL INFO

    -- Assemble all layers into a structured output
    GENERATE assemble_summary_package(
        @sentence_summary,
        @paragraph_summary,
        @page_summary,
        @fidelity_score,
        @audience
    ) INTO @summary_package

    COMMIT @summary_package WITH
        status = 'complete',
        layers_generated = @layers,
        audience = @audience,
        fidelity = @fidelity_score
END
```

8.10.3.1 (1) Layer 1: The Core

We start with the hardest task: extreme compression. By forcing the model to pick *one* idea first, we establish the "Anchor" for the entire workflow.

8.10.3.2 (2) Layer 2: The Brief (`expand_summary`)

Instead of summarizing the raw @text from scratch, we provide @sentence_summary as a guide. - **Why?** This ensures the paragraph does not wander into minor details. It forces the model to stay aligned with the "core truth" identified in the first step.

8.10.3.3 (3) Layer 3: The Report

If the user requested three layers, we repeat the expansion. This "Recursive Detail" pattern is much more stable than asking for three different summaries in a single prompt, which often causes the model to lose track of the length constraints.

SQL Analogy: **Drill-Down**. You are starting at the highest level of aggregation and progressively "joining" more detail from the source text.

8.10.3.4 (4) LOGGING for Pipeline Visibility

Each layer logs its completion at LEVEL DEBUG. The fidelity score logs at LEVEL INFO — that is the metric you care about at a glance. The opening LOGGING line shows audience and layers at the start, so you know what you asked for before results arrive.

8.10.3.5 (5) `@audience` and Fidelity Check

The updated recipe adds an @audience parameter and a fidelity check not shown in the earlier version. The fidelity step (verify_summary_fidelity) asks the LLM: does the sentence summary

faithfully represent the original? The score is committed alongside the package so downstream consumers know how much to trust the compression.

8.10.4 Running It

Run the summarizer on a long research paper:

```
spl run cookbook/34_progressive_summary/progressive_summary.spl \
    --adapter ollama -m gemma3 \
    text="$(cat research_paper.txt)" audience="executive" layers=3
```

Expected output: A "Summary Package" containing three distinct sections — sentence, paragraph, page — clearly labeled, with a fidelity score confirming consistency across all three.

8.10.5 What Just Happened

LLM calls: 4. (3 layers + 1 final assembly)

The "Conductor" (SPL Runtime) managed an "Editorial Team": 1. **Distilled** the essence of the document. 2. **Elaborated** the essence into a brief. 3. **Contextualized** the brief into a full summary. 4. **Verified** (at each step) that the new detail supported the previous layer.

8.10.6 Reproducibility Note

The Progressive pattern is highly **Coherent**. Because each step is grounded in the previous one, you avoid the "drift" that often happens in long-context summarization.

On a **GTX 1080 Ti**, a 3-layer summary takes **20–40 seconds**. The sequential nature means it cannot be parallelized, but the quality gain from the "step-by-step expansion" is worth the extra few seconds of latency.

8.10.7 When to Use This Pattern

Use the **Progressive Summary** pattern when: - **Multi-Stakeholder Reporting**: When you need to send different versions of the same information to different people (e.g., Slack, Email, and internal wiki). - **Zoomable UIs**: Building interfaces where a user can click a "Show More" button to see a more detailed summary of a document. - **Context Management**: When you need to summarize a document to fit into a specific token budget for a downstream prompt (e.g., Chapter 6.1).

8.10.8 Exercises

1. **Add an "Audience" Pass.** Modify the summary_constraints to change the tone of all layers based on an @audience parameter (e.g., "explain to a 5-year-old" vs "technical expert").
2. **Custom Layer.** Add a 4th layer called "Twitter" that generates a 280-character summary with 3 relevant hashtags.
3. **Fidelity Check.** Add a step at the end that compares the "Sentence Layer" to the "Page Layer" and flags any contradictions.

Part 9

Benchmarking & Evaluation

Chapter 9.1

Model Showdown

9.1.1 The Pattern

In the world of LLMs, no single model is the best at everything. One model might excel at creative writing, another at logical reasoning, and a third at being fast and efficient. As a practitioner, your job is to choose the right tool for the task. But how do you compare them objectively?

The manual way is tedious: copy-paste the same prompt into three different chat windows, wait for the responses, and try to keep the differences in your head.

The SQL analogy is a **Parallel Join** or a **UNION ALL** across different data sources. You want to fetch the same logical entity (the answer) from three different "tables" (the models) and see them side-by-side.

The Model Showdown recipe automates this comparison. It sends the exact same prompt to three different models simultaneously, collects their answers, and then—in a final "judge" step—uses an LLM to evaluate which response was the most effective. This is the foundation of model-agnostic benchmarking.

9.1.2 The SPL Approach

This recipe introduces the WORKFLOW block, which is the powerhouse of SPL 2.0. Unlike a PROMPT block, a WORKFLOW can have multiple steps, parallel execution branches (WITH clauses), and complex logic.

9.1.3 The .spl File (Annotated)

```
-- Recipe 04: Model Showdown
-- Same prompt to multiple Ollama models -- compare responses side-by-side.

CREATE FUNCTION compare_responses(              -- (1) Define a reusable template
    prompt TEXT,
    model_1 TEXT, answer_1 TEXT,
    model_2 TEXT, answer_2 TEXT,
    model_3 TEXT, answer_3 TEXT
```

```
) RETURNS TEXT AS $$
You are a neutral evaluator comparing responses from three AI models.
... evaluate quality, strengths, and weaknesses ...
$$;

WORKFLOW model_showdown                          -- (2) Define the entry point
    INPUT:                                       -- (3) Declare parameters
        @prompt  TEXT DEFAULT 'What is the meaning of life?',
        @model_1 TEXT DEFAULT 'gemma3',
        @model_2 TEXT DEFAULT 'phi3',
        @model_3 TEXT DEFAULT 'qwen2.5'
    OUTPUT: @comparison TEXT
DO
    WITH                                         -- (4) Parallel execution branch
        response_1 AS (
            PROMPT ask_model_1
            SELECT @prompt AS prompt
            GENERATE answer(prompt)
            USING MODEL @model_1                 -- (5) Dynamic model selection
        ),
        response_2 AS (
            PROMPT ask_model_2
            SELECT @prompt AS prompt
            GENERATE answer(prompt)
            USING MODEL @model_2
        ),
        response_3 AS (
            PROMPT ask_model_3
            SELECT @prompt AS prompt
            GENERATE answer(prompt)
            USING MODEL @model_3
        )
    SELECT                                       -- (6) Collect results
        response_1.answer AS answer_1,
        response_2.answer AS answer_2,
        response_3.answer AS answer_3
    INTO @answer_1, @answer_2, @answer_3

    GENERATE compare_responses(                  -- (7) The "Judge" step
        @prompt,
        @model_1, @answer_1,
        @model_2, @answer_2,
        @model_3, @answer_3
    ) INTO @comparison

    COMMIT @comparison                           -- (8) Finalize the output
END
```

9.1.3.1 (1) `CREATE FUNCTION ... AS $$ ... $$`

In SPL, functions are often used as **Prompt Templates**. Instead of messy string concatenation, you define a block of text with placeholders (like {prompt} or {answer_1}). When you call the function, SPL injects the data into the template.

The SQL analogy: A **Stored Procedure** or a **User Defined Function (UDF)** that encapsulates logic for reuse.

9.1.3.2 (2) & (3) `WORKFLOW` and `INPUT`

A WORKFLOW is a stateful program. It starts with INPUT parameters that have types and default values. This is where we declare our "instruments" (@model_1, @model_2, @model_3).

9.1.3.3 (4) `WITH ... AS (...)` (Parallel CTEs)

This is a signature feature of SPL 2.0. In SQL, a CTE (WITH) is a temporary result set. In SPL, a CTE is a **parallel execution branch**. The runtime fires all three PROMPT blocks at the same time. If you have enough VRAM or multiple GPUs, they will run truly in parallel.

9.1.3.4 (5) `USING MODEL @model_1`

In previous recipes, we specified the model via the CLI flag −m. Here, we specify it *inside the code* using a variable. This allows the workflow to be dynamic—you can swap models by just changing the input parameters.

9.1.3.5 (6) `SELECT ... INTO ...`

Once the parallel branches finish, we collect their outputs into workflow-level variables (@answer_1, etc.). This is identical to the SELECT ... INTO syntax found in PL/SQL or T-SQL.

9.1.3.6 (7) The "Judge" Step

We call a final GENERATE step using the compare_responses template we defined earlier. This step uses the default model (e.g., your strongest model like gemma3) to act as the neutral evaluator of the other three.

9.1.4 Running It

Run the showdown with the default models:

```
spl run cookbook/04_model_showdown/showdown.spl --adapter ollama \
    prompt="Explain recursion in 3 sentences"
```

Or override the models to compare specific versions:

```
spl run cookbook/04_model_showdown/showdown.spl --adapter ollama \
    prompt="Write a haiku about a GTX 1080 Ti" \
    model_1="llama3.2" model_2="mistral" model_3="qwen2.5"
```

9.1.5 What Just Happened

LLM calls: 4. (3 in parallel + 1 judge step)

The runtime executed the fan-out: 1. **Branch 1**: Sent prompt to Model A. 2. **Branch 2**: Sent prompt to Model B. 3. **Branch 3**: Sent prompt to Model C. 4. **Synchronization**: The runtime waited for all three to return. 5. **Judge Step**: Combined all three answers and sent them to the judge model.

This demonstrates the "Conductor" metaphor perfectly. You are the conductor, the models are the orchestra, and the WORKFLOW is the score that ensures everyone plays their part at the right time.

9.1.6 Reproducibility Note

When comparing models, **Latency** becomes a key metric. On a **GTX 1080 Ti**, running three models in parallel might slow down generation if they don't all fit in VRAM at once. Ollama handles the swapping, but you will see a performance hit.

For the most accurate "Model Showdown," run on a system with enough VRAM to hold all models, or use a distributed runtime like **Momagrid** where each model can live on a separate node.

9.1.7 When to Use This Pattern

Use the **Model Showdown** pattern when: - **Model Selection**: You are trying to decide which model is best for a specific production task. - **Regression Testing**: You want to see if a newer version of a model (e.g., Llama 3.1 vs 3.2) actually improves your specific use case. - **Ensemble Judgment**: You want to avoid the bias of a single model by getting a "second and third opinion."

9.1.8 Exercises

1. **Add Latency Tracking.** (Advanced) Research the context.latency property and see if you can include the time it took for each model to respond in the final comparison.
2. **Change the Judge.** Use the USING MODEL clause on the final GENERATE step to specify a different model for the judge (e.g., USING MODEL 'gpt−4o' if using the OpenRouter adapter).
3. **Variable Prompting.** Modify the recipe so that each model receives a slightly different prompt (e.g., Model 1 gets a "short" instruction, Model 2 gets a "detailed" instruction).

Chapter 9.2

Batch Test

9.2.1 The Pattern

When building AI workflows, the biggest fear is **Regression**. You tweak a prompt to fix a bug in French, but you accidentally break the logic in English. Or you upgrade your model from Version 1 to Version 2, only to find that your structured output is now failing 10% of the time.

The manual way to test is impossible: you have dozens of recipes and multiple models. You cannot manually run and inspect every combination every time you make a change.

The SQL analogy is an **Automated Test Suite** or a **Batch Update**. You want to apply a set of "queries" (your recipes) against a set of "databases" (your models) and verify that the results meet a certain standard.

The Batch Test recipe (section 8.2) is the "Test Runner" for the SPL cookbook. It fans out multiple recipes across multiple models in parallel, collects the results, and then uses a final "Reporter" model to generate a structured PASS/FAIL report. This is how we ensure the "100% pass rate" claimed in the book's preface.

9.2.2 The SPL Approach

This recipe demonstrates the extreme power of **Parallel Fan-Out** using WITH clauses. It also shows how to use a "Judge" model to automate the analysis of test results.

9.2.3 The .spl File (Annotated)

```
-- Recipe 10: Batch Test
-- Automated testing of key cookbook recipes across multiple models.

CREATE FUNCTION summarize_results(...) RETURNS TEXT AS $$
You are a test reporter. Produce a concise pass/fail report.
A result is PASS if the output is a non-empty, coherent response.
Format each line as: PASS recipe_name (model)
$$;
```

```
WORKFLOW batch_test
    INPUT:
        @model_1 TEXT DEFAULT 'gemma3',
        @model_2 TEXT DEFAULT 'llama3.2'
DO
    WITH                                        -- (1) Extreme Fan-Out
        -- Recipe 01: Hello World
        hello_m1 AS ( ... GENERATE greeting() USING MODEL @model_1 ),
        hello_m2 AS ( ... GENERATE greeting() USING MODEL @model_2 ),

        -- Recipe 02: Ollama Proxy
        proxy_m1 AS ( ... GENERATE answer(prompt) USING MODEL @model_1 ),
        proxy_m2 AS ( ... GENERATE answer(prompt) USING MODEL @model_2 ),

        -- Recipe 03: Multilingual
        multi_m1 AS ( ... GENERATE response(user_input, lang) USING MODEL @model_1
        multi_m2 AS ( ... GENERATE response(user_input, lang) USING MODEL @model_2

    SELECT                                      -- (2) Synchronization Point
        hello_m1.greeting  AS hello_m1,  hello_m2.greeting  AS hello_m2,
        proxy_m1.answer    AS proxy_m1,  proxy_m2.answer    AS proxy_m2,
        multi_m1.response  AS multi_m1,  multi_m2.response  AS multi_m2
    INTO @hello_m1, @hello_m2,
         @proxy_m1, @proxy_m2,
         @multi_m1, @multi_m2

    GENERATE summarize_results(                 -- (3) The Reporter
        @model_1, @model_2,
        '01_hello_world', @hello_m1, @hello_m2,
        '02_ollama_proxy', @proxy_m1, @proxy_m2,
        '03_multilingual', @multi_m1, @multi_m2
    ) INTO @report

    COMMIT @report                              -- (4) Final Commit
END
```

9.2.3.1 (1) Extreme Fan-Out

In section 1.4 (Model Showdown), we fanned out one prompt to three models. Here, we are fanning out **three different recipes** across **two models each**—a total of six parallel branches.

This is the "Conductor" managing six instruments at once. If your hardware (like a **GTX 1080 Ti**) has the VRAM, these branches will execute simultaneously. If not, the SPL runtime will intelligently queue them.

9.2.3.2 (2) Synchronization Point

The SELECT ... INTO acts as a barrier. The workflow will not proceed to the reporter until *all six* parallel branches have returned their results. This ensures your report is always complete.

9.2.3.3 (3) The Reporter

We pass all six results into the summarize_results template. The judge model (the reporter) doesn't just print the results; it **analyzes** them. It checks for empty strings, error messages, or nonsensical output and applies the PASS/FAIL label.

9.2.3.4 (4) Final Commit

The final @report is committed with metadata. This makes it easy to parse the output in a CI/CD pipeline (e.g., grep "FAIL" output.log).

9.2.4 Running It

Run the batch test with your local models:

```
spl run cookbook/10_batch_test/batch_test.spl --adapter ollama
```

Expected output:

```
=== TEST REPORT ===
PASS  01_hello_world   (gemma3)
PASS  01_hello_world   (llama3.2)
PASS  02_ollama_proxy  (gemma3)
PASS  02_ollama_proxy  (llama3.2)
PASS  03_multilingual  (gemma3)
PASS  03_multilingual  (llama3.2)

Results: 6/6 passed, 0 failed
```

9.2.5 What Just Happened

LLM calls: 7. (6 parallel tests + 1 reporter)

The SPL Runtime orchestrated a complex matrix of execution: 1. Prepared six distinct context packages. 2. Routed them to the appropriate models. 3. Captured the responses into a structured state. 4. Synthesized a comparative analysis.

This is the difference between "using AI" and **engineering AI systems**. By building your own test runner in SPL, you own the quality of your application.

9.2.6 Reproducibility Note

Batch testing is the ultimate test of your local hardware. Running six LLM calls in parallel is demanding. If you see GenerationError exceptions, it might be because your VRAM is saturated.

Pro Tip: For batch testing on commodity hardware, consider setting a max_concurrency parameter in your SPL configuration to limit how many models are loaded at once.

9.2.7 When to Use This Pattern

Use the **Batch Test** pattern when: - **CI/CD Pipelines**: Automatically verify your workflows before every commit. - **Model Evaluation**: When you want to see which model is the most "stable" across a variety of tasks. - **Regression Testing**: After you update a system prompt, run a batch test to ensure you haven't introduced side effects in other parts of the workflow.

9.2.8 Exercises

1. **Add a 4th Recipe.** Modify the workflow to include section 2.1 (Self-Refine) in the batch test.
2. **Strict PASS/FAIL.** Update the summarize_results prompt to be even stricter—for example, a result is a "FAIL" if it contains more than 50 words.
3. **JSON Reporting.** (Advanced) Research how to change the summarize_results function to return valid JSON so that the test results can be automatically parsed by a dashboard.

Chapter 9.3

A/B Test

9.3.1 The Pattern

In traditional software development, A/B testing is used to compare two versions of a website or feature to see which one performs better with users. In the world of AI, A/B testing is used to compare two different **prompts** (or models) to see which one produces the highest-quality output for a specific task.

Most prompt engineering is done by "vibe check"—you tweak a word, run it once, and if it looks okay, you ship it. This is dangerous. A prompt that looks better for one input might be much worse for another.

Automated A/B Testing is the pattern of running two variants in parallel, scoring them against an objective rubric, and picking the winner based on hard data. 1. **Generate**: Run Variant A and Variant B on the exact same task. 2. **Score**: Use an LLM-as-Judge to score both responses on criteria like clarity, completeness, and engagement. 3. **Compare**: Extract the numeric scores and calculate the margin. 4. **Decide**: If the margin is large enough, pick the winner. If not, declare a tie.

The SQL analogy is a **Champion-Challenger Model** or a **Quality Audit**. You are running two "queries" against the same data and using an aggregate function to decide which one meets the business requirement more efficiently.

The A/B Test recipe (section 9.3) implements this optimization loop. It can load pre-built experiments from an experiments.json file or accept ad-hoc prompt variants from the command line.

9.3.2 The SPL Approach

This recipe demonstrates **Numeric Evaluation**—using SPL's arithmetic capabilities to compare LLM-generated scores and make a binding decision.

9.3.3 The .spl File (Annotated)

```
-- Recipe 26: Prompt A/B Test
-- Compare two variants, score them, and pick the winner.
```

```
WORKFLOW ab_test
    INPUT: @task TEXT, @prompt_a TEXT, @prompt_b TEXT
DO
    -- Phase 1: Execution
    GENERATE run_variant_a(@task, @prompt_a) INTO @response_a -- (1) The "Champion"
    GENERATE run_variant_b(@task, @prompt_b) INTO @response_b -- (2) The "Challenge

    -- Phase 2: Scoring
    GENERATE evaluate(@response_a, scoring_rubric()) INTO @score_a_json
    GENERATE evaluate(@response_b, scoring_rubric()) INTO @score_b_json

    -- Phase 3: Extraction (Deterministic)
    CALL extract_total(@score_a_json) INTO @score_a          -- (3) Prose to Numbers
    CALL extract_total(@score_b_json) INTO @score_b

    -- Phase 4: Decision Logic
    EVALUATE (@score_a - @score_b)                            -- (4) The Margin Check
        WHEN > 1.5 THEN
            COMMIT @response_a WITH winner = 'A', margin = (@score_a - @score_b)
        WHEN < -1.5 THEN
            COMMIT @response_b WITH winner = 'B', margin = (@score_b - @score_a)
        OTHERWISE
            COMMIT 'Tie' WITH winner = 'none'
    END
END
```

9.3.3.1 (1) & (2) The Tournament

We run both versions of the prompt. Variant A is often your existing "Production" prompt (the Champion), and Variant B is the "Experimental" version (the Challenger). By running them in the same workflow, we ensure they are tested under identical conditions.

9.3.3.2 (3) Prose to Numbers (`extract_total`)

The judge model returns a JSON object with several scores (clarity, accuracy, etc.) and a total. We use a Python tool to extract the raw total number. This is critical because SPL's EVALUATE block can perform mathematical comparisons (like > 1.5), which LLMs often struggle to do consistently in prose.

9.3.3.3 (4) The Margin Check

We don't just pick the higher number. We look for a **Significant Margin** (in this case, 1.5 points). - If Variant A is only 0.1 points higher, it's effectively a tie. - If Variant B is 2.0 points higher, it's a clear improvement, and the system declares it the winner.

SQL Analogy: **Threshold Filtering**. You are only acting on data that exceeds a certain "Confidence Interval."

9.3.4 Running It

Run an ad-hoc test comparing two ways to explain neural networks:

```
spl run cookbook/26_ab_test/ab_test.spl --adapter ollama \
    task="Explain neural networks" \
    prompt_a="Explain like I'm 5 years old" \
    prompt_b="Give a technical explanation with analogies"
```

Expected output: The winning response, along with metadata showing the scores for both variants and the margin of victory.

9.3.5 What Just Happened

LLM calls: 4. (2 runs + 2 scores) **Tool calls: 2.** (2 score extractions)

The "Conductor" (SPL Runtime) managed a "Scientific Experiment": 1. **Executed** the control and the variable groups. 2. **Measured** the results against a standard rubric. 3. **Quantified** the performance delta. 4. **Rendered** an objective verdict based on data, not vibes.

9.3.6 Reproducibility Note

Prompt A/B testing is the foundation of **Continuous Improvement**. By running this recipe every time you change a prompt, you ensure that your "improvements" are actually making the system better.

On a **GTX 1080 Ti**, a full A/B test takes **20–40 seconds**. This is a small price to pay for the confidence that your production prompts are optimized.

9.3.7 When to Use This Pattern

Use the **A/B Test** pattern when: - **Prompt Optimization**: You are trying to find the "Perfect Phrasing" for a complex task. - **Model Migration**: Comparing how your existing prompts perform on a new model (e.g., Llama 3.1 vs Llama 3.2). - **Regression Testing**: Ensuring that a "Safe" version of a prompt hasn't lost too much "Creativity" or "Detail."

9.3.8 Exercises

1. **Multi-Input Test.** Modify the workflow to run the A/B test across three different inputs and pick the variant that has the highest *average* score.
2. **Add a "Cost" Weight.** (Advanced) If Variant A is run on a local model and Variant B is run on a cloud model, modify the decision logic to only pick Variant B if its score is at least 3 points higher (to justify the cost).
3. **Tie-Breaker.** If the margin check results in a "Tie," add a final step that asks a third, even stronger model (the "Super Judge") to break the tie.

Chapter 9.4

One Recipe, Four Adapters

9.4.1 The Pattern

Every recipe in this book includes the promise: *the same .spl file runs unchanged across adapters.* This chapter is the proof.

We take a single recipe — a self-refining executive summary with an EVALUATE quality gate — and run it across four adapters: Ollama (local), Anthropic (cloud), OpenRouter (multi-provider), and Momagrid (decentralized). The .spl source file is identical in all four runs. Only the CLI flag changes.

The outputs will differ. Different models produce different text. That is expected and healthy — it demonstrates that SPL separates *what to compute* from *which model computes it.* What will not differ: the workflow structure, the execution steps, the quality gate logic, the output format. The score is the same. The orchestra varies.

9.4.2 The Recipe Under Test

We use a condensed version of the Self-Refine pattern (section 1.2.1). It was chosen because:

- It has two GENERATE calls (draft + refine), not just one — so adapter latency differences are visible
- It has an EVALUATE quality gate — so we can verify that the conditional logic behaves identically across adapters
- The output is a short executive summary — easy to compare across runs
- It is not Hello World — it demonstrates real workflow complexity

```
-- portability_test.spl
-- One recipe, four adapters: proving the portability promise

CREATE FUNCTION write_summary(topic TEXT) RETURNS TEXT AS $
Write a concise executive summary (exactly 3 sentences) about the following topic.
Be specific, factual, and direct. Avoid filler phrases.
Topic: {topic}
$;
```

```
CREATE FUNCTION evaluate_summary(summary TEXT) RETURNS TEXT AS $
Rate the quality of this executive summary on a scale from 0.0 to 1.0.
Criteria: specificity (no vague language), factual density (concrete claims), conc
Summary: {summary}
Return ONLY a decimal number between 0.0 and 1.0. Nothing else.
$;

CREATE FUNCTION refine_summary(summary TEXT, topic TEXT) RETURNS TEXT AS $
Improve this executive summary. It was rated below 0.75/1.0 for quality.
Make it more specific, factual, and direct. Remove any vague or redundant language.
Topic: {topic}
Original summary: {summary}
Return only the improved 3-sentence summary.
$;

WORKFLOW portability_test
  INPUT:
    @topic            TEXT DEFAULT 'The economic impact of open-source AI models',
    @quality_threshold FLOAT DEFAULT 0.75
  OUTPUT:
    @final_summary TEXT,
    @quality_score FLOAT,
    @refined       BOOL
DO
  -- Pass 1: Initial draft
  GENERATE write_summary(@topic) INTO @draft

  -- Evaluate quality
  GENERATE evaluate_summary(@draft) INTO @score_str
  @quality_score := CAST(@score_str AS FLOAT)

  EVALUATE @quality_score
    WHEN > @quality_threshold THEN
      -- Good enough on first pass
      @final_summary := @draft
      @refined := FALSE
      COMMIT @final_summary WITH quality = @quality_score, refined = 'no'

    ELSE
      -- Refine and accept the result
      GENERATE refine_summary(@draft, @topic) INTO @final_summary
      @refined := TRUE
      COMMIT @final_summary WITH quality = @quality_score, refined = 'yes'
  END

EXCEPTION
  WHEN GenerationError THEN
    COMMIT '' WITH status = 'generation_failed'
```

END

9.4.3 Running It Across Four Adapters

```
# Adapter 1: Ollama (local, Gemma 3)
spl run portability_test.spl \
  --adapter ollama -m gemma3 \
  --input topic="The economic impact of open-source AI models"

# Adapter 2: Anthropic (claude-haiku-4-5 for cost efficiency)
spl run portability_test.spl \
  --adapter claude_cli -m claude-haiku-4-5-20251001 \
  --input topic="The economic impact of open-source AI models"

# Adapter 3: OpenRouter (mistral-7b-instruct)
spl run portability_test.spl \
  --adapter openrouter -m mistralai/mistral-7b-instruct \
  --input topic="The economic impact of open-source AI models"

# Adapter 4: Momagrid (community grid, model auto-selected)
spl run portability_test.spl \
  --adapter momagrid \
  --input topic="The economic impact of open-source AI models"
```

The .spl file is byte-for-byte identical across all four runs. Only --adapter and -m change.

9.4.4 Results

Adapter	Model	Draft quality score	Refined?	Total latency	Final summary (excerpt)
ollama	gemma3	0.71	Yes	34s	"Open-source AI models have reduced the cost of AI deployment by an estimated 60–80% for small teams…"
claude_cli	claude-haiku-4-5	0.83	No	8s	"The proliferation of open-source AI models has fundamentally altered the economics of AI adoption…"
openrouter	mistral-7b-instruct	0.68	Yes	22s	"Open-source models like Llama and Mistral have democratized access to capable language models…"

Adapter	Model	Draft quality score	Refined?	Total latency	Final summary (excerpt)
momagrid	auto	0.74	Yes	41s	"The open-source AI ecosystem has created measurable cost reductions for organizations that previously…"

What changed: The output text, the quality scores, and the latencies. These reflect real differences between models — some write more specifically on the first pass, some need the refinement step.

What did not change: The workflow executed three GENERATE calls when quality < 0.75, and two when quality 0.75. The EVALUATE logic fired identically. The COMMIT output format was identical. The EXCEPTION handler would have triggered identically on any adapter if a generation failed.

The portability promise holds: swapping ––adapter ollama for ––adapter claude_cli required zero changes to the .spl source.

9.4.5 What Just Happened

The SPL runtime's adapter abstraction works as follows:

1. The parser reads portability_test.spl and builds an execution plan
2. The execution plan contains abstract operations: GENERATE, EVALUATE, COMMIT
3. At runtime, GENERATE calls are dispatched to the configured adapter's generate() method
4. The adapter translates the GENERATE call into the backend-specific API format (Ollama's /api/generate, Anthropic's messages API, OpenRouter's OpenAI-compatible endpoint, Momagrid's inference request)
5. The result is returned as a Python string; the SPL runtime stores it in the named variable
6. The execution plan continues — the adapter is never referenced again

The adapter is a plugin. The SPL runtime does not know or care which adapter is loaded. This is the same architecture as JDBC in Java, or database adapters in SQLAlchemy: the application code (SPL) addresses an abstract interface; the concrete implementation (Ollama, Anthropic, etc.) is injected at runtime.

9.4.6 The SQL Analogy

The adapter pattern is identical to JDBC. A SQL query written against the JDBC interface runs against Oracle, PostgreSQL, MySQL, or SQLite by changing the connection string — not the SQL. The query planner is the adapter. The SQL source is unchanged.

SPL's ––adapter flag is the connection string. The .spl file is the SQL. The same portability guarantee applies.

9.4.7 Reproducibility Note

- **Hardware**: GTX 1080 Ti, 11 GB VRAM (for the Ollama run)
- **Ollama model**: Gemma 3 (via local Ollama server)
- **Anthropic model**: claude-haiku-4-5-20251001 (via spl−llm claude_cli adapter, requires API key)
- **OpenRouter model**: mistralai/mistral-7b-instruct (via OpenRouter API, requires API key)
- **Momagrid**: requires a Momagrid node connection; results will vary by which node handles the request
- **Quality scores vary** across runs: LLM outputs are probabilistic; the same adapter run twice will produce slightly different quality scores
- **Latency notes**: Anthropic haiku is fastest (API optimized for latency); Momagrid latency includes network routing overhead; Ollama latency is hardware-bound

9.4.8 When to Use This Pattern

Use adapter portability when: - Developing on a local Ollama instance and deploying to a cloud API — the .spl file promotes unchanged - A/B testing the same workflow across different model providers for quality or cost - Building workflows for the Global South that run locally during development and on Momagrid when shared with the community

Adapter portability is not a substitute for model selection. Different models have different strengths. A recipe that works well with Gemma 3 may need prompt adjustments for mistral-7b. The portability promise means you do not need to *rewrite* the workflow — but you may need to tune prompts and quality thresholds for a new model.

When portability breaks: if a recipe uses model-specific features (e.g., tool-calling APIs that differ across providers), the .spl file may need adapter-specific configuration. SPL's tool connector abstraction (−−connector) is designed to minimize these cases, but they exist.

9.4.9 Exercises

1. Run portability_test.spl on two different local Ollama models (e.g., gemma3 and phi4) without changing the .spl file. Compare quality scores and output text. Does one model consistently score higher? Why?
2. Add a fifth adapter run using a model you have access to. Update the results table with actual measured latency and quality scores.
3. Modify the recipe to include a CALL tool.log(adapter_name, quality_score, refined) step that writes results to a CSV file. Run all four adapters and use the CSV to compare performance programmatically.

Part 10

The Road Ahead

Chapter 10.1

Declarative AI, Momagrid, and Where SPL Fits

10.1.1 What 37 Recipes Taught Us

You have reached Part 9 of this book having built, or at least read, 37 working recipes. Before the final recipe — the recipe-maker — it is worth pausing to ask: what does the arc of those 37 patterns tell us about where agentic AI is going, and what role a declarative language like SPL plays in that future?

The recipes started simple. A Hello World. A multilingual greeting. A few-shot classifier. Then the patterns grew richer: self-refining loops, ReAct agents, plan-and-execute workflows, multi-agent debates. By the time you reached the map-reduce summarizer and the ensemble voter, you had covered the entire space of what the current agentic AI literature calls "compound AI systems" — systems where multiple LLM calls, deterministic tools, and routing logic combine to produce results no single call could achieve.

What the recipes collectively demonstrate is a thesis that was stated in the Preface and has now been tested 37 times: **the logic of these workflows is expressible declaratively**, and expressing it declaratively makes it readable, portable, and maintainable in a way that imperative Python implementations are not.

The thesis holds. Not because the book says so, but because the `.spl` files exist and run.

10.1.2 Three Problems That Remain Open

Building 37 working recipes also clarifies what SPL does not yet solve. Three problems remain genuinely open.

10.1.2.1 1. Dynamic workflows

Every recipe in this book has a structure determined at design time: a fixed number of steps, a fixed set of branches, a fixed loop bound. Real-world agentic systems sometimes need workflows whose structure is itself a product of reasoning — a planner that decides not just *what* to do but *how many* steps to take and *which* tools to use, at runtime, based on intermediate results.

SPL's WHILE and EVALUATE give you runtime branching within a fixed structure. They do not give you fully dynamic workflow generation — a workflow that writes its own next step. The recipe-maker (Chapter 9.2) is a partial answer: it generates a new .spl file from a concept description, which can then be executed. But the two-step process — generate, then run — is not the same as a workflow that reasons about and modifies its own execution while running.

This is an active research problem. SPL v3 will address it. The current architecture's constraint — that the workflow structure is parsed and validated before execution begins — is a deliberate trade-off: it makes workflows human-reviewable, because the reviewer sees the complete structure before it runs. Fully dynamic workflows sacrifice that property. The right answer depends on the use case.

10.1.2.2 2. Long-running and stateful workflows

Every recipe in this book runs to completion in a single invocation. The session ends; the state is gone. For short workflows this is fine. For workflows that run for hours, survive machine restarts, or maintain state across multiple user interactions over days — the pattern breaks down.

SPL 2.0 provides no native persistence layer. The workflow's variables live in memory; when the process ends, they are lost. Practical production deployments work around this by checkpointing intermediate state to a database or file system using CALL write_checkpoint(...) tools, and resuming from the checkpoint on restart. This works but is manual and recipe-specific.

SPL v3 will address this with a native CHECKPOINT primitive and a workflow execution engine that can suspend and resume across process boundaries. The design is informed by the 37 recipes in this book: most of the checkpoint patterns that emerged naturally in complex recipes will become first-class language features.

10.1.2.3 3. Multi-tenant and multi-user workflows

Every recipe in this book is single-user: one person, one workflow, one output. Multi-tenant systems — where many users run the same workflow with their own data, credentials, and isolation requirements — require infrastructure that the current SPL runtime does not provide. This is not a language problem; it is a deployment problem. But it is a real barrier to production use at scale.

Momagrid's architecture addresses this directly: each workflow execution is isolated to a node, credentials are never included in the .spl file, and the adapter layer handles authentication at the node level. The Momagrid deployment model is inherently multi-tenant by design. As the network grows, this becomes the recommended production deployment model for SPL workflows at scale.

10.1.3 The Momagrid Vision: Status and Timeline

Momagrid is not a future promise. It is operational today on modest hardware.

The current network consists of independently operated GPU nodes running the Momagrid adapter, connected through a lightweight coordination layer. A workflow submitted to the grid is matched to an available node with sufficient VRAM for the requested model, executed in isolation, and the result returned to the submitter. The submitter never knows which node ran their workflow, and the node operator never sees the submitter's data — the protocol is designed for mutual opacity.

What the network needs to fulfill the democratic AI vision is scale: more nodes, more geographic diversity, more operator-owned hardware in the regions where cloud AI is expensive or unavailable. A student in Hefei with a GTX 1080 Ti can contribute a node today. A researcher in Nairobi with a used gaming PC can do the same. The coordination layer handles discovery, matching, and result routing. The operator sets their availability and model catalog; the grid does the rest.

Three things are needed to reach the critical scale where Momagrid becomes a meaningful alternative to cloud APIs:

1. **More recipes** — more `.spl` workflows that demonstrate what can be run on commodity hardware. This book is the first layer of that catalog.
2. **More node operators** — people and institutions willing to contribute inference capacity to the grid. The GTX 1080 Ti benchmark is the floor; any GPU above it can participate.
3. **More adapter integrations** — connections between the Momagrid grid and the tools people already use: databases, document stores, web search, enterprise APIs.

The recipe-maker in Chapter 9.2 is the mechanism for the first need. The second and third are community problems. If you have built something with SPL that you want to contribute to the catalog, or if you want to operate a Momagrid node, the project is open.

10.1.4 The Type Progression: Why "Simple" Works Forever

The 37 recipes in this book all work with text. That is where AI workflows live in 2026. But the language is designed to stay simple as the type system grows.

Here is the full progression, where it is now, and where it is going:

Version	Types	Era
SPL v1.0	TEXT	Prompt engineering
SPL v2.0	TEXT, NUMBER, BOOL, LIST	Agentic workflows
SPL v3.0	+ IMAGE, AUDIO, VIDEO	Multimodal AI
SPL v4.0	+ STREAM, GRAPH, SENSOR	Physical world — IoT, camera feeds, real-time data

In each version, the *language* stays the same. WHILE, EVALUATE, CALL, GENERATE, EXCEPTION — unchanged. The *world it can describe* grows larger.

A v2.0 workflow that calls GENERATE summarize(@text) becomes a v3.0 workflow that calls GENERATE describe_image(@photo) by changing one type declaration and one function name. The control flow, the CALL/GENERATE split, the adapter abstraction — none of it changes.

This is why "Simple" is the right name and not a temporary one. A language named "Structured Prompt Language" would have aged out the moment multimodal arrived — prompts are text, and text is just one modality. "Simple Programming Language" makes no such promise about modality. It promises only one thing: **that expressing what you want should never be harder than the thing itself.**

That promise holds for text. It holds for images. It holds for audio, video, and sensor streams. Simplicity is not a feature — it is the design commitment that all features are measured against.

```
-- v3.0: the workflow looks familiar
WORKFLOW analyze_product_photo
    INPUT:  @photo IMAGE, @question TEXT
    OUTPUT: @answer TEXT
DO
    GENERATE describe_image(@photo) INTO @description
    GENERATE answer(@question, @description) INTO @answer
    COMMIT @answer
END
```

Same grammar. New type. Zero relearning.

10.1.5 What SPL Is Not Trying to Be

Clarity about what SPL does not aim to replace is as important as understanding what it does.

SPL is not trying to replace Python. Python is the right tool for imperative logic, dynamic workflows, and anything that requires a full programming language's expressiveness. SPL calls Python functions through CALL. The two languages are complementary, not competing.

SPL is not trying to replace LangChain, AutoGen, or CrewAI. Those frameworks serve use cases that require dynamic agent construction, complex memory systems, and tight integration with specific enterprise platforms. If your system needs those capabilities, those frameworks are the right choice.

SPL is not trying to be a research language. It is a production tool. The design choices — human-reviewable syntax, adapter portability, the CALL/GENERATE split, explicit exception handling — are all oriented toward systems that real engineers maintain over time, not experiments that run once.

What SPL is trying to be: the simplest possible language for expressing the canonical agentic workflow patterns in a form that a SQL practitioner can read, a business stakeholder can review, and a system administrator can deploy portably across environments. That is a well-defined, non-trivial problem. The 37 recipes in this book are evidence that it is solvable.

10.1.6 On Adoption: An Honest Assessment

A reader who has reached this chapter has invested time in learning a language that is not yet widely adopted. That investment deserves a direct answer to the question the book has been avoiding.

SPL is not a mainstream tool. It is an open-source project with a small but growing user base, a published runtime, and a production-tested recipe catalog. It is not backed by a foundation or a large enterprise. It does not have a Stack Overflow tag with ten thousand questions. If your team evaluates SPL and asks "who else is using this?" — the honest answer is: early adopters, researchers, and practitioners who prioritized declarativeness over ecosystem size.

This is a real risk. A language that does not reach critical adoption becomes a maintenance liability. The roadmap items above — Momagrid scale, SPL v3, the specialty SPL model — are

not decoration. They are the work required to build the ecosystem that makes the adoption risk acceptable.

Here is where the project stands:

Milestone	Status
SPL 2.0 runtime published	Complete
37-recipe cookbook	Complete
Momagrid adapter live	Complete
Parser fixes for recipes 13, 14, 19	In progress
PARALLEL DO for concurrent execution	Planned
Native CHECKPOINT for long-running workflows	Planned (v3)
Code-RAG for Text2SPL	Deferred — awaiting spec stability
Specialty SPL model (Qwen-Coder fine-tuned)	Deferred — awaiting training corpus

If your team evaluates SPL and decides it is not the right choice, here is what you take away regardless: the CALL/GENERATE discipline (separating deterministic operations from probabilistic ones), the habit of writing workflows as declarative artifacts reviewable by non-engineers, and the pattern library in this book. Every one of these transfers to LangGraph, AutoGen, or CrewAI. The mental model is portable even when the syntax is not.

10.1.7 What Comes Next for You

You have 37 recipes. You have the recipe-maker. You have the adapter abstraction that lets you develop locally and deploy to the grid without changing your workflow.

The next step is yours to choose:

If you are a SQL practitioner building your first agentic system: pick the recipe closest to your use case, run it, read the annotated .spl, and modify it for your data. The pattern is the scaffold. Your domain knowledge is the dish.

If you are in the Global South with commodity hardware: set up Ollama, install spl−llm, and run section 1.1.1. The stack runs on a GTX 1080 Ti. If that is more than you have, try section 1.1.1 with a smaller model. The barrier to entry is a pip install, not a credit card.

If you are an engineering leader evaluating orchestration tools: the SPL repository contains all 37 recipes, the full runtime, and the benchmark data. The LOC comparison to equivalent LangGraph and AutoGen implementations is in the arxiv paper. The governance argument — that non-technical stakeholders can review an SPL script and approve it — is testable on the first recipe you run.

If you want to contribute: the catalog grows one recipe at a time. The recipe-maker generates the scaffold; your domain knowledge writes the recipe. Open source, CC-BY licensed, globally accessible.

Next: Chapter 9.2 — The Recipe Maker: SPL Eating Its Own Cake

Chapter 10.2

The Recipe Maker: SPL Eating Its Own Cake

10.2.1 The Concept: What 37 Recipes Taught Us

You have now built 37 recipes. Each one follows the same rhythm.

You identify a task. You find some data. You write a tools.py to load and process that data deterministically. You write a .spl workflow that CALLs the tools for everything code does better, and GENERATEs with the LLM for everything that genuinely requires reasoning. You write a readme.md. You run it, taste it, fix it, run it again.

After doing this 37 times, a pattern emerges that is hard to miss: **every recipe is the same shape**. The data files change. The domain changes. The workflow steps change. But the structure — data catalog, tool layer, workflow, documentation — is invariant.

A SQL practitioner would recognize this immediately. You have discovered a schema. You have been hand-writing the same INSERT statement 37 times. The next move is obvious: write a procedure that generates the INSERT for you.

That procedure is **recipe 00: the recipe-maker**.

The recipe-maker is SPL eating its own cake. It is a workflow that takes a plain-language concept as input and produces a complete, runnable recipe as output — the .spl file, the tools.py, the sample data, and the documentation. It applies everything the cookbook taught: CALL for deterministic operations, GENERATE only where reasoning is required, grounding with real data before asking the LLM to think.

This is not a parlor trick. It is the natural destination of a mature declarative system: **the abstractions become generative**. Once you can describe a pattern precisely enough to teach it to a reader, you can describe it precisely enough to execute it as code.

10.2.2 Cooking as Science and Art

Before the recipe chapter, a word about expectations — because the recipe-maker is frequently misunderstood on first encounter.

Cooking is both a science and an art. The science gives you reliability: the soufflé rises every time, the bread proofs in 90 minutes, the sauce reduces to the right consistency. The art gives you excellence: the seasoning is right, the plating is thoughtful, the meal is worth eating.

A recipe in a cookbook does the science for you. It tells you the ratios, the temperatures, the timing. It does not give you the art. That requires a cook — someone who tastes as they go, adjusts, learns what "just right" feels like in this kitchen with this oven at this altitude.

The recipe-maker is a recipe for writing recipes. It does the science: it generates a structurally complete workflow, a tools file with the right shape, sample data, and documentation. It solves the **bootstrap problem** — the hardest moment in writing any recipe is facing an empty directory. The recipe-maker eliminates the blank page.

What it does not do is give you the finished dish. The generated workflow will need tuning. The tools will need edge-case handling. The sample data will cover the happy path but not the corners. The CALL/GENERATE split will be close but occasionally wrong. This is expected and normal. The recipe-maker compresses the bootstrap problem, not the craft problem.

Think of it as a first draft from a very well-read sous-chef who has studied all 37 recipes and understands the pattern cold, but has never tasted *your* dish before.

10.2.3 The Full Lifecycle

The recipe-maker introduces a lifecycle that goes beyond the single-run model of the earlier chapters. Where every recipe from 01 to 37 is a linear pipeline — input → generate → commit — the recipe-maker exposes a cycle:

```
concept


GENERATE (plan + components)


output_dir (workflow.spl, tools.py, data.json, readme.md, recipe_plan.md)


—feedback: taste the cake ←          human (default) or llm-judge

    fail → iterate → taste again

    pass


        —reflect: document lessons learned


        —publish: write publication docs + upload to registry
```

Four phases, two gates. The −−reflect and −−publish flags only fire on a passing taste — you do not document or distribute a recipe that has not been validated. This is a deliberate design choice:

the registry should contain only recipes that work.

Version 1 of the recipe-maker implements the **generate phase** only. The −−feedback, −−reflect, and −−publish phases are scaffolded in the workflow as commented-out blocks, visible and ready to activate. They are described here because they shape how you work with the generated output even before v2 exists: generate, taste manually, iterate, then publish when it's right.

10.2.4 The CLI Contract

The recipe-maker introduces five new CLI concepts that will become the standard interface for production SPL workflows:

```
spl run cookbook/00_recipe_maker/recipe_maker.spl \
    --adapter ollama -m gemma3 \
    --tools cookbook/00_recipe_maker/tools.py \
    concept="Customer churn predictor with SHAP explainability" \
    output_dir="cookbook/38_churn_predictor/" \
    dataset="data/customers.csv" \
    resources="references/churn_taxonomy.md" \
    feedback=human
```

Each flag has a role:

Flag	Layer	Purpose
−−adapter / −m	Generative	Which LLM does the creative work
−−tools	Deterministic	Which Python tools handle the code work
dataset=	Data	Local file (JSON/CSV/TXT) or connection string the recipe will operate on
resources=	Knowledge	Reference material, domain docs, related work — grounding for the planner
output_dir=	Artifacts	Where generated files are written
feedback=	Validation	human (default, stops for review) or llm−judge (v2)
concept=	Intent	Plain-language description — what the recipe should do

If you squint, this is a compiler invocation. concept is the source. output_dir is the binary. −−adapter is the compiler backend. −−tools is the standard library. dataset and resources are the headers.

10.2.5 section ?0: Recipe Maker

10.2.5.1 The Pattern

You have been writing the same structural shape 37 times. Each time: create a data directory, write a tools.py with a load_X(), a list_X(), and processing tools, write a .spl workflow that CALLs the tools and GENERATEs with the LLM, write a readme.md.

The naive approach is to keep writing this by hand. It is not painful exactly — each recipe is different enough that hand-crafting is satisfying. But it is slow, and the structural parts (file layout, tool scaffolding, readme sections) are not where the creativity lives. They are ceremony.

The recipe-maker is the ceremony automator. Given a concept, it generates the complete recipe skeleton — not perfect, not finished, but structurally sound and immediately runnable. You spend your time on the craft: tuning the GENERATE step names, adding edge cases to the data, adjusting the CALL/GENERATE split. The scaffolding is already there.

10.2.5.2 The SPL Approach

The recipe-maker is itself a recipe: CALL to load the pattern catalog and user-supplied context, GENERATE to plan and create each component, CALL to write all artifacts to disk and notify for review.

It demonstrates the core principle of the entire cookbook in miniature: **every step that does not require reasoning is a CALL; every step that does is a GENERATE**. The file I/O, the pattern lookup, the artifact writing, the review formatting — all CALL. The planning, the workflow generation, the tools generation, the data generation, the documentation — all GENERATE, each grounded by the deterministic context loaded before it.

10.2.5.3 The .spl File (Annotated)

```
-- Recipe 00: Recipe Maker (Meta-Recipe)
-- SPL 2.0 eating its own cake.

-- System context functions injected as LLM system prompts — zero LLM cost.
-- These ground every GENERATE step with precise expectations.

CREATE FUNCTION spl_syntax_guide()          -- teaches the LLM exact SPL syntax
RETURNS TEXT AS $$ ... $$;

CREATE FUNCTION tools_py_guide()             -- teaches the LLM the @spl_tool pattern
RETURNS TEXT AS $$ ... $$;

CREATE FUNCTION recipe_quality_rubric()      -- defines what "complete recipe" means
RETURNS TEXT AS $$ ... $$;

WORKFLOW recipe_maker
    INPUT:  @concept     TEXT DEFAULT '',        -- the recipe idea, in plain langua
            @output_dir  TEXT DEFAULT 'cookbook/output/',
            @dataset     TEXT DEFAULT '',         -- optional: path to real data
            @resources   TEXT DEFAULT '',         -- optional: reference material
```

```
            @feedback     TEXT DEFAULT 'human'     -- validation mode
    OUTPUT: @review_notification TEXT
DO
    -- Phase 1: Load context -- all CALL, zero LLM cost
    CALL load_patterns('')     INTO @patterns_catalog  -- 11 patterns from the cookb
    CALL load_dataset(@dataset)   INTO @dataset_context  -- real data as grounding
    CALL load_resources(@resources) INTO @resources_context

    -- Phase 2: Plan -- LLM understands intent and chooses the right pattern
    -- Grounded by: pattern catalog + quality rubric + user data context
    GENERATE plan_recipe(
        @concept,
        @patterns_catalog,
        @dataset_context,
        @resources_context,
        recipe_quality_rubric()
    ) INTO @recipe_plan

    -- Phase 3: Generate components -- each GENERATE grounded by the plan
    -- SPL syntax guide prevents the LLM from inventing non-existent syntax
    GENERATE generate_spl_workflow(
        @recipe_plan, @concept, spl_syntax_guide()
    ) INTO @spl_content

    -- Tools guide ensures @spl_tool decorator, str parameters, str return types
    GENERATE generate_tools_py(
        @recipe_plan, @concept, @dataset_context, tools_py_guide()
    ) INTO @tools_content

    -- Dataset grounded by user-supplied data if provided
    GENERATE generate_sample_data(
        @recipe_plan, @concept, @dataset_context
    ) INTO @data_content

    -- Readme grounded by the actual generated artifacts, not the plan
    GENERATE generate_readme(
        @recipe_plan, @concept, @spl_content, @tools_content,
        recipe_quality_rubric()
    ) INTO @readme_content

    -- Phase 4: Write artifacts -- all CALL, zero LLM cost
    CALL write_artifact(@output_dir, 'workflow.spl',     @spl_content)
INTO @_
    CALL write_artifact(@output_dir, 'tools.py',         @tools_content) INTO @_
    CALL write_artifact(@output_dir, 'data.json',        @data_content)
INTO @_
    CALL write_artifact(@output_dir, 'readme.md',        @readme_content) INTO @_
    CALL write_artifact(@output_dir, 'recipe_plan.md',   @recipe_plan)
```

```
INTO @_

    -- Phase 5: Summarise and notify -- all CALL, zero LLM cost
    CALL list_artifacts(@output_dir)
INTO @artifact_list
    CALL notify_review(@output_dir, @artifact_list, @feedback)   INTO @review_noti

    COMMIT @review_notification WITH
        status     = 'awaiting_review',
        concept    = @concept,
        output_dir = @output_dir

    --   v2 Placeholders (feedback → reflect → publish)
    -- [v2] GENERATE validate_artifacts(...) INTO @validation
    -- [v2] EVALUATE @validation
    --          WHEN 'pass' THEN
    --              GENERATE reflect(...)   INTO @reflection
    --              GENERATE publish_writeup(...) INTO @writeup
    --              CALL register_recipe(...)
    --          WHEN 'fail' THEN RETRY
    -- [v2] END

EXCEPTION
    WHEN GenerationError THEN
        COMMIT 'Recipe generation failed. Check the concept description and try aga
            WITH status = 'error', concept = @concept
END
```

The SQL analogy: load_patterns() is a lookup join against the pattern catalog. plan_recipe() is the query planner. The four generate_* steps are the column projections. The write_artifact() calls are the INSERT INTO OUTPUT TABLE. notify_review() is the audit log entry.

10.2.5.4 Running It

```
spl run cookbook/00_recipe_maker/recipe_maker.spl \
    --adapter ollama -m gemma3 \
    --tools cookbook/00_recipe_maker/tools.py \
    concept="Interview simulator for data science roles with scoring rubric" \
    output_dir="/tmp/spl_recipes/ds_interview/"
```

Expected output (abbreviated):

```
RECIPE GENERATION COMPLETE

GENERATED ARTIFACTS in /tmp/spl_recipes/ds_interview/

  data.json                          2,841 bytes
  readme.md                          4,203 bytes
  recipe_plan.md                     1,876 bytes
```

```
  tools.py                                  5,512 bytes
  workflow.spl                              3,940 bytes

  Total: 5 file(s), 18,372 bytes


NEXT STEPS — Human Review Required:

  1. Review the generated artifacts in:
     /tmp/spl_recipes/ds_interview/

  2. Run the generated workflow:
     spl run /tmp/spl_recipes/ds_interview/workflow.spl
       --adapter ollama -m <model>
       --tools /tmp/spl_recipes/ds_interview/tools.py

  3. Taste the cake — does it produce the expected output?

  4. If satisfied: re-run with --reflect and --publish flags (v2).
     If not: edit the artifacts and re-run with your corrections.

  Status: awaiting_human_review
```

Total LLM calls: **5** (plan + spl + tools + data + readme). Everything else is deterministic Python.

10.2.5.5 What Just Happened

The workflow made exactly 5 LLM calls, in sequence:

1. **plan_recipe** — the most important call. The LLM read the pattern catalog, the quality rubric, and the concept description. It produced a structured plan naming the pattern (multi_persona), the workflow inputs and outputs, the GENERATE steps needed, the tools required, and the shape of the sample data. This plan becomes the grounding context for every subsequent call.

2. **generate_spl_workflow** — the LLM wrote the .spl file, guided by the recipe plan and the spl_syntax_guide() context function that teaches exact SPL syntax. Without the syntax guide, the LLM would invent plausible but invalid clauses.

3. **generate_tools_py** — the LLM wrote the tools file, guided by the tools_py_guide() context that specifies @spl_tool decoration, str parameter types, and str return types. The dataset context (if provided) shaped the data-loading functions.

4. **generate_sample_data** — the LLM generated realistic sample records that match the data schema described in the plan.

5. **generate_readme** — the LLM documented the recipe, receiving the actual generated .spl and tools.py content as input rather than just the plan. This means the readme reflects what was actually generated, not what was intended.

All five calls are grounded: none asks the LLM to reason from scratch. Each receives the accumulated context from the steps before it. This is the chain pattern (recipe 09) applied to the recipe-making process itself.

After the five LLM calls, write_artifact() is called five times (one per file), list_artifacts() produces the bundle summary, and notify_review() formats the human-facing output. Zero additional LLM cost.

10.2.5.6 Reproducibility Note

Latency varies significantly with concept complexity. A simple extraction recipe concept ("Extract structured fields from invoice text") plans and generates in 45–90 seconds on a GTX 1080 Ti with gemma3. A complex multi-persona concept ("Interview simulator with role catalog, candidate profi takes 2–4 minutes — the plan step alone requires reasoning across all 11 patterns and the full quality rubric.

CV% is higher than most earlier recipes because the plan step involves genuine reasoning that varies run-to-run. Two runs of the same concept will produce structurally similar but not identical output. This is expected: the recipe-maker is a creative tool, not a deterministic one. What matters is whether the output is runnable and close to the target, not whether it is identical across runs.

Run with a stronger model (−m mistral or −−adapter claude_cli −m claude−sonnet−4−6) for more reliable plan quality on complex concepts.

10.2.5.7 When to Use This Pattern

The recipe-maker is the right tool when you are facing a blank directory and a concept description. Use it to:

- **Bootstrap a new recipe** from a concept you have not implemented before
- **Recreate an existing recipe** to measure how close the generator gets — this is the calibration test: run the recipe-maker against a concept whose ground truth you know
- **Iterate on an existing recipe** by providing the original .spl and tools.py as resources= — the generator will treat them as reference and produce a closer first draft
- **Teach the pattern to a new team member** — recipe_plan.md explains the LLM's reasoning at each step, making it a readable design document even before the recipe is complete

Do not use the recipe-maker when:

- The recipe is a minor variation of an existing one — editing the original is faster than generating from scratch
- The concept is vague — the quality of the plan is proportional to the specificity of the concept; "something with sentiment" will produce a worse plan than "batch sentiment classifier for customer suppor
- You need a production-ready artifact on the first run — the recipe-maker produces a baseline, not a finished dish

10.2.5.8 Exercises

1. **The calibration run.** Run the recipe-maker with concept="Customer support triage with order lookup an and output_dir pointing to a fresh directory. Compare the generated workflow to 28_support_triage. Count how many lines you would need to change to reach parity. This number is your current recipe-maker quality score.

2. **Dataset grounding.** Re-run the calibration with dataset="cookbook/28_support_triage/orders.json" added. Does the generated data.json more closely match the original? Does the tools.py get the schema right?
3. **Concept refinement.** Take a concept that produced a poor plan in your first run. Improve the concept description — add domain vocabulary, specify the pattern explicitly, name the data schema. Observe how much the plan quality improves. The lesson: the recipe-maker multiplies the quality of your concept, it does not replace it.
4. **Swap the model.** Run the same concept with −−adapter ollama −m mistral and then with −−adapter claude_cli −m claude−sonnet−4−6. Compare the plans. Stronger models produce better structural reasoning; the pattern catalog and rubric grounding matter more on weaker models.

10.2.6 A Note on What This Means

Thirty-seven recipes in, you have a tool that writes recipes. This is not the end of the cookbook — it is the beginning of a second order of productivity.

The recipes you write by hand will always be better than the recipes the recipe-maker generates on the first pass. You know the domain. You understand the edge cases. You have taste. The recipe-maker does not have taste; it has pattern recognition and grounding.

What the recipe-maker gives you is **the bootstrap**. The blank-page problem vanishes. The structural ceremony is automated. The first draft exists in the time it takes to run five LLM calls. What remains is the craft: reading recipe_plan.md, running the generated workflow, tasting the output, adjusting the seasoning, running again.

This is the conductor's work. The score is written. Now it needs to be played.

This concludes A Cookbook of SPL Recipes. See Chapter 9.1 — The Road Ahead for the forward-looking context behind everything you have built.

Part 11

Tools & Community

Chapter 11.1

Recipe Kitchen: A Browser UI for SPL

11.1.1 What Is Recipe Kitchen?

Every recipe in this book runs from the command line. That is by design — SPL is a language for developers and data engineers who are comfortable in a terminal. But not every team member who needs to *run* a workflow is the same person who *writes* it.

Recipe Kitchen is a Streamlit application that puts all 40 cookbook recipes one click away. No terminal, no flags, no adapter syntax. You select a recipe, adjust its inputs, and press Run.

It is also the place where new recipes are born. The Recipe Maker tab takes a plain-English description and calls the Text2SPL compiler (Recipe #22) to generate a working SPL workflow. You review the code in a built-in editor, run it, and — if you like what you see — save it to a file.

11.1.2 Starting the Kitchen

```
pip install streamlit          # one-time setup
cd references/SPL20
streamlit run ../../scripts/recipe_kitchen.py
```

Your browser opens at http://localhost:8501. The kitchen has three tabs.

11.1.3 Tab 1 — Playground

The Playground is for running existing recipes interactively.

How it works:

1. Filter by category (basics, patterns, reasoning, multi-agent, …)
2. Select a recipe from the dropdown — title and description appear immediately
3. Edit any input parameters. The kitchen reads the args field from cookbook_catalog.json and renders every key=value pair as a live text field. Change the topic, the threshold, the model — without touching a config file.

4. Press **Run Recipe**. Output, stderr (if any), exit code, and elapsed time appear inline.

The base command is shown read-only above the parameter fields. You always see exactly what will be executed.

Example: Recipe #7 (Progressive Summary) has a @source_text input. In the Playground, that becomes an editable text box. Paste any document, click Run, and the progressive-summary workflow runs against your content — the same SPL, your data.

11.1.4 Tab 2 — Recipe Maker

Recipe Maker bridges the gap between intent and implementation.

The workflow:

1. Describe what you want in plain English. Be specific: "Summarize a support ticket, classify its urgency (low/medium/high), and draft a reply if urgency is high."
2. Choose a mode: auto (compiler decides), prompt (single GENERATE), or workflow (multi-step WORKFLOW).
3. Choose an adapter and model. claude_cli with a capable model produces the best SPL for complex workflows.
4. Press **Generate SPL**. The Text2SPL compiler (Recipe #22) generates a .spl file and loads it into the inline editor.
5. Read the code. Edit it if needed — the editor is fully writable.
6. Press **Run** to execute. Or press **Save** to write it to a named .spl file in the cookbook directory.

Recipe Maker does not write to cookbook_catalog.json automatically. When a generated recipe proves useful, add it to the catalog manually and it will appear in the Playground on the next page load.

11.1.5 Tab 3 — Bake-All

Bake-All runs every active recipe in the catalog and saves a timestamped log.

When to use it:

- After updating the SPL runtime: verify all 40 recipes still pass
- After adding a new recipe: confirm it does not break anything downstream
- On a schedule (cron job or CI): track regression over time

How it works:

Press **Bake All Active Recipes**. The kitchen calls run_all.py as a subprocess with a 600-second timeout. When the run completes, the log is saved to cookbook/out/run_all_YYYYMMDD_HHMMSS.md.

The Log Browser shows every saved log. Select one to see an approximate pass/fail count (based on / /ERROR markers in the log) and the full output.

11.1.6 Design Notes

Why Streamlit? SPL recipes are Python subprocesses under the hood. Streamlit makes it trivial to wrap any subprocess in a browser UI — no JavaScript, no REST API, no front-end build step. The entire kitchen is ~350 lines of Python.

Why not a web server? Recipe Kitchen is a local tool. It runs on your machine, reads your filesystem, and executes your SPL runtime. It is not designed to be deployed publicly. For team use, run it on a shared machine and access it over your local network — Streamlit's ––server.address flag makes this straightforward.

The catalog as single source of truth. The Playground and Bake-All both read cookbook_catalog.json. Add a recipe to the catalog and it appears everywhere immediately — no UI code changes required. This is the same principle that drives SPL itself: declare the data, let the system handle the display.

11.1.7 The Vision: A Community Portal

Recipe Kitchen is the seed of something larger.

The current version runs locally. A future version could connect to a community catalog — a shared registry of SPL recipes contributed by users around the world. Browse recipes by category, adapter, or star rating. Fork a recipe, modify it in the editor, and submit it back. Rate recipes you find useful.

This is not a feature for v1.0. It is the direction.

SPL's value compounds with adoption. Every recipe published to a community catalog makes SPL more useful for the next person. The Recipe Kitchen UI is the interface that makes contributing to that catalog accessible to non-developers — the analyst who built a great summarization workflow, the support engineer who automated ticket triage, the teacher who built a quiz generator for their classroom.

The language is the grammar. The recipes are the vocabulary. The kitchen is the door.

Chapter 11.2

SPL Cheatsheet

All SPL 2.0 syntax on one page.

11.2.1 Core Statements

Statement	Purpose
PROMPT name	Single-shot LLM invocation
SELECT ... GENERATE fn()	Context assembly + generation
WORKFLOW name	Multi-step agentic workflow
PROCEDURE name(args)	Reusable sub-workflow
CREATE FUNCTION name	Prompt template definition

11.2.2 WORKFLOW Structure

```
WORKFLOW workflow_name
  INPUT:
    @param1 TYPE DEFAULT value,
    @param2 TYPE DEFAULT value
  OUTPUT:
    @result1 TYPE,
    @result2 TYPE
DO
  -- body
  COMMIT @result1 WITH key = value
EXCEPTION
  WHEN ErrorType THEN
    -- handler
END
```

11.2.3 Keywords and Clauses

11.2.3.1 Data flow

Keyword	Meaning
INTO @var	Store result in variable
:=	Assign value to variable
CAST(x AS TYPE)	Type conversion
||	String concatenation

11.2.3.2 Generation

Keyword	Meaning
GENERATE fn(@arg)	Call LLM via named function template
CALL tool.fn(@arg)	Call deterministic Python tool (no tokens)
SELECT expr AS alias	Assemble context for next GENERATE

11.2.3.3 Control flow

Keyword	Meaning
WHILE condition DO ... END	Loop while condition is true
EVALUATE @var WHEN ... END	Branch on variable value
WHEN > threshold	Numeric comparison in EVALUATE
WHEN 'value'	String match in EVALUATE
ELSE	Default branch in EVALUATE
BREAK	Exit WHILE loop
RETRY	Re-execute current step (in EXCEPTION handler)

11.2.3.4 State

Keyword	Meaning
COMMIT @var WITH key=val	Write output and metadata
STORE @var IN memory.key	Write to named memory slot
memory.get('key')	Read from named memory slot

11.2.3.5 Exception handling

Keyword	Meaning
EXCEPTION	Begin exception handler block
WHEN ErrorType THEN	Handle specific error type

11.2.4 Built-in Error Types

Error	When it fires
GenerationError	LLM call failed or returned malformed output
MaxIterationsReached	WHILE loop hit its bound without completing
BudgetExceeded	Token or cost limit exceeded
ContextLengthExceeded	Input exceeded model's context window
HallucinationDetected	Downstream validator flagged hallucination
ValidationError	Structured output failed schema validation
TimeoutError	Operation did not complete within time limit
FileNotFoundError	Tool connector could not find the specified file

11.2.5 Tool Connectors (Planned)

```
spl run script.spl --connector pdf=pymupdf --connector transcribe=whisper
```

Function	Operation	Default backend
tool.pdf_to_md(file)	PDF → Markdown	pymupdf
tool.md_to_pdf(text)	Markdown → PDF	pandoc
tool.transcribe(file)	Audio → Text	whisper
tool.tts(text)	Text → Audio	piper
tool.ocr(image)	Image → Text	tesseract
tool.caption(image)	Image → Caption	llava
tool.read_file(path)	File → Text	built-in
tool.write_file(path, text)	Text → File	built-in
tool.get_user_input(prompt)	Console → Text	built-in
tool.display(text)	Text → Console	built-in

11.2.6 Adapter CLI Flags

```
spl run script.spl \
  --adapter ollama \          # or: claude_cli, openrouter, momagrid
  -m gemma3 \                 # model name (adapter-specific)
  --input @param=value \      # override INPUT parameters
  --tools tools.py \          # Python tool definitions
  --connector pdf=pymupdf     # tool connector backend
```

Adapter	Backend
ollama	Local Ollama server (open models)
claude_cli	Anthropic API (Claude models)
openrouter	OpenRouter API (multi-provider)
momagrid	Momagrid decentralized grid

11.2.7 CALL vs GENERATE

	CALL	GENERATE
What it does	Calls a Python function	Calls an LLM
Output	Deterministic	Probabilistic
Token cost	Zero	Non-zero
Use for	Computation, I/O, tools	Generation, reasoning, evaluation

Rule: If a Python function can do it reliably, use CALL. Reserve GENERATE for tasks that require language understanding.

11.2.8 SQL → SPL Mapping

SQL / PL-SQL	SPL equivalent
SELECT col FROM table	SELECT @var AS alias
INSERT INTO result	COMMIT @var WITH key=val
CALL stored_proc()	CALL tool.fn() or PROCEDURE
IF / CASE WHEN	EVALUATE @var WHEN ... END
WHILE / LOOP	WHILE condition DO ... END
BEGIN ... EXCEPTION ... END	WORKFLOW ... EXCEPTION WHEN ... END
CREATE PROCEDURE	WORKFLOW or PROCEDURE
CREATE FUNCTION	CREATE FUNCTION (prompt template)
EXECUTE / CALL	spl run script.spl --adapter X

11.2.9 Input / Output Types

Type	SPL keyword
Text	TEXT
Integer	INT
Floating-point	FLOAT
Boolean	BOOL
JSON object	JSON
List	LIST

11.2.10 Recipe Template Structure

Every recipe follows this eight-section structure:

1. **The Pattern** — what problem this solves
2. **The SPL Approach** — why SPL makes it natural
3. **The .spl File (Annotated)** — full source with comments
4. **The SQL Analogy** — structural parallel for SQL practitioners
5. **Running It** — exact commands and expected output
6. **What Just Happened** — execution trace
7. **Reproducibility Note** — latency, hardware, variability
8. **When to Use This Pattern** — use cases, anti-patterns, alternatives

Chapter 11.3

SPL Standard Library

Every SQL database ships with a standard library of functions — UPPER, ROUND, COALESCE, JSON_GET — so practitioners never need to rewrite the same utility logic. SPL 2.0 follows the same philosophy. The SPL Standard Library provides built-in functions available in every workflow without any −−tools flag or import statement.

```
-- These just work -- no --tools needed
CALL to_float(@score_str) INTO @score
CALL upper(@name)         INTO @name_upper
CALL coalesce(@a, @b, 'default') INTO @result
CALL json_get(@payload, 'status') INTO @status
```

The stdlib is auto-loaded when the spl package is imported. It is the equivalent of SQL's built-in function set.

11.3.1 Type Conversion

The most common need: LLMs return everything as text. These functions cast that text to the type you need.

Function	SQL equivalent	Description
to_float(value)	CAST(x AS FLOAT)	Extract first decimal number; returns 0.0 on failure
to_int(value)	CAST(x AS INTEGER)	Truncate to integer; returns 0 on failure
to_text(value)	CAST(x AS TEXT)	Any value → string
to_bool(value)	CAST(x AS BOOLEAN)	'true'/'false'; truthy: 1, true, yes, on, t, y

to_float is deliberately lenient — it extracts the first numeric token from the string, so LLM output like "Quality score: 0.85\n" or "I rate this 0.85 out of 1.0" both return 0.85.

```
-- Pattern: LLM returns a score string; compare it numerically
GENERATE rate_quality(@text) INTO @score_str
```

```
CALL to_float(@score_str) INTO @score

EVALUATE @score
    WHEN > 0.8 THEN COMMIT @text WITH grade = 'A'
    WHEN > 0.6 THEN COMMIT @text WITH grade = 'B'
    ELSE       COMMIT @text WITH grade = 'C'
END
```

11.3.2 String Functions

Mirrors the SQL string function set exactly.

Function	SQL equivalent	Description
upper(value)	UPPER(x)	Uppercase
lower(value)	LOWER(x)	Lowercase
trim(value)	TRIM(x)	Strip leading/trailing whitespace
ltrim(value)	LTRIM(x)	Strip leading whitespace
rtrim(value)	RTRIM(x)	Strip trailing whitespace
length(value)	LEN(x) / LENGTH(x)	Character count
substr(value, start, length)	SUBSTR(x, s, n)	1-based substring; omit length for rest of string
replace(value, old, new)	REPLACE(x, old, new)	Replace all occurrences
concat(a, b, ...)	CONCAT(...)	Concatenate strings
instr(value, search)	INSTR(x, s)	1-based index of first match; 0 if not found
lpad(value, width, fill)	LPAD(x, w, f)	Left-pad to width
rpad(value, width, fill)	RPAD(x, w, f)	Right-pad to width
split_part(value, delim, n)	SPLIT_PART(x, d, n)	1-based nth token after splitting
reverse(value)	REVERSE(x)	Reverse a string

11.3.3 Pattern Matching

Function	SQL equivalent	Description
like(value, pattern)	x LIKE p	SQL wildcards: % = any chars, _ = any char
startswith(value, prefix)	—	Returns 'true' or 'false'
endswith(value, suffix)	—	Returns 'true' or 'false'
contains(value, substring)	—	Returns 'true' or 'false'
regexp_match(value, pattern)	REGEXP_LIKE(x, p)	Python regex; returns 'true' or 'false'

Pattern functions return the strings 'true' and 'false' so they compose with EVALUATE:

```
CALL startswith(@category, 'error') INTO @is_error
EVALUATE @is_error
    WHEN 'true' THEN COMMIT @event WITH severity = 'high'
    ELSE        COMMIT @event WITH severity = 'normal'
END
```

11.3.4 Numeric Functions

Function	SQL equivalent	Description
abs_val(value)	ABS(x)	Absolute value
round_val(value, decimals)	ROUND(x, n)	Round to n decimal places (default 0)
ceil_val(value)	CEIL(x)	Smallest integer value
floor_val(value)	FLOOR(x)	Largest integer value
mod_val(dividend, divisor)	MOD(a, b)	Remainder
power_val(base, exponent)	POWER(x, n)	Exponentiation
sqrt_val(value)	SQRT(x)	Square root
sign_val(value)	SIGN(x)	−1, 0, or 1
clamp(value, lo, hi)	—	Constrain to [lo, hi] range

11.3.5 Conditional Functions

Function	SQL equivalent	Description
coalesce(a, b, ...)	COALESCE(...)	First non-null, non-empty argument
nullif(value, compare)	NULLIF(x, y)	Returns '' if value equals compare
iif(condition, true_val, false_val)	IIF(c, t, f)	Inline if; truthy: 1, true, yes

```
-- Use coalesce to provide fallback values from LLM output
GENERATE extract_summary(@doc) INTO @summary
CALL coalesce(@summary, @doc) INTO @final   -- fall back to original if summary is
```

11.3.6 Null and Empty Checks

Function	SQL equivalent	Description
isnull(value)	IS NULL	'true' if None or empty string
nvl(value, default)	NVL(x, d)	Return default if null/empty (Oracle-style)
isblank(value)	—	'true' if empty or only whitespace

11.3.7 Text Aggregates

Useful for measuring LLM output properties without calling another LLM.

Function	Description
word_count(value)	Whitespace-delimited token count
char_count(value)	Characters excluding whitespace
line_count(value)	Newline-separated line count

```
-- Enforce output length without a second LLM call
GENERATE write_summary(@doc) INTO @summary
CALL word_count(@summary) INTO @words

EVALUATE @words
    WHEN > 150 THEN
        GENERATE shorten(@summary) INTO @summary
    ELSE
        -- length is acceptable
END
```

11.3.8 JSON Functions

Essential for structured-output workflows where LLMs return JSON payloads.

Function	SQL equivalent	Description
json_get(json, key)	JSON_VALUE(x, path)	Extract field; supports dot notation 'a.b'
json_set(json, key, value)	—	Set a top-level key; returns updated JSON string
json_keys(json)	—	Comma-separated list of top-level keys
json_length(json)	JSON_ARRAY_LENGTH(x)	Number of keys (object) or items (array)
json_pretty(json)	—	Pretty-print with 2-space indent

```
-- Extract fields from a structured LLM response
GENERATE classify(@text) INTO @json_response
CALL json_get(@json_response, 'category') INTO @category
CALL json_get(@json_response, 'confidence') INTO @conf_str
CALL to_float(@conf_str) INTO @confidence
```

11.3.9 Date and Time

Function	SQL equivalent	Description
now_iso()	NOW()	Current UTC time as YYYY−MM−DDTHH:MM:SS
date_format_val(date, fmt)	DATE_FORMAT(x, f)	Reformat ISO date with strftime format string
date_diff_days(date_a, date_b)	DATEDIFF(a, b)	Days between two ISO dates (a − b)

11.3.10 Hashing

Function	SQL equivalent	Description
md5_hash(value)	MD5(x)	MD5 hex digest — useful for deduplication keys
sha256_hash(value)	SHA2(x, 256)	SHA-256 hex digest

11.3.11 List / Array Helpers

SPL 2.0 has a native LIST type. [] creates an empty list; the list_∗ functions operate on it. Internally lists are JSON arrays — no delimiter ambiguity, no split-on-comma surprises.

Function	Description
list_append(list, item)	Append an item; returns updated list
list_concat(list, separator)	Join all elements into a single TEXT string
list_count(list)	Number of elements
list_get(list, index)	0-based element access

```
@items := []
@items := list_append(@items, 'first')
@items := list_append(@items, 'second')
@n     := list_count(@items)                    -- '2'
@text  := list_concat(@items, ', ')             -- 'first, second'
@item  := list_get(@items, 0)                   -- 'first'
```

The map-reduce pattern (Chapter 7.1) is the canonical use: accumulate chunk summaries into a LIST during the Map phase, then list_concat before the Reduce GENERATE call.

11.3.12 File I/O

write_file and read_file are built-in — no −−tools flag, no tools.py entry needed. They are deterministic CALL operations: zero LLM tokens, always.

Function	Description
write_file(path, content)	Write content to file; creates parent dirs automatically. Returns 'written: <path>'.
read_file(path)	Read file contents as TEXT. Returns '' if file does not exist.

```
-- Save an intermediate result for inspection
CALL write_file(f'{@log_dir}/summary_{@i}.md', @chunk_summary) INTO @_

-- Load a prompt template from disk
CALL read_file('prompts/system_prompt.md') INTO @system_prompt
GENERATE analyze(@doc, @system_prompt) INTO @result
```

INTO @_ discards the return value — the SPL equivalent of Python's _ = fn(). Use it when the side-effect (writing the file) is what you want and the return value is not needed.

File paths are relative to the working directory where spl run is invoked. Use the @log_dir INPUT parameter pattern to keep paths configurable:

```
WORKFLOW my_workflow
    INPUT:
        @document TEXT,
        @log_dir TEXT DEFAULT 'my_workflow/logs'
    OUTPUT: @result TEXT
DO
    -- ... processing ...
    CALL write_file(f'{@log_dir}/result.md', @result) INTO @_
END
```

11.3.13 Stdlib vs. UDFs

The stdlib covers routine operations including file I/O, list manipulation, string processing, JSON parsing, type conversion, and datetime. When you need domain-specific logic — PII detection, database lookups, external API calls, or custom computation — write a UDF. See Chapter 10.4 for how to extend SPL with your own tools.

The decision rule: if it has a clear, general-purpose definition independent of your domain, it belongs in stdlib. If it requires your schema, your credentials, or your business rules, it belongs in a tools.py UDF.

Chapter 11.4

Extending SPL with User-Defined Functions

11.4.1 CALL Is SPL's Design Strength

Every SPL workflow makes exactly two kinds of operations. GENERATE calls an LLM — probabilistic, creative, slow, expensive. CALL calls a Python function — deterministic, testable, instant, free. This distinction is not a syntax choice. It is an architectural principle.

The Anthropic editorial review of SPL 2.0 called this split out explicitly as the language's most important strength:

> *CALL vs. GENERATE is the clearest design decision in SPL. It forces the workflow author to be explicit about which operations are deterministic and which are probabilistic. Most LLM frameworks blur this line — SPL draws it in ink.*

CALL is SPL's bridge into the Python ecosystem. The @spl_tool decorator is a Foreign Function Interface: any Python callable — from a one-liner to a 500-line class method that queries a database, calls a REST API, runs a scikit-learn model, or reads a file — becomes a first-class SPL operation with a single line.

```
@spl_tool
def lookup_customer(customer_id: str) -> str: ...    # database query
```

```
CALL lookup_customer(@id) INTO @customer_json          — deterministic, zero LLM cos
GENERATE write_response(@customer_json) INTO @reply  — LLM for the language part o
```

The entire Python ecosystem — NumPy, pandas, SQLAlchemy, Requests, scikit-learn, OpenCV, PyTorch inference, any internal SDK — is one decorator away. This is what makes SPL production-grade: not that it can call LLMs, but that it can precisely control *when* it does.

11.4.2 What Becomes Possible

Because any Python function can be a CALL target, SPL workflows can:

Operation	Python library	Example CALL
Query a database	sqlalchemy, psycopg2	CALL db_lookup(@id) INTO @row
Call a REST API	httpx, requests	CALL fetch_weather(@city) INTO @forecast
Run ML inference	scikit−learn, transformers	CALL classify_intent(@text) INTO @intent
Read/write files	pathlib, csv, openpyxl	CALL load_csv(@path) INTO @data
Validate data	pydantic, jsonschema	CALL validate_schema(@json) INTO @errors
Send a notification	slack−sdk, smtplib	CALL notify_slack(@message) INTO @status
Compute embeddings	sentence−transformers	CALL embed(@text) INTO @vector
Detect PII	presidio, regex	CALL detect_pii(@text) INTO @report
Cache results	redis, diskcache	CALL cache_get(@key) INTO @cached
Hash / sign data	hashlib, hmac	CALL sign_payload(@data) INTO @sig

None of these require changes to SPL itself. The language stays small. The ecosystem grows without limit.

11.4.3 How CALL Dispatches

When SPL encounters CALL foo(@arg), it resolves the function name in this order:

1. **Registered Python tool** — loaded via −−tools tools.py or from stdlib
2. **SPL PROCEDURE** — defined earlier in the same .spl file
3. **LLM fallback** — treated as a GENERATE if nothing else matches

UDFs take priority over everything else. If you register a tool named upper, it overrides the stdlib upper. Use this consciously — override when you need domain-specific behaviour; otherwise rely on stdlib.

11.4.4 The `@spl_tool` Decorator

The entire UDF system is a single decorator. Any Python callable decorated with @spl_tool becomes available as a CALL target:

```
# tools.py
from spl.tools import spl_tool

@spl_tool
def word_count(text: str) -> int:
    """Count words in a string."""
    return len(text.split())

— workflow.spl
CALL word_count(@response) INTO @n_words
```

That is the complete API. No registration table to maintain, no base classes to inherit, no configuration files. The decorator handles everything.

11.4.5 section 11.4 — The Definitive Example

section 11.4 (Tool-Use / Function-Call) is the reference implementation of the UDF pattern. It embodies the core architectural principle:

> **LLMs for language. Python for computation.**

```python
# cookbook/36_tool_use/tools.py

@spl_tool
def sum_values(csv: str) -> str:
    """Sum a comma-separated list of numbers."""
    values = [float(v.strip()) for v in csv.split(",") if v.strip()]
    return str(sum(values))

@spl_tool
def percentage_change(old: str, new: str) -> str:
    """Compute percentage change from old to new."""
    old_f = float(old.replace(",", "").strip())
    new_f = float(new.replace(",", "").strip())
    if old_f == 0:
        return "N/A"
    change = ((new_f - old_f) / abs(old_f)) * 100
    sign = "+" if change >= 0 else ""
    return f"{sign}{change:.2f}"

@spl_tool
def format_currency(amount: str, symbol: str = "USD") -> str:
    """Format as currency string: 'USD 1,234.56'."""
    val = float(amount.replace(",", "").strip())
    return f"{symbol} {val:,.2f}"
```

The SPL workflow delegates all arithmetic to these tools and sends only the narrative summary to the LLM:

```
WORKFLOW sales_analysis
  INPUT:  @sales TEXT, @prev_total TEXT, @period TEXT
  OUTPUT: @report TEXT
DO
  -- Deterministic math -- zero LLM tokens, no hallucination risk
  CALL sum_values(@sales)                      INTO @total
  CALL average_values(@sales)                  INTO @avg
  CALL percentage_change(@prev_total, @total)  INTO @growth
  CALL format_currency(@total)                 INTO @total_fmt

  -- LLM only writes the narrative
  GENERATE sales_report(@period, @total_fmt, @avg, @growth) INTO @report
  COMMIT @report
END
```

Run it:

```
spl run cookbook/36_tool_use/tool_use.spl \
    --adapter ollama -m gemma3 \
    --tools cookbook/36_tool_use/tools.py \
    sales="1200,1450,1380,1600,1750,1900" \
    prev_total="7800" \
    period="H1 2025"
```

11.4.6 Writing Good UDFs

11.4.6.1 Accept and return strings

SPL variables are strings. Tools receive strings and should return strings (or Python primitives that get coerced to strings). Avoid returning complex objects.

```
# Good — returns a string
@spl_tool
def extract_domain(email: str) -> str:
    return email.split("@")[-1] if "@" in email else ""

# Also fine — Python int/float returned, coerced by executor
@spl_tool
def word_count(text: str) -> int:
    return len(text.split())
```

11.4.6.2 Fail safely

LLM output is unpredictable. Guard against malformed input rather than raising exceptions that halt the workflow.

```
@spl_tool
def parse_score(text: str) -> float:
    """Extract a 0-1 score from LLM text. Returns 0.0 on any failure."""
    import re
    m = re.search(r"\b(0?\.\d+|1\.0|[01])\b", str(text))
    return float(m.group()) if m else 0.0
```

11.4.6.3 Pure functions when possible

Tools without side effects are trivially testable and composable. Reserve I/O (database, API, file) for tools that genuinely need it.

```
# Pure — easy to test in isolation
@spl_tool
def normalize_name(name: str) -> str:
    return " ".join(w.capitalize() for w in name.strip().split())

# Impure — I/O justified; wrap carefully
@spl_tool
```

```
def lookup_customer(customer_id: str) -> str:
    row = db.query("SELECT name, tier FROM customers WHERE id = ?", customer_id)
    return json.dumps(row) if row else ""
```

11.4.6.4 Document with docstrings

The docstring becomes the tool's description in spl explain output. Write it for the workflow author, not the Python author.

```
@spl_tool
def detect_language(text: str) -> str:
    """Detect the ISO 639-1 language code for the input text.

    Returns a 2-letter code like 'en', 'fr', 'zh'.
    Returns 'unknown' if detection confidence is below threshold.
    Requires: pip install langdetect
    """
    ...
```

11.4.7 Custom Name

If the Python function name cannot be used in SPL (conflicts with a keyword, or you want a shorter name), pass name= to the decorator:

```
@spl_tool(name="pct_change")
def compute_percentage_change_with_guard(old: str, new: str) -> str:
    ...
```

```
CALL pct_change(@prev, @curr) INTO @delta
```

11.4.8 Multiple Tools in One File

A single tools.py can register any number of tools. Group them by domain:

```
# tools.py — order management tools

@spl_tool
def lookup_order(order_id: str) -> str:
    """Return JSON with order status and items for an order ID."""
    ...

@spl_tool
def extract_order_numbers(text: str) -> str:
    """Extract all ORD-XXXXXX order numbers from text as a comma-separated list."""
    import re
    return ", ".join(re.findall(r"ORD-\d{6}", text))

@spl_tool
```

```
def format_order_summary(order_json: str) -> str:
    """Format an order JSON payload as a human-readable summary."""
    ...
```

Load all of them with a single flag:

```
spl run workflow.spl --tools tools.py
```

11.4.9 Async Tools

For I/O-bound operations (HTTP calls, database queries), define the tool as an async def. The SPL executor runs inside asyncio, so async tools compose naturally:

```
import httpx
from spl.tools import spl_tool

@spl_tool
async def fetch_webpage(url: str) -> str:
    """Fetch the text content of a webpage (first 4000 chars)."""
    async with httpx.AsyncClient(timeout=10) as client:
        r = await client.get(url)
        return r.text[:4000]
```

```
CALL fetch_webpage(@url) INTO @page_text
GENERATE summarize(@page_text) INTO @summary
```

11.4.10 Stdlib vs. UDF: The Decision Rule

Question	Answer
Is it a general-purpose string/numeric/date operation?	Use **stdlib**
Does it require external I/O (DB, API, file)?	Write a **UDF**
Is it domain-specific (PII, financial, medical)?	Write a **UDF**
Could it appear in any SQL database's built-in list?	Use **stdlib**
Does it encode business rules specific to your org?	Write a **UDF**

The stdlib exists so you never write to_float or upper again. UDFs exist so you can encode the things that only your team knows.

11.4.11 Testing Your Tools

Because tools are plain Python, they test with plain pytest — no LLM runtime needed:

```
# test_tools.py
from tools import extract_order_numbers, parse_score

def test_extract_orders():
```

```
    text = "Customer mentioned ORD-123456 and ORD-789012 were delayed."
    assert extract_order_numbers(text) == "ORD-123456, ORD-789012"

def test_parse_score_lenient():
    assert parse_score("Quality: 0.87 out of 1.0") == 0.87
    assert parse_score("no number here") == 0.0
    assert parse_score("") == 0.0

pytest test_tools.py -v
```

Testing tools in isolation — before wiring them into a workflow — is the fastest feedback loop in SPL development. Fix the Python logic here, not inside a 30-second LLM run.

11.4.12 The Guardrails Recipe: A Real UDF Pattern

section 4.2 (Guardrails Pipeline) shows UDFs doing exactly what an LLM should not: deterministic pattern matching.

```
# cookbook/18_guardrails/tools.py (excerpt)

@spl_tool
def classify_input_keywords(text: str) -> str:
    """Keyword pre-screen: returns 'harmful', 'off_topic', or 'safe'."""
    lower = text.lower()
    for kw in _HARMFUL_KEYWORDS:
        if kw in lower:
            return "harmful"
    for kw in _OFF_TOPIC_KEYWORDS:
        if kw in lower:
            return "off_topic"
    return "safe"

@spl_tool
def detect_pii(text: str) -> str:
    """Regex PII detection. Returns 'pii_found' or 'clean'."""
    ...
```

The EVALUATE in the workflow branches on these exact string values — no LLM involved, no hallucination risk, microsecond latency:

```
CALL classify_input_keywords(@user_input) INTO @keyword_class
EVALUATE @keyword_class
    WHEN 'harmful'   THEN COMMIT 'I cannot help with that.' WITH status = 'blocked'
    WHEN 'off_topic' THEN COMMIT 'Outside my scope.'         WITH status = 'blocked
    OTHERWISE              -- pass through to LLM classification
END
```

This is the UDF pattern at its best: deterministic gates that protect LLM calls, not replace them.

Chapter 11.5

The Digital Duck Ecosystem

SPL is the conductor. The digital-duck libraries are the orchestra.

When you run an SPL workflow, you are not just running SPL. The runtime delegates to a set of purpose-built, open-source Python libraries — each one a clean abstraction over a class of infrastructure concern. This appendix introduces them.

The goal of each dd−∗ library is the same: **one consistent API, many swappable backends**. Just as SPL lets you swap −−adapter ollama for −−adapter anthropic without changing your .spl file, the dd−∗ libraries let you swap SQLite for DuckDB, FAISS for ChromaDB, or sentence-transformers for OpenAI embeddings — without changing the SPL layer above them.

This is the same principle at every level of the stack: declarative at the top, swappable at the bottom.

11.5.1 The Stack at a Glance

```
                    SPL workflows (.spl)
              GENERATE · CALL · EVALUATE · STORE

                      SPL runtime (spl/)


dd-logging      dd-config       dd-db / dd-vectordb / dd-embed
dd-llm          dd-cache        dd-extract / dd-format / dd-dogic
```

The SPL runtime calls into dd−∗ libraries for infrastructure concerns. Your .spl code never sees the library — it sees only the SPL primitive (GENERATE, CALL, @memory[...]). The library swap happens in configuration, not in code.

11.5.2 The Libraries

11.5.2.1 `dd-config` — Configuration Management

Purpose: Load, merge, and validate configuration from any format — YAML, JSON, TOML, INI, or .env — with dot-path access and environment variable interpolation.

Key API:

```
from dd_config import Config

cfg = Config.load("~/.spl/config.yaml")
adapter = cfg.get("adapter", default="ollama")
cfg.validate(schema)
```

SPL use: The ~/.spl/config.yaml file that drives spl run defaults — adapter, model, log level, storage path — is loaded via dd−config. Change the format of your config file without touching SPL.

11.5.2.2 `dd-logging` — Shared Logging

Purpose: Consistent, timestamped-file logging for CLI and long-running applications. Hierarchical logger support with automatic handler management.

Key API:

```
from dd_logging import setup_logging, get_logger

setup_logging(log_dir="~/.spl/logs", log_level="INFO")
log = get_logger("memory_chat")
log.info("Response ready")
```

SPL use: Every LOGGING ... LEVEL INFO statement in a workflow maps to a dd−logging call. Log files in ~/.spl/logs/ are created and managed by this library.

11.5.2.3 `dd-db` — Relational Database Abstraction

Purpose: Unified SQL interface over nine relational backends — SQLite, DuckDB, PostgreSQL, MySQL, Snowflake, BigQuery, ClickHouse, SQL Server, Oracle — returning results as pandas DataFrames.

Key API:

```
from dd_db import SQLiteDB, DuckDB, PostgresDB

db = SQLiteDB("~/.spl/memory.db")
db.connect()
df = db.run_query("SELECT value FROM spl_kv WHERE key = :k", {"k": "chat_user_prof
db.disconnect()
```

SPL use: The STORAGE-typed workflow input (@memory STORAGE(sqlite, '~/.spl/memory.db')) is backed by dd−db. The spl memory CLI commands (list, get, set, delete) use the same library. Swap sqlite for duckdb or postgres in the STORAGE declaration and the SPL code is unchanged.

11.5.2.4 `dd-embed` — Embedding Abstraction

Purpose: Shared embedding model adapter layer with seven built-in backends — sentence-transformers, HuggingFace, Ollama, OpenAI, OpenRouter, Gemini, Voyage — plus a disk-persistent embedding cache.

Key API:

```
from dd_embed import get_adapter

adapter = get_adapter("sentence_transformers", model_name="all-MiniLM-L6-v2")
result = adapter.embed(["Who is Wen?", "What is SPL?"])
vectors = result.embeddings   # numpy array, shape (2, 384)
```

SPL use: spl doc−rag add and spl code−rag import use dd−embed to convert text into vectors before storing them. The embedding model is locked in vector_config.json at index creation time — swap it by deleting the index and re-importing with a different provider.

11.5.2.5 `dd-vectordb` — Vector Database Abstraction

Purpose: Unified semantic search interface over four vector backends — in-memory NumPy, FAISS, ChromaDB, and Qdrant — with consistent add/search/delete API and metadata filtering.

Key API:

```
from dd_vectordb import FAISSVectorDB, Document, SearchResult

db = FAISSVectorDB(dimension=384)
db.add_documents([Document(id="p1", text="SPL is declarative", vector=vec)])
results: list[SearchResult] = db.search(query_vector, k=3)
db.save("~/.spl/vectors.faiss")
```

SPL use: spl doc−rag uses FAISSVectorDB for document retrieval. spl code−rag uses ChromaVectorDB for SPL example lookup. The rag.query(context.question, top_k=3) call in a PROMPT statement routes through this library. Swap FAISS for Qdrant in configuration without touching any .spl files.

11.5.2.6 `dd-llm` — LLM Abstraction

Purpose: Shared LLM adapter layer with six built-in backends — Claude CLI, Anthropic API, OpenAI, Gemini, OpenRouter, Ollama — with multi-provider retry, exponential backoff, and automatic fallback.

Key API:

```
from dd_llm import get_adapter, LLMResponse

adapter = get_adapter("ollama", model="phi4")
response: LLMResponse = adapter.call("Explain quantum computing in one sentence.")
print(response.text, response.tokens_used)
```

SPL use: Currently, SPL has its own adapter layer (spl/adapters/). Migrating to dd−llm is on the roadmap — it would consolidate 13 SPL adapters into the shared library, giving all dd−∗ tools and SPL a unified LLM backend.

11.5.2.7 `dd-cache` — Caching Abstraction

Purpose: Backend-swappable caching with TTL support — in-memory, SQLite disk cache, and Redis — with a get_or_set() pattern for lazy population and automatic cache statistics.

Key API:

```
from dd_cache import DiskCache

cache = DiskCache("~/.spl/cache.db", ttl=3600)
result = cache.get_or_set("embed:Who is Wen?", lambda: adapter.embed(["Who is Wen?'
```

SPL use: spl run −−cache enables response caching for GENERATE calls. The backing store will migrate to dd−cache — letting users choose in-memory caching for development or Redis for production without changing the SPL layer.

11.5.2.8 `dd-extract` — Document Extraction

Purpose: Abstraction layer for extracting structured text from PDFs and other document formats, returning clean Markdown.

Key API:

```
from dd_extract import PDFExtractor

extractor = PDFExtractor()
markdown = extractor.extract("report.pdf")
```

SPL use: section 3.2 (Chat Over PDF) and future document-processing recipes will use dd−extract to pull text out of PDFs before indexing them with spl doc−rag add.

11.5.2.9 `dd-format` — Document Formatting

Purpose: Convert Markdown to PDF, DOCX, and HTML output formats.

Key API:

```
from dd_format import markdown_to_pdf, markdown_to_docx

markdown_to_pdf("# Report\nSPL is great.", "output.pdf")
markdown_to_docx(text, "output.docx")
```

SPL use: Planned for report-generation recipes — workflows that produce structured Markdown outputs and need to render them as polished documents.

11.5.2.10 `dd-dogic` — Document Conversion Pipeline

Purpose: Orchestrates document conversion pipelines using a PocoFlow (Extract → Transform → Format) pattern — converting between PDF, Markdown, HTML, and DOCX.

Key API:

```
from dd_dogic import convert

convert("report.pdf", output_format="markdown")
```

SPL use: High-level document processing pipelines that span extraction, transformation, and formatting. dd−dogic composes dd−extract and dd−format into a single orchestrated flow — the same "composition layer" philosophy that SPL applies to LLM workflows.

11.5.3 Integration Status

Library	Status in SPL
dd−config	Integrated — drives ~/.spl/config.yaml
dd−logging	Integrated — backs all LOGGING statements
dd−db	Integrated — backs STORAGE-typed inputs and spl memory CLI
dd−embed	Integrated — backs spl doc−rag and spl code−rag
dd−vectordb	Integrated — backs semantic search in RAG recipes
dd−llm	Roadmap — will consolidate 13 SPL LLM adapters
dd−cache	Roadmap — will back spl run −−cache
dd−extract	Roadmap — will back PDF ingestion in document recipes
dd−format	Roadmap — will back Markdown-to-document output recipes
dd−dogic	Roadmap — will back multi-format document pipelines

11.5.4 Where to Find Them

All dd−∗ libraries are developed by the Digital Duck team and hosted at:

```
https://github.com/digital-duck/
```

Each library follows the same conventions: clean abstract base class, multiple concrete adapters, consistent get_adapter() factory, zero mandatory dependencies (extras install what you need). The same philosophy that makes SPL portable makes every library in the stack portable.

If you build tools on top of SPL — custom adapters, new storage backends, domain-specific RAG pipelines — the dd−∗ libraries are the natural foundation. The stack is open, documented, and designed to be extended.

Chapter 11.6

For Decision-Makers

Written for CTOs, engineering leads, and technical decision-makers evaluating AI orchestration tools for their teams. Covers the governance case for SPL, competitive positioning, deployment model, and the honest adoption risk.

11.6.1 The Question That Matters

When evaluating an AI orchestration tool, the instinct is to ask "which framework is most capable?" The right question is different: **who on your team can build, read, and maintain these workflows — and who cannot?**

Imperative frameworks (LangGraph, AutoGen, CrewAI) require Python proficiency, framework-specific abstractions, and careful state management. Your senior software engineers can build with them. Your data analysts, BI developers, domain experts, and SQL practitioners cannot — not without significant retraining.

SPL changes that denominator.

11.6.2 Who Can Contribute With SPL

If your team includes SQL practitioners — and most data-adjacent teams do — they can write, read, and review SPL workflows without Python training. The syntax maps directly to what they already know:

What they know	What it maps to in SPL
SELECT query	SELECT context assembly
Stored procedure	WORKFLOW
IF / CASE WHEN	EVALUATE ... WHEN
BEGIN ... EXCEPTION ... END	WORKFLOW ... EXCEPTION ... END
CREATE FUNCTION	CREATE FUNCTION (prompt template)

The transfer is structural, not a metaphor. A data engineer who has written PL/SQL will recognize the SPL pattern within the first recipe.

More importantly: **business stakeholders can read an SPL script and approve it** without executing it. The intent is explicit in the syntax. "Generate a draft, evaluate its quality, refine if below threshold, commit the result" is readable by anyone. The equivalent Python implementation is not. This is a governance model that imperative code cannot provide, regardless of how good the documentation is.

11.6.3 The Three Roles, One Language

The readability benefit is not uniform across all roles. Different people find different things in the same SPL file:

Role	What they read in SPL
Business Analyst	WORKFLOW, INPUT, OUTPUT, GENERATE, prompt text inside CREATE FUNCTION — the *what*
Implementer	CALL, USING MODEL, WHILE, EXCEPTION WHEN — the *how*; Python tools for deterministic computation
Governance Approver	INPUT/OUTPUT contract, EXCEPTION handlers, COMMIT WITH status= — the audit trail

This has a practical consequence for the development cycle. The conventional path with Python-based frameworks involves three sequential handoffs:

```
sequenceDiagram
    participant BA as Business Analyst
    participant Dev as Developer
    participant GA as Governance Approver
    BA->>Dev: requirements doc
    Dev->>Dev: translate + build (Python / LangGraph)
    Dev->>GA: explanation doc
    GA->>Dev: review + change requests
    Dev->>BA: re-spec or accept?
```

Three handoffs. Three translation layers. Each one is a delay and a potential source of drift between intent and implementation.

With SPL, all three roles work from the same file:

```
flowchart LR
    BA["Business Analyst"]
    Dev["Implementer"]
    GA["Governance Approver"]
    SPL(["workflow.spl"])
    BA <-->|read + write| SPL
    Dev <-->|read + write| SPL
    GA <-->|read + audit| SPL
```

Whether this potential is fully realised depends on team adoption and workflow complexity — but the structural conditions for it are present in the language design.

11.6.4 Documentation That Stays Current

Python-based AI workflows accumulate a documentation problem over time. The TypedDict drifts from the README. The routing logic is explained in a Confluence page that was last updated six months ago. The developer who understood the original graph left the team.

The load-bearing structural elements of an SPL recipe are resistant to this problem because they are executable, not descriptive:

SPL construct	What it documents — and cannot misrepresent
CREATE FUNCTION f(...) AS $$ prompt $$	Exactly what the AI is asked, in plain English
INPUT: @param TYPE DEFAULT v	The full parameter contract — types and defaults
GENERATE f() USING MODEL @m	Which model is responsible for which task
CALL python_tool()	Where deterministic logic replaces LLM inference
EXCEPTION WHEN X THEN	Every known failure mode and its handling
COMMIT ... WITH status=	The observable output contract

These cannot drift from the running behaviour because they *are* the running behaviour. Prose comments can still go stale, as in any language. But the contract — inputs, outputs, exceptions, model assignments — is always in sync with what executes.

11.6.5 A Concrete Comparison: Ensemble Voting in SPL and LangGraph

Chapter 6.3 includes a detailed implementation of the ensemble voting recipe in both SPL and LangGraph, using the same algorithm, the same defaults, and the same log output. The comparison is included as working, runnable code.

The headline numbers from that comparison:

Dimension	SPL	LangGraph
Non-comment lines	**107**	**213**
State management	implicit @variables	explicit TypedDict with 14 fields
Loop	WHILE @i < @n DO	conditional edge + routing function
Model dispatch	GENERATE ... USING MODEL @m	ChatOllama(model=m).invoke(prompt)
CLI entry point	INPUT: block	10-line argparse block

Dimension	SPL	LangGraph
Graph wiring	sequential — read top to bottom	add_node, add_edge, compile()

The 2× line difference is a symptom of a deeper design difference: SPL makes the workflow the primary artefact; LangGraph makes the graph the primary artefact. LangGraph is the right choice when you need fully dynamic agent construction, complex memory systems, or tight enterprise platform integration. For the large majority of production AI workflow patterns — all 37 recipes in this book — the logic is expressible declaratively, and the declarative form is readable by the whole organisation, not just the engineers who built it.

11.6.6 Deployment and Promotion Model

The same .spl file promotes unchanged through Dev, Test, and Production by swapping CLI flags:

```
# Development — local Ollama, fast iteration
spl run workflow.spl --adapter ollama -m gemma3 --tools dev_tools.py

# Test — cloud model, production tools, test data
spl run workflow.spl --adapter claude_cli -m claude-haiku-4-5 --tools prod_tools.py

# Production — same file, production adapter and data
spl run workflow.spl --adapter openrouter -m gpt-4o --tools prod_tools.py --dataset
```

No environment-specific code paths. No re-review across environments. The artifact that your team reviewed in Dev is byte-for-byte identical to what runs in Production. The only changes are external configuration.

11.6.7 Lines of Code: SPL vs Imperative Frameworks

Across the 37 recipes in this book, SPL consistently produces 5–7× fewer lines of code than equivalent LangGraph or AutoGen implementations for the same workflow patterns. The comparison is documented in the accompanying arxiv paper.

The LOC reduction is not the primary benefit — it is a symptom of the primary benefit, which is that the declarative abstraction removes boilerplate that does not express business logic. The code that remains is the code that matters.

11.6.8 The Efficient Runtime: Token Cost Is a Real Budget Line

This is the most consequential design decision in SPL, and the one that most directly affects your team's AI operating costs.

Every token sent to an LLM for a deterministic operation — something that could have been computed by code — is a token wasted in latency, dollars, and unpredictability. SPL makes the efficient choice the *natural* choice.

11.6.8.1 Two Execution Modes, One Language

Every operation in an SPL workflow belongs to exactly one category:

Category	Characteristic	SPL keyword	Token cost
Deterministic	Logic can be expressed as code — precise, reproducible	CALL	Zero
Probabilistic	Requires judgment, generation, or ambiguity resolution	GENERATE	LLM tokens

The author decides at write time. The runtime executes both seamlessly in the same workflow.

Consider chunk_plan(@document): counting tokens, estimating boundaries, deciding chunk sizes. Pure computation. No judgment needed. **CALL it. Zero tokens.**

Consider summarize_chunk(@chunk): language understanding, compression, judgment. **GENERATE it.**

```
WHILE @chunk_index < @chunk_count DO
    CALL extract_chunk(@document, @chunk_index, @chunk_count) INTO @chunk
-- deterministic, 0 tokens
    GENERATE summarize_chunk(@chunk) INTO @chunk_summary
-- probabilistic, costs tokens
    @summaries := list_append(@summaries, @chunk_summary)
    @chunk_index := @chunk_index + 1
END
```

Same workflow. Two execution models. Token cost concentrated exactly where intelligence is genuinely needed.

11.6.8.2 The Decision Rule

> *"If you can write it as code — write it as code. Reserve GENERATE for what only a language model can do."*

Clear logic + clear pattern → CALL. Ambiguity + judgment + generation → GENERATE.

This discipline produces workflows that are fast where speed is possible, intelligent where intelligence is required, and auditable throughout. For cost-conscious teams, it is also the single biggest lever on inference spend.

11.6.8.3 Polyglot Tools

The −−tools mechanism accepts tools written in any language. The @spl_tool Python decorator is the most convenient path, but the underlying contract is simple: a function that takes strings and returns a string.

Language	Mechanism
Python	@spl_tool decorator
Rust	FFI or subprocess — high-performance parsing, cryptography
Go	subprocess / gRPC — concurrent I/O, network tools
Java / JVM	subprocess / gRPC — enterprise systems, existing libraries
JavaScript	subprocess / Node.js — web scraping, JSON manipulation
Any binary	If it reads stdin and writes stdout, it works

Existing enterprise logic written in any language becomes a first-class CALL-able tool in SPL — without rewriting it in Python and without spending a single LLM token on it.

11.6.8.4 Momagrid: The Fourth Dimension

The three dimensions above describe *what* runs and *how*. Momagrid answers *where*.

Rather than a single machine or a centralized cloud, Momagrid treats the world's compute as a shared resource, routing workflow steps to wherever execution is cheapest, fastest, or most private. For SPL, this changes the deployment model fundamentally: a workflow written for local execution runs on Momagrid without modification — the runtime handles dispatch, scheduling, and result aggregation.

For teams evaluating infrastructure spend: Momagrid is the path from "cloud API cost" to "community compute at commodity pricing."

11.6.9 Competitive Landscape

Dimension	SPL	LangGraph	AutoGen	CrewAI
Language required	SPL (SQL-like)	Python	Python	Python
SQL practitioners	Yes	No	No	No
Business Analyst readable	Yes	No	No	Partial[1]
Governance auditable	Yes	No	No	No
End-user transparent	Yes	No	No	No
Docs cannot drift	Yes	No	No	No
Adapter portability	Yes	No	No	No
Commodity GPU	Yes	Vary[2]	Vary[2]	Vary[2]
Dynamic agents	Limited	Yes	Yes	Yes
Ecosystem maturity	Early	Mature	Mature	Mature

[1] *CrewAI's YAML agent/task config is more approachable than LangGraph graph wiring, but covers roles rather than full workflow logic.*

[2] *Depends on the model chosen; smaller open-weight models run on commodity hardware, frontier API models do not.*

SPL occupies a distinct niche: declarative, portable, accessible to non-Python teams, and designed for workflows whose logic is expressible without dynamic agent construction. LangGraph, AutoGen,

and CrewAI are the right choice when you need fully dynamic agent construction, complex memory systems, or tight enterprise platform integration.

Most agentic workflows are not in that category. The 37 recipes in this book cover the patterns that represent the large majority of production AI workflow use cases — and all of them are expressible declaratively.

11.6.10 The Adoption Risk: An Honest Assessment

SPL is not yet a mainstream tool. It does not have a large enterprise backer, a mature ecosystem, or a large Stack Overflow community. A team that adopts SPL today is making a bet on a language that is still establishing its user base.

This is a real risk, and it deserves a direct response:

What you get if SPL succeeds: a team where SQL practitioners contribute AI workflows, where stakeholders can review and approve automation artifacts, and where the same workflow runs portably from local development to cloud to decentralized grid.

What you get if SPL does not reach mainstream adoption: the mental model and the pattern library. The CALL/GENERATE discipline, the declarative workflow structure, and the 37 patterns in this book transfer directly to LangGraph or AutoGen. The .spl files can be mechanically translated. The thinking does not need to be.

The mitigation strategy: start with internal workflows that do not require production-grade SLAs. Run SPL in parallel with your existing framework on one use case. Evaluate the governance and maintainability benefits against the adoption risk on real work, not benchmarks.

11.6.11 Summary: When to Choose SPL

Choose SPL when: - Your team includes SQL practitioners you want to leverage for AI workflow development - Stakeholder review and approval of workflow logic is a governance requirement - Portability across inference backends (local, cloud, decentralized) is a design requirement - The workflow patterns in this book cover your use cases (they cover most common patterns)

Evaluate alternatives when: - You need fully dynamic agent construction at runtime - Your team is entirely Python-proficient and portability is not a priority - You need production SLAs backed by a mature enterprise support ecosystem today

Chapter 11.7

Glossary

Key terms used throughout this book, in alphabetical order.

Adapter The component of the SPL runtime that translates a workflow's declarative instructions into API calls for a specific LLM provider or infrastructure backend. Adapters handle authentication, prompt formatting, response parsing, and error mapping — so the .spl file does not have to. Built-in adapters include ollama, openrouter, claude_cli, gemini, and momagrid. Swapped with −−adapter at the command line.

Agent / Agentic Workflow A workflow in which one or more LLM calls make decisions that affect subsequent steps — routing logic, loop termination, planning — rather than simply producing output. An agentic workflow *acts* on intermediate results, not just generates them.

CALL The SPL statement for invoking a deterministic Python function registered with the @spl_tool decorator. CALL is zero-token, machine-speed, and perfectly reproducible. It should be used for everything a Python function can do: parsing, arithmetic, file I/O, API lookups, string manipulation. See also: GENERATE.

Commit / COMMIT The SPL statement that finalizes workflow output and writes it to the result. A WORKFLOW must end with a COMMIT on every execution path. The committed value becomes the workflow's observable output.

Context Window The maximum number of tokens an LLM can process in a single call — inputs plus output combined. SPL workflows that accumulate large intermediate results in variables (especially inside WHILE loops) can exhaust the context window. The correct pattern is to pass lightweight summaries between steps, not raw accumulated output.

CREATE FUNCTION The SPL statement for defining a named prompt template that can be injected as system context into a GENERATE call. Analogous to a SQL user-defined function (UDF). A CREATE FUNCTION body contains free-form prompt text with {parameter} substitution slots; it is injected as the system role instruction when the function is called in a GENERATE statement.

Declarative Programming A style of programming in which you describe *what* you want rather than *how* to produce it. SQL is the canonical example: you specify the desired result set, and the query optimizer decides how to retrieve it. SPL applies this principle to LLM workflows: you specify the intent of each step, and the runtime (adapter) handles provider-specific execution details.

EVALUATE The SPL statement for branching workflow execution based on the content of an LLM-generated value. Analogous to a semantic WHERE or CASE clause. Unlike a deterministic conditional (IF value == 'x'), EVALUATE matches against language model output, which means the branch taken is probabilistic — the same input may route differently across runs. This is expected behavior for semantic branching.

EXCEPTION The SPL block that handles runtime errors and failure modes. Analogous to BEGIN...EXCEPTION...END in PL/SQL. Named exception types (e.g., GenerationError, MaxIterationsReached, BudgetExceeded) allow the workflow to handle different failures differently — graceful degradation, partial commit, early termination — without crashing the entire workflow.

The AI Quartet The four-member research team that produced this book: Wen Gong (human author and architect), Claude (Anthropic), Gemini (Google), and Z.ai (ZhiPu). Each brought a distinct perspective: Wen's architectural vision and lived experience, Claude's technical precision, Gemini's editorial breadth, and Z.ai's adversarial critical review.

GENERATE The SPL statement for invoking an LLM. GENERATE costs tokens, takes time, and is probabilistic — the same input will produce slightly different output across runs. It should be reserved for steps that genuinely require language understanding, reasoning, or generation. For everything code can do, use CALL instead.

GTX 1080 Ti The reference hardware used for all benchmarks in this book — an NVIDIA graphics card from 2017 with 11 GB VRAM, available on the secondhand market for under $200. Its use is deliberate: it represents the floor of what is achievable on commodity hardware, and it is the GPU a student in the Global South might realistically own or afford.

Hallucination An LLM's generation of plausible-sounding but factually incorrect content. SPL does not eliminate hallucination, but it provides tools to detect and handle it: EVALUATE for semantic checking, EXCEPTION WHEN HallucinationDetected, and the CALL/GENERATE split that keeps deterministic facts in Python functions rather than asking the LLM to produce them.

Imperative Programming A style of programming in which you specify *how* to produce a result, step by step. Python, Java, and most general-purpose programming languages are imperative. LLM framework code (LangChain, AutoGen, CrewAI) is typically imperative: you write the loop, the state machine, the API calls, the error handling. Contrast with *declarative programming*.

LLM (Large Language Model) A neural network trained on large text corpora that can generate, classify, summarize, translate, and reason about text. In SPL, LLMs are invoked exclusively through GENERATE statements. They are the "orchestra" — the models are the musicians, SPL is the score, and the human is the conductor.

Model The specific LLM variant invoked by a GENERATE step. Specified with −−model or −m at the CLI, or with USING MODEL @var inside a workflow. Different models have different capability, cost, and latency profiles. The workflow does not change when the model changes.

Momagrid A decentralized inference network where GPU nodes contributed by independent operators (students, hobbyists, cooperatives, universities) form a shared compute grid. Anyone with a GPU can contribute inference capacity; anyone with an SPL workflow can run it against the grid. Accessed via −−adapter momagrid. Part of the full SPL + Momagrid stack for democratic AI access.

Ollama An open-source tool for running LLMs locally on your own hardware, without API keys or cloud accounts. The default development adapter in this book (−−adapter ollama). Used with

locally pulled models like Gemma 3, Phi-4, Llama 3.2, Mistral, and Qwen.

OpenRouter A cloud API gateway that provides access to a wide range of LLMs (OpenAI, Anthropic, Meta, Mistral, and others) through a single API endpoint. Accessed via ––adapter openrouter.

Part A grouping of related chapters in this book, organized by theme: Foundations, Basics, Agentic Patterns, Reasoning, Safety & Reliability, Memory & Retrieval, Multi-Agent Systems, Applications, Benchmarking & Evaluation, and The Road Ahead. Each Part corresponds to a section divider in the printed book.

PROCEDURE A named, reusable SPL sub-workflow that can be CALLed from within another workflow. Analogous to a SQL stored procedure. Enables modular composition of complex workflows from smaller, independently testable units.

PROMPT The SPL statement for a single, stateless LLM interaction — no workflow state, no loop, no COMMIT. A PROMPT block defines a template and executes it in one shot. Used for simple, single-step generation tasks where a full WORKFLOW would be over-engineering.

Recipe A self-contained unit in this book: a specific pattern, demonstrated with a complete .spl file, annotated line by line, with sample data, a tools.py for deterministic operations, and a readme.md with usage instructions. Designed so you can open any recipe and run it independently.

SELECT The SPL statement for assembling the context that flows into a GENERATE call — the variables, computed values, and system roles that form the LLM prompt. Analogous to the SELECT clause in SQL: you specify what you want to surface, and the adapter handles how to package it for the specific model being called.

SPL (Structured Prompt Language) The declarative language this book teaches. SPL 2.0 extends SPL 1.0's context management primitives (PROMPT, SELECT, GENERATE) with agentic orchestration features (WORKFLOW, EVALUATE, WHILE, EXCEPTION, PROCEDURE). The language is model-agnostic and adapter-portable: the same .spl file runs against any supported backend.

spl-llm The Python package that provides the spl command-line tool. Installed with pip install spl–llm. Provides the runtime that parses .spl files, resolves adapters, manages workflow state, and executes CALL and GENERATE steps.

@spl_tool The Python decorator that registers a Python function as a CALL-able tool in an SPL workflow. Functions decorated with @spl_tool must accept string parameters and return strings. They are loaded at workflow invocation time with ––tools path/to/tools.py.

WHILE The SPL looping construct for iterating a block of steps as long as a condition holds. Used for retry loops, multi-round patterns (debate, refinement), and step-by-step execution over a plan. The loop body should be lightweight: decide or route, not generate large artifacts. Heavy generation belongs outside loops.

WORKFLOW The SPL construct for a named, parameterized, multi-step agentic process. Analogous to a SQL stored procedure. Defines INPUT parameters (with optional defaults), an OUTPUT variable, a DO body (the steps), and an EXCEPTION block (error handling). The primary unit of composition in SPL 2.0.

Terms in italics within definitions are themselves defined in this glossary.

Chapter 11.8

About the Author

A portrait of the person building SPL and Momagrid — physicist, engineer, and advocate for AI as a public utility.

11.8.1 From Anhui to CERN: A Journey of Access

Wen Gong's journey began in a rural village in Anhui, China. In a place where poverty was the baseline and the horizon was often limited to the next field, there was only one path out: education. It was a path he took with a physicist's focus—leading from the University of Science and Technology of China (USTC) to a Ph.D. in nuclear physics in the United States, and eventually to the halls of Lawrence Berkeley National Laboratory and the experimental pits of CERN in Geneva.

This origin story is not mere background; it is the "initial condition" for everything Wen builds today.

Having seen how institutions and access can transform a life, Wen views the current state of AI with a physicist's skepticism and an engineer's pragmatism. Today, the power of AI is locked behind barriers of technical fluency, expensive cloud accounts, and complex programming frameworks. Wen's mission is to dissolve those barriers—not as an act of charity, but as a "debt repayment" to the systems of education and open research that once gave him a chance.

11.8.2 The Physicist's Instinct: Declarative Thinking

Wen's career has spanned the evolution of modern computing—from witnessing the birth of the World Wide Web at CERN to seeing the first distributed computing projects like SETI@home. But it was his training under the legacy of Werner Heisenberg at the Max-Planck-Institut that shaped his deepest technical instinct: **the declarative approach.**

In physics, you cannot always specify the "how"—the unobservable path of a particle. Instead, you formalize the "what"—the observables—and let the underlying mathematics resolve the rest. This is the core philosophy of **SPL 2.0 (Structured Prompt Language)**. Just as SQL allows a data engineer to query millions of rows without knowing the disk's rotation speed, SPL allows an AI practitioner to orchestrate complex "agentic" workflows without writing low-level Python "glue" code.

11.8.3 The AI Quartet and the 48-Hour Sprint

Wen is a practitioner of the **Human × AI Partnership**. This book was produced by a four-member research team he calls the **AI Quartet**: Wen Gong (human author and architect), Claude (Anthropic's AI assistant, for technical execution and precision), Gemini (Google's AI assistant, for breadth and recent literature), and Z.ai (ZhiPu's AI assistant from China, for critical adversarial review). Each brings a different "frequency" to the work:

- **Wen** — vision, architectural intuition, and lived experience building declarative systems
- **Claude** — technical execution, code correctness, and structural precision
- **Gemini** — breadth of reference, research coverage, and editorial feedback
- **Z.ai** — adversarial review, critical assessment, and cross-cultural perspective

This book itself is a product of this collaboration. It was drafted, refined, and validated in an iterative loop that mirrors the "Conductor" metaphor: Wen provides the vision and the "score," while the AI members of the AI Quartet execute, critique, and improve the performance. It is a model of how the next generation of engineers — especially those in the Global South — can scale their impact using commodity hardware like a used GTX 1080 Ti and a declarative language like SPL.

11.8.4 The Vision: AI as a Public Utility

Wen Gong believes that AI computation should be as accessible as the Web. Through **Momagrid** and **SPL 2.0**, he is building a "resilience layer" for AI—a decentralized, open-source infrastructure where a child in a rural village can "conduct their own symphony" of intelligence using a smartphone and a simple, declarative score.

Wen Gong is currently an AI engineer, a former application developer at Oracle and cloud data engineer at Vanguard, and a lifelong builder committed to making AI a universal tool for human potential.